Informatik Einführungskurs



m.rummens@th-bingen.de

TH-Bingen

2026/2027

Übersicht

- 1. Komponenten eines Rechners
- 2. Von-Neumann-Rechner
- 3 Speicher & Daten – Was ist ein Bit, Byte, Wort?
- 4 Zahlensysteme – Dual, Oktal, Hexadezimal
- 5 Programmiersprachen – Unterschiede & Einsatzzwecke
- 6. Flip-Flops – Wie speichert ein Computer ein Bit?
- 7. Kommunikation im System – Bus, Ring, Netzwerk
- 8. Abstrakte Maschinen (APIs, Cloud und die moderne Welt)
- 9. Kryptographie und Blockchain

Intro meiner Person

- 30 Jahre alt
- Wohne in **München** aber aufgewachsen in **Bad Kreuznach**
- Arbeite bei **Airbus Defence and Space**
  - Duales Studium Bachelor in Informationstechnik
  - 2 Jahre im AI-Team als Technical Expert
  - 5 Jahre im Restricted Cloud Team als Solution Architekt und Squad Lead
  - Seit 2025 Head of Containers and PaaS
  - Master in IT-Management berufsbegleitend
- **Hobbies**: Laufen, Smart Home, 3D Druck und Reisen
- **Informatik** seit 15+ Jahre und beruflich seit 10 Jahren



Gründe Informatik zu verstehen



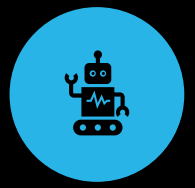
BERUFLICHE MÖGLICHKEITEN



PROBLEME LÖSEN



KREATIVITÄT



AUTOMATISIERUNG



VERSTÄNDNIS DER TECHNOLOGIE



HOHE NACHFRAGE

Warum Informatik in der Verfahrenstechnik?

- Verfahrenstechnik nutzt **computerbasierte Simulationen** und Regelsysteme zur Prozessoptimierung .
- Rechner ermöglichen **Echtzeit-Überwachung und Automatisierung** (Sensoren/Aktoren steuern Anlagen).
- Absolventen arbeiten als Entwickler von Simulationsmodellen und Leitsystemen.
- *Beispiel: Simulationsprogramme bilden chemische Reaktionen nach, dienen als Planungs- und Regelungsmodell.*

Quellen und nützliche Links



google.com

Google Suche Auf gut Glück!

Google gibt es auch auf: English Afrikaans

16.10.2026

If you want the first character to remain lowercase in the resulting camel case string with spaces, you can modify the function to capitalize only the first character of the first word. Here's the updated function:

python Copy code

```python
def snake_case_to_camel_case_with_space(snake_case_string):
    words = snake_case_string.split('_')
    camel_case_with_space = words[0] + ' ' + ' '.join([word.capitalize() for word in
    return camel_case_with_space

# Example usage
snake_case_string = "snake_case_string"
camel_case_with_space = snake_case_to_camel_case_with_space(snake_case_string)
print(camel_case_with_space)  # Output: "snake Case String"
```

ChatGPT

In this modified version, the first word is not capitalized, and only subsequent words are capitalized. This ensures that the first character remains lowercase in the resulting string.

16.10.2026

Wir machen dich erfolgreich.

Auf Studyflix finden über 6 Millionen Schüler/innen und Studierende kostenfreie Lernvideos, Ausbildungsplätze und Jobs.



Lernen und verstehen

Über 6.000 hochwertige Lernvideos für dich.

zum Lernportal

Ausbildung & duales Studium

Finde den perfekten Ausbildungsberuf für dich.

zum Ausbildungsportal

Klassisches Studium

Finde einen Studiengang und eine Hochschule, die zu dir passen.

zum Studienfinder

Jobs, Praktika & Co

Finde den perfekten Beruf für dich.

zum Jobportal

Was willst du lernen?

Zu jedem Thema erklären wir die wichtigsten Inhalte mit hochwertig animierten Videos. Du kannst aus über 6.000 Videos wählen.
Jetzt neu: Teste dein Wissen mit interaktiven Quizzes zu vielen Themen!

Schüler/in | Student/in | Azubi

Studyflix

INFORMATI

Code Issues Pull requests Actions Projects Wiki Security Insights Settings

grundlagen-der-informatik Public

Pin | Watch 0 | Fork 0 | Star 0

main | 1 Branch | 0 Tags | Go to file t | Add file | Code

Marcel Rummens First version of Slide deck — 8c9de82 · 1 minute ago — 4 Commits

.gitignore	Initial commit	4 minutes ago
Informatik Einführungskurs.pdf	First version of Slide deck	1 minute ago
LICENSE.md	Create LICENSE.md	4 minutes ago
README.md	Revise README title and add chapter list	2 minutes ago

README | License

Grundlagen der Informatik

Vorlesung Grundlagen der Informatik, ursprünglich entwickelt für die TH-Bingen

Kapitel Übersicht

1. Komponenten eines Rechners
2. Von-Neumann-Rechner 3 Speicher & Daten – Was ist ein Bit, Byte, Wort? 4 Zahlensysteme – Dual, Oktal, Hexadezimal 5 Programmiersprachen – Unterschiede & Einsatzzwecke
3. Flip-Flops – Wie speichert ein Computer ein Bit?
4. Kommunikation im System – Bus, Ring, Netzwerk
5. Abstrakte Maschinen (APIs, Cloud und die moderne Welt)
6. Kryptographie und Blockchain

About

Vorlesung Grundlagen der Informatik, ursprünglich entwickelt für die TH-Bingen

Readme
View license
Activity
0 stars
0 watching
0 forks

Releases

No releases published
Create a new release

Packages

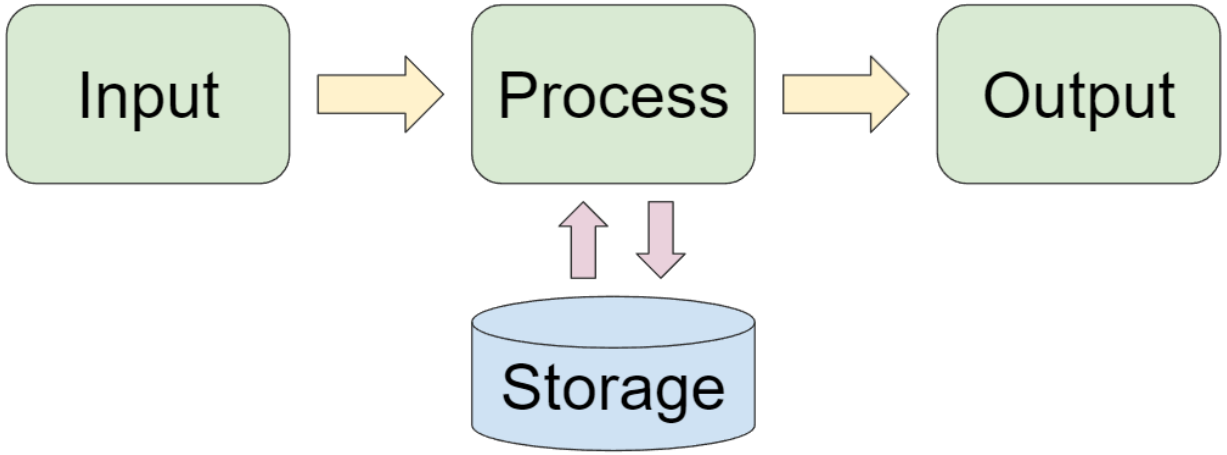
No packages published
Publish your first package

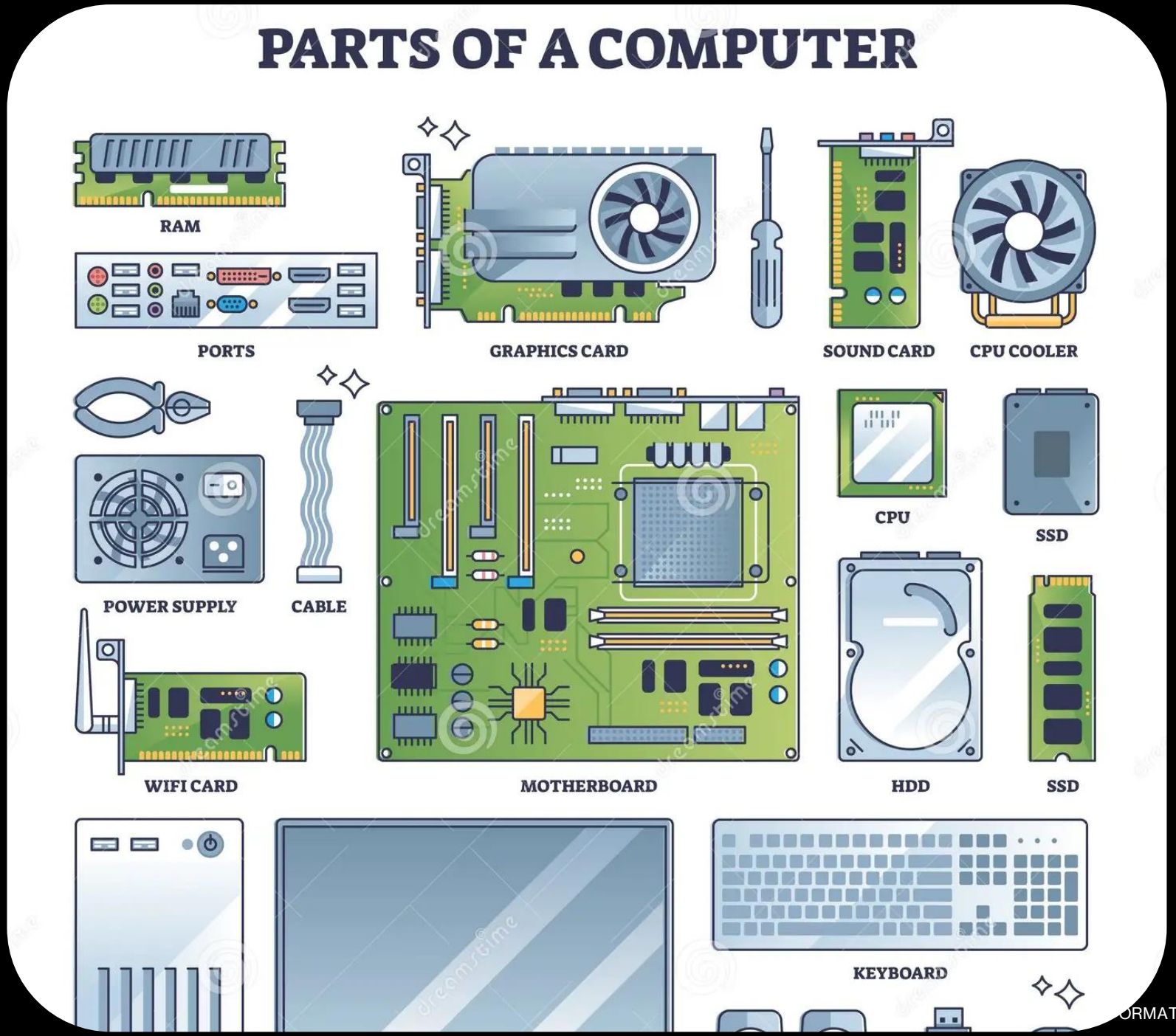
https://github.com/rummens/grundlagen-der-informatik

1. Komponenten eines Rechners

Computer als Datenverarbeiter

- Definition: Ein Computer ist ein Gerät, das mittels programmierbarer Rechenvorschriften Daten verarbeitet .
- EDV (elektronische Datenverarbeitung): Erfassung, Bearbeitung, Transport und Ausgabe von Daten .
- *Beispiel: Messwert einer Anlage (Input) wird berechnet (Prozessor) und als Anzeige oder Steuersignal (Output) ausgegeben.*
- Daten kommen z.B. von Sensoren, Tastaturen etc.; Ausgaben z.B. Bildschirm, Aktoren, Drucker.




PARTS OF A COMPUTER
RAM
PORTS
GRAPHICS CARD
SOUND CARD
CPU COOLER
POWER SUPPLY
CABLE
CPU
SSD
WIFI CARD
MOTHERBOARD
HDD
SSD
KEYBOARD

Wichtigsten Komponenten erklärt

- CPU (Central Processing Unit): Zentrale Recheneinheit mit Rechenwerk (ALU) und Steuerwerk. Führt Arithmetik/Logik aus.
- GPU (Graphics Processing Unit): Spezialisierte Recheneinheit für Grafik oder hoch parallelisierbare Aufgaben (z.B. KI)
- Arbeitsspeicher (RAM): Temporärer Speicher für laufende Programme und Daten.
- Massenspeicher (Festplatte/SSD): Dauerhafte Speicherung großer Datenmengen und Programme (großer, langsamer Speicher).
- Ein-/Ausgabewerk (I/O): Schnittstellen für Benutzereingaben (Tastatur, Maus, Sensoren) und Ausgaben (Bildschirm, Drucker, Aktoren) .
- Bus-System: “Datenautobahn” zwischen CPU, Speicher und I/O .

Merke

- Analogien: CPU ≈ Gehirn (verarbeitet Signale), RAM ≈ Kurzzeitgedächtnis (flüchtige Infos), Festplatte ≈ Archiv (dauerhafte Speicherung), I/O ≈ Sinne und Muskulatur.
- Missverständnisse: Der PC „denkt“ nicht selbst – er führt nur programmierte Befehle aus. Ohne Software passiert nichts.
- Moderne Prozessleitsysteme und SPS arbeiten nach ähnlichen Prinzipien (Inputs → Controller (Rechner) → Outputs). Ein Steuergerät enthält Mikroprozessor, Speicher und I/O.

2. Von-Neumann-Rechner

John von Neumann (1903–1957)

- Geboren in Budapest; Mathematik- und Chemieingenieurstudium; Promotion 1926
- Beiträge zu: Mengenlehre, Quantenmechanik, Operatoralgebren
- Begründer der modernen **Spieltheorie** (Minimax-Theorem, Buch mit Morgenstern)
- Mitarbeit am **Manhattan-Projekt** (Monte Carlo, Implosions-technik)
- Entwickler der **Von-Neumann-Architektur** (EDVAC-Entwurf, IAS-Computer)
- Pionier bei **selbstreproduzierenden Automaten**
- Bekannt für brillanten Intellekt & humorvolle Zitate
- Zahlreiche Auszeichnungen (Medal of Freedom, Enrico-Fermi-Preis)





Der universale Rechner





Sumlock ANITA Mk VII (1961)

Von-Neumann-Rechner Anforderungen



1. Einheitlicher Speicher für Daten und Programme

Daten und Befehle müssen im **gleichen Speicher** abgelegt werden können.



2. Zentrale Verarbeitungseinheit

Rechenwerk, um arithmetische Operationen auszuführen

Steuerwerk zur Kontrolle des Programms



3. Ein-/Ausgabeeinheit

Schnittstellen, um Informationen von außen in das System einzubringen (Eingabe) und Ergebnisse auszugeben (Ausgabe).



4. Gemeinsames Bussystem

Datenbus: transportiert Daten und Befehle zwischen Komponenten.

Steuerbus: überträgt Steuersignale (z. B. Lese-/Schreibbefehle).



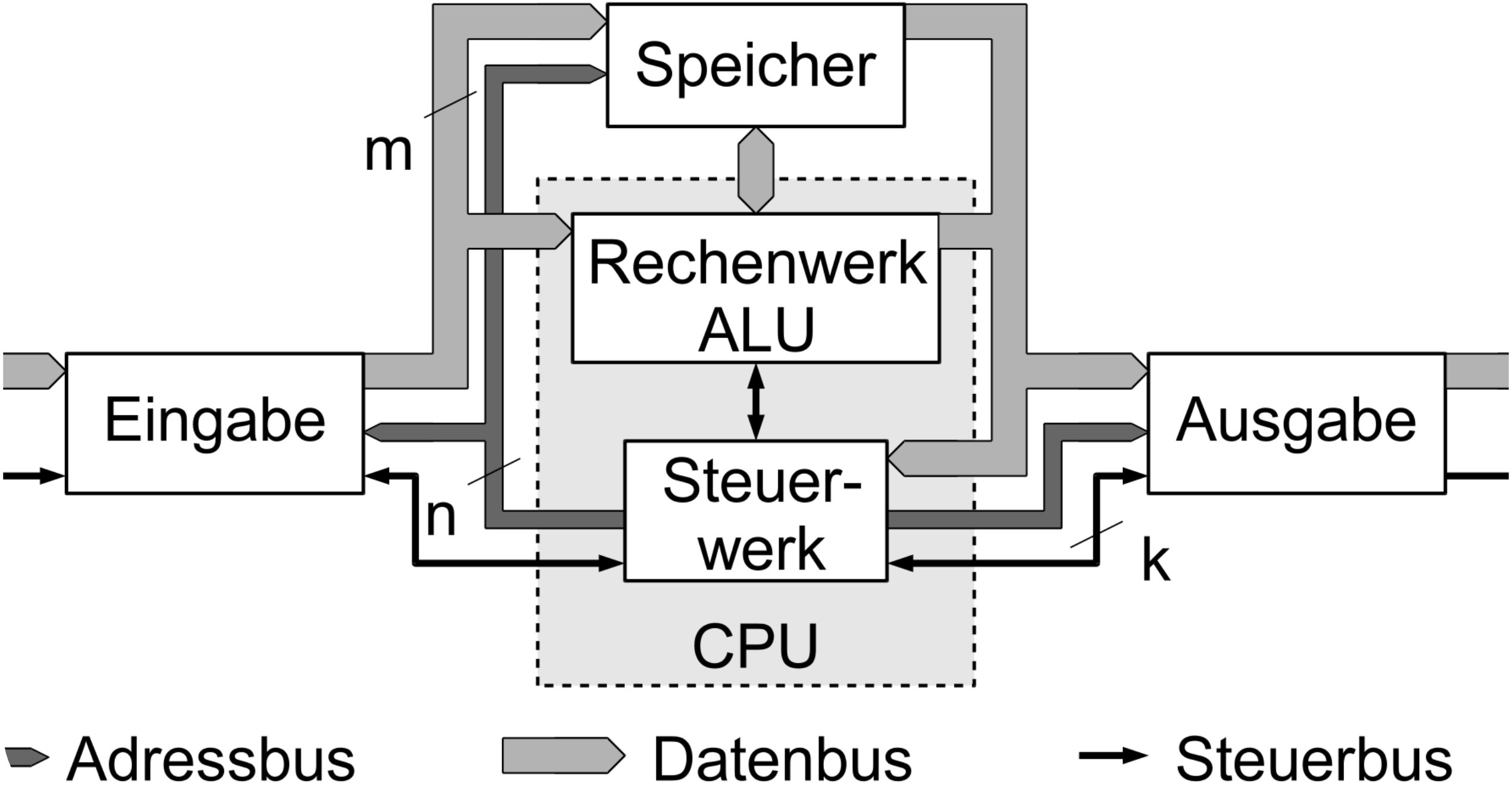
5. Sequentielle Abarbeitung von Befehlen

Die Verarbeitung erfolgt in einem **Befehlshol- und Ausführungszyklus** (Fetch-Decode-Execute).



6. Universalität

Das System muss **beliebige Programme** ausführen können, solange sie im Speicher vorhanden sind – unabhängig von der konkreten Aufgabe.


Speicher
m
Rechenwerk
ALU
Eingabe
Ausgabe
Steuer-
werk
n
k
CPU
Adressbus
Datenbus
Steuerbus

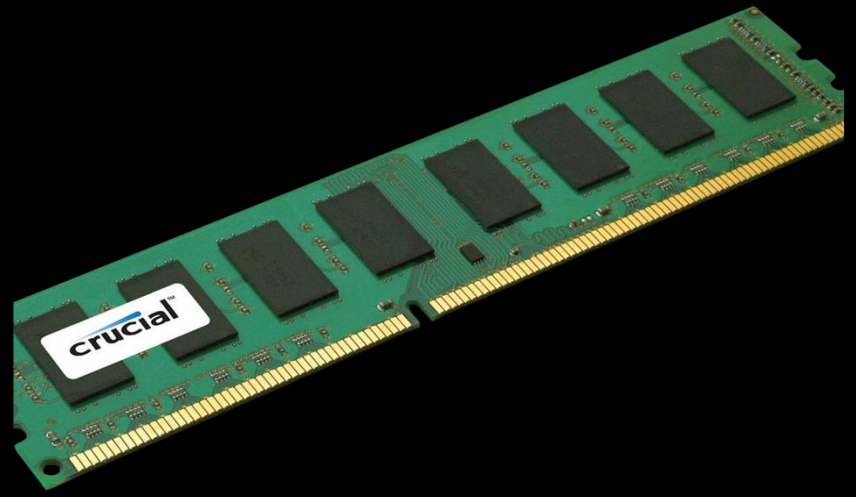


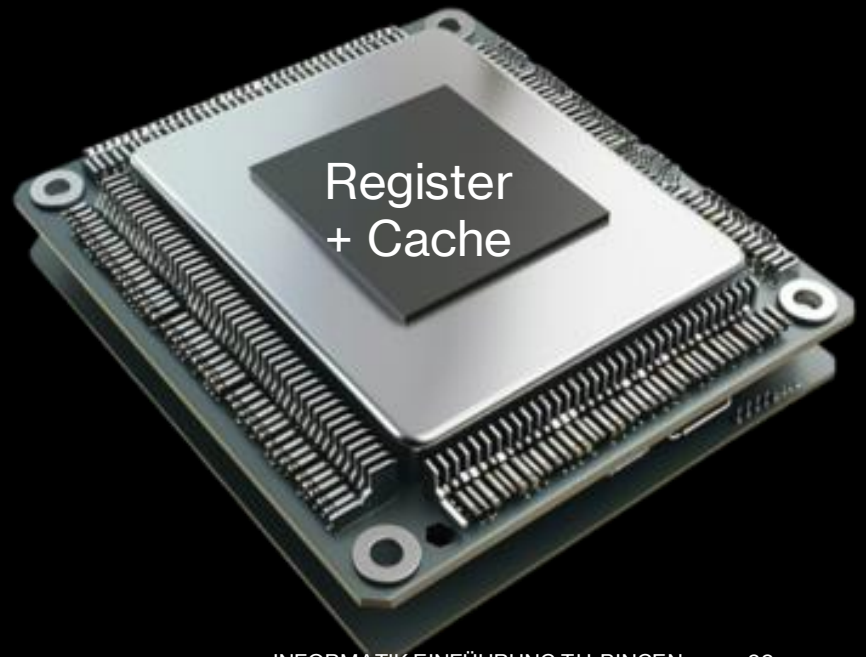
2.1 Speicherwerk

Unterschiedliche Arten von Speicher

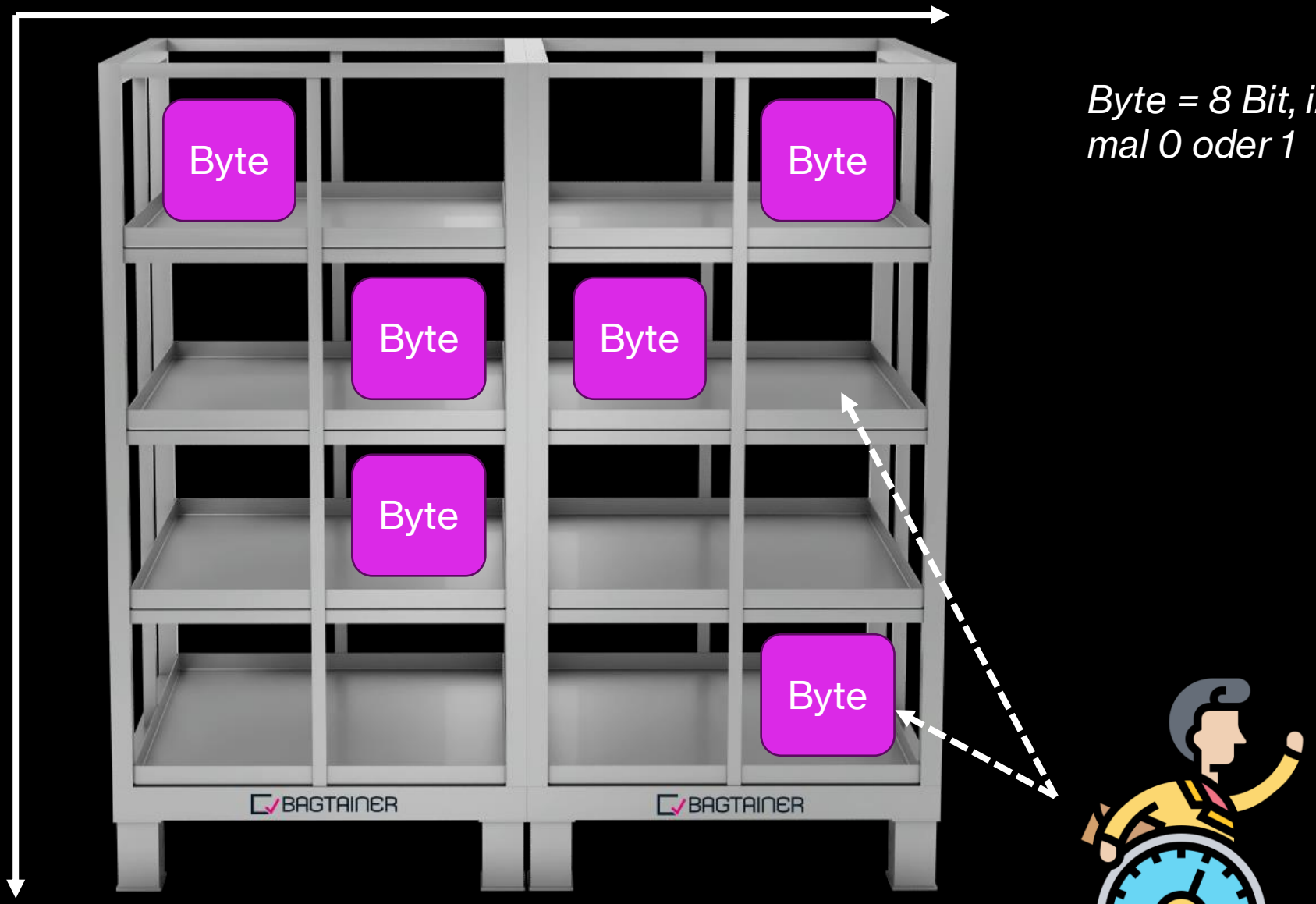






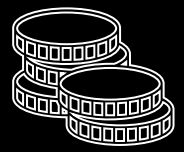


Größe, i.e. Kapazität

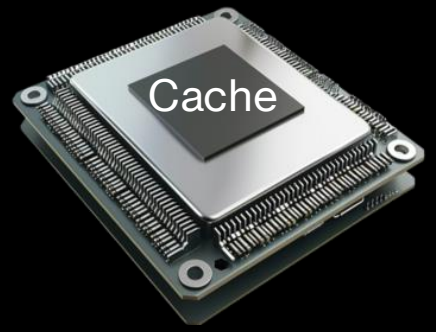


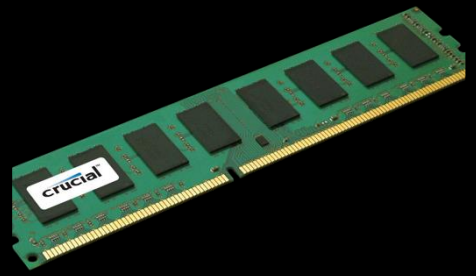
Byte = 8 Bit, i.e. 8 mal 0 oder 1

Kosten













Speed





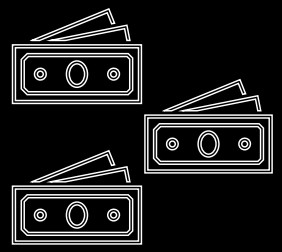


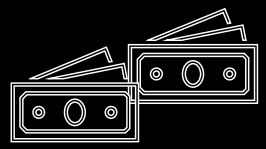


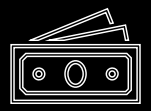


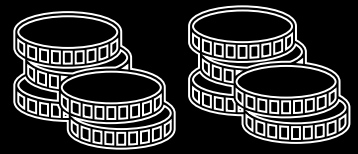
Kapazität	XXXXS (B)	XXS (MB)	M (GB)	L (GB oder TB)	XXXXL (TB)

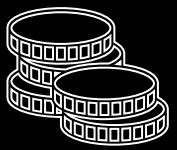
Kosten









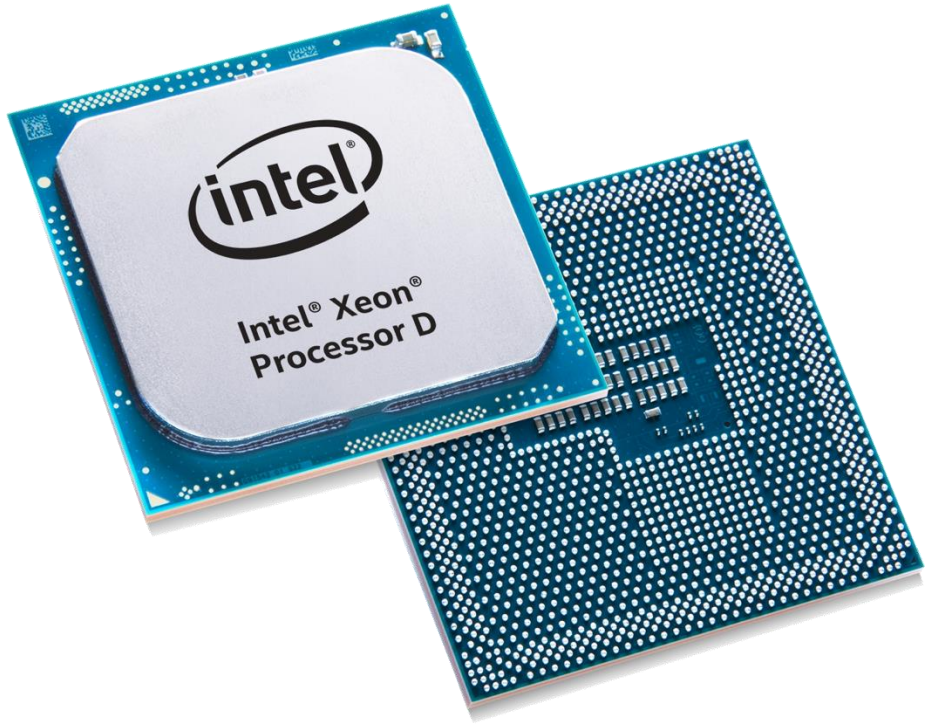


RAM vs. ROM vs. Permanent

Merkmal	RAM (Random Access Memory)	ROM (Read Only Memory)	Permanenter Speicher (z. B. SSD, HDD)
Flüchtigkeit	Flüchtig – Daten gehen beim Ausschalten verloren	Nicht flüchtig – Daten bleiben erhalten	Nicht flüchtig – Daten bleiben erhalten
Schreib-/Lesbarkeit	Lese- **und** schreibbar	Meist nur lesbar (Programmierung selten)	Lese- und schreibbar
Verwendungszweck	Temporärer Arbeitsspeicher für laufende Prozesse	Dauerhafte Speicherung von Firmware/Startprogrammen	Dauerhafte Speicherung von Dateien, Programmen und Betriebssystem

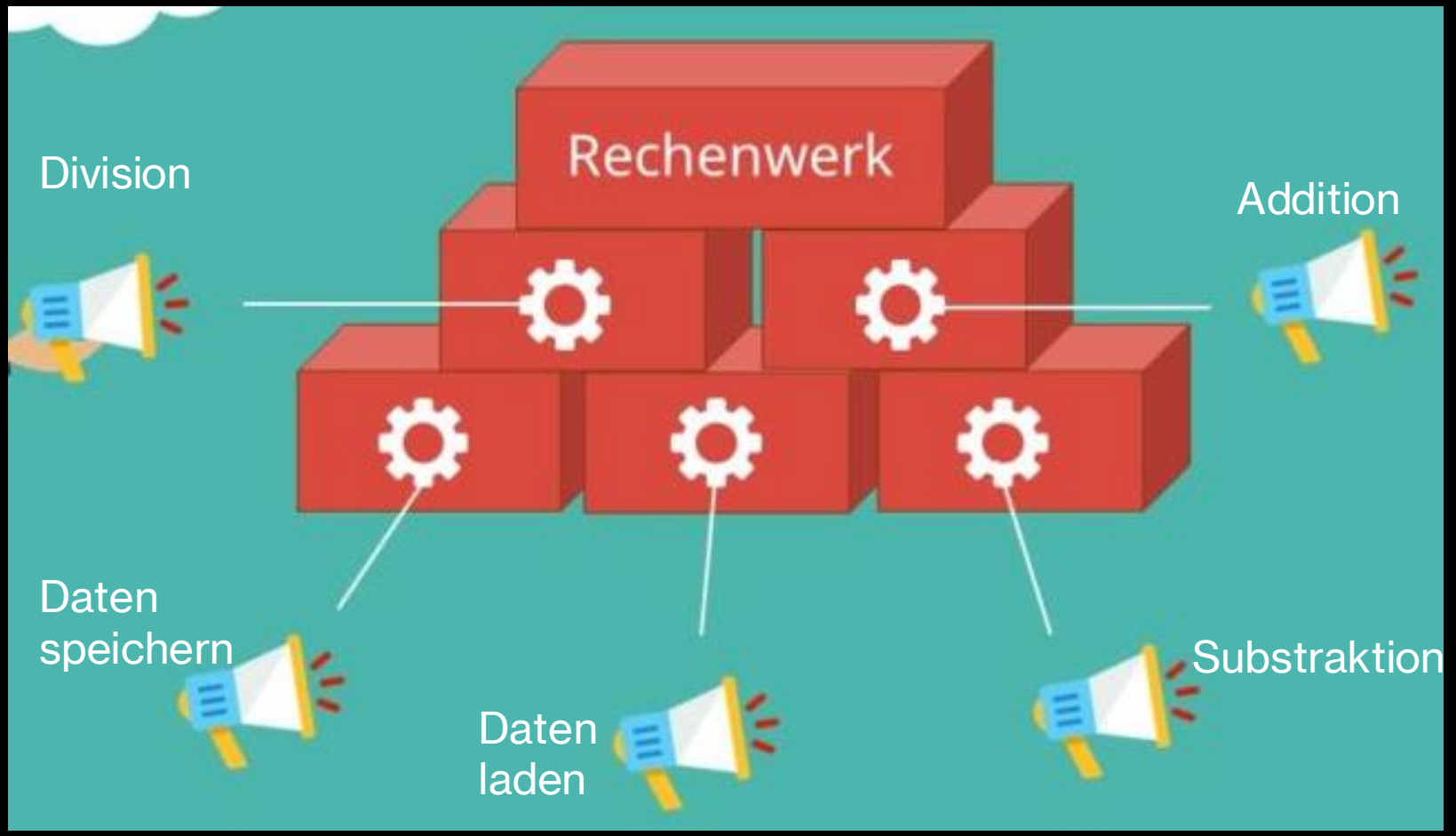
Aufgabe

- Wofür würdet ihr die verschiedenen Speicherarten einsetzen?
- Welche habt ihr bereits aktiv genutzt?



2.2 Rechenwerk

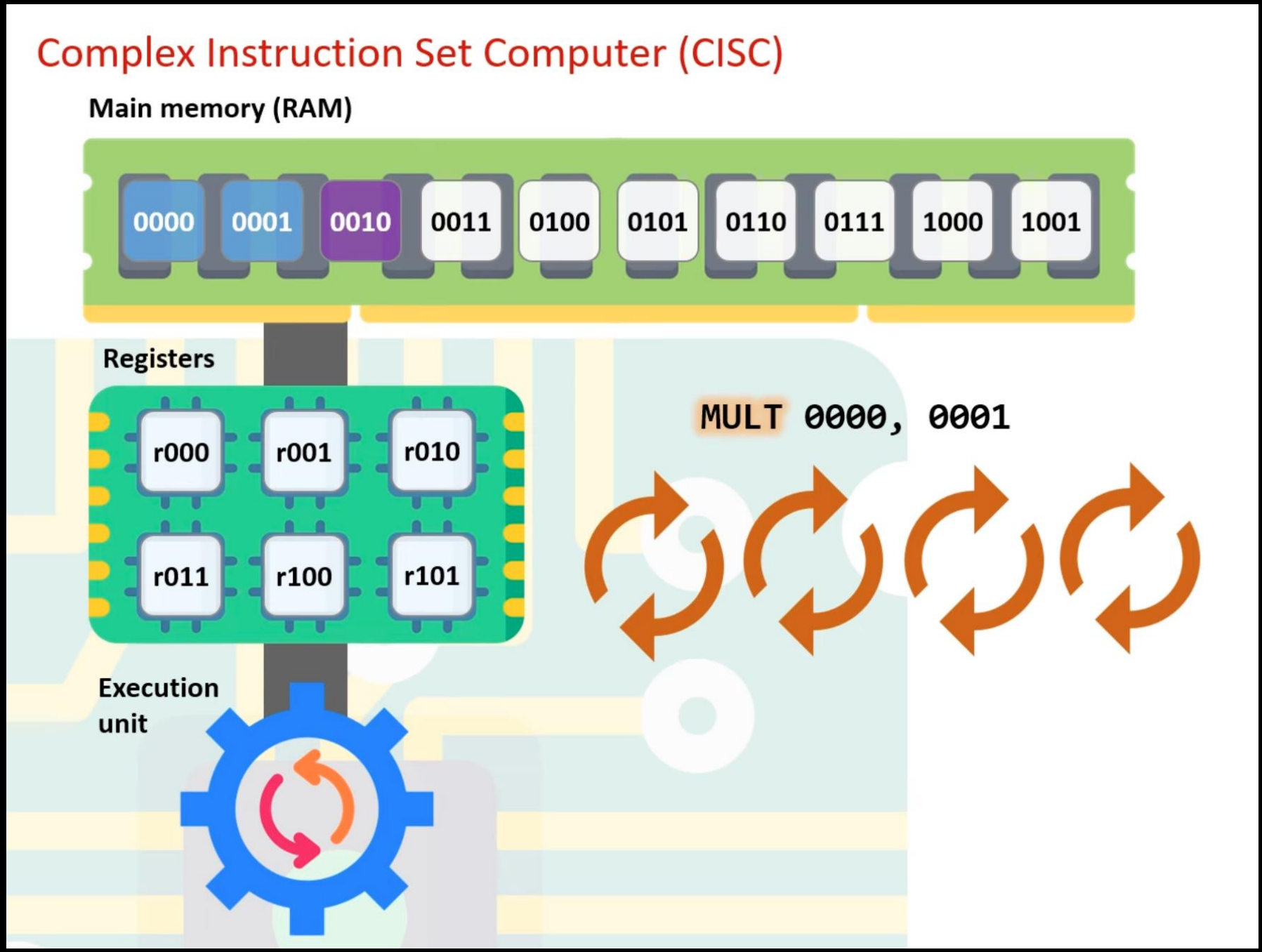
Funktionseinheiten / Instructions



x86

1000+
Instructions

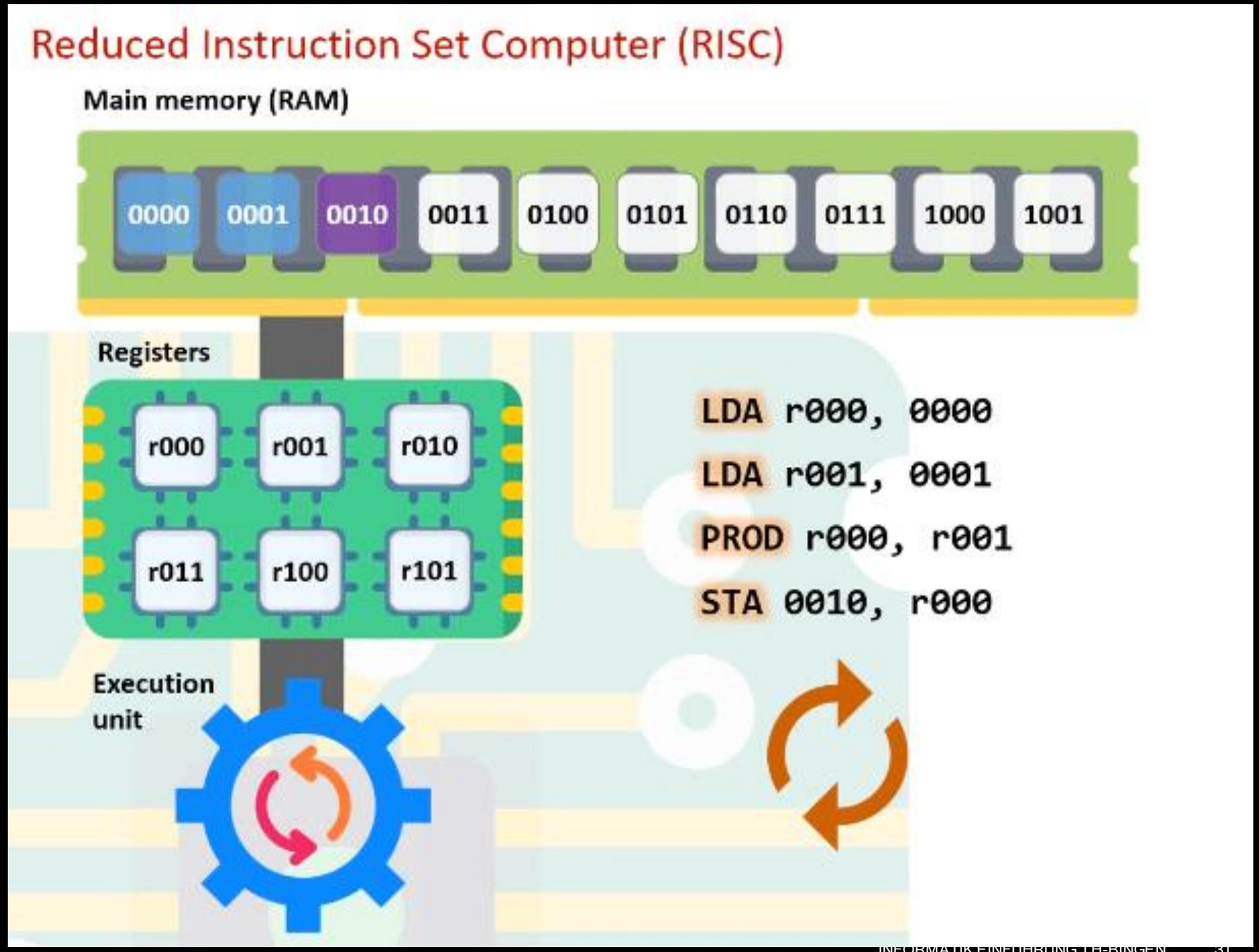
- Mikroprogramm anstatt fester Schaltungen
- Mehr Platz und Komplexität
- Flexibler
- Langsamer (mehrere Takte pro Instruction)

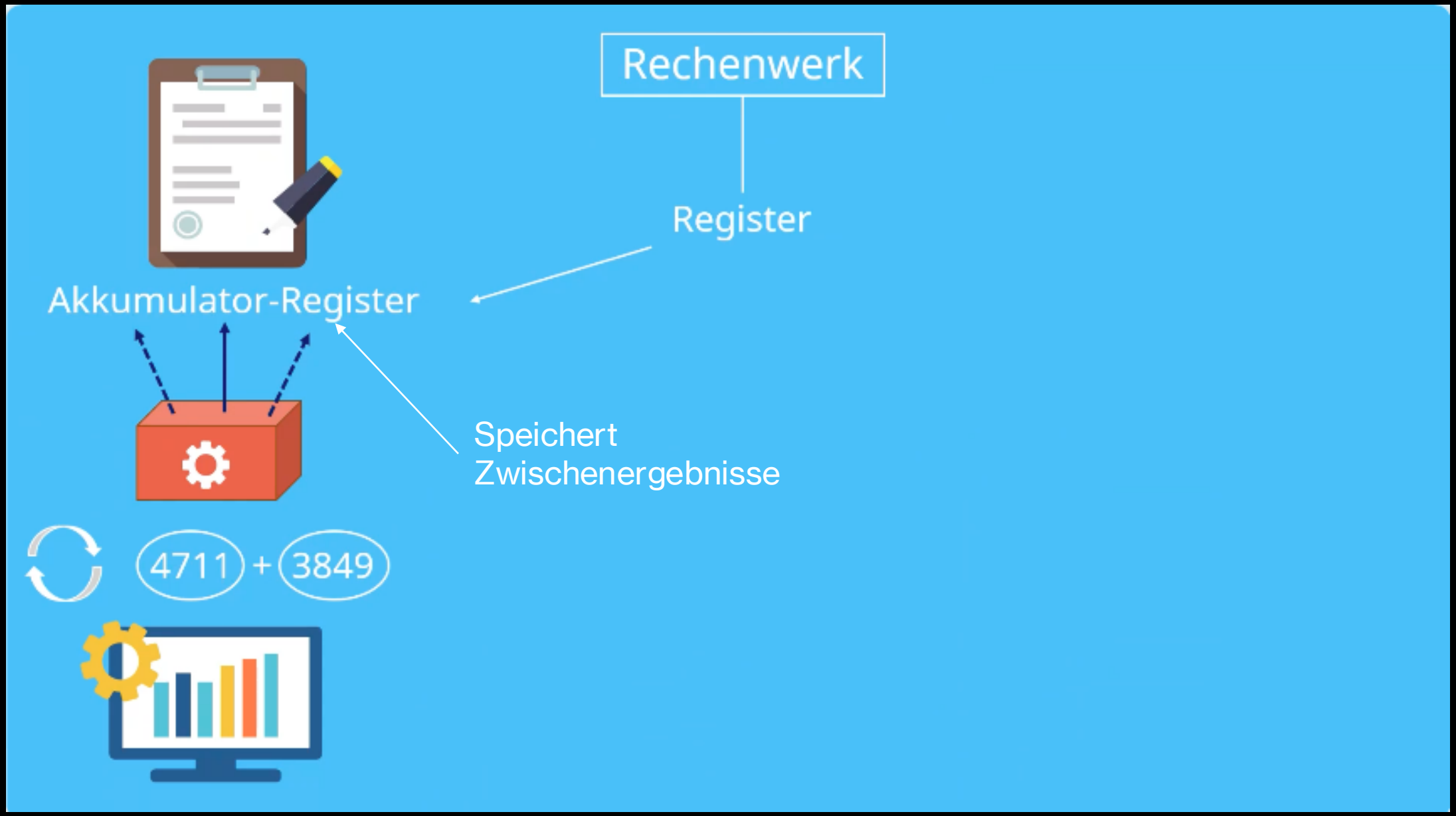


ARM

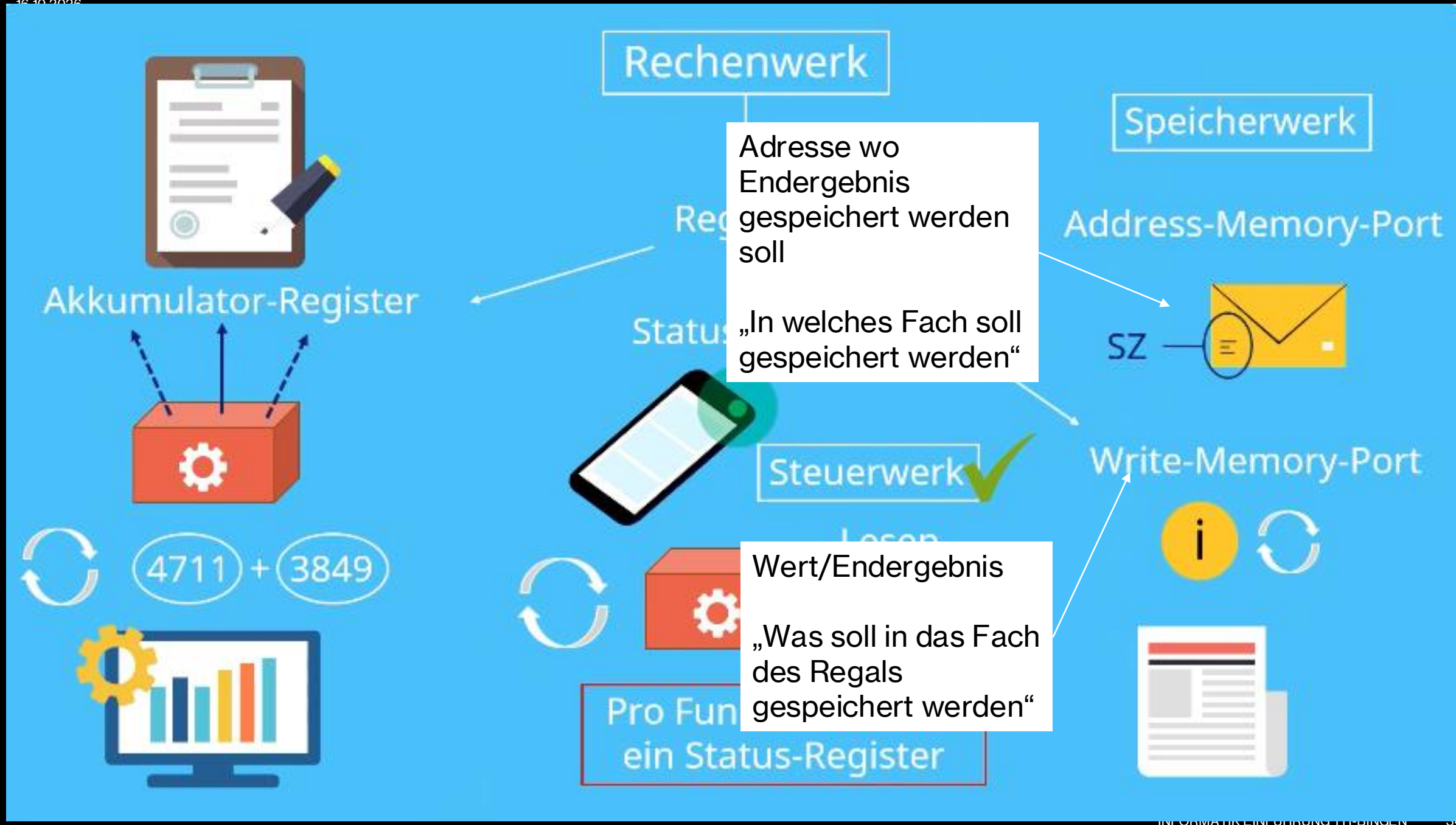
<300
Instructions

- Fest installierte Schaltungen
- Weniger Platz
- Energieeffizienter
- Schneller (nur ein Takt pro Instruction)




Rechenwerk
Register
Akkumulator-Register
Speichert
Zwischenergebnisse
4711 + 3849

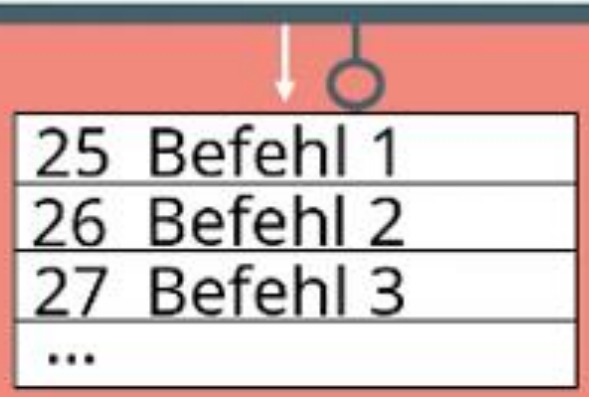

Rechenwerk
Register
Akkumulator-Register
Status-Register
Steuerwerk
Kommunikation mit Steuerwerk: Signalisiert, ob Funktionseinheit beschäftigt oder frei ist
4711 + 3849
Lesen
Schreiben
Langweilen
Pro Funktionseinheit ein Status-Register


Rechenwerk
Speicherwerk
Adresse wo Endergebnis gespeichert werden soll
„In welches Fach soll gespeichert werden“
Address-Memory-Port
SZ
Akkumulator-Register
4711 + 3849
Steuerwerk
Write-Memory-Port
Wert/Endergebnis
„Was soll in das Fach des Regals gespeichert werden“
ein Status-Register

Maschinencode a.k.a. Assembler

- Low Level Sprache, welche Funktionseinheiten aufruft
- Alle High Level Sprachen (z.B. Python, Java, C etc.) werden in dieses übersetzt (teils mittels eines Compilers)
- Auch Betriebssystem ist nichts anderes als einer sehr großen Anzahl an Programmen, welche als Assembler ausgeführt werden

INCREMENT
INIT 1000
SPRUNG 1005
SPRUNG 1004, 1005
AUSGABE 1000
DECREMENT





Real liegen Befehle binär vor

SZ 1000	INIT 1004
SZ 1001	SPRUNG 1004, 1005
SZ 1002	INCREMENT 1004
SZ 1003	AUSGABE 1004
SZ 1004	5
SZ 1005	AUSGABE 1006
SZ 1006	1

Maschinencode



25 Befehl 1
26 Befehl 2
27 Befehl 3
...

Real liegen Befehle binär vor

„Programm“

SZ 1000	INIT 1004
SZ 10	PRUNG 1004, 1005
S	INCREMENT 1004
SZ 1003	AUSGABE 1004
SZ 1004	5 → 0
SZ 1005	AUSGABE 1006
SZ 1006	1

Es gibt zwei Formen des Sprungbefehls

1) SPRUNG <Nummer SZ>

Sprung erfolgt direkt

2) SPRUNG <Nummer SZ1, SZ2>

Bedingung:

Sprung zur SZ2, wenn der gespeicherte Wert in SZ1 = 0 ist

„Programm"

SZ 1000	INIT 1004
SZ 1001	SPRUNG 1004, 1005
SZ 1002	INCREMENT 1004
SZ 1003	AUSGABE 1004
SZ 1004	5 → 0 ✓
SZ 1005	AUSGABE 1006
SZ 1006	1



„Programm"

SZ 1000	INIT 1004
SZ 1001	SPRUNG 1004, 1005
SZ 1002	INCREMENT 1004
SZ 1003	AUSGABE 1004
SZ 1004	5 → 0 ✓
SZ 1005	AUSGABE 1006
SZ 1006	1

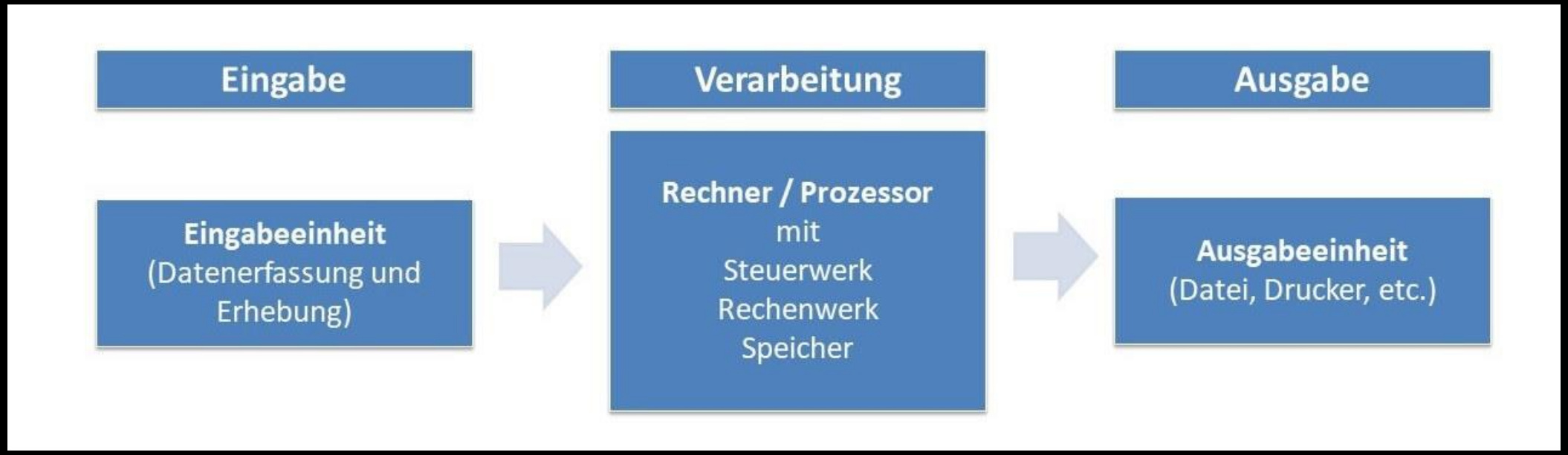
Aufgabe

Wie wäre der Ablauf des vorherigen Programms wenn die Speicherzelle 1004 den Wert ”1” enthalten würde?

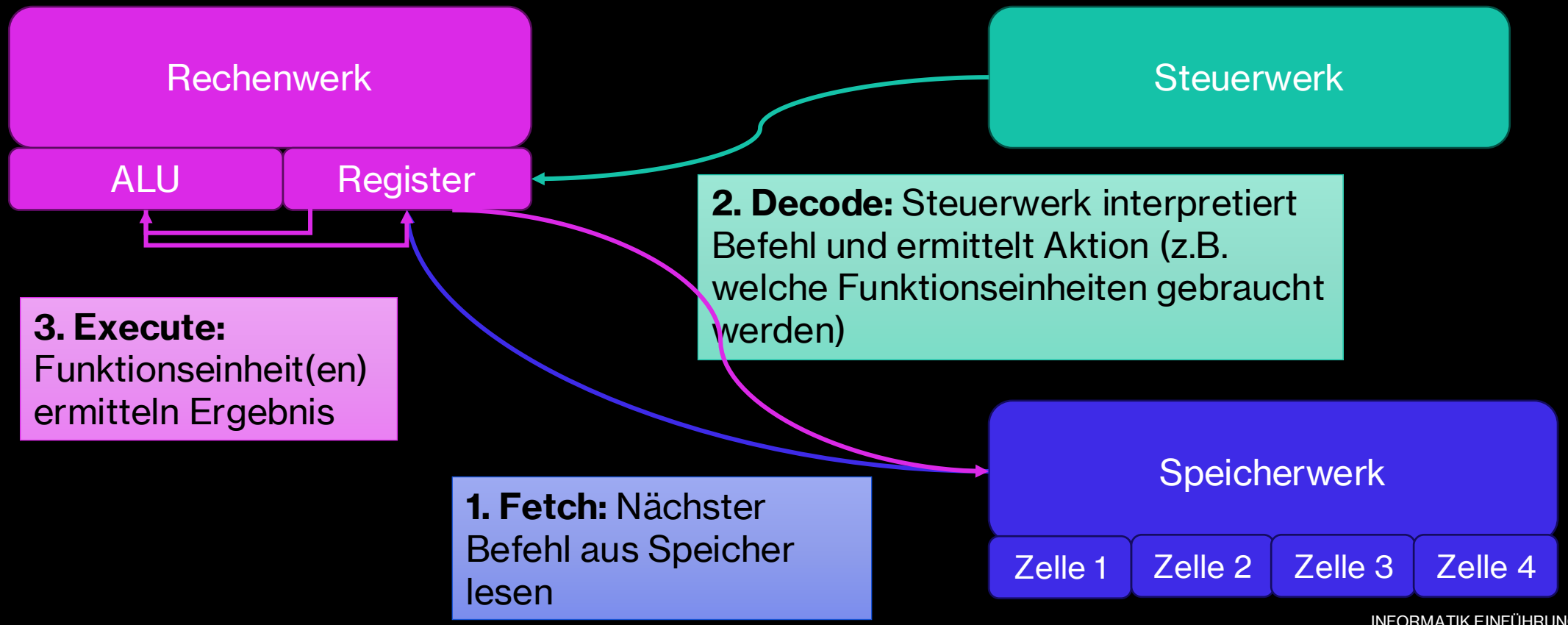


2.3 Steuerwerk

Grundsatz jeglicher Datenverarbeitung - EVA-Prinzip

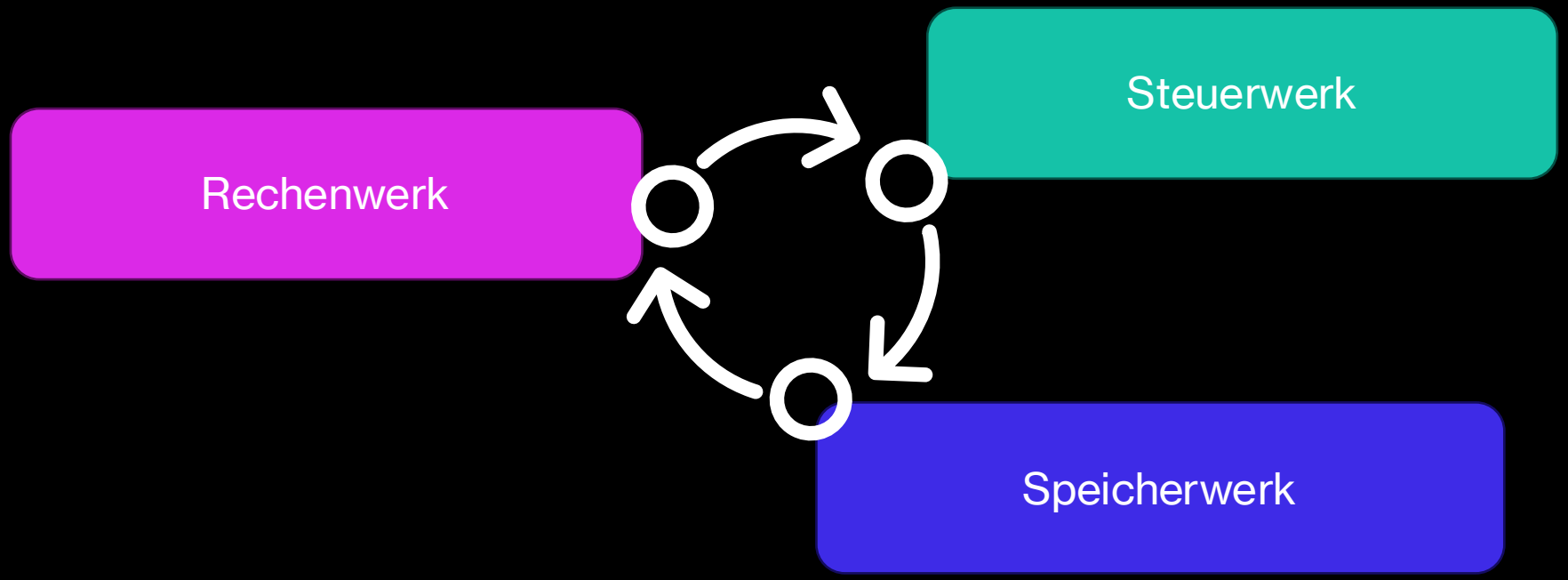


Fetch–Decode–Execute

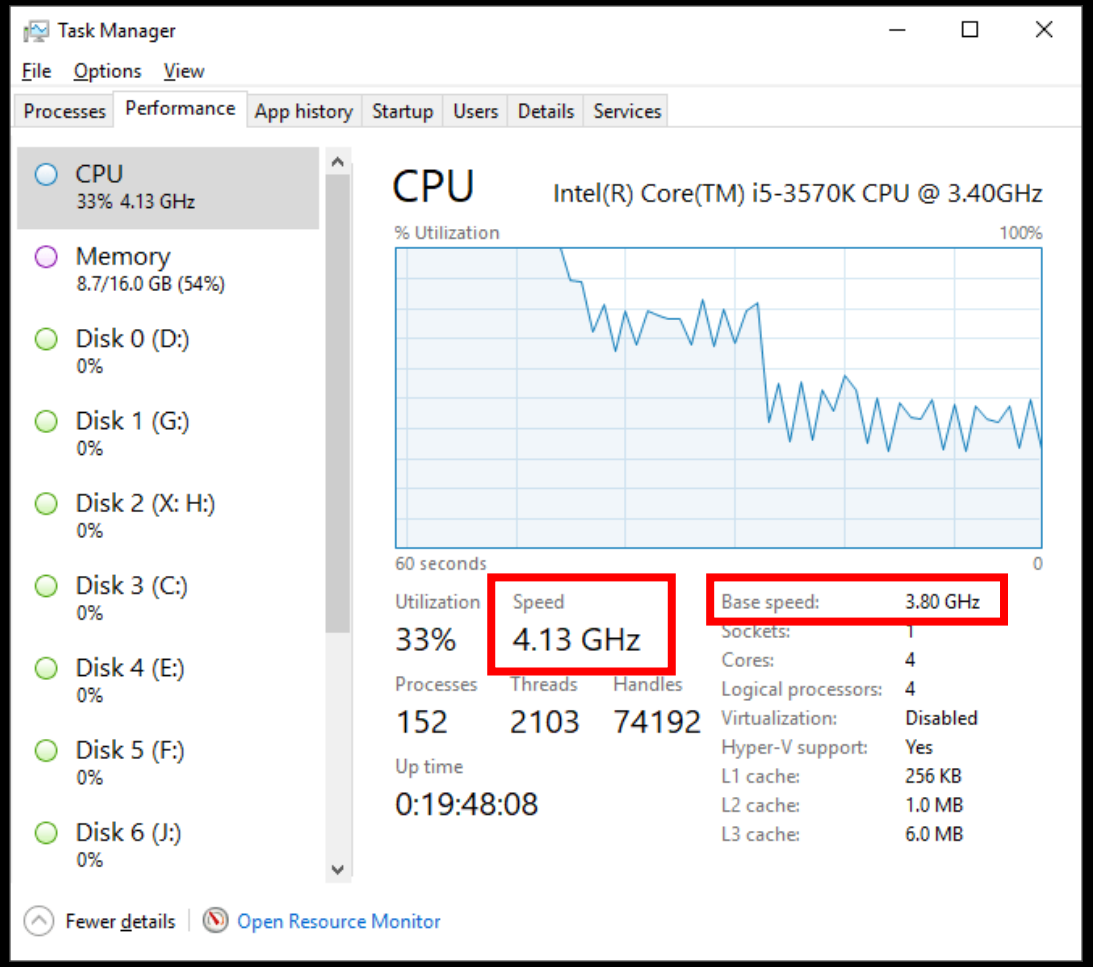


Der CPU-Takt

1 Takt = 1 Fetch-Decode-Execute Zyklus:

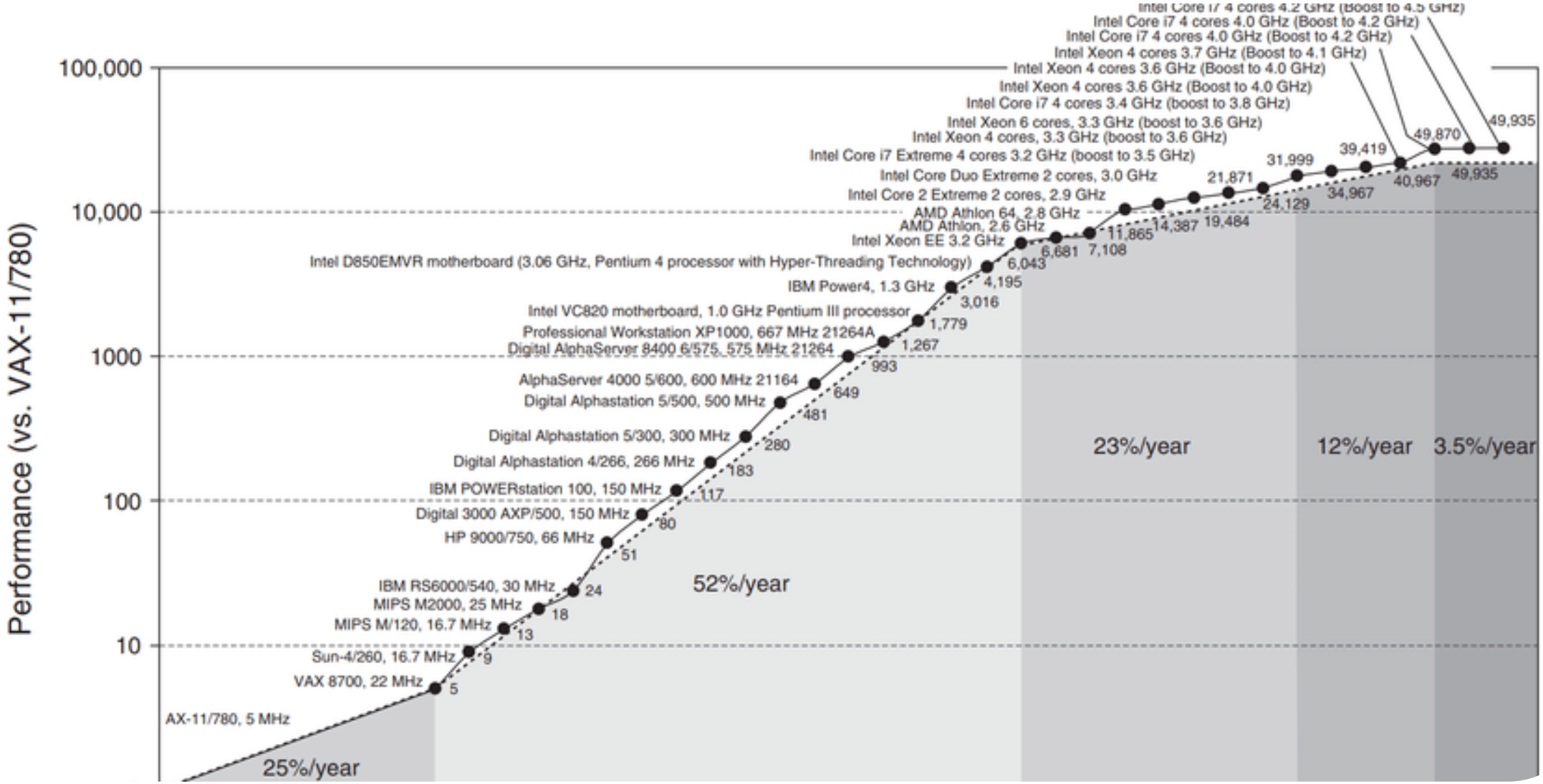


Wie viel Takte schaffe ich pro Sekunde?



PRODUCT DETAILS

name	AMD Ryzen 5 7600
Product series	AMD Ryzen 5 7000
Architecture	Zen 4
CPU core count	6
Thread count	12
Reference clock frequency	3.8GHz
Default thermal design power	65W
Maximum boost clock frequency	5.1GHz
CPU Platform	Socket AM5
Maximum operating temperature	95°C
L2 cache	6MB
L3 cache	32MB
Cooling solutions	AMD Wraith Stealth
System memory type	Supports up to DDR5-5200MHz.
Graphics card model	AMD Radeon™ Graphics
Supported technologies	Supports x86-64, AES, AMD-V, AVX, AVX2, AVX512, FMA3, MMX-plus, SHA, SSE, SSE2, SSE3, SSE4.1, SSE4.2, SSE4A, SSSE3, etc.

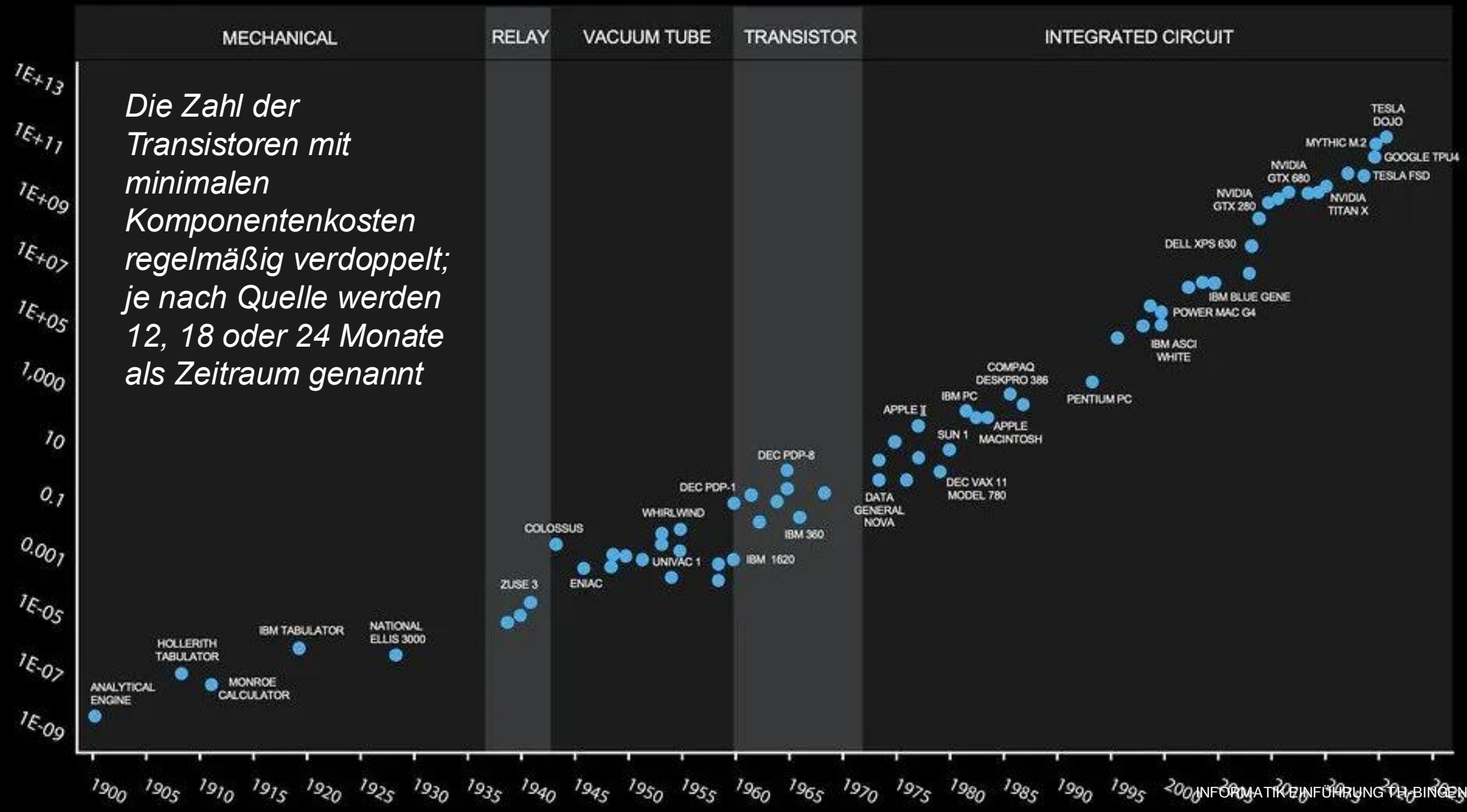

Performance (vs. VAX-11/780)
100,000
10,000
1000
100
10
AX-11/780, 5 MHz
VAX 8700, 22 MHz
5
Sun-4/260, 16.7 MHz
9
MIPS M/120, 16.7 MHz
13
MIPS M2000, 25 MHz
18
IBM RS6000/540, 30 MHz
24
HP 9000/750, 66 MHz
51
Digital 3000 AXP/500, 150 MHz
80
IBM POWERstation 100, 150 MHz
117
Digital Alphastation 4/266, 266 MHz
183
Digital Alphastation 5/300, 300 MHz
280
Digital Alphastation 5/500, 500 MHz
481
AlphaServer 4000 5/600, 600 MHz 21164
649
Digital AlphaServer 8400 6/575, 575 MHz 21264
993
Professional Workstation XP1000, 667 MHz 21264A
1,267
Intel VC820 motherboard, 1.0 GHz Pentium III processor
1,779
IBM Power4, 1.3 GHz
3,016
Intel D850EMVR motherboard (3.06 GHz, Pentium 4 processor with Hyper-Threading Technology)
4,195
Intel Xeon EE 3.2 GHz
6,043
AMD Athlon, 2.6 GHz
6,681
AMD Athlon 64, 2.8 GHz
7,108
Intel Core 2 Extreme 2 cores, 2.9 GHz
11,865
Intel Core Duo Extreme 2 cores, 3.0 GHz
14,387
Intel Core i7 Extreme 4 cores 3.2 GHz (boost to 3.5 GHz)
19,484
Intel Xeon 4 cores, 3.3 GHz (boost to 3.6 GHz)
21,871
Intel Xeon 6 cores, 3.3 GHz (boost to 3.6 GHz)
24,129
Intel Core i7 4 cores 3.4 GHz (boost to 3.8 GHz)
31,999
Intel Xeon 4 cores 3.6 GHz (Boost to 4.0 GHz)
34,967
Intel Xeon 4 cores 3.6 GHz (Boost to 4.0 GHz)
39,419
Intel Xeon 4 cores 3.7 GHz (Boost to 4.1 GHz)
40,967
Intel Core i7 4 cores 4.0 GHz (Boost to 4.2 GHz)
49,870
Intel Core i7 4 cores 4.0 GHz (Boost to 4.2 GHz)
49,935
Intel Core i7 4 cores 4.2 GHz (Boost to 4.5 GHz)
49,935
25%/year
52%/year
23%/year
12%/year
3.5%/year

Beispiele für verschiedene Frequenzen

Frequenz	Formel	Taktdauer
1 Hz	1 / 1 = 1 s	1 Sekunde pro Zyklus
1 kHz	1 / 1.000 = 0,001 s	1 Millisekunde pro Zyklus
1 MHz	1 / 1.000.000 = 0,000001 s	1 Mikrosekunde pro Zyklus
1 GHz	1 / 1.000.000.000 = 0,000000001 s	1 Nanosekunde pro Zyklus
3 GHz	1 / 3.000.000.000 ≈ 0,33 ns	ca. 0,33 Nanosekunden pro Zyklus

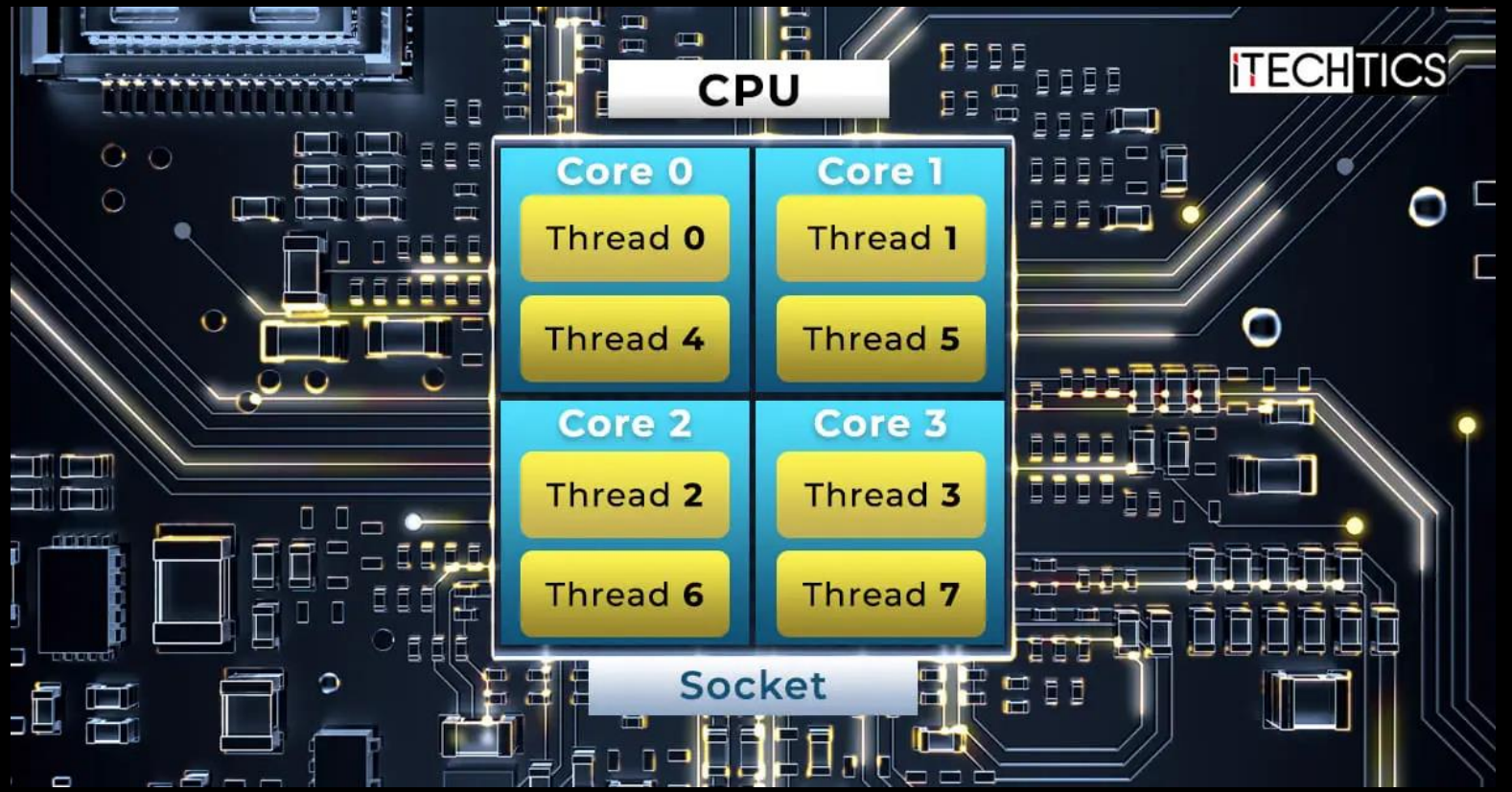
FUTURE VENTURES

122 YEARS OF MOORE'S LAW

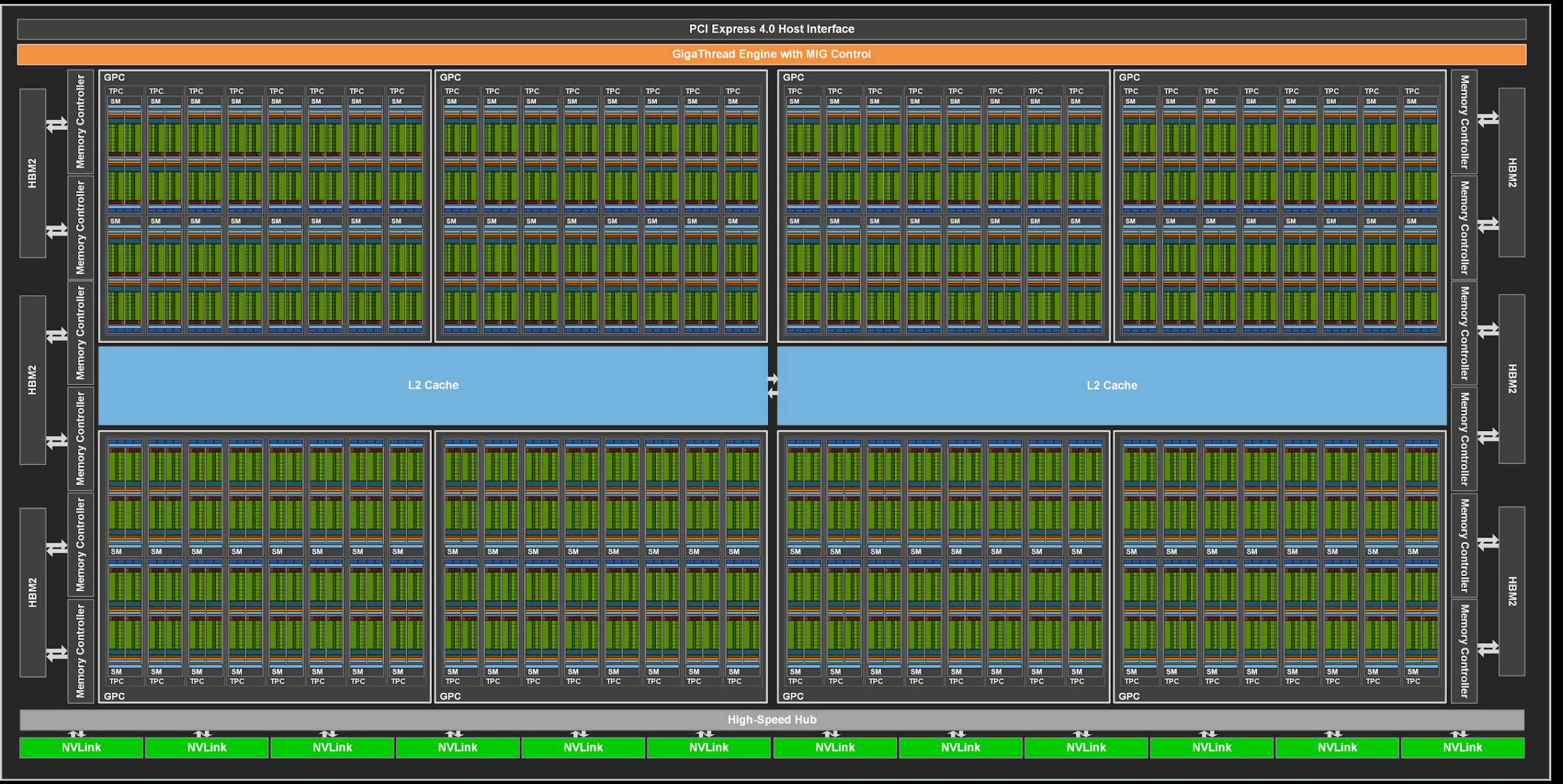


Die Zahl der Transistoren mit minimalen Komponentenkosten regelmäßig verdoppelt; je nach Quelle werden 12, 18 oder 24 Monate als Zeitraum genannt

Wie kann ich Arbeiten parallelisieren?



1 Thread = eine Einheit, die jeweils einen Takt ausführen kann



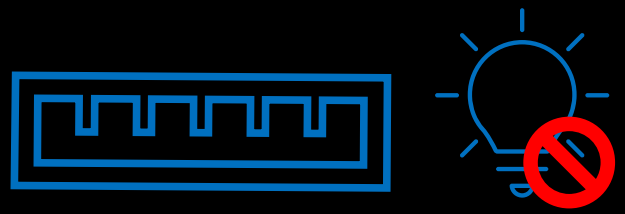
GPUs haben deutlich mehr Kerne → sehr gut für parallele Aufgaben (Grafik, KI)

Kann ich eine unendlich schnelle CPU bauen?



Jede Operation erzeugt Wärme → muss gekühlt werden

bessere Kühlung → mehr Leistung



Strom fließt deutlicher langsamer als Lichtgeschwindigkeit

→ Register und Cache muss physikalisch nahe an Recheneinheit sein, sonst „dauert“ es zu lange Dinge zu laden/speichern

Fiber optic cable



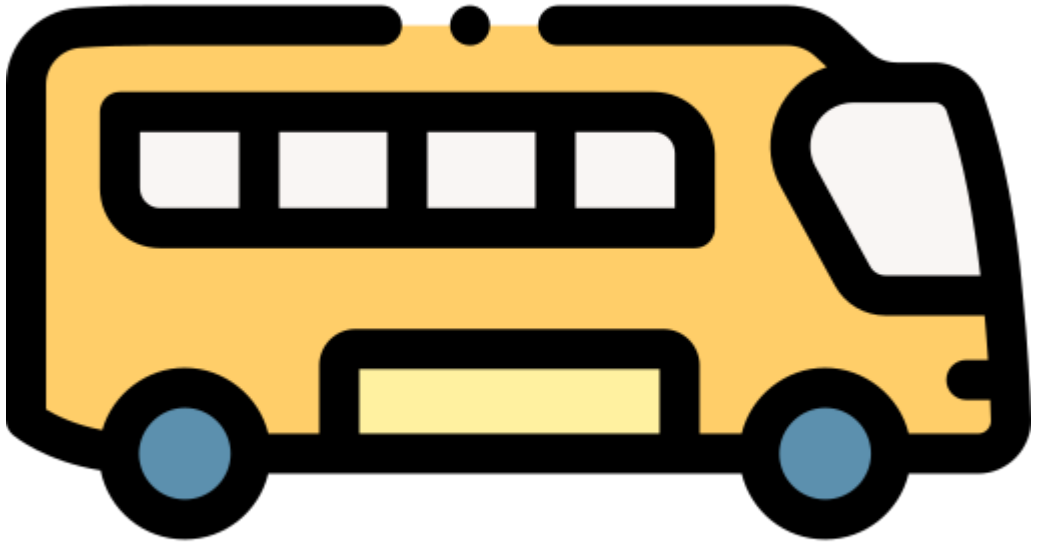
Copper Cable



Aufgabe

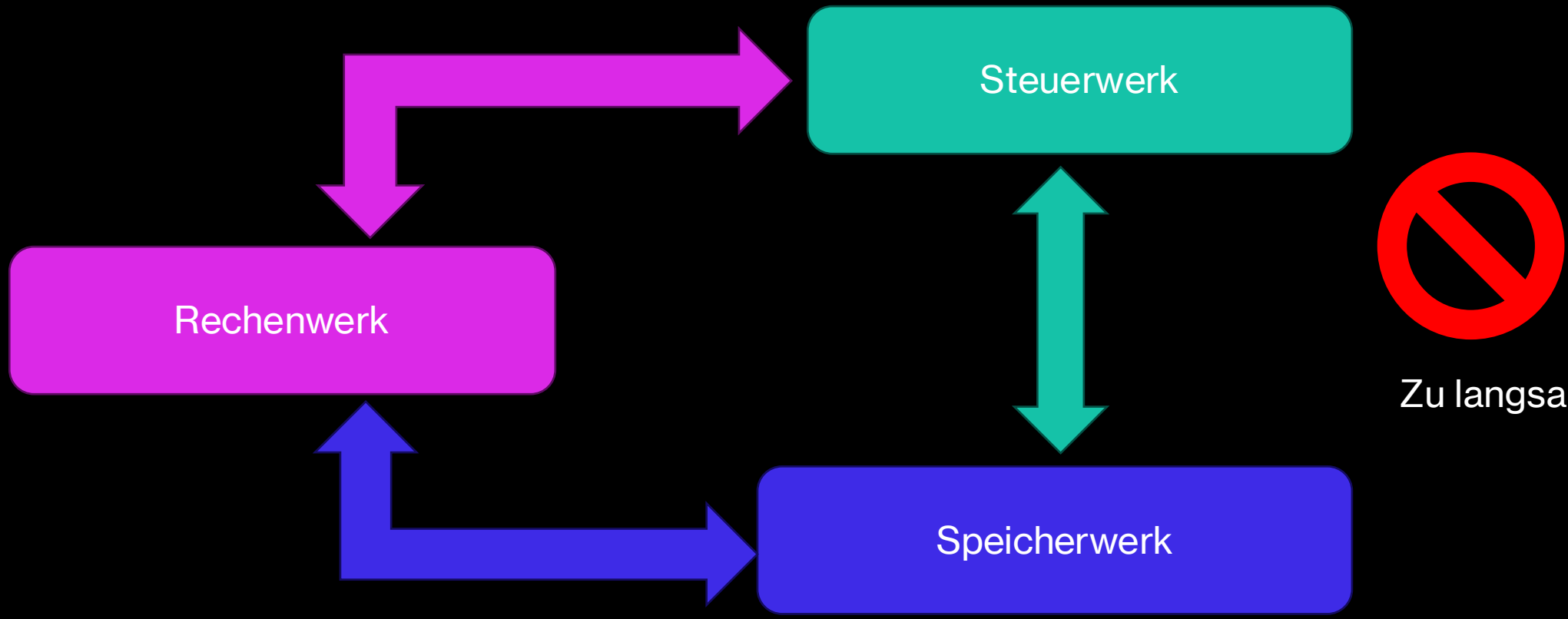
Stellen Sie den Rechenvorgang 5 + 3 als Rezept dar.

Formulieren Sie in kurzen Schritten (wie in einem Kochrezept), was der Computer tut: Welche ‘Zutaten’ (Zahlen), welche Arbeitsschritte (Rechenoperationen) und welches Ergebnis



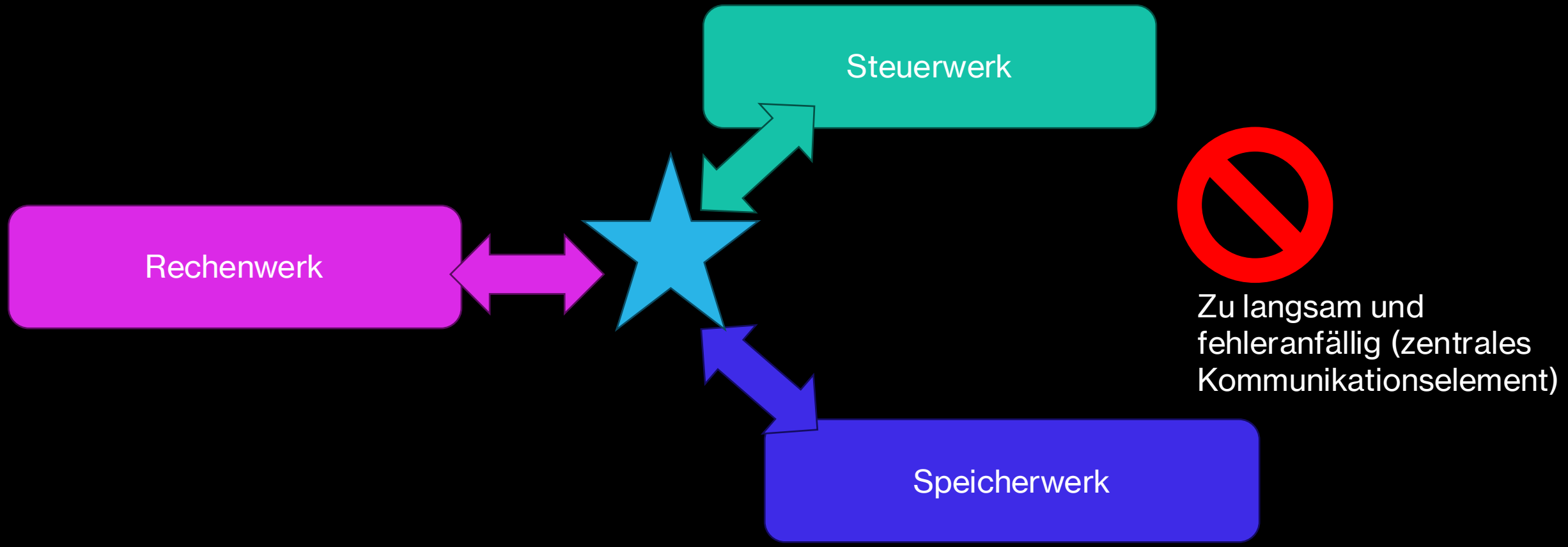
2.4 Kommunikation / Bussysteme

Wie reden die Systeme untereinander?

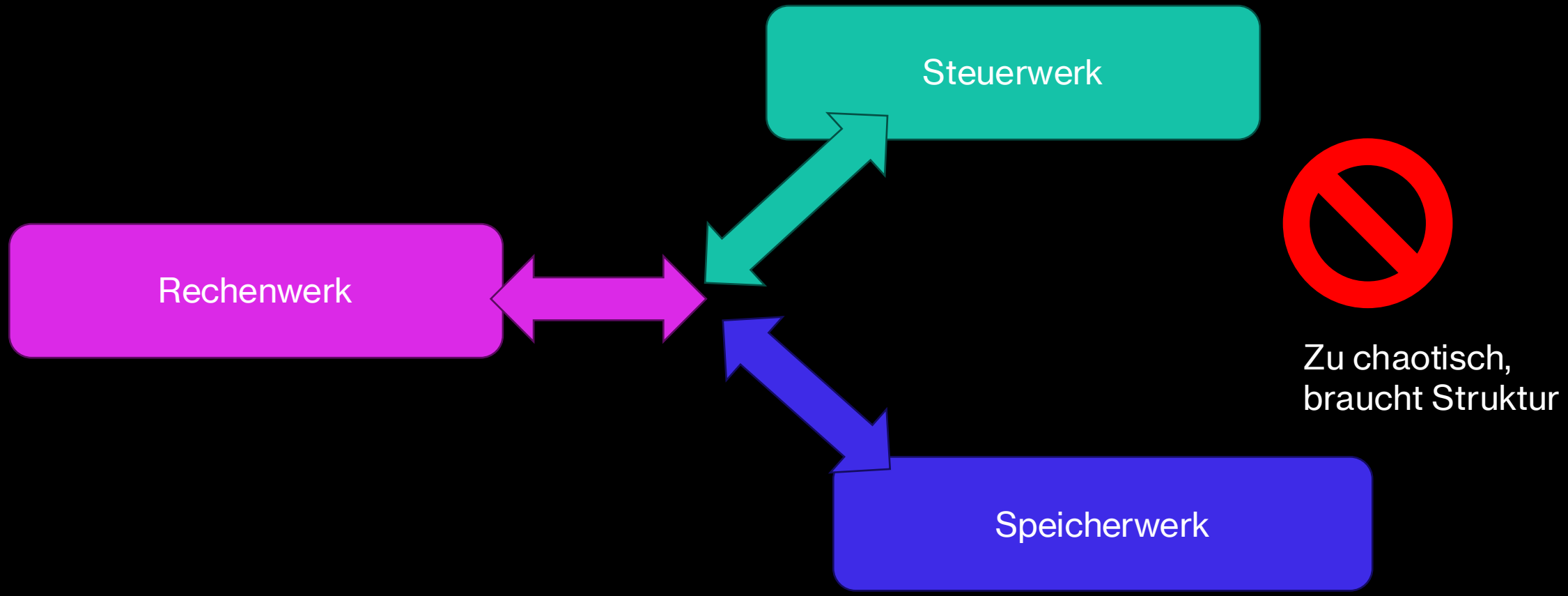


Zu langsam und ineffizient

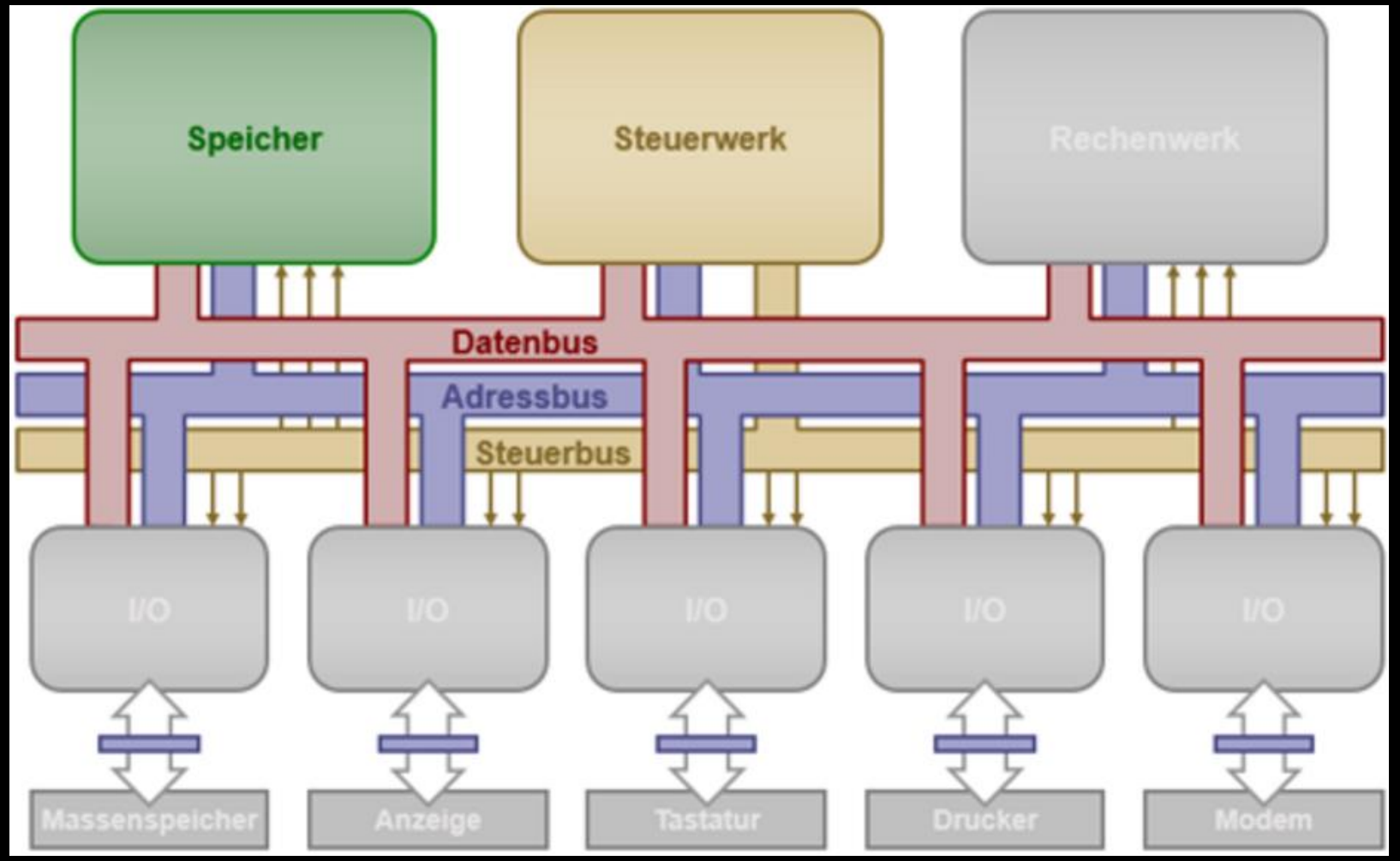
Wie reden die Systeme untereinander?

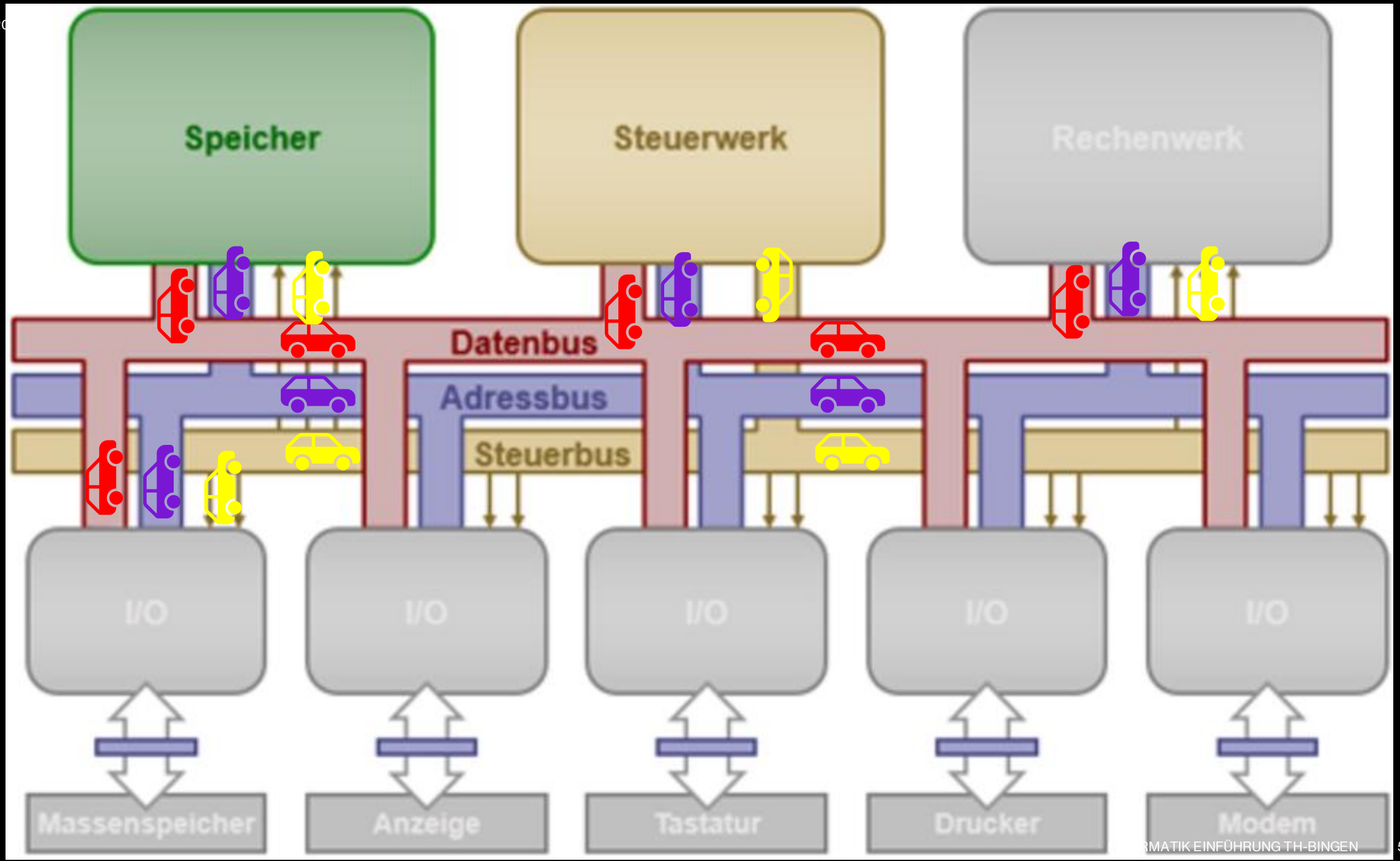


Wie reden die Systeme untereinander?



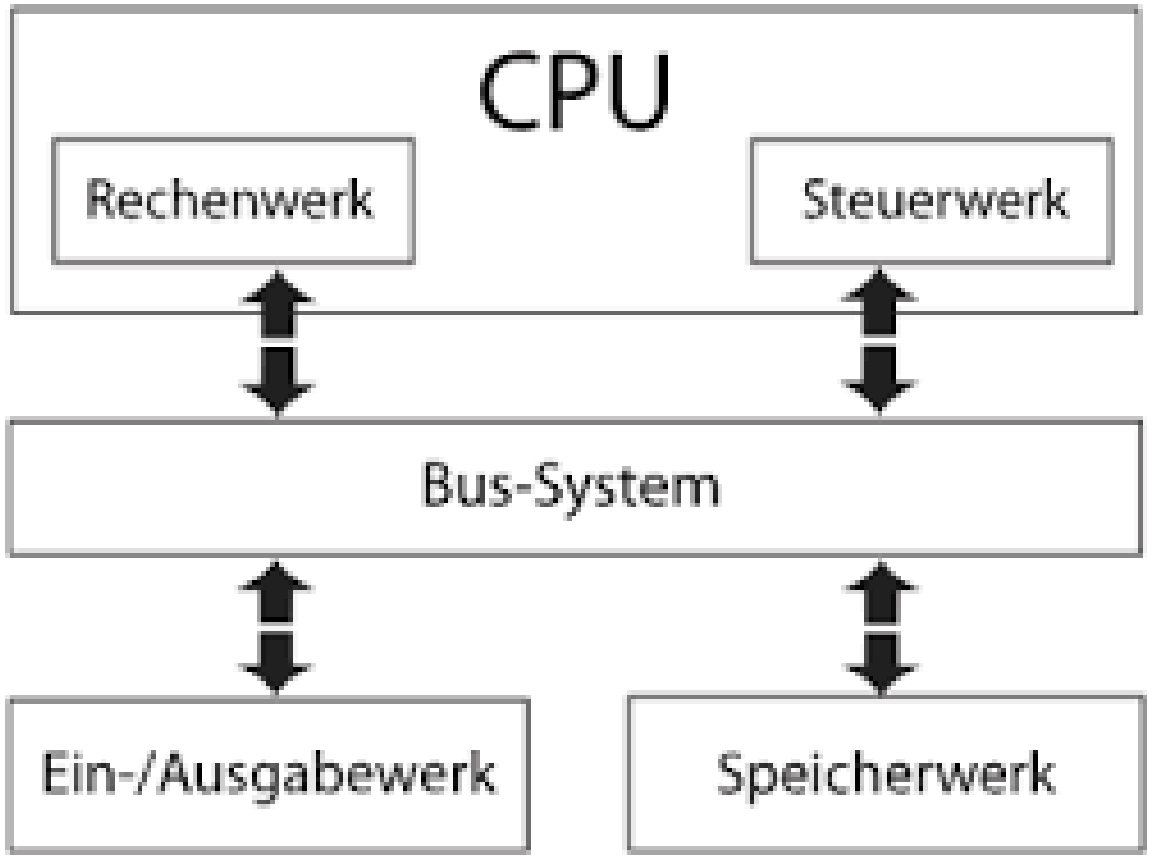
Lösung: Das Bussystem




Speicher
Steuerwerk
Rechenwerk
Datenbus
Adressbus
Steuerbus
I/O
I/O
I/O
I/O
I/O
Massenspeicher
Anzeige
Tastatur
Drucker
Modem

Das Bussystem

- Verbindet CPU, Speicher und I/O.
- Ist wie eine Art “Autobahn” auf welcher Pakete (*Autos*) von A nach B fahren können
- Spezielle Pakete (*Autos*) müssen dabei über spezielle “Autobahnspuren” fahren, z.B. Datenpakete über Datenbus oder Addresspakete über Addressbus
- Dabei dient:
  + Adressbus: Austausch von Start und Zieladresse von Daten (z.B. Lese bzw. Schreibziele im Speicher) – meist einseitig (CPU zu Speicher)
  + Datenbus: Die gelesenen bzw. zu schreibende Daten – meist bidirektional
  + Steuerbus: Überträgt Kontrollsignale (z.B. “Jetzt lesen“, „Jetzt schreiben“ oder „Nächster Takt“) – meist einseitig (Steuerwerk zu anderen Komponenten)



2.5 Zusammenfassung und Unterschiede

Vorteile des von Neumann Rechners

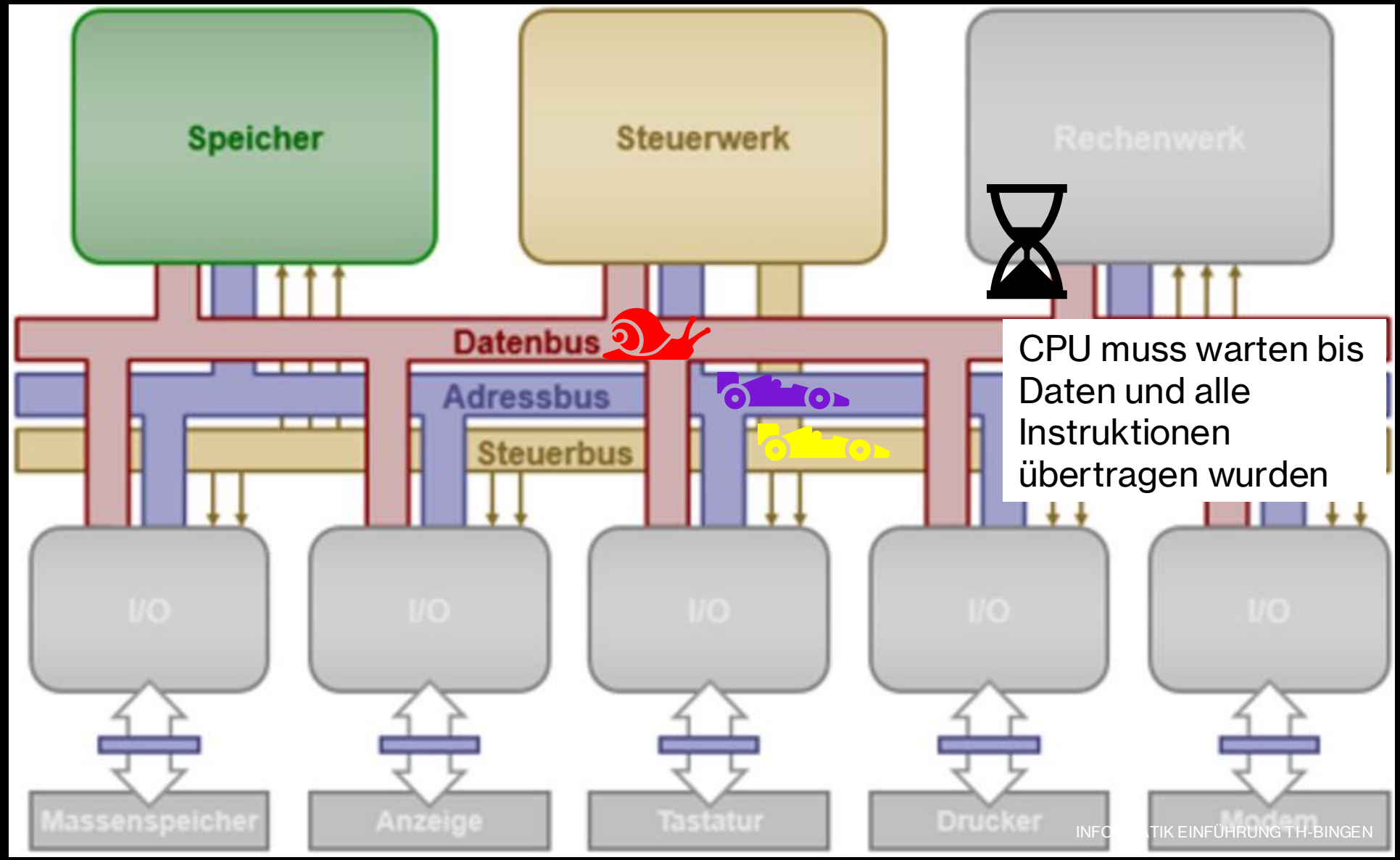
- **Einfache Architektur**
  + Nur ein Speicher für Daten und Programme → weniger Hardware-Komplexität.
- **Geringere Kosten**
  + Weniger Speicherleitungen und Komponenten nötig → kostengünstiger in der Herstellung.
- **Flexibilität bei der Programmierung**
  + Programme können wie Daten behandelt werden → Selbstmodifizierender Code möglich.
- **Einheitlicher Speicherzugriff**
  + CPU muss sich nur auf einen Speicher und einen Bus konzentrieren.
- **Einfaches Design von Steuerungen**
  + Ein Steuerbus steuert alles → einfachere Kontrolle der Speicher- und Datenoperationen.
- **Weit verbreitet**
  + Standardarchitektur für die meisten PCs, eingebetteten Systeme und Lernmaterialien → große Community und viele Ressourcen.

Nachteile des von Neumann Rechners

- **Von-Neumann-Flaschenhals**
  + Daten und Programme teilen sich denselben Bus → CPU muss oft warten, bis Daten oder Instruktionen übertragen sind.
- **Langsamer bei parallelen Zugriffen**
  + Keine gleichzeitige Instruktions- und Datenverarbeitung möglich.
- **Begrenzte Speicherbandbreite**
  + Wenn viele Daten verarbeitet werden müssen, kann der einzelne Bus zum Engpass werden.
- **Selbstmodifizierender Code kann riskant sein**
  + Flexibilität ermöglicht Fehler oder Sicherheitsprobleme, wenn Programme ihre eigenen Instruktionen verändern.
- **Nicht optimal für Hochleistungsanwendungen**
  + Bei Echtzeit- oder Hochgeschwindigkeitsanwendungen kann die Architektur die Leistung begrenzen.

Von-Neumann-Flaschenhals

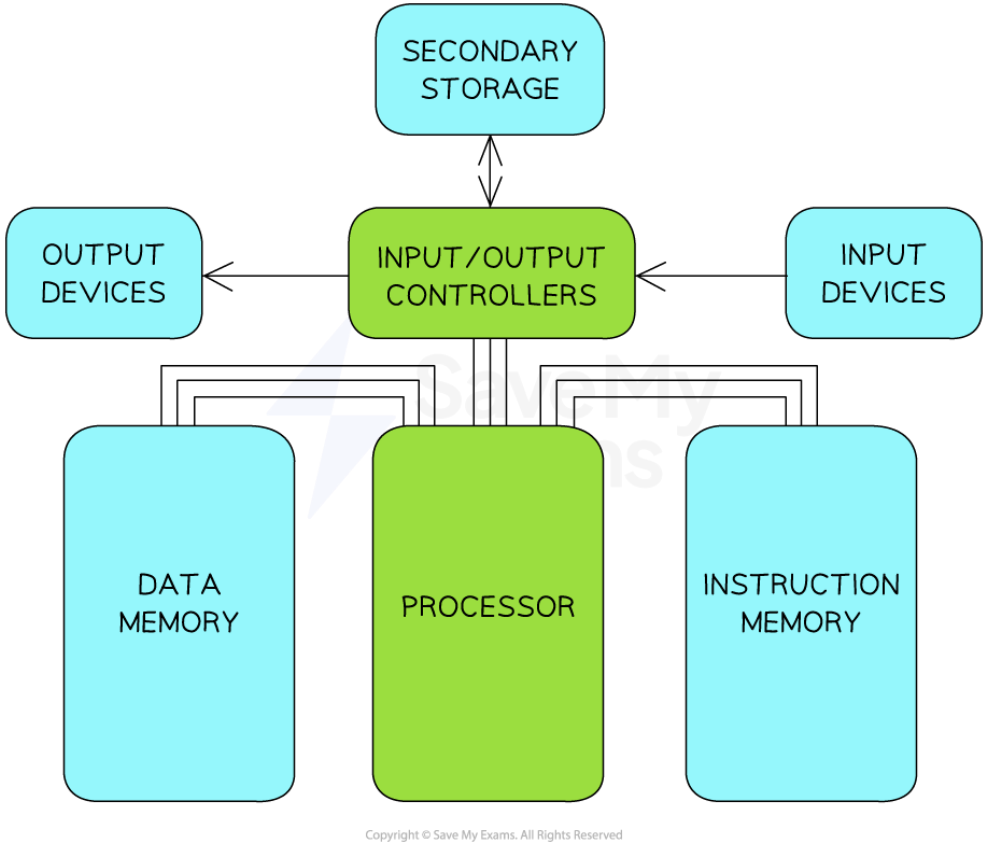
“Breite“ des Busses ist begrenzt



Der Harvard Computer

- **Speicher:** Getrennte Speicher für **Daten** und **Programme**.
- **Busstruktur:** Getrennte Busse (“Autobahnen“) für **Daten** und **Instruktionen**.
- **Folge:** CPU kann gleichzeitig Daten lesen/schreiben **und** eine Instruktion holen.
- **Vorteil:** Höhere Geschwindigkeit, kein Flaschenhals bei parallelem Zugriff.
- **Nachteil:** Komplexere Hardware, mehr Speicherleitungen nötig.

→ Hat sich im Alltag nicht durchgesetzt, wird eher in Hochleistungsumgebungen eingesetzt



Quiz

Multiple Choice

1 Minute

Welche Hauptkomponenten gehören zum klassischen Von-Neumann-Rechner?

a) Rechenwerk (ALU)

b) Speicherwerk

c) Steuerwerk

d) Eingabe- und Ausgabewerk

Welche Nachteile hat die Von-Neumann-Architektur?

a) „Von-Neumann-Flaschenhals“: begrenzte Geschwindigkeit durch gemeinsame Busse.

b) Programme und Daten können nicht im gleichen Speicher liegen.

c) Potenzielle Sicherheitsprobleme, da Daten als Programm interpretiert werden können.

d) Keine Möglichkeit, parallele Prozesse auszuführen.

Quiz

Wahr/Falsch

1 Minute

- Das Steuerwerk sorgt für die eigentliche Datenspeicherung, während das Rechenwerk die Steuerung übernimmt.



- Der Von-Neumann-Rechner basiert auf dem Prinzip, dass Programme und Daten im gleichen Speicher abgelegt werden.



- HDD-Festplatten sind besonders schnell, jedoch teurer während SSD-Festplatten eine günstigere aber langsamere Alternative darstellen.



- Das Rechenwerk (ALU) führt logische und arithmetische Operationen aus.



Diskussion

- Warum wird die Von-Neumann-Architektur auch heute noch gelehrt, obwohl moderne Prozessoren oft komplexere Architekturen (z. B. Harvard, superskalar, Mehrkernsysteme) verwenden?

- In welchen Situationen könnte der „Von-Neumann-Flaschenhals“ in industriellen Prozessen besonders kritisch werden?

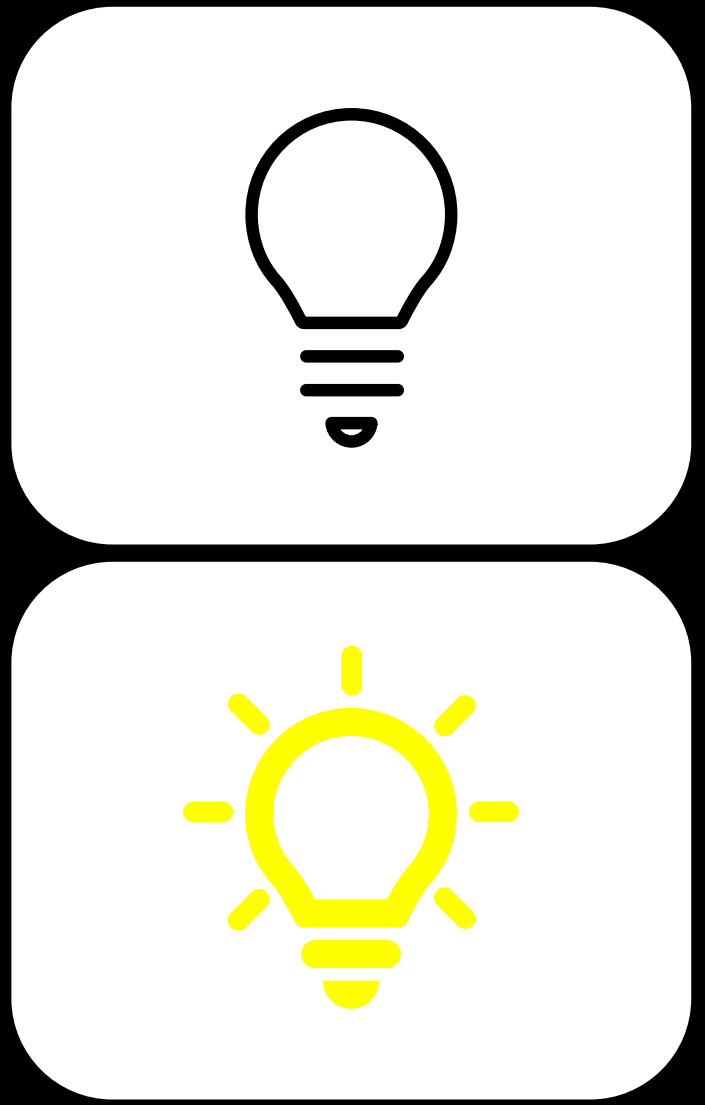
- Diskutieren Sie: Warum ist es für Verfahrenstechniker sinnvoll, zumindest das Grundkonzept der Von-Neumann-Architektur zu kennen, auch wenn Sie keine Informatiker sind?

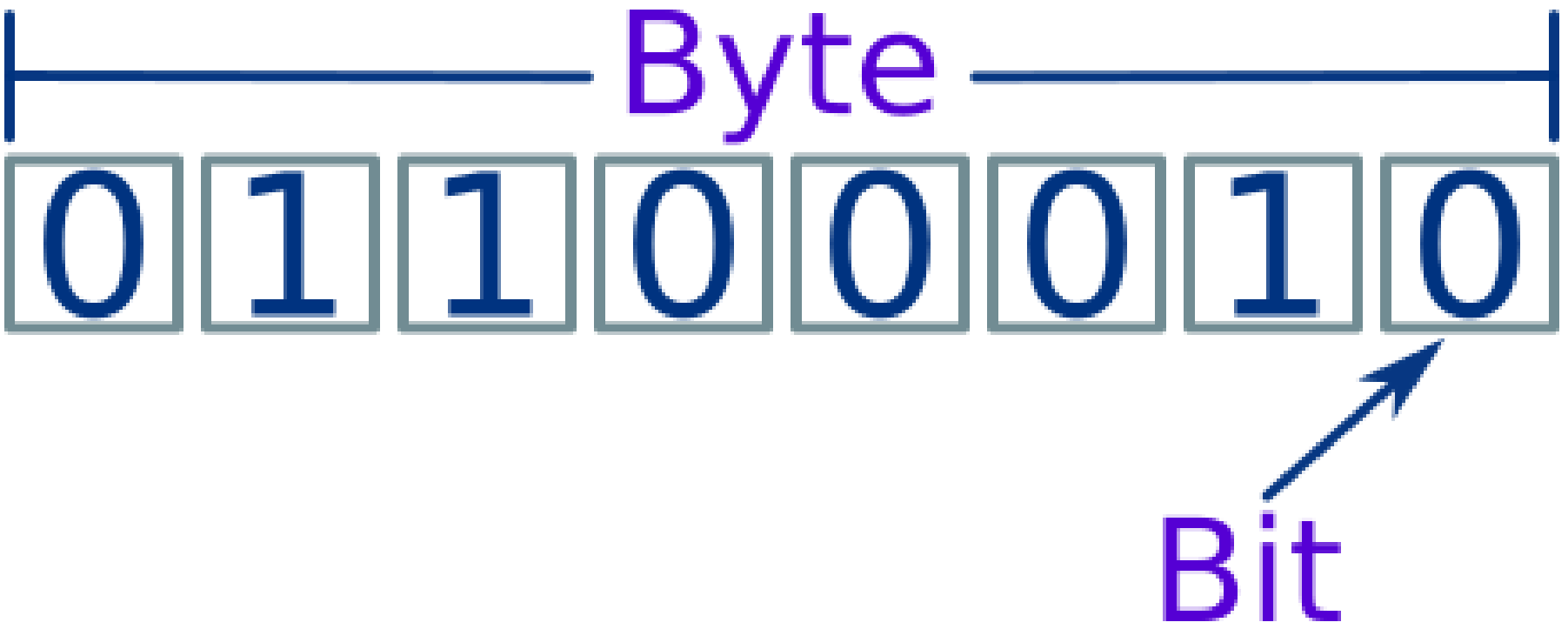
5 Minuten

3 Speicher & Daten – Was ist ein Bit, Byte, Wort?

Das Bit

- Ein Bit ist die kleinste Informationseinheit und kann genau zwei Zustände annehmen (z.B. „An/Aus“, „Ja/Nein“ oder „0/1“)
- Mit jedem zusätzlichen Bit verdoppeln sich die möglichen Werte (n Bits kodieren 2^n Zustände)
- So kann man beispielsweise mit 8 Bits (einem Byte) $2^8 = 256$ Werte darstellen.



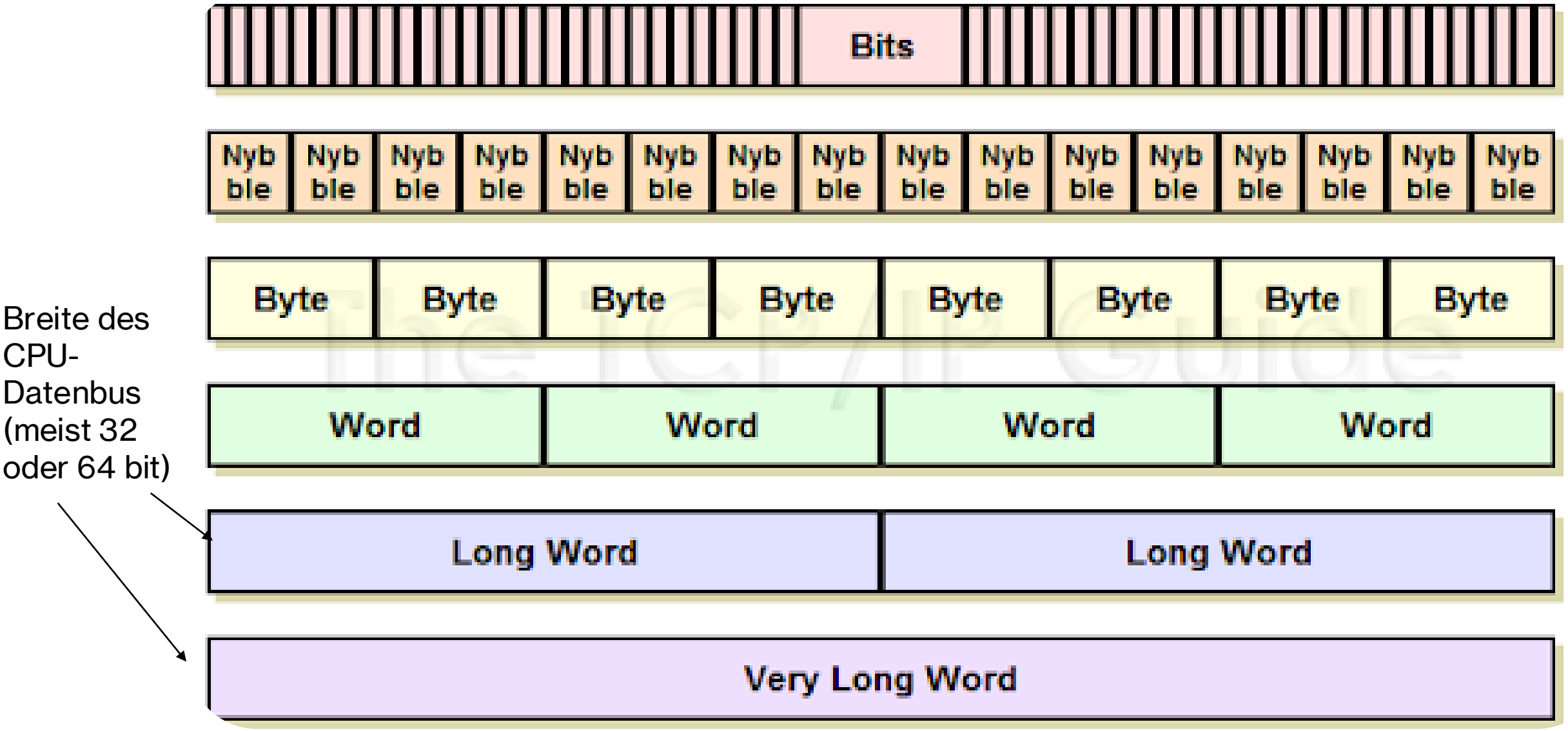


Das Byte

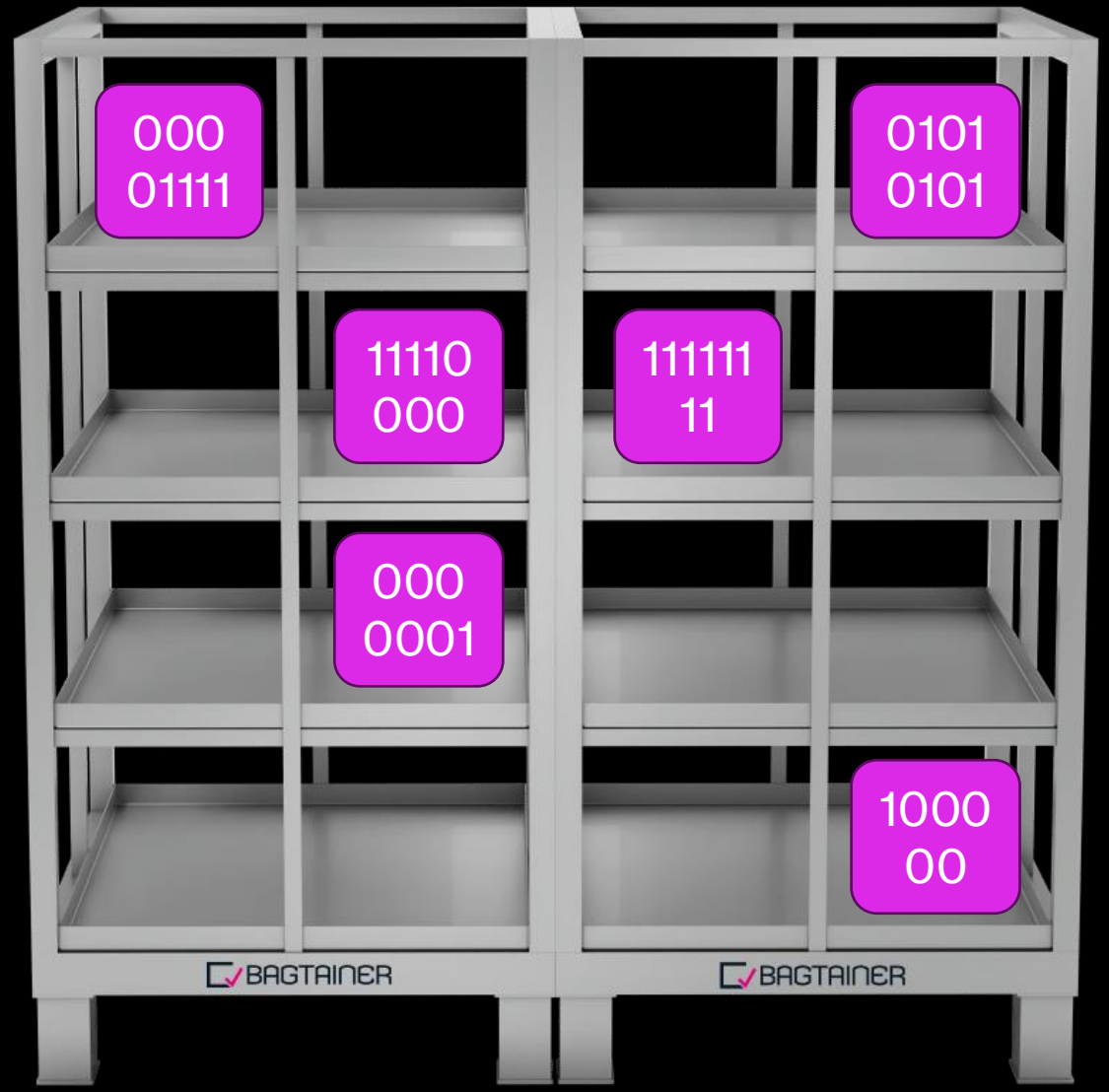
Ein Byte sind 8 Bit

1024 bytes	=	**1 KB**
1024 KB	=	**1 MB**
1024 MB	=	**1 GB**
1024 GB	=	**1 TB**
1024 TB	=	**1 PB**

KB	=	**Kilobyte**
MB	=	**Megabyte**
GB	=	**Gigabyte**
TB	=	**Terabyte**
PB	=	**Petabyte**


Bits
Nybble Nybble Nybble Nybble Nybble Nybble Nybble Nybble Nybble Nybble Nybble Nybble Nybble Nybble Nybble Nybble
Byte Byte Byte Byte Byte Byte Byte Byte
Word Word Word Word
Long Word Long Word
Very Long Word
Breite des CPU-Datenbus (meist 32 oder 64 bit)

Der Speicher als Regal



Byte = 8 Bit, i.e. 8 mal 0 oder 1

Wie viele Bytes bzw. Bits bekomme ich in ein 4x4 Regal/Speicher?
Wie viele in einen Speicher der 1 KB fasst?

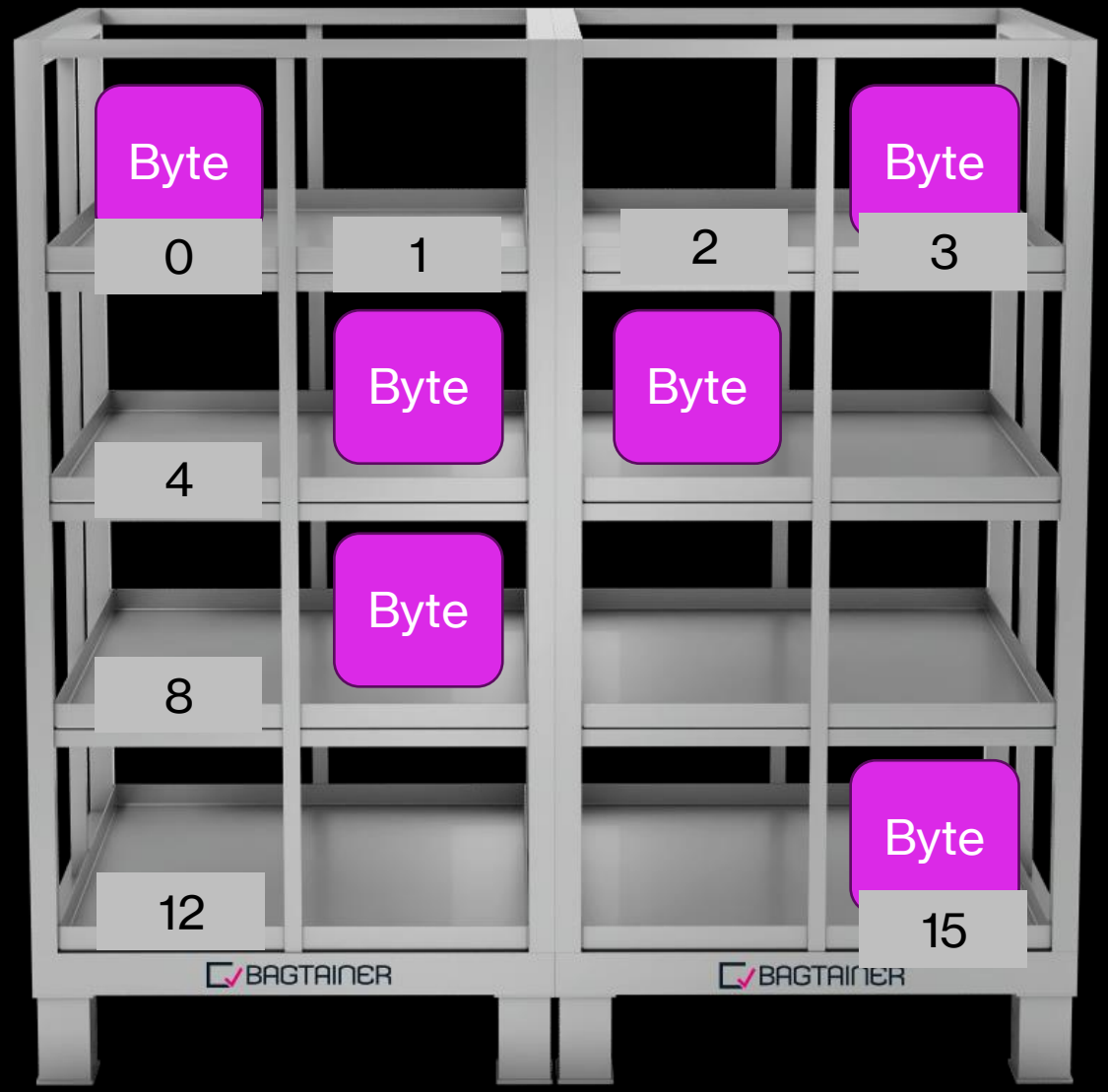
Lösung:
16 Bytes (128 Bits)
1024 Bytes (8192 Bits)

Adresse nicht gleich Daten

<u>Nicht vergessen:</u>
Adresse von Regalfach ist nicht gleich Inhalt von Regalfach

Wie viele Speicher Adressen kann ich mit 8 Bit abbilden?
Wie viel mit 32 Bit?

Lösung:
256 Bytes
4GB



Es werden 4 Bits benötigt, um alle 16 (2^4) Adressen dieses Speichers abzubilden

Wie lässt sich Text darstellen/speichern?

- Mittels einer Kodierung – der ASCII Tabelle
- Jedes Zeichen (z.B. Groß- oder Kleinbuchstaben) erhält eine Zahl / ein Byte

Zeichen	ASCII (dezimal)	ASCII (binär, 8 Bit)
H	72	01001000
A	65	01000001
L	76	01001100
L	76	01001100
O	79	01001111

Wie viele Bytes und Bits brauche ich, um das Wort „HALLO“ darzustellen?

Lösung:
5 Bytes (40 Bits)

ASCII Tabelle

Dezimal	Binär		Dezimal	Binär		Dezimal	Binär	
32	00100000	SP	64	01000000	@	96	01100000	`
33	00100001	!	65	01000001	A	97	01100001	a
34	00100010	"	66	01000010	B	98	01100010	b
35	00100011	#	67	01000011	C	99	01100011	c
36	00100100	$	68	01000100	D	100	01100100	d
37	00100101	%	69	01000101	E	101	01100101	e
38	00100110	&	70	01000110	F	102	01100110	f
39	00100111	'	71	01000111	G	103	01100111	g
40	00101000	(	72	01001000	H	104	01101000	h
41	00101001	)	73	01001001	I	105	01101001	i
42	00101010	*	74	01001010	J	106	01101010	j
43	00101011	+	75	01001011	K	107	01101011	k
44	00101100	,	76	01001100	L	108	01101100	l
45	00101101	-	77	01001101	M	109	01101101	m
46	00101110	.	78	01001110	N	110	01101110	n
47	00101111	/	79	01001111	O	111	01101111	o
48	00110000	0	80	01010000	P	112	01110000	p
49	00110001	1	81	01010001	Q	113	01110001	q
50	00110010	2	82	01010010	R	114	01110010	r
51	00110011	3	83	01010011	S	115	01110011	s
52	00110100	4	84	01010100	T	116	01110100	t
53	00110101	5	85	01010101	U	117	01110101	u
54	00110110	6	86	01010110	V	118	01110110	v
55	00110111	7	87	01010111	W	119	01110111	w
56	00111000	8	88	01011000	X	120	01111000	x
57	00111001	9	89	01011001	Y	121	01111001	y
58	00111010	:	90	01011010	Z	122	01111010	z
59	00111011	;	91	01011011	[	123	01111011	{
60	00111100	<	92	01011100	\	124	01111100	|

Unicode & UTF-8: Der globale Standard

- **Herausforderung der Globalisierung:**
  - Einschränkungen von ASCII und Codepages stießen an Grenzen
  - Unmöglichkeit, Texte mit Zeichen aus verschiedenen Sprachen in einer Datei zu kombinieren
  - Datenkorruption oder -verlust bei Nicht-ASCII-Zeichen
- **Einführung von Unicode:**
  - Universeller Zeichensatz, umfasst Zeichen aus allen Schreibsystemen weltweit
  - Weist jedem Zeichen einen eindeutigen Code-Punkt zu, unabhängig von Plattform, Programm oder Sprache
  - Kontinuierlich erweitert
  - Verantwortlich: Unicode Consortium

UTF-8 (Unicode Transformation Format - 8-bit)

- Am weitesten verbreitete Kodierung von Unicode
- **Variable Längen-Codierung:** Ein Zeichen kann je nach Code-Punkt 1 bis 4 Bytes belegen
- **Funktionsweise:**
  - **1 Byte:** ASCII-Bereich (U+0000 bis U+007F)
    - Vollständig rückwärtskompatibel mit ASCII
  - **2 Bytes:** U+0080 bis U+07FF (Lateinisch, Griechisch, Kyrillisch, diakritische Zeichen)
  - **3 Bytes:** U+0800 bis U+FFFF (Basic Multilingual Plane, CJK)
  - **4 Bytes:** U+10000 bis U+10FFFF (Emojis, seltene CJK-Zeichen)

Unicode Code-Punkt Bereich	Zeichen-Anzahl	UTF-8 Byte-Sequenz	Beispielzeichen	Code-Punkt	UTF-8 (Dezimal)
U+0000 - U+007F *(ASCII-kompatibel)*	128	0xxxxxxx *(1 Byte)*	A	U+0041	65
U+0080 - U+07FF *(Lateinisch-1 Ergänzungen, Griechisch, Kyrillisch)*	1920	110xxxxx 10xxxxxx *(2 Bytes)*	ß	U+00DF	195 159
U+0800 - U+FFFF *(Grundlegende Mehrsprachige Ebene, CJK)*	61440	1110xxxx 10xxxxxx 10xxxxxx *(3 Bytes)*	字	U+5B57	229 189 151
U+10000 - U+10FFFF *(Ergänzende Ebenen, Emojis)*	1048576	11110xxx 10xxxxxx 10xxxxxx 10xxxxxx *(4 Bytes)*	😂	U+1F602	240 159 152 130

Aufgabe

Schreiben Sie den Satz ”Guten Tag” mit Hilfe der ASCII Tabelle.

Wie viele Bytes werden für die Darstellung benötigt?

5 Minuten

Lösung

Zeichen	ASCII-Wert (dezimal)	ASCII-Wert (binär)	
G	71	01000111	
u	117	01110101	
t	116	01110100	
e	101	01100101	→ 9 Bytes
n	110	01101110	
(Leer)	32	00100000	
T	84	01010100	
a	97	01100001	
g	103	01100111	

Gemeinsame Übung

5 Minuten

Python Skript zur Darstellung von UTF-8 Encoding:

```
umlaute_sonderzeichen = ['ä', 'ö', 'ü', 'ß', '€']
print("\nExploration von Umlauten und Sonderzeichen:")
for char in umlaute_sonderzeichen:
    code_punkt = ord(char)
    print(f"Zeichen: '{char}', Code-Punkt: {code_punkt},
Zurückgewandelt: '{chr(code_punkt)}'")

# Beispiel für einen Bereich von Großbuchstaben
print("\nGroßbuchstaben A-Z und ihre Code-Punkte:")
for i in range(ord('A'), ord('Z') + 1):
    print(f"Zeichen: '{chr(i)}', Code-Punkt: {i}")
```



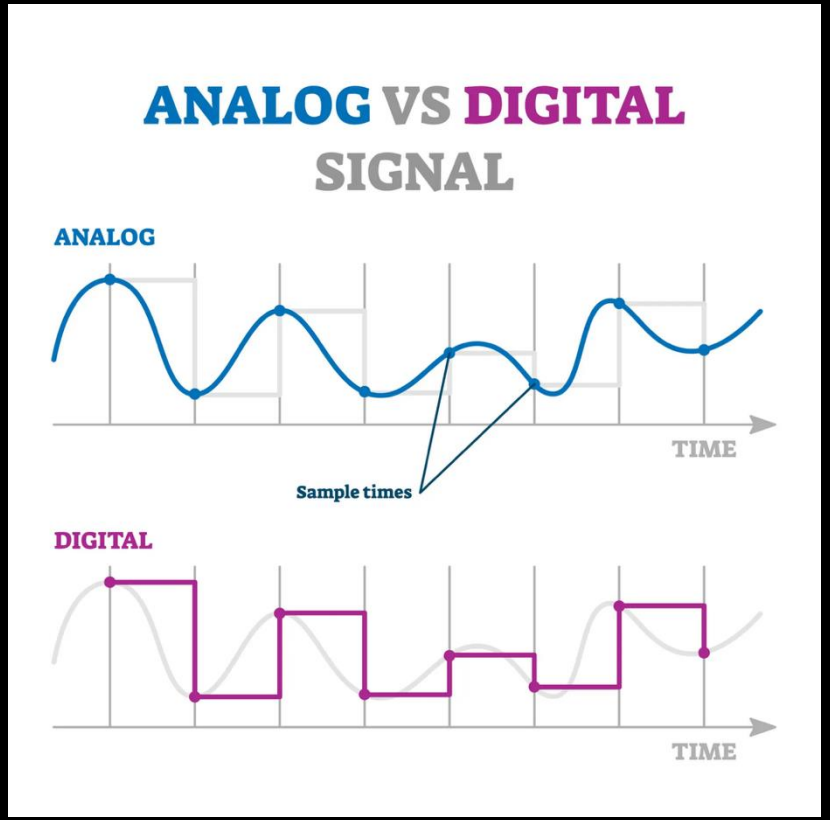
Gutes Video zur Erklärung

https://www.youtube.com/watch?v=jARCjroHRCw

4 Zahlensysteme – Dual, Oktal, Hexadezimal

Analog vs. Digital

- **Analogie:** Ein Thermometer (analog) zeigt kontinuierlich eine Temperatur an. Eine digitale Anzeige (z. B. auf einem Display) zeigt diskrete, also in Stufen gemessene Werte.
- **Prozessleittechnik (PLT):** Moderne Anlagensteuerungen (SPS) und Sensoren verarbeiten digitale Signale.
- **Beispiel:** Ein Sensor misst einen Druck. Dieser analoge Wert wird durch einen **Analog-Digital-Wandler (ADC)** in eine digitale, binäre Zahl umgewandelt. Diese Zahl muss verstanden werden, um den Prozess zu steuern.



Verschiedene Zahlensysteme

- **Zahlensysteme** sind Methoden, um Zahlen mithilfe einer festen Menge von **Symbolen** darzustellen.
- Jedes Zahlensystem hat eine **Basis** (engl. *base*), die angibt, **wie viele verschiedene Ziffern** es enthält und auf welchem **Vielfachensystem** es beruht.
- Die **Basis** ist die Anzahl der verschiedenen Ziffern, die im System verwendet werden.
- Beispiel:
  - Basis 10 → Ziffern **0–9**
  - Basis 2 → Ziffern **0 und 1**

Wichtige Zahlensysteme

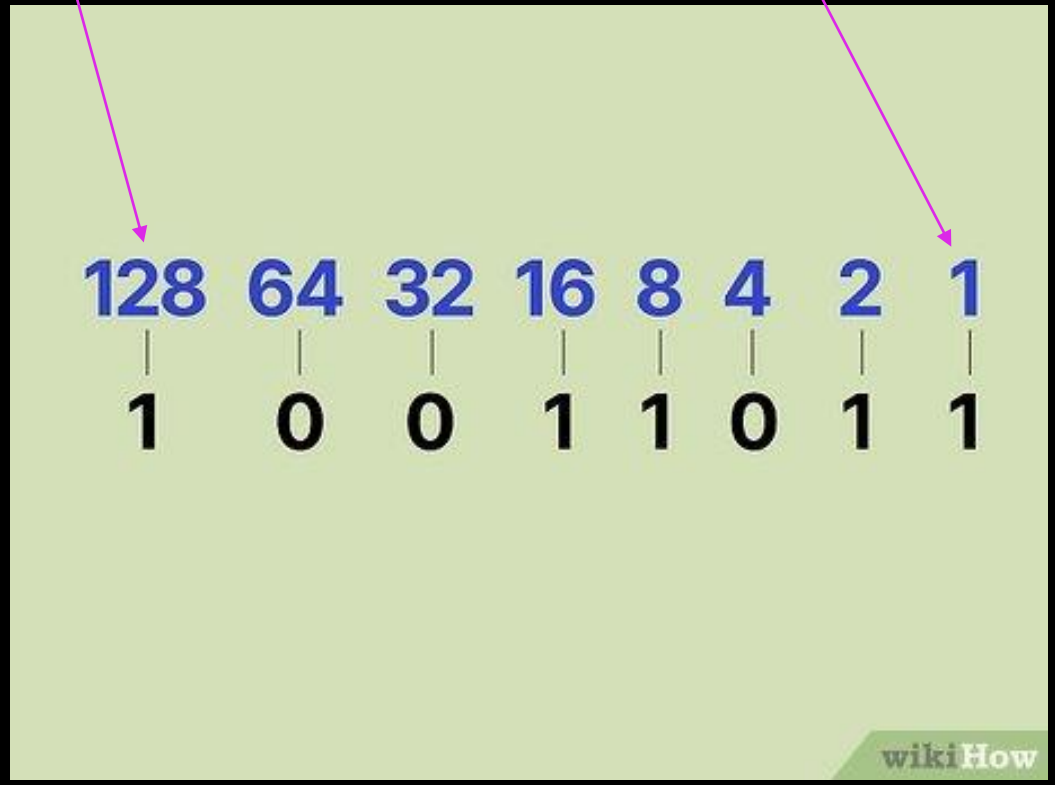
- **Dezimalsystem (Basis 10)**
  - Unser Alltagssystem, Ziffern 0–9.
- **Binärsystem (Basis 2)**
  - Nur 0 und 1, wird in Computern genutzt.
- **Oktalsystem (Basis 8)**
  - Ziffern 0–7, wird manchmal in der Informatik verwendet.
- **Hexadezimalsystem (Basis 16)**
  - Ziffern 0–9 und Buchstaben A–F, sehr praktisch für kompakte Darstellung von Binärzahlen.

4.1 Binärsystem

Das Binärsystem

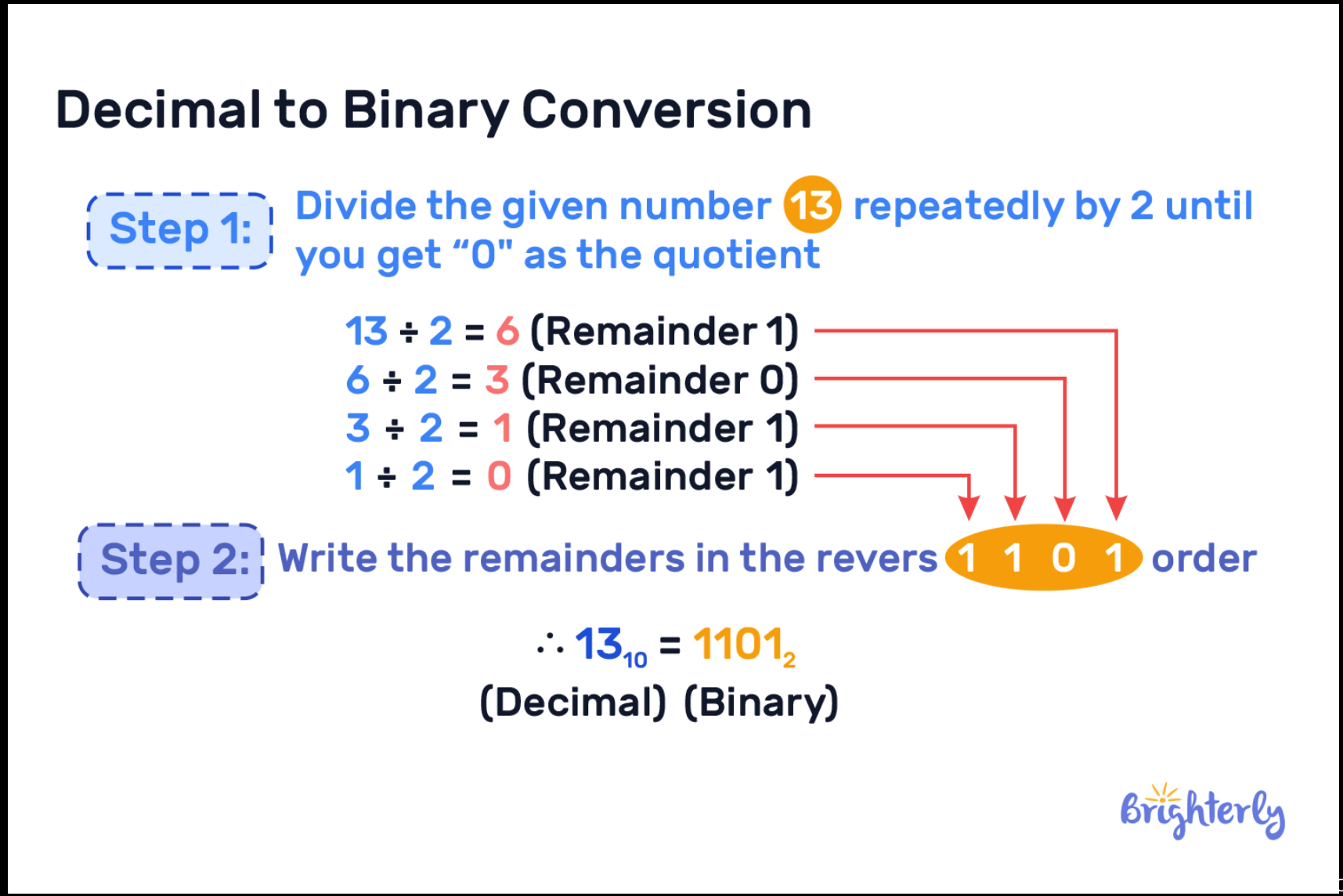
Most Significant Bit (MSB)
→ Immer ganz links
→ hat den größten Wert (128)

Least Significant Bit (MSB)
→ Immer ganz rechts
→ hat den kleinsten Wert (1)



- **Problem:** Elektrische Schaltungen können nur zwei stabile Zustände zuverlässig darstellen: Strom an/aus, hohe/niedrige Spannung.
- **Lösung:** Das Dualsystem oder **Binärsystem** (Basis 2). Es verwendet nur die Ziffern **0** und **1**.
- **Stellenwertsystem:** Jede Stelle hat einen Wert, der einer Potenz von 2 entspricht (z. B. 2^0, 2^1, 2^2).
- **Beispiel:** Die Zahl 1101 im Binärsystem ist im Dezimalsystem: $1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 = 8 + 4 + 0 + 1 = 13$.
- Standard Prefix: 0b (z.B. *0b1101 → 13*)

Konvertierung



Konvertierung

Video

BINÄR → DEZIMAL (STELLENWERTVERFAHREN)

Beispiel: 101010_2 → $?_{10}$

Stelle	Ziffer	× Potenz	Wert	
5	1	× $2^5 = 32$	**32**	Most Significant Bit (MSB)
4	0	× $2^4 = 16$	0	
3	1	× $2^3 = 8$	**8**	
2	0	× $2^2 = 4$	0	
1	1	× $2^1 = 2$	**2**	
0	0	× $2^0 = 1$	0	Least Significant Bit (LSB)

Summe: $32+8+2 = 42_{10}$ ✓

Zweierpotenzen: 1, 2, 4, 8, 16, 32, 64, 128, 256…

Aufgabe

- Konvertieren Sie die dezimalen Zahlen in Binär:
  - 16
  - 50
  - 124

10 Minuten

Lösungen

- **16 in Binär**
- $16 = 2^4$
- Binär: **10000**

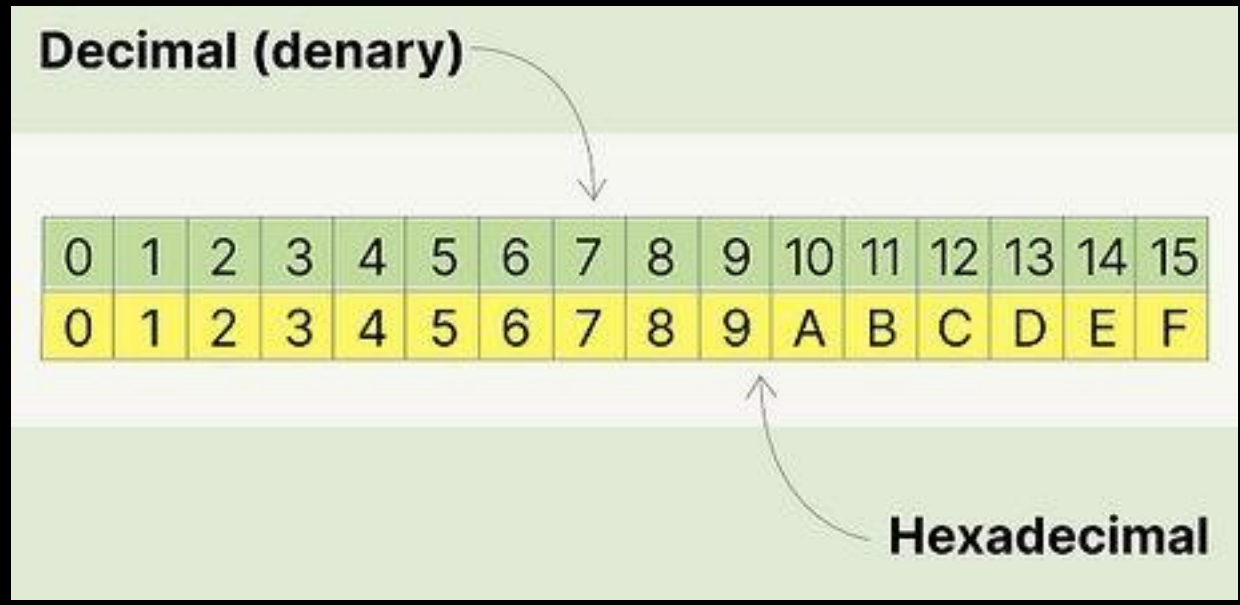
50 in Binär
50 ÷ 2 = 25 Rest **0**
25 ÷ 2 = 12 Rest **1**
12 ÷ 2 = 6 Rest **0**
6 ÷ 2 = 3 Rest **0**
3 ÷ 2 = 1 Rest **1**
1 ÷ 2 = 0 Rest **1**
Von unten nach oben gelesen: **110010**

124 in Binär
124 ÷ 2 = 62 Rest **0**
62 ÷ 2 = 31 Rest **0**
31 ÷ 2 = 15 Rest **1**
15 ÷ 2 = 7 Rest **1**
7 ÷ 2 = 3 Rest **1**
3 ÷ 2 = 1 Rest **1**
1 ÷ 2 = 0 Rest **1**
Von unten nach oben gelesen: **1111100**

4.2 Hexadezimal

Das Hexadezimalsystem

- **Problem:** Lange binäre Zahlen sind unübersichtlich und fehleranfällig. Ein 8-Bit-Wert wie 10111100 ist schwer zu lesen.
- **Lösung:** Das **Oktalsystem** (Basis 8) und das **Hexadezimalsystem** (Basis 16) dienen als kompakte Kurzschreibweisen.
- **Hexadezimalsystem:** Verwendet die Ziffern 0-9 und A-F für die Werte 10-15.
- **Vorteil:** Eine hexadezimale Ziffer entspricht genau vier Binärziffern (z. B. F = 1111). Eine oktale Ziffer entspricht drei Binärziffern. Das macht die Umrechnung sehr einfach.
- Standard Prefix: 0x (z.B. *0x0D → 13*)



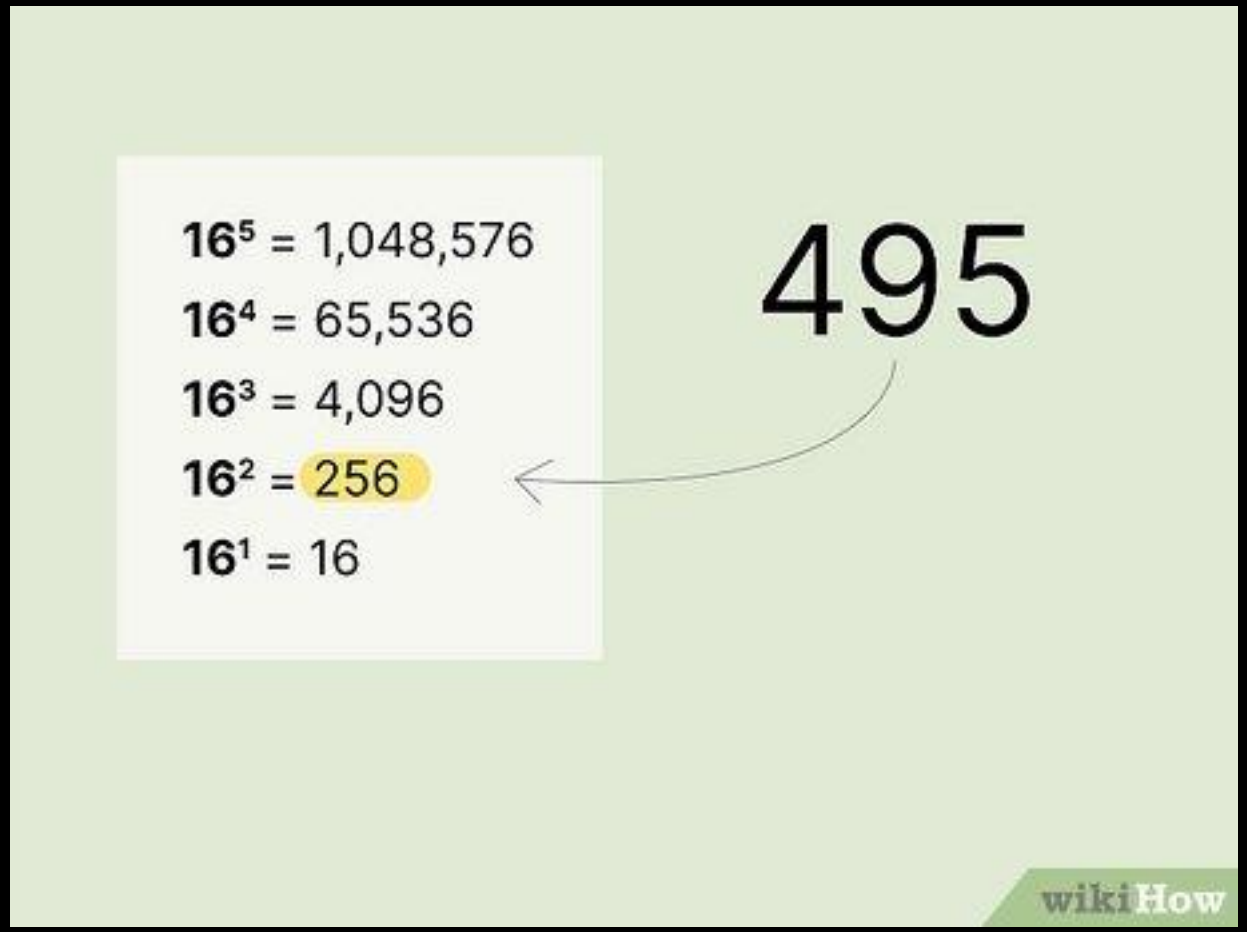
Schreibweise Hexadezimal

16^3	16^2	16^1	16^0	
4096	*256*	*16*	*0*	
1	*10*	*12*	*8*	→ **6856**
1	**A**	**C**	**8**	

Most Significant Bit (MSB)
→ **Immer ganz links**
→ **hat den größten Wert (4096)**

Least Significant Bit (LSB)
→ **Immer ganz rechts**
→ **hat den kleinsten Wert (8)**

Konvertierung



Zwischen welche Hexa-Faktoren passt unsere Zahl?
Hier zwischen 16^2 und 16^3, da:
$256 < 495 < 4096$

→ 16^2 ist unser erster Wert

Konvertierung – Beispiel 495

Video

1. Erster Wert ermitteln

2. Zahl durch Faktor teilen (495 / 256 = 1 mit Rest 239)

3. Rest übertragen

Faktor von 16	Rest	Ergebnis	Hex
256	495	1	1
16	239	14	E
	15	15	F

4. Rest durch nächsten Faktor teilen (239 / 16 = 14 mit Rest 15)

5. Rest übertragen (Rest < 16 → Fertig)

6. Ergebnisse in Hex übersetzen und von oben nach unten ablesen

→ 1 E F

Konvertierung – Beispiel 812

Video

Faktor von 16	Rest	Ergebnis	Hex
256	812	3	3
16	44	2	2
	12	12	C

→ **3 2 C**

Aufgabe

- Konvertieren Sie die dezimalen Zahlen in Hexadezimal:
  - 16
  - 124
  - 513
  - 5130

10 Minuten

Lösungen (1/2)

Faktor von 16	Rest	Ergebnis	Hex
16	16	0	1
	0	0	0

→ **1 0**

Faktor von 16	Rest	Ergebnis	Hex
16	124	7	7
	12	12	C

→ **7 C**

Lösungen (2/2)

Faktor von 16	Rest	Ergebnis	Hex
256	513	2	2
16	1	0	0
	1	1	1

→ **2 0 1**

Faktor von 16	Rest	Ergebnis	Hex
4096	5130	1	1
256	1034	4	4
16	10	0	0
	10	10	A

→ **1 4 0 A**

HEXADEZIMALSYSTEM

Dezimalsystem = 10 Ziffern -> 0, 1, 2, 3, 4, 5, 6, 7, 8, 9

Binärsystem = 2 Ziffern -> 0, 1

Hexadezimalsystem = 16 Ziffern -> 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F

Gutes Video zur Erklärung

https://www.youtube.com/watch?v=nbUiXka4tj0

4.3 Festkomma & Gleitkomma

Festkomma & Gleitkomma

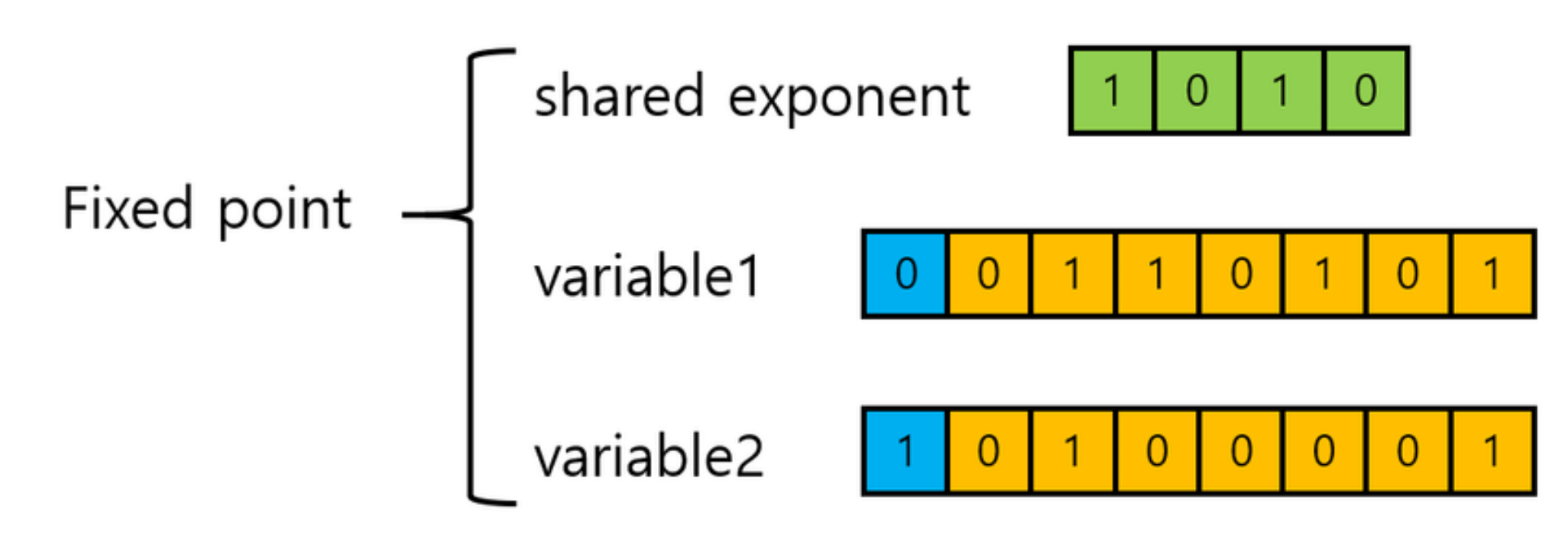
- Wir wissen bereits, dass ein Computer nur Nullen und Einsen kennt.

- **Frage:** Wie speichert man dann Messwerte wie die Temperatur eines Reaktors (123,45 °C) oder den Volumenstrom (0,05 m3/s), wenn es kein Komma gibt?
- **Problem:** Ganzzahlen (z. B. 123) sind einfach. Für Kommazahlen (auch als **Fließ-** oder **Reelle Zahlen** bekannt) braucht man eine spezielle Methode.
- **Analogie:** Wie würde man eine Messung wie 123,45 cm auf einem Lineal mit nur Millimeter-Markierungen speichern? Und wie würde man die Entfernung zur Sonne darstellen?

Methode 1: Festkommazahlen (Fixed-Point)

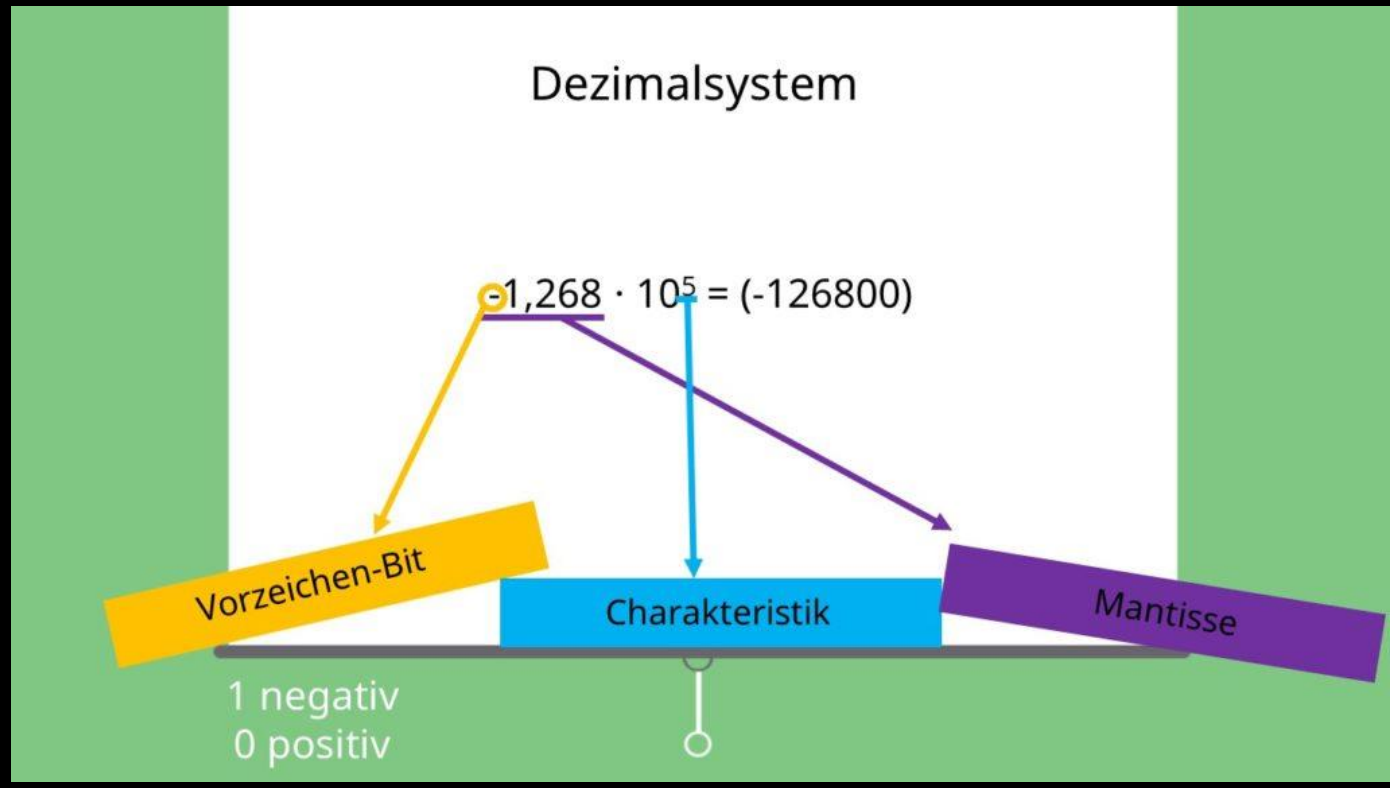
- **Konzept:** Man legt fest, wo das Komma sitzen soll, also wie viele Stellen hinter dem Komma gespeichert werden. Es ist immer an der gleichen, "festen" Stelle.
- **Beispiel:** Währungswerte. Man speichert alles in Cent (1,50€ wird zu 150). Das Komma ist gedanklich immer nach zwei Stellen.
- **Vorteile:**
  - Sehr einfach und schnell zu verarbeiten.
  - Keine Rundungsfehler durch die Basis (10 vs. 2).
  - Präzise für einen begrenzten Bereich (z. B. 0,00 bis 999,99).
- **Nachteile:**
  - **Begrenzter Bereich:** Man muss sich zwischen sehr großen Zahlen (großer Vorkomma-Teil) und hoher Genauigkeit (großer Nachkomma-Teil) entscheiden.
  - Schlecht für sehr kleine oder sehr große Werte.

Methode 1: Festkommazahlen (Fixed-Point)

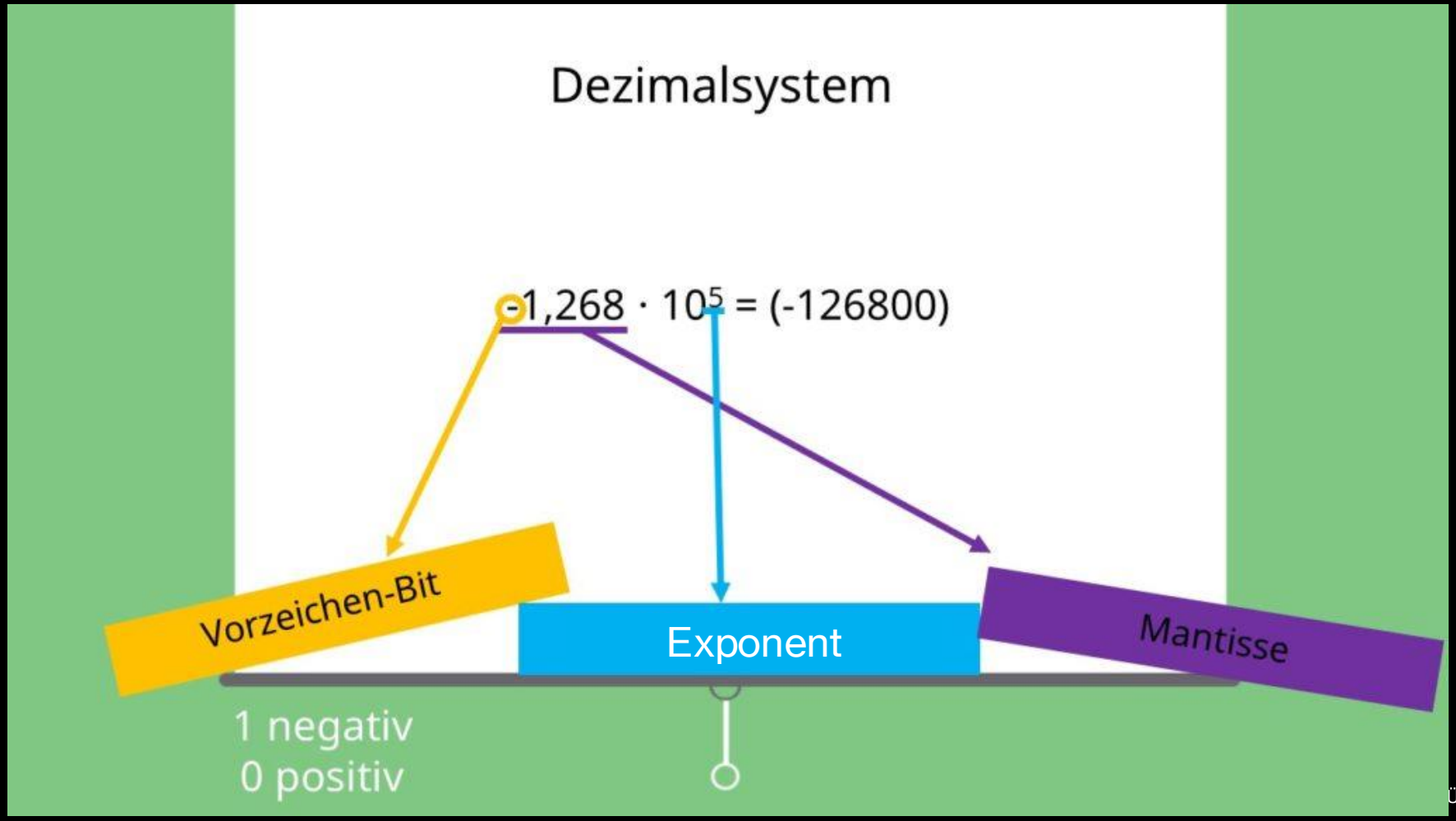


Methode 2: Gleitkommazahlen (Floating-Point)

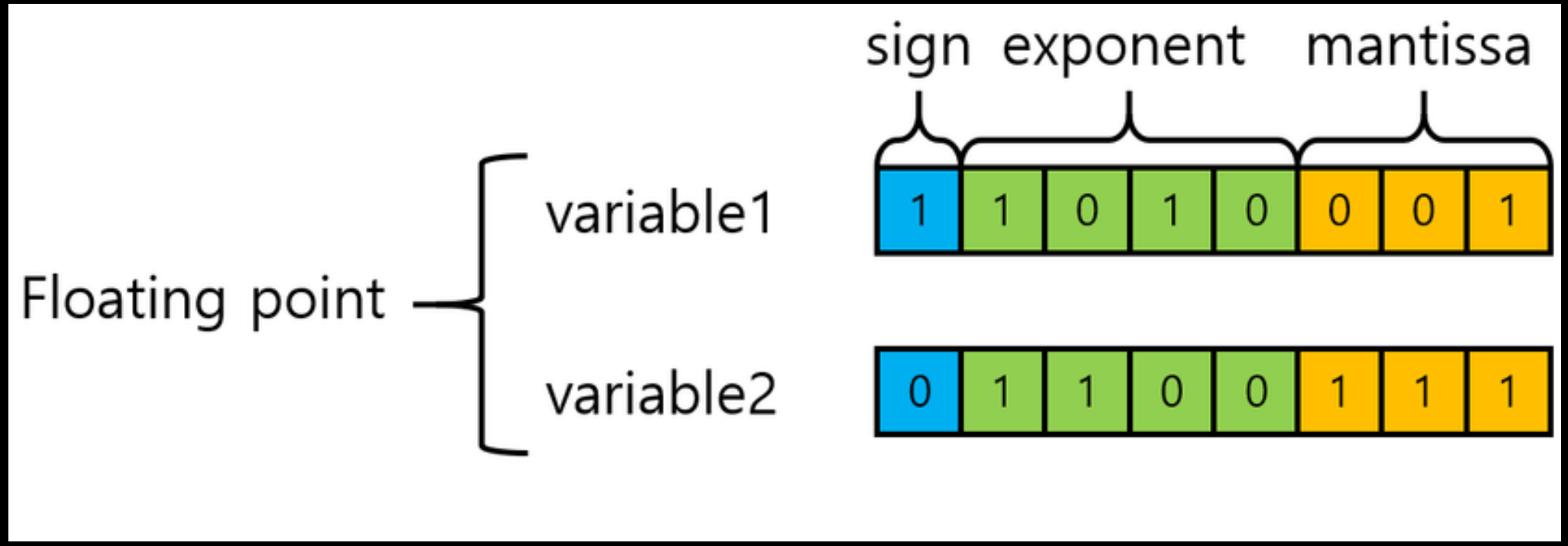
- **Konzept:** Das Komma kann „gleiten“. Ähnlich wie bei der wissenschaftlichen Schreibweise ($1{,}23 \times 10^3$). Die Zahl wird in drei Teile zerlegt.
- **Vorteile:**
  - Riesiger Zahlenbereich, von extrem klein bis extrem groß.
  - Universell einsetzbar für wissenschaftliche Berechnungen.
- **Nachteile:**
  - Komplexere Arithmetik.
  - **Genauigkeitsprobleme:** Nicht alle Zahlen lassen sich exakt darstellen (z. B. 0,1 im Dezimalsystem ist eine unendliche Binärfolge). Dies führt zu Rundungsfehlern.



Methode 2: Gleitkommazahlen (Floating-Point)



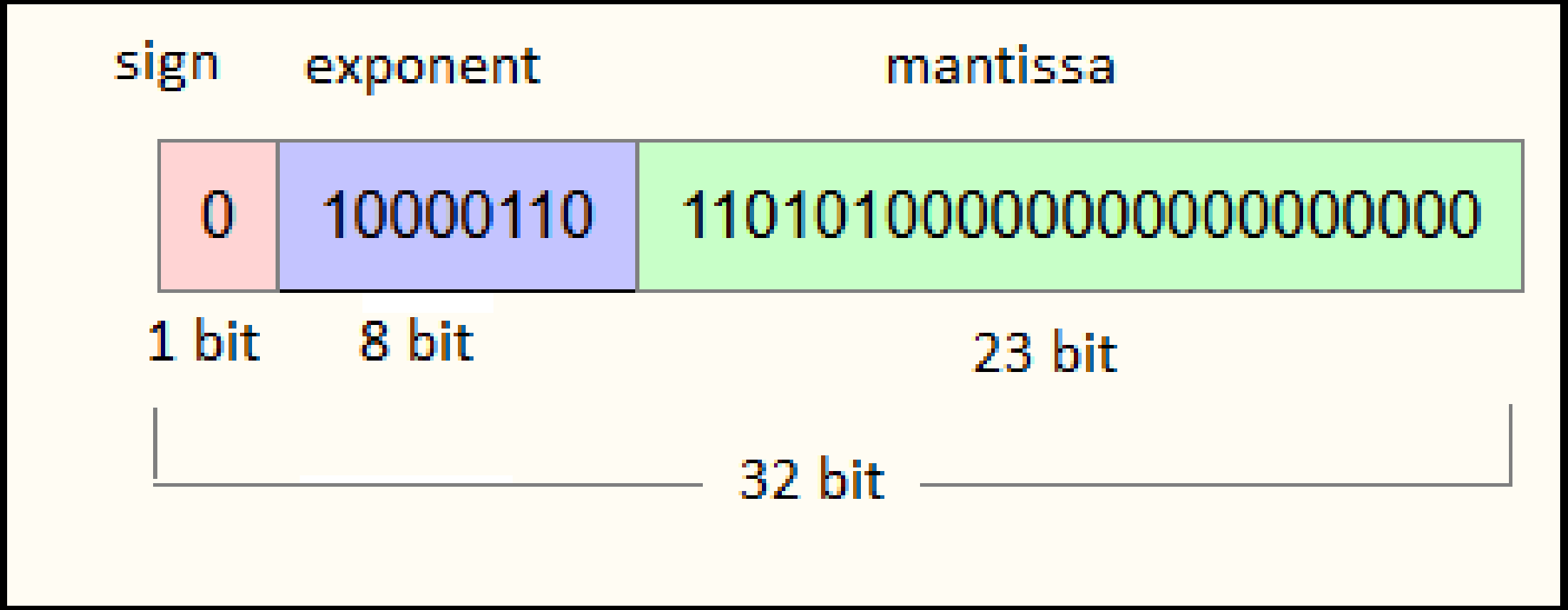
Methode 2: Gleitkommazahlen (Floating-Point)



Der IEEE-754 Standard

- Die meisten Computer nutzen den IEEE-754-Standard, der genau festlegt, wie die drei Teile (Vorzeichen, Exponent, Mantisse) in 32 oder 64 Bit gespeichert werden.
- **32 Bit (Single Precision):**
  - 1 Bit für das Vorzeichen
  - 8 Bit für den Exponenten
  - 23 Bit für die Mantisse
- **64 Bit (Double Precision):**
  - 1 Bit für das Vorzeichen
  - 11 Bit für den Exponenten
  - 52 Bit für die Mantisse
- **Merksatz:** Mehr Bits bedeuten größere Zahlen und/oder eine höhere Genauigkeit. Für die meisten wissenschaftlichen Berechnungen wird 64-Bit genutzt.

Der IEEE-754 Standard



Warum das für Sie wichtig ist: Die Tücken der Gleitkomma-Arithmetik

- **Beispiel 1: Numerische Simulationen**
  - In CFD-Simulationen (Strömungsmechanik) oder chemischen Prozesssimulationen werden Tausende von Gleichungen gelöst.
  - Jede einzelne Berechnung kann einen winzigen Rundungsfehler erzeugen. Über Tausende von Schritten können sich diese Fehler akkumulieren und zu **falschen Ergebnissen** oder sogar zum **Abbruch der Simulation** führen (sog. Divergenz).
- **Beispiel 2: Sensorik & Steuerung**
  - Ein PID-Regler im Prozessleitsystem rechnet ständig mit Messwerten, z. B. 25.001 °C.
  - Wenn zwei fast gleiche Werte voneinander subtrahiert werden (sog. **Auslöschung**), kann die Genauigkeit drastisch sinken, was das Regelverhalten beeinträchtigen kann.
- **Lösung:** Man nutzt 64-Bit-Gleitkommazahlen (Double Precision) und wählt robuste Algorithmen.

Gemeinsame Übung

5 Minuten

- Online Tool zur Darstellung von Gleitkommazahlen:
  - h-schmidt.net/FloatConverter/IEEE754.html

- Geben Sie die Zahl 0.5 ein. Was sehen Sie?
- Geben Sie die Zahl 0.1 ein. Was fällt auf?

- **Diskussion:** Warum ist die Zahl 0.1 im Binärsystem ungenau, obwohl 0.5 exakt darstellbar ist? (Hinweis: Die Basis 2).

Quiz

Multiple Choice

1 Minute

Die Binärzahl 1010 entspricht welcher Zahl im Hexadezimalsystem?

a) A

b) 12

c) 10

d) B

Welche Aussagen über Gleitkommazahlen sind korrekt?

a) Sie können sehr große und sehr kleine Werte darstellen.

b) Sie haben eine feste Anzahl von Nachkommastellen.

c) Sie können Rundungsfehler enthalten.

d) Sie werden in Computern oft nach dem IEEE-754-Standard gespeichert.

Quiz

Wahr/Falsch

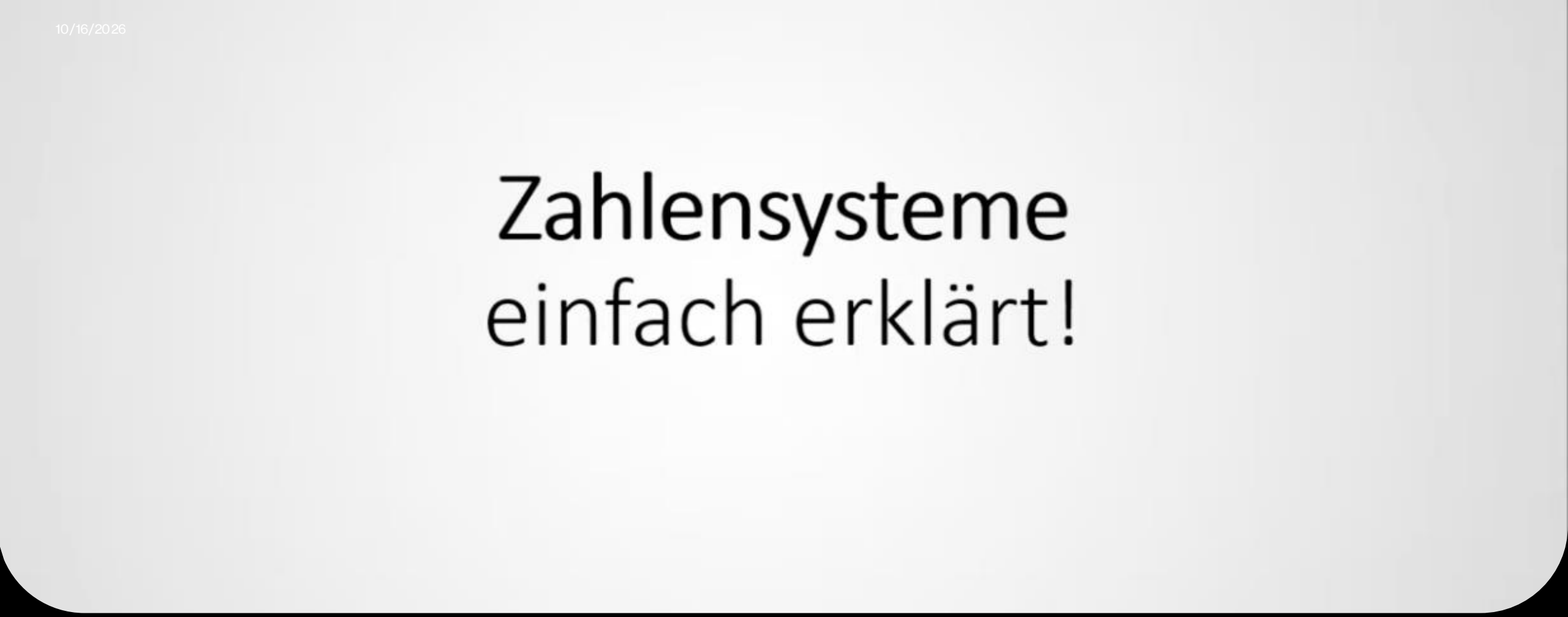
1 Minute

- Ein Byte besteht aus 16 Bits. 
- Die Zahl 1111 im Binärsystem entspricht F im Hexadezimalsystem. 
- Ein „Wort“ ist in der Informatik immer genau 32 Bit groß. 
- Festkommazahlen eignen sich besonders gut für präzise Berechnungen mit festen Nachkommastellen (z. B. Geldbeträge), während Gleitkommazahlen besser für sehr große oder sehr kleine Werte geeignet sind. 

Diskussion

5 Minuten

- Warum arbeitet ein Computer intern im Binärsystem, obwohl wir Menschen Zahlen normalerweise im Dezimalsystem darstellen?
- Das Hexadezimalsystem wird oft als „Abkürzung“ für Binärzahlen verwendet. Diskutieren Sie: Welche Vorteile bietet es gegenüber einer reinen Binärdarstellung?
- In der Verfahrenstechnik werden oft Messwerte und Zustände gespeichert. Warum ist es wichtig, die Begriffe **Bit, Byte und Wort** zu verstehen, wenn man Speicherbedarf oder Datenübertragungen einschätzen möchte?
- Diskutieren Sie anhand eines Beispiels: Welche praktischen Probleme könnten auftreten, wenn ein System mit 8-Bit-Wörtern durch eines mit 64-Bit-Wörtern ersetzt wird (z. B. Kompatibilität, Speicherbedarf, Geschwindigkeit)?



Gutes Video zur Erklärung

https://www.youtube.com/watch?v=Y_NhZm_kQ7U

5 Programmiersprachen – Unterschiede & Einsatzzwecke

Was sind Programmiersprachen?

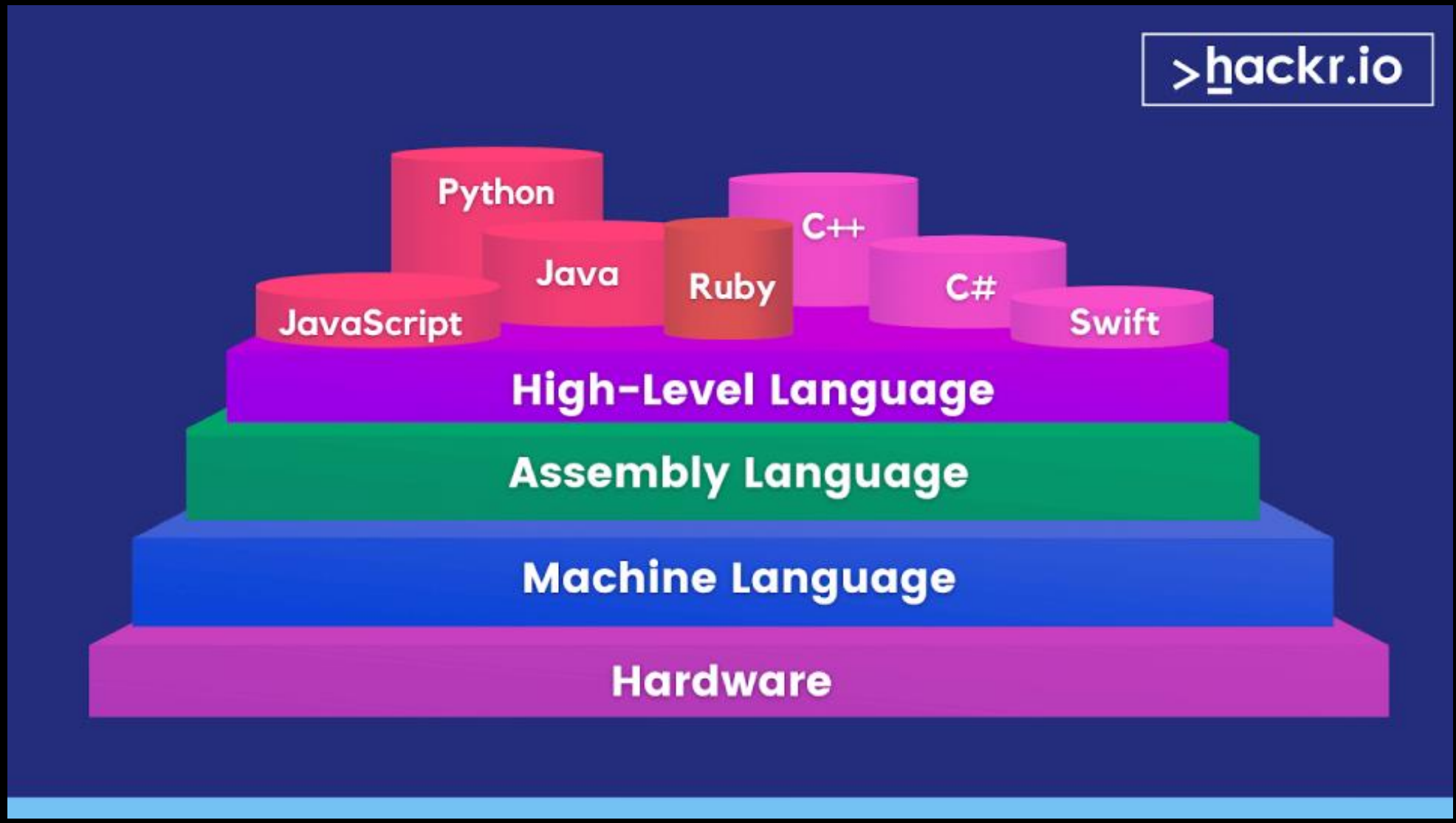
- **Was sind Programmiersprachen?**
  - Kommunikationsmittel zwischen Menschen und Computer.
  - Präzise "Kochrezepte" für Computer, die Anweisungen wörtlich verstehen.

- **Häufige Missverständnisse:**
  - "Man muss ein Mathematik-Genie sein": Irrtum, logisches Denken ist wichtiger.
  - "Programmierung ist langweilig/nur für IT-Spezialisten": Kreatives Werkzeug zur Problemlösung.
  - "Automatisierung führt zu Arbeitsplatzverlusten": Verändert die Arbeit, ermöglicht Fokus auf höherwertige Aufgaben.
  - **Herausforderung:** Hohe Präzision im Gegensatz zu natürlicher Sprache; erfordert "algorithmisches Denken".

Definition einer Programmiersprache

- **Definition:** Künstliche Sprache zur Spezifikation von Computerprogrammen und Anweisungen für Berechnungen.
- **Formale Sprachen:** Präzise, eindeutige Spezifikation (im Gegensatz zu natürlichen Sprachen).
- **Zwei Säulen:**
  - **Syntax (Grammatik):** Regeln, wie Anweisungen (Quellcode) gebildet werden (z.B. Semikolon am Ende, Einrückung).
  - **Semantik (Bedeutung):** Verhalten des Codes bei Ausführung – was passiert.
    - *Wichtig:* Semantische Fehler führen zu falschem Verhalten, auch bei korrekter Syntax.
- **Abstraktionsgrad von Hardware:**
  - **Low-Level-Sprachen:** Wenige Abstraktionen, hardwarenah, direkte Kontrolle (z.B. Maschinencode, Assembler).
  - **High-Level-Sprachen:** Viele Abstraktionen, näher an menschlicher Sprache, einfacher zu lernen/debuggen, portabler (z.B. Python, Java).
    - *Hinweis:* "High-Level" ist relativ (z.B. C war High-Level, heute oft Mid-Level).
- **Sprachwahl:** Immer ein Kompromiss zwischen Hardwarekontrolle, Ausführungsgeschwindigkeit und Entwicklerproduktivität.

Leichter lesbar für Menschen



Leichter lesbar für Maschine

Langsamer / Flexibel

Schneller / Weniger flexibel

Vergleich: Hochsprachen vs. Maschinensprachen (Ausführung)

- **Übersetzung in Maschinencode:** Computer verstehen nur Binärcode.
- **Kompromiss:** Entwicklungsgeschwindigkeit vs. Ausführungsgeschwindigkeit.
- **Moderne Entwicklung:** Grenzen verschwimmen (z.B. Java JVM mit JIT-Kompilierung).

Interpretierte Sprachen:

Prozess: Quellcode *Zeile für Zeile* zur Laufzeit durch "Interpreter" übersetzt und direkt ausgeführt.
Vorteile: Sehr schneller Entwicklungszyklus (sofortiges Testen/Debugging).
Nachteile: Langsamere Ausführung (Übersetzung bei jedem Programmlauf).
Beispiele: Python, JavaScript.



Kompilierte Sprachen:

Prozess: Gesamter Quellcode *einmalig* vor Ausführung durch "Compiler" in Maschinencode übersetzt.
Vorteile: Sehr schnelle Ausführung, effizienter Ressourceneinsatz.
Nachteile: Längerer Entwicklungsprozess (jede Änderung erfordert Neukompilierung).
Beispiele: C, C++. Java ist Hybrid (Bytecode-Kompilierung, dann Interpretation/JIT-Kompilierung durch JVM).









3 Sprachen im Vergleich

Python

Eigenschaften:

- **Interpretiert**: Schneller Edit-Test-Debug-Zyklus.
- High-Level & Objektorientiert: Vereinfacht Codierung, abstrahiert Hardware.
- **Dynamische Typisierung**: Variablentypen zur Laufzeit bestimmt, beschleunigt Entwicklung.
- **Hohe Lesbarkeit**: Einfache, klare Syntax (Einrückung statt Klammern).
- **Umfangreiches Ökosystem**: Große Standardbibliothek, viele Module/Pakete (Code-Wiederverwendung).

Produktivitätssteigerung: Kürzere Programme, weniger Entwicklungszeit durch High-Level-Datentypen und dynamische Typisierung.
Bedeutung für VT: Brückensprache zwischen Ingenieurwissen und digitaler Transformation (Industrie 4.0, Datenwissenschaft, KI).

Typische Anwendungsbereiche:

- **Datenanalyse & Maschinelles Lernen (ML)**: De-facto-Standard in Datenwissenschaft (NumPy, Pandas, TensorFlow).
- **Automatisierung & Scripting**: Repetitive Aufgaben, Dateiverwaltung, Web-Scraping.
- **Simulation & Modellierung**
- **Webentwicklung** (Backend): Django, Flask.
- Allgemeine Zweckmäßigkeit: Vielseitige "***General-Purpose"-Sprache***“.

Java

Eigenschaften:

- **Objektorientiert**: Fördert modularen, wiederverwendbaren Code.
- **Plattformunabhängigkeit** ("Write Once, Run Anywhere"): Bytecode läuft auf jeder JVM-fähigen Plattform.
- **Robust & Sicher**: Keine direkten Pointer, Schutz vor Speicherlecks, robuste Fehlerbehandlung.
- **Multithreading-Fähigkeit**: Gleichzeitige Ausführung mehrerer Aufgaben (Transaktionsverarbeitung, Echtzeit-Monitoring).
- **Umfangreiches Ökosystem**: Große Standardbibliothek, viele Frameworks/Tools.

- **Strategische Bedeutung:** Plattformunabhängigkeit reduziert Komplexität der Systemintegration in der Fertigung.
- **Rückgrat für Industriesysteme:** Zuverlässigkeit, Sicherheit, Portabilität für Bestandsmanagement, Echtzeit-Monitoring, Smart Manufacturing.

Typische Anwendungsbereiche:

- **Industrielle Automatisierung** & Steuerung: Echtzeit-Monitoring, Prozesskontrolle.
- **Enterprise-Systeme**: Große Geschäftsanwendungen (ERP, Supply Chain Management).
- **Internet of Things**: Intelligente Fertigungsanlagen, nahtlose Kommunikation.
- **Mobile Anwendungen:** Primäre Sprache für native Android-Apps.
- **Webanwendungen** (Backend): Serverseitige Webentwicklung.

C

Eigenschaften:

- **"Mid-Level"-Sprache**: Mischung aus High-Level-Abstraktion und Low-Level-Hardwarekontrolle.
- **Prozedural**: Fokus auf Funktionen und sequenzielle Ausführung.
- **Effizienz & Performance**: Effizienter Ressourceneinsatz.
- **"Mutter vieler Sprachen":** Syntax beeinflusste C++, Java, Python.

- **Strategische Bedeutung:** Hardwarenähe ermöglicht schnelle und präzise Kontrolle
- **Bedeutung für VT:** Nutzung in vielen embedded systems (i.e. Systeme in anderen Produkten, z.B. Fertigungsmaschine)

Typische Anwendungsbereiche:

- **Echtzeitsysteme & Embedded** Systems: Präzise Zeitsteuerung, direkter Hardwarezugriff (Mikrocontroller, Automobil, Luft- u. Raumfahrt).
- **Betriebssysteme & Compiler**: Die meisten sind in C/C++ geschrieben.
- **Prozessleitsysteme (PLCs)**: Integration in industrielle Automatisierung.
- **Hochleistungsrechnen & Simulationen**: Effiziente Leistung, Echtzeitverarbeitung.
- **Spiele-Engines & Grafikanwendungen**: Geschwindigkeit, Hardwarekontrolle.

C++

Eigenschaften:

- **Erweiterung von C ("C with Classes"):** Fügt OOP und High-Level-Funktionen hinzu.
- **"Middle-Level"-Sprache:** Kombiniert High-Level (OOP, Templates) mit Low-Level (Speicherverwaltung).
- **Sehr schnelle Ausführung:** Gehört zu den schnellsten High-Level-Sprachen (kompiliert).27
- **Manuelle Speicherverwaltung:** Direkte Kontrolle, aber potenzielle Fehlerquellen.

- **Strategische Bedeutung:** Hardwarenähe ermöglicht schnelle und präzise Kontrolle
- **Bedeutung für VT:** Nutzung in vielen embedded systems (i.e. Systeme in anderen Produkten, z.B. Fertigungsmaschine)

Typische Anwendungsbereiche:

- **Echtzeitsysteme & Embedded** Systems: Präzise Zeitsteuerung, direkter Hardwarezugriff (Mikrocontroller, Automobil, Luft- u. Raumfahrt).
- **Betriebssysteme & Compiler**: Die meisten sind in C/C++ geschrieben.
- **Prozessleitsysteme (PLCs)**: Integration in industrielle Automatisierung.
- **Hochleistungsrechnen & Simulationen**: Effiziente Leistung, Echtzeitverarbeitung.
- **Spiele-Engines & Grafikanwendungen**: Geschwindigkeit, Hardwarekontrolle.

Merkmal	Python	Java	C	C++
Typ	Interpretiert (High-Level)	Kompiliert zu Bytecode (High-Level)	Kompiliert (Mid-Level)	Kompiliert (Mid-Level)
Lesbarkeit	Sehr hoch (Englisch-ähnliche Syntax, klare Struktur)	Hoch (objektorientiert, klare Naming Conventions)	Mittel (näher an Hardware, weniger Abstraktion)	Mittel (komplexer als C durch OOP-Features)
Ausführungsgeschwindigkeit	Langsamer (durch Interpretation, dynamische Typisierung)	Schnell (durch JVM, JIT-Kompilierung)	Sehr schnell (direkte Maschinencode-Generierung)	Extrem schnell (optimiert für Performance)
Hardwarenähe / Kontrolle	Gering (hohe Abstraktion)	Mittel (JVM schirmt Hardware ab)	Hoch (direkter Speicherzugriff, Systemprogrammierung)	Sehr hoch (direkter Speicherzugriff, Systemprogrammierung)
Entwicklungszeit	Sehr schnell (Rapid Application Development, große Bibliotheken)	Schnell (gute Tools, robustes Ökosystem)	Langsamer (manuelle Speicherverwaltung, weniger Abstraktion)	Langsamer (komplexere Features, manuelle Speicherverwaltung)
Typische Anwendungsfälle in VT	Datenanalyse, KI, Simulation, Automatisierungsskripte, Prototyping	Industrielle Steuerung, IoT, Enterprise-Systeme, große verteilte Anwendungen	Embedded Systems, Betriebssysteme, Treiber, Echtzeit-Steuerung	Echtzeitsysteme, Hochleistungsrechnen, Embedded Systems, Prozessleitsysteme

Python – Berechnung von Mittelwert

```python
# Python-Beispiel: Mittelwertberechnung
import statistics  # Oder import numpy as np für größere Datenmengen

daten = [10.5, 12.0, 11.2, 13.5, 10.8]
mittelwert = statistics.mean(daten)  # Oder np.mean(daten)
# Alternative ohne Import: mittelwert = sum(daten) / len(daten)

print(f"Der Mittelwert in Python ist: {mittelwert}")
```

Java – Berechnung von Mittelwert

```java
// Java-Beispiel: Mittelwertberechnung
public class MittelwertJava {
    public static void main(String[] args) {
        double[] daten = {
            10.5,
            12.0,
            11.2,
            13.5,
            10.8
        };
        double summe = 0;
        for (int i = 0; i < daten.length; i++) {
            summe += daten[i];
        }
        double mittelwert = summe / daten.length;
        System.out.println("Der Mittelwert in Java ist: " + mittelwert);
    }
}
```

C – Berechnung von Mittelwert

```c
// C-Beispiel: Mittelwertberechnung
#include <stdio.h>


int main() {
	float daten[] = {10.5f, 12.0f, 11.2f, 13.5f, 10.8f};
	int anzahl = sizeof(daten) / sizeof(daten[0]); // Anzahl der Elemente berechnen
	float summe = 0.0f;
	for (int i = 0; i < anzahl; i++) {
		summe += daten[i];
	}
	float mittelwert = summe / anzahl; // Achtung: Hier float-Division sicherstellen
	printf("Der Mittelwert in C ist: %f\n", mittelwert);
	return 0;
}
```

Zusammenfassung

- **Programmiersprachen:** Formale Anweisungssätze für Computer, definiert durch Syntax und Semantik.
- **Kompromiss:** Lesbarkeit/Entwicklungsgeschwindigkeit vs. Ausführungsgeschwindigkeit/Hardwarenähe.
- **Sprachen im Überblick:**
  - **Python:** Ideal für schnelle Entwicklung, Datenanalyse, KI, Automatisierung (hohe Lesbarkeit, großes Ökosystem).
  - **Java:** Robust, plattformunabhängig, wichtig für große, verteilte Systeme und industrielle Steuerung (guter Kompromiss aus Performance und Portabilität).
  - **C/C++:** Unverzichtbar für hardwarenahe Programmierung, Echtzeitsysteme, Embedded Systems (maximale Kontrolle und Geschwindigkeit).
- **Kernbotschaften für Verfahrenstechniker:**
  - **Essentielle Kernkompetenz:** Programmierung ist entscheidend in Industrie 4.0.
  - **Keine "beste" Sprache:** Immer die "passendste" für die Aufgabe.
  - **Grundlagen wichtiger als Syntax:** Verständnis von Syntax, Semantik, Abstraktionsebenen, Ausführungsmodellen.
  - **Fördert Problemlösung:** Übertragbare Fähigkeiten wie kritisches Denken und Problemlösung.

- **Programmieren ist eine Denkweise:** Zerlegen komplexer Probleme, präzise Anweisungen formulieren.

Quiz

- **Welche Programmiersprache ist besonders bekannt für ihre Plattformunabhängigkeit ("Write Once, Run Anywhere")?**
  a) C++
  b) Java
  c) Python
  d) Assembler

- **Welches Merkmal ist typisch für eine Low-Level-Sprache im Vergleich zu einer High-Level-Sprache?**
  a) Hohe Lesbarkeit
  b) Automatische Speicherverwaltung
  c) Direkter Hardwarezugriff
  d) Schneller Entwicklungszyklus

5 Minuten

Quiz

5 Minuten

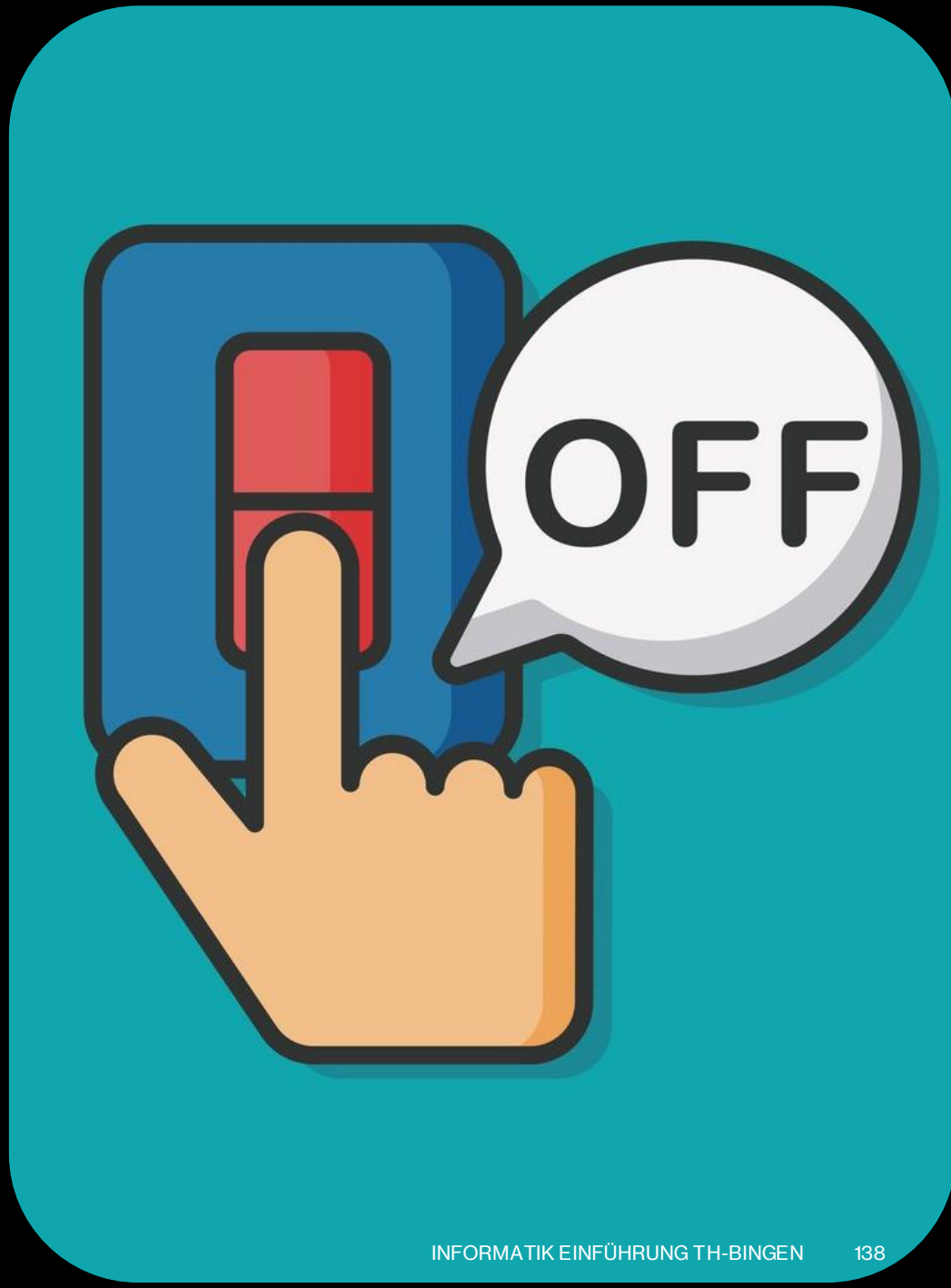
- **Für welche Anwendung in der Verfahrenstechnik wäre Python aufgrund seiner Stärken am besten geeignet?**
  a) Programmierung eines Mikrocontrollers für eine Echtzeitsteuerung
  b) Entwicklung eines Betriebssystems
  c) Analyse großer Sensordatenmengen zur Prozessoptimierung
  d) Entwicklung einer Game Engine

- **4. Welche Sprache wird oft als “Sprache der Systeme” bezeichnet, da sie traditionell für Betriebssysteme und Embedded-Programmierung verwendet wird?**
  a) C
  b) Java
  c) Python
  d) SQL

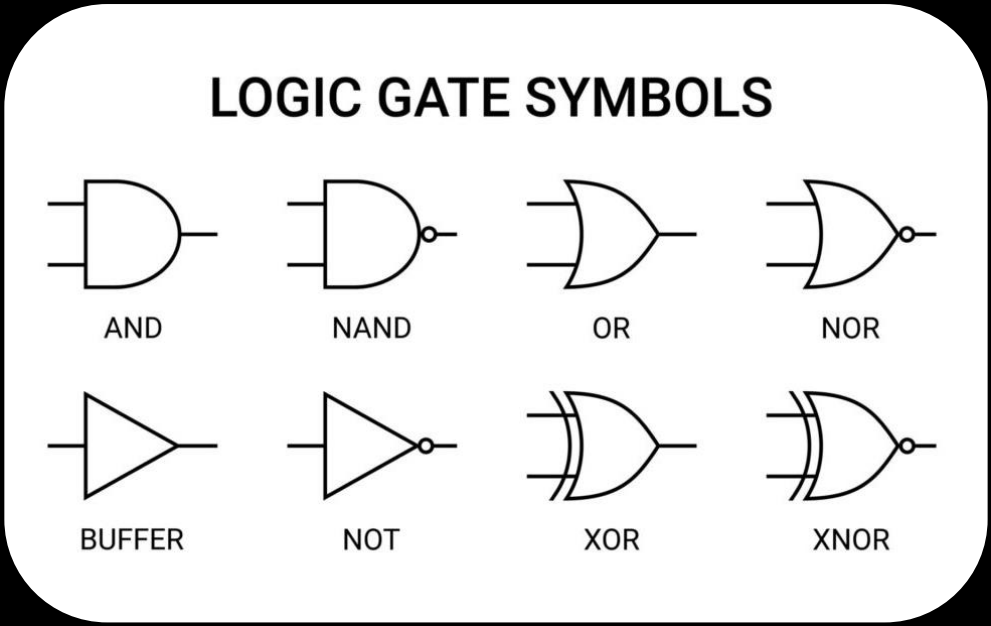
6. Flip-Flops – Wie speichert ein Computer ein Bit?

Logikgrundlagen – Die Sprache der Digitaltechnik

- **Das Binärsystem: Nullen und Einsen**
  - Computer nutzen das Binärsystem (Zweiersystem/Dualsystem) mit nur zwei Ziffern: 0 und 1.
  - 0 = "falsch", "aus", "kein Strom"
  - 1 = "wahr", "an", "Strom fließt"
- **Bit: Kleinste Informationseinheit**
  - Jede 0 oder 1 ist ein "Bit" (Binary Digit).
  - Elementarer Baustein für alle komplexeren Daten (Zahlen, Texte, Bilder).
- **Physikalische Repräsentation:**
  - Elektrische Spannungspegel oder Stromflüsse.
  - Beispiel: +5 Volt = "1", 0 Volt = "0" (TTL-Logik).



Logikgatter: Bausteine der digitalen Welt



- **Was sind Logikgatter?**
  - Fundamentale elektronische Schaltungen.
  - Verarbeiten ein oder mehrere binäre Eingangssignale.
  - Liefern ein einziges binäres Ausgangssignal.
  - Setzen Boolesche Operationen in Hardware um.
- **Wichtige elementare Logikgatter:**
  - **UND-Gatter (AND):** Ausgang 1, wenn *alle* Eingänge 1.
  - **ODER-Gatter (OR):** Ausgang 1, wenn *mindestens einer* der Eingänge 1.
  - **NICHT-Gatter (NOT) / Inverter:** Kehrt das Eingangssignal um (1 wird 0, 0 wird 1).
- **Kombination von Gattern:** Ermöglicht komplexere logische Funktionen.

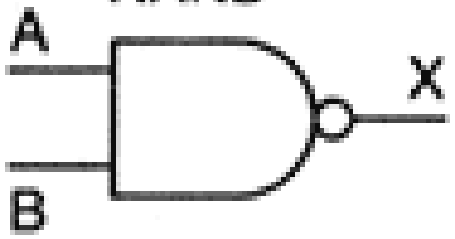
NOT



A	X
0	1
1	0

(a)

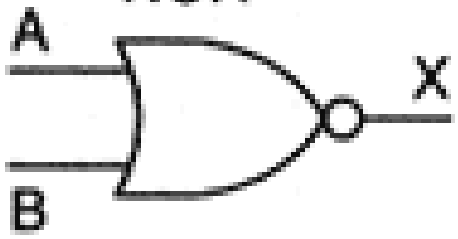
NAND



A	B	X
0	0	1
0	1	1
1	0	1
1	1	0

(b)

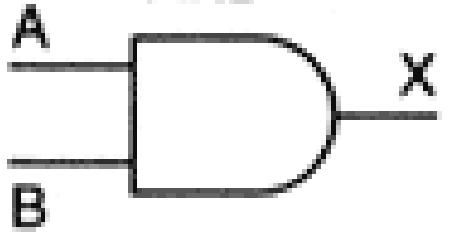
NOR



A	B	X
0	0	1
0	1	0
1	0	0
1	1	0

(c)

AND



A	B	X
0	0	0
0	1	0
1	0	0
1	1	1

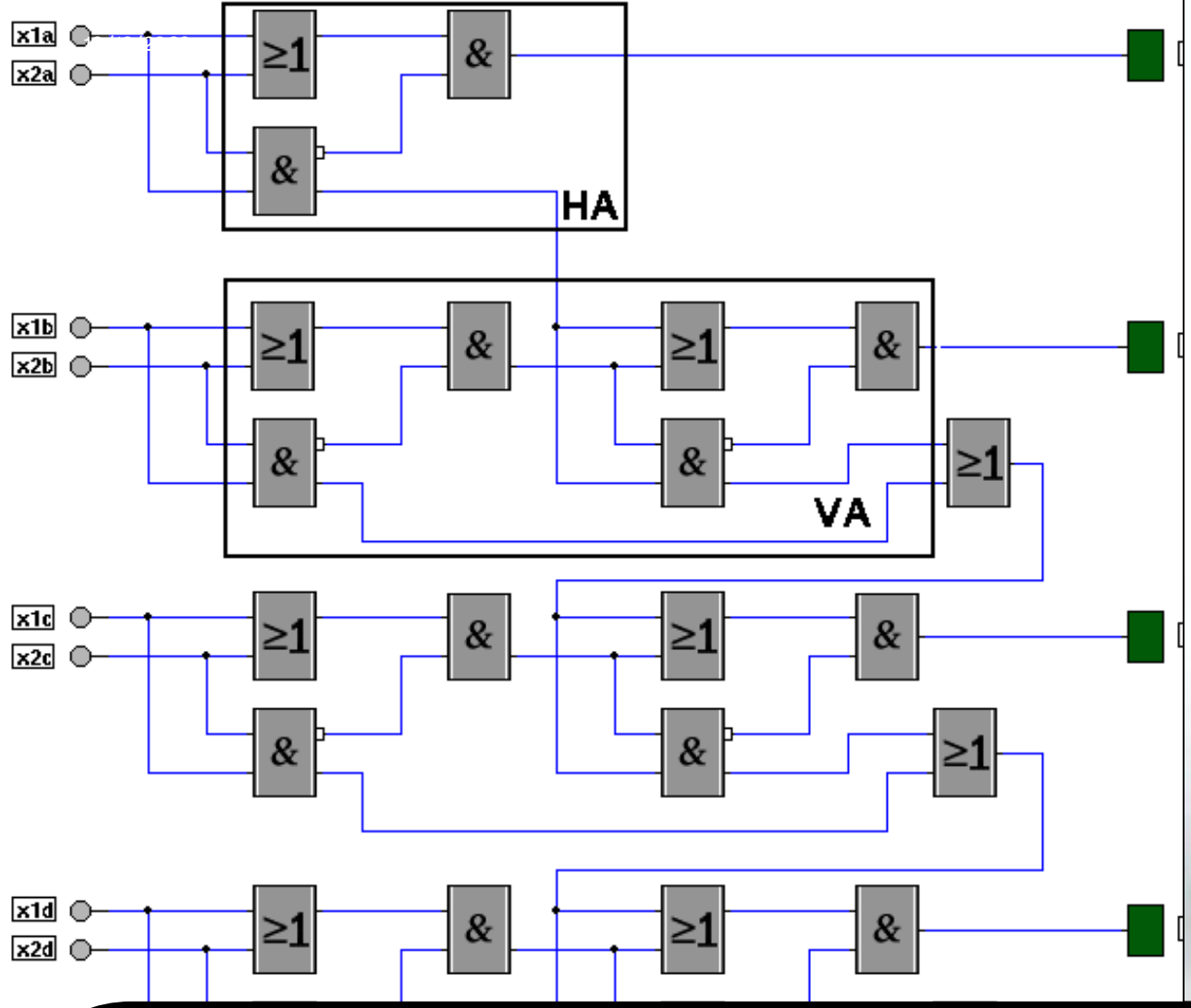
(d)

OR



A	B	X
0	0	0
0	1	1
1	0	1
1	1	1

(e)





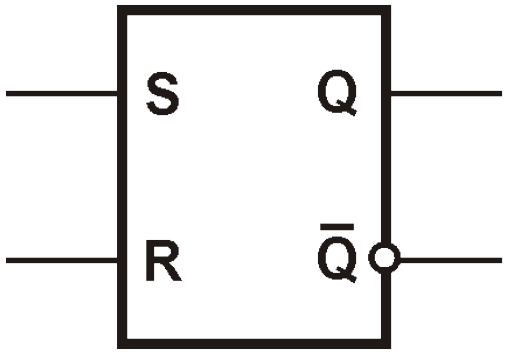
4-Bit Addierer

RS-Flip-Flop: Der einfache Speicherbaustein

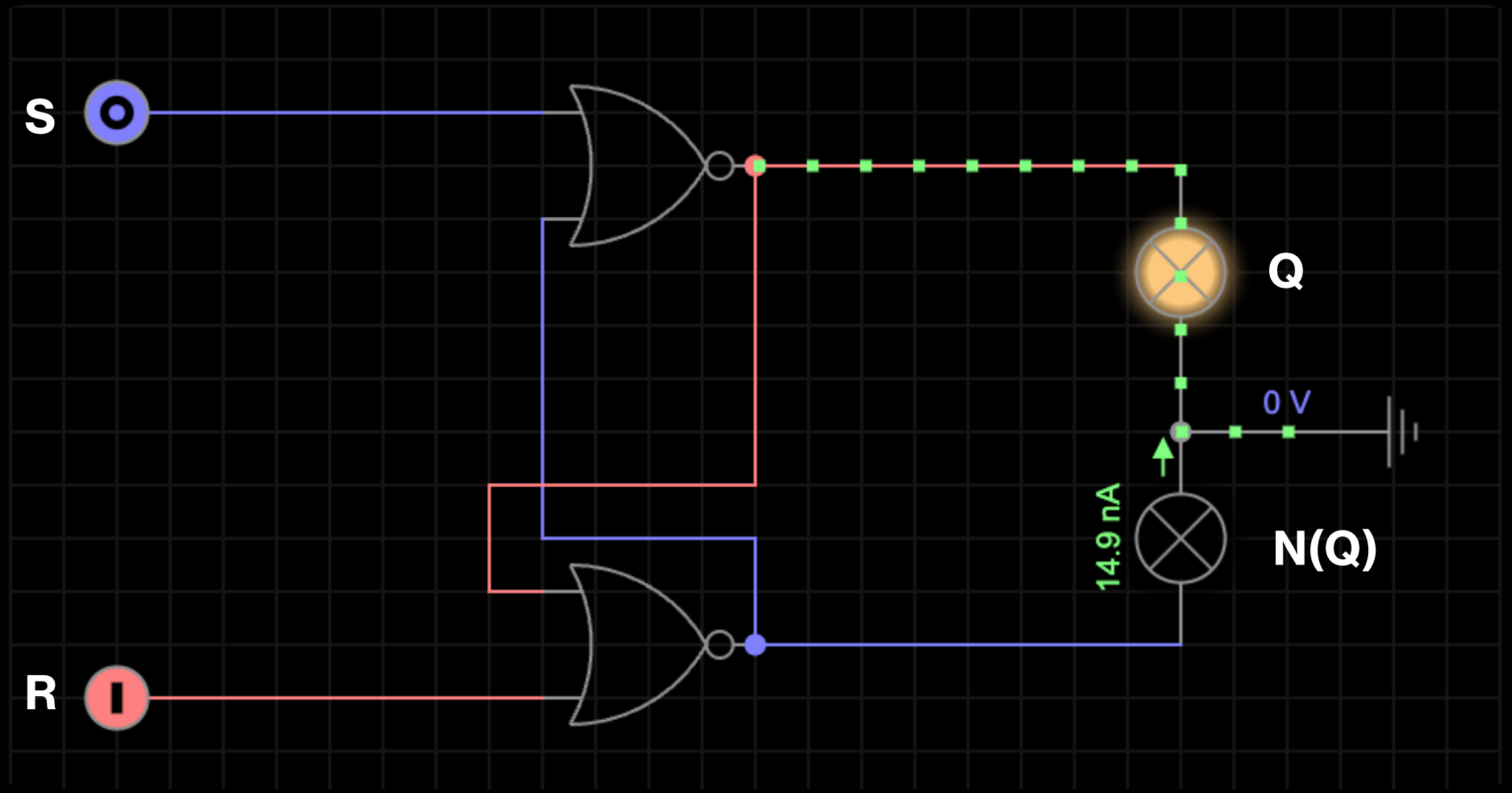
- **Ziel:** Speichern eines Zustands (0 oder 1)
- **Konzept der bistabilen Kippstufe:**
  - Ein Flip-Flop ist ein grundlegendes Speicherelement.
  - "Bistabil": Kann zwei stabile Zustände annehmen (z.B. "0" oder "1") und in diesem verbleiben.
  - Speichert genau ein Bit Information.
- **Aufbau aus NOR- oder NAND-Gattern:**
  - Einfaches RS-Flip-Flop (Set-Reset Flip-Flop) aus zwei rückgekoppelten NOR- oder NAND-Gattern.
  - Rückkopplung ermöglicht Speicherfähigkeit (Ausgang hängt vom vorherigen Zustand ab).

RS-Flip-Flop: Funktionsweise

- **Eingänge:** S (Set) und R (Reset).
- **Ausgänge:** Q und Q' (Q negiert).
- **Funktionsweise (Wahrheitstabelle für NOR-Gatter-Aufbau):**
  - **Setzen (S=1, R=0):** Ausgang Q wird 1.
  - **Rücksetzen (S=0, R=1):** Ausgang Q wird 0.
  - **Speichern (S=0, R=0):** Behält den aktuellen Zustand bei.
- **Das "verbotene" Zustandsproblem:**
  - Tritt auf, wenn S und R gleichzeitig aktiv sind (z.B. S=1, R=1 bei NOR-FF).
  - Ausgänge Q und Q' sind nicht mehr invertiert.
  - Führt zu undefiniertem, unvorhersehbarem Zustand (Metastabilität) bei Rückkehr in den Speicherzustand.



S	R	Q	$\bar{Q}$	
L	L	Q	$\bar{Q}$	Zustand wird gespeichert
H	L	H	L	Ausgang setzen
L	H	L	H	Ausgang rücksetzen
H	H	L	L	nicht speicherbar (unbestimmt)



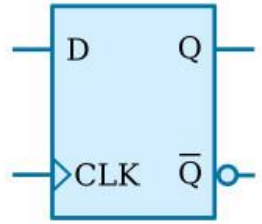
https://everycircuit.com/circuit/6687150077640704/rs-flip-flop

D-Flip-Flop: Der zuverlässige Datenspeicher

- **Motivation:**
  - Löst das Problem des "verbotenen Zustands" des RS-Flip-Flops.
  - Sorgt für robustere und vorhersehbarere Speicherung.
- **Aufbau:**
  - Ableitung aus dem RS-Flip-Flop.
  - R-Eingang über ein NICHT-Gatter (Inverter) mit dem S-Eingang verbunden.
  - Nur noch ein Dateneingang: D.
  - Verhindert, dass interne Set- und Reset-Eingänge gleichzeitig aktiv sind.

D-Flip-Flop: Funktionsweise

- **Eingänge:** Daten-Eingang (D) und Takteingang (C oder T für Clock/Takt).
- **Varianten der Datenübernahme:**
  - **Taktzustandsgesteuertes D-Latch:**
    - Ausgang Q folgt D, solange Takt aktiv (z.B. C=1). ("transparente Phase")
    - Letzter D-Wert wird gespeichert, wenn Takt inaktiv (z.B. C=0). ("Haltephase")
  - **Taktflankengesteuertes D-Flip-Flop (häufiger):**
    - Wert am D-Eingang wird nur bei einer *Taktflanke* (z.B. steigende Flanke von 0 auf 1) an Q übernommen.
    - Zwischen den Flanken bleibt der Zustand unverändert.
    - Symbolisiert durch kleines Dreieck am Takteingang.
    - Taktsignal als "Dirigent" für synchrone Aktualisierung in komplexen Systemen.

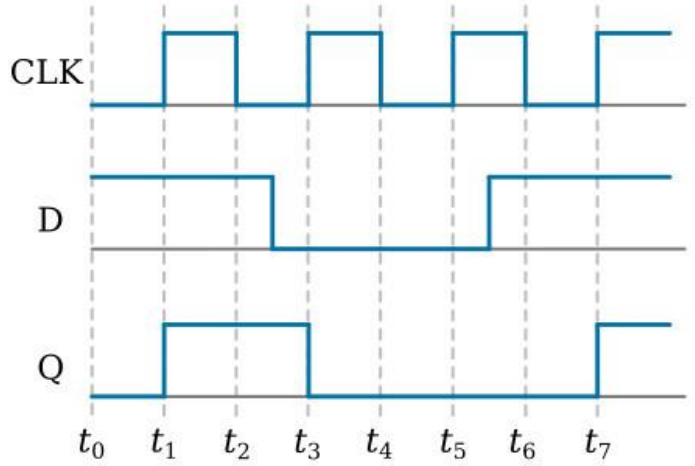


(A)

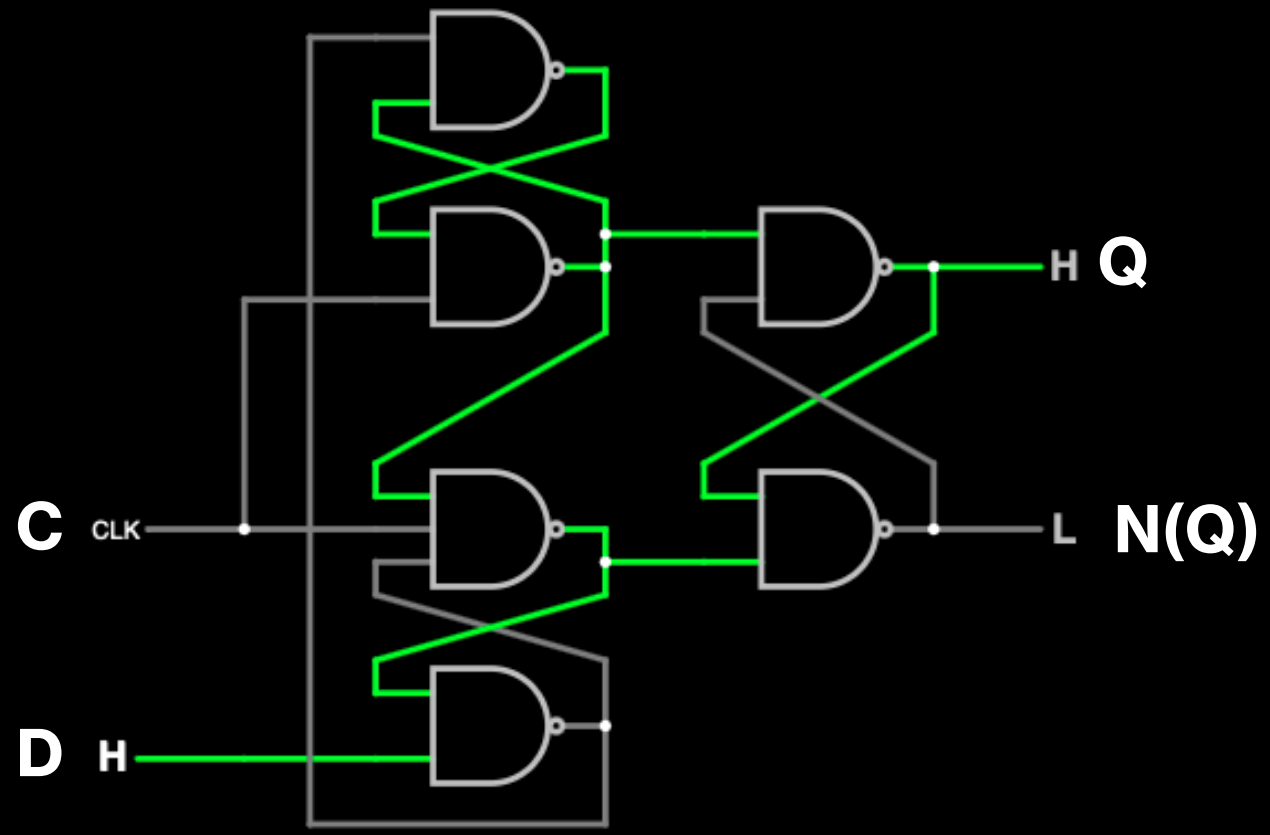
CLK	D	Q_{next}	Comment
Rising edge	0	0	Store 0
Rising edge	1	1	Store 1
Non-rising	X	Q	No change

Q_{next} - "after the clock transition" output
Q - the current output
X - the signal is irrelevant

(B)



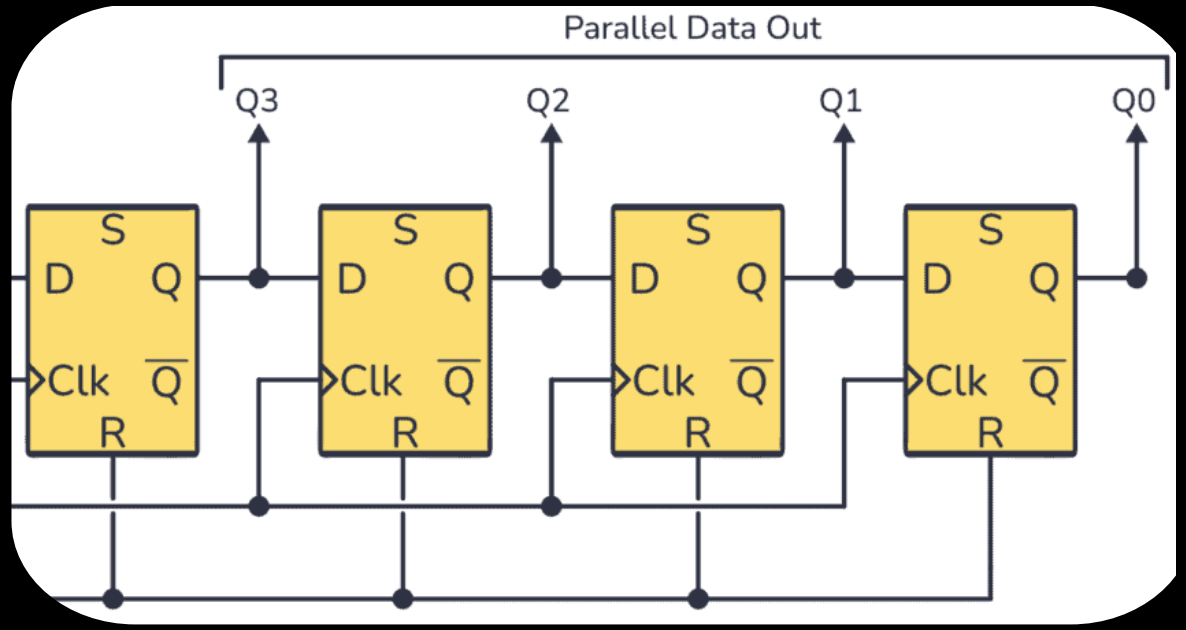
(C)



https://www.falstad.com/circuit/e-edgedff.html



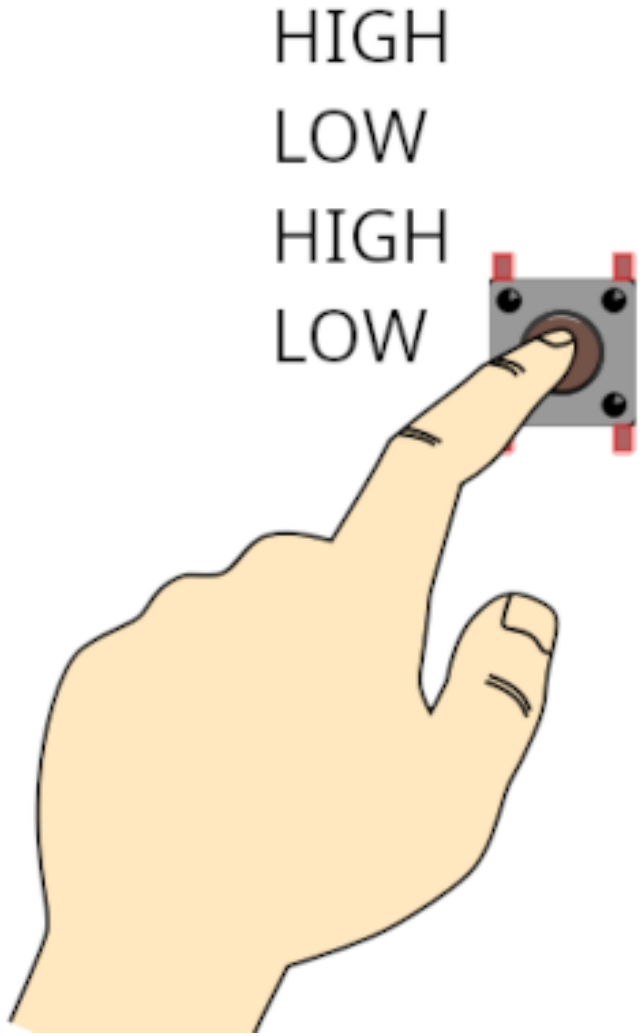
Flip-Flops als Speicherzelle: Vom Bit zum Register



- **Ein Flip-Flop als 1-Bit-Speicher:**
  - Kleinste Einheit, die ein Bit (0 oder 1) speichern kann.
  - Elementarer "Gedächtnisbaustein" in digitalen Schaltungen.
- **Register: Speicherung von Datenworten:**
  - Mehrere Flip-Flops werden zu einem "Register" aneinandergereiht.
  - Beispiel: 4-Bit-Register speichert ein 4-Bit-Datenwort.
  - Schieberegister: Bits werden bei jedem Takt verschoben (serielle Datenübertragung, Verzögerung).
- **Anwendungen in Computern:**
  - Grundlage für statischen RAM (SRAM) in Cache-Speichern und Prozessorregistern.
  - Hohe Geschwindigkeit für schnelle Zugriffe.

Praktische Anwendungen in der Verfahrenstechnik

- **Taster-Entprellung:**
  - Mechanische Taster "prellen" (erzeugen mehrere Signale).
  - Flip-Flop speichert nur den ersten stabilen Zustand, ignoriert Prellimpulse.
  - Wandelt unreine Signale in saubere digitale Zustände um.
- **Zustandsregister in SPS-Programmen:**
  - Flip-Flops oder logische Äquivalente speichern Anlagenzustände (z.B. "Produktion läuft", "Reinigung aktiv").
  - Realisiert Sicherheitsverriegelungen (z.B. Pumpe läuft nicht, wenn Ventil geschlossen).
  - Fundamental für sequentielle Prozessabläufe und Sicherheitsfunktionen.



Quiz

Multiple Choice

1 Minute

Welche Aussagen über ein RS-Flip-Flop sind korrekt?

a) Es speichert ein Bit Information.

b) Es besitzt die Eingänge S (Set) und R (Reset).

c) Der Zustand hängt nur von den aktuellen Eingängen ab.

d) Bei S=1 und R=1 tritt ein undefinierter Zustand auf.

Welche Eigenschaften treffen auf ein D-Flip-Flop zu?

a) Es hat nur einen Dateneingang (D).

b) Es kann nicht durch ein Taktsignal gesteuert werden.

c) Es übernimmt den Wert am Eingang D bei einer Taktflanke.

d) Es ist schneller, wenn S und R gleichzeitig aktiv sind.

Quiz

Wahr/Falsch

1 Minute

- Flip-Flops werden in der Praxis nur in Computern eingesetzt, nicht in industriellen Anlagen. ✗
- Ein Flip-Flop kann genau ein Bit Information speichern. ✓
- Das gleichzeitige Aktivieren von S=1 und R=1 beim RS-Flip-Flop führt zu einem stabilen, definierten Zustand. ✗
- Ein D-Flip-Flop löst das Problem des “verbotenen Zustands” beim RS-Flip-Flop. ✓

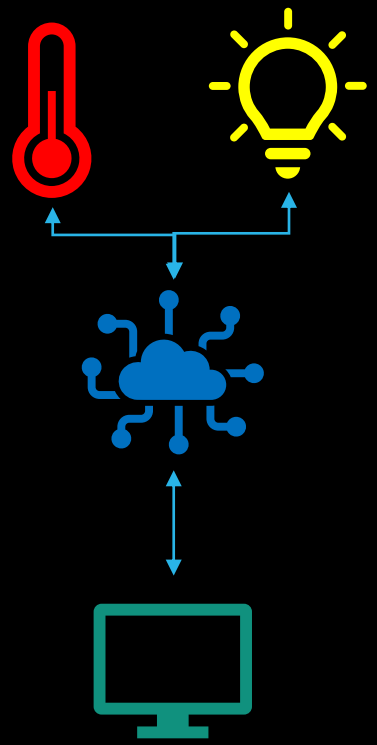
Diskussion

- Warum reicht in der Verfahrenstechnik nicht immer ein einfaches Signal “an/aus”? Wieso muss ein System Zustände speichern können?
- Welche Probleme könnten in einer industriellen Anlage entstehen, wenn ein Flip-Flop in einem undefinierten Zustand (z. B. RS mit S=1 und R=1) bleibt?
- Diskutieren Sie: Warum ist die Taktsteuerung bei D-Flip-Flops für moderne digitale Systeme unverzichtbar?
- Flip-Flops sind die Basis für Register und Speicher. Welche Auswirkungen hätte es auf Prozessleitsysteme, wenn Flip-Flops nicht zuverlässig arbeiten würden?

5 Minuten

7.1 Kommunikation im System – Bus, Ring, Netzwerk

Einführung: Warum Kommunikation wichtig ist



- Kommunikation ist das Fundament komplexer Systeme: Computer, Netzwerke, Industrieanlagen.
- In der Verfahrenstechnik: Reibungslose Datenübertragung zwischen Komponenten ist entscheidend für Betrieb und Sicherheit.
- **Beispiele:**
  - Sensoren erfassen Daten (Temperatur, Druck, Füllstände).
  - Daten gehen an Steuerungen (SPS) zur Verarbeitung.
  - Steuerungen senden Befehle an Aktoren (Ventile, Pumpen).
- **Relevanz für Verfahrenstechniker:** Verständnis der Datenströme ist grundlegend für Planung, Betrieb und Wartung moderner Anlagen.
- **Industrie 4.0:** Zunehmende Digitalisierung und Automatisierung erfordern robuste und effiziente Kommunikationsnetzwerke.
- Verständnis von Topologien und Bussystemen ist praktische Notwendigkeit.

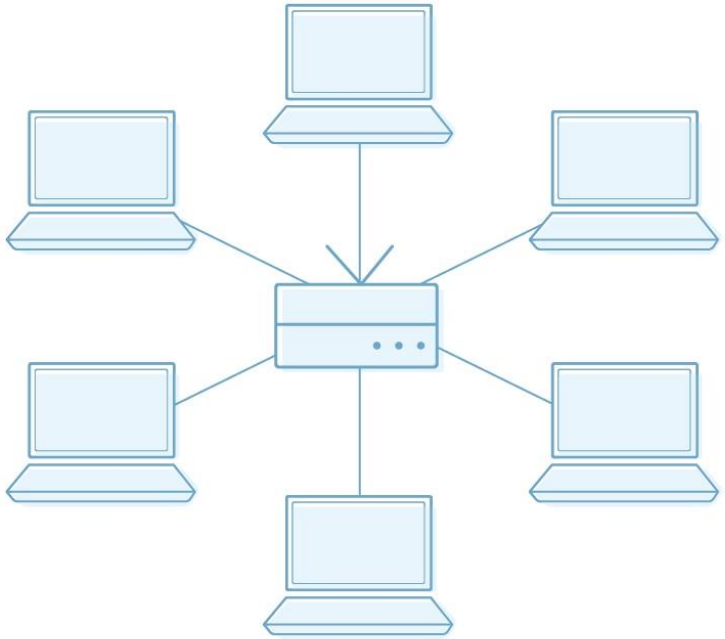
Netzwerktopologien: Wie Geräte miteinander verbunden sind

- **Definition:** Eine Netzwerktopologie beschreibt die Art und Weise, wie Geräte in einem Kommunikationsnetzwerk miteinander verbunden sind.
- **Physikalische Topologie:** Tatsächliche Verkabelung und Anordnung der Geräte.
- **Logische Topologie:** Wie Daten durch das Netzwerk fließen, unabhängig von der physikalischen Verbindung.
- **Wahl der Topologie:** Entscheidend für Leistung, Zuverlässigkeit und Skalierbarkeit.

Logical vs Physical Network Diagrams

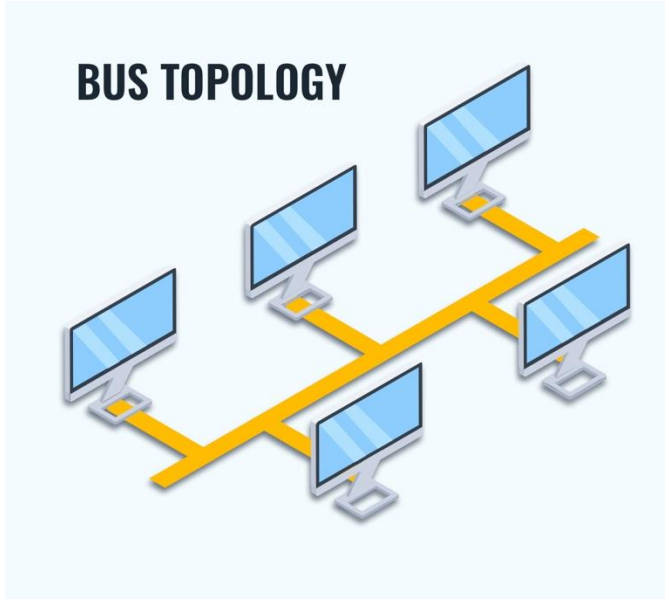
Logical Network Diagram

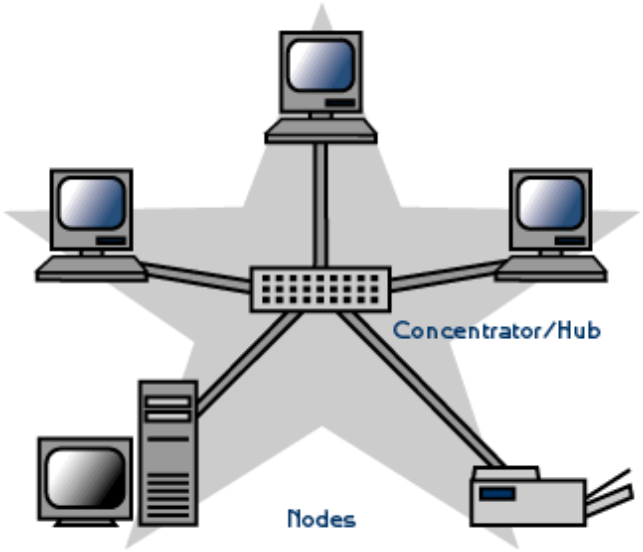
- Shows how data behaves between network devices
- Lines represent data flow, not physical cables

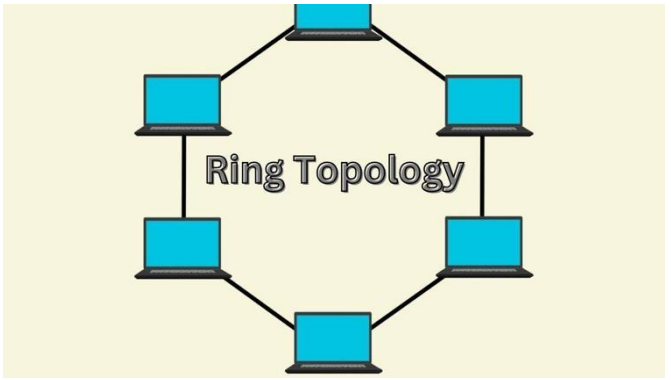


Physical Network Diagram

- Represents your network's physical devices and cables
- Lines represent cable connections rather than data flow



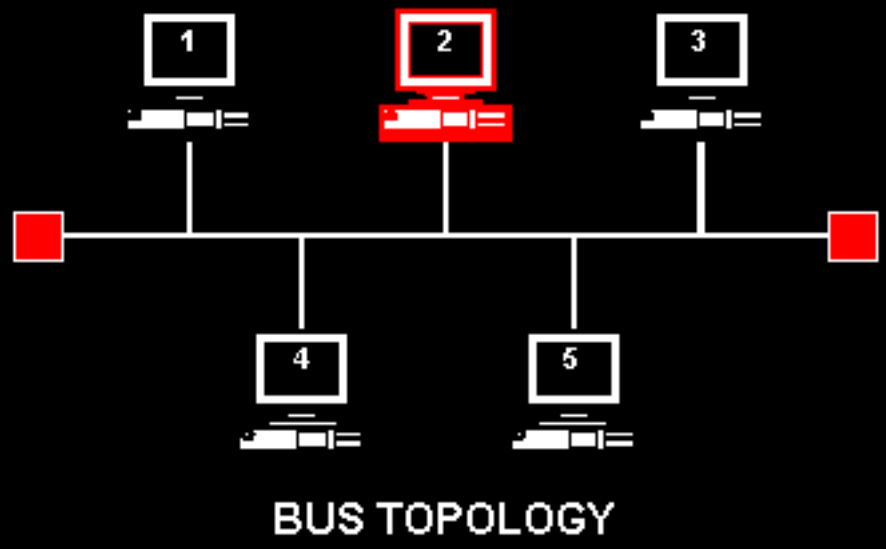




3 verschiedene Topologien

Bus-Topologie: Die gemeinsame Straße

- **Funktionsweise:** Alle Geräte sind über ein einziges, gemeinsames Hauptkabel (Bus) verbunden.
- Daten werden seriell gesendet und von allen Geräten empfangen; nur das adressierte Gerät verarbeitet sie.
- Abschlusswiderstände (Terminatoren) an den Enden verhindern Signalreflexionen.
- **Analogie:** Eine Einbahnstraße oder eine einzige Telefonleitung, die sich alle teilen.

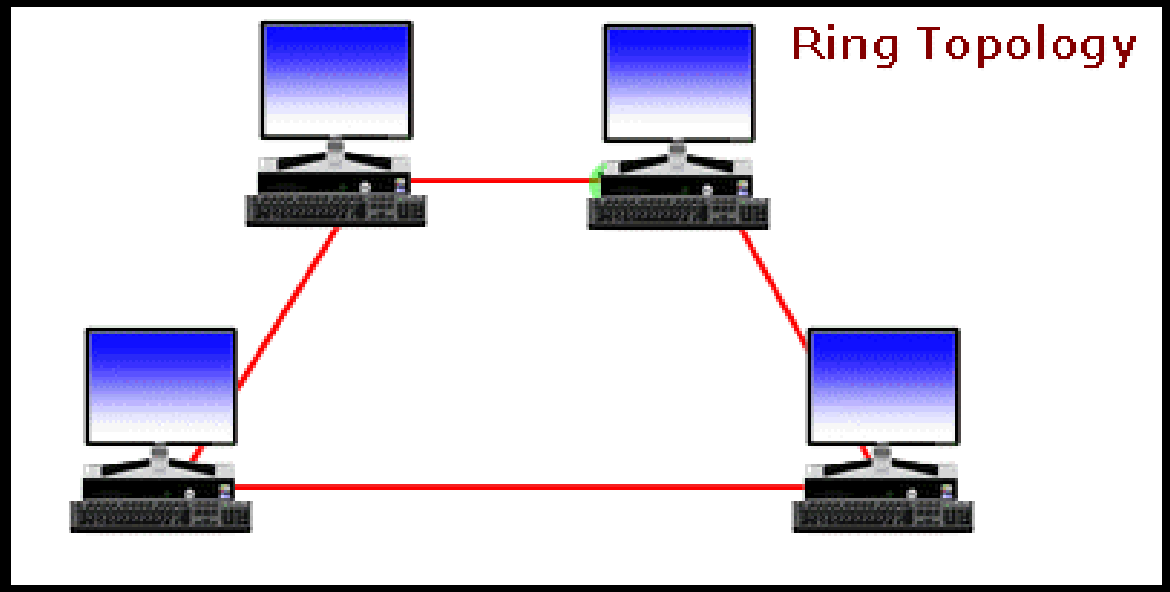


Bus-Topologie: Die gemeinsame Straße

Vorteile:	Einfachheit und Kosteneffizienz (weniger Verkabelung).
	Einfache Installation und Hinzufügen neuer Geräte.
	Effiziente Nutzung der Bandbreite für kleine bis mittelgroße Netzwerke.
Nachteile:	**Single Point of Failure (SPOF):** Ausfall des Hauptkabels legt gesamtes Netzwerk lahm.
	Begrenzte Kabellänge und Geräteanzahl (Signalqualität).
	Datenkollisionen erfordern spezielle Techniken (z.B. CSMA/CD).
	Sicherheitsprobleme (Daten können abgefangen werden).
	Wartung/Erweiterung oft mit Netzwerkausfall verbunden.

Ring-Topologie: Der Datenkreislauf

- **Funktionsweise:** Jedes Gerät ist mit genau zwei anderen Geräten verbunden, bildet einen geschlossenen Kreis.
- Daten wandern sequenziell von Knoten zu Knoten, typischerweise in einer Richtung (unidirektional).
- Oft mit "Token"-Verfahren: Nur Gerät mit Token darf senden, minimiert Kollisionen.
- **Analogie:** Staffellauf (Stab = Token) oder Rundbrief.

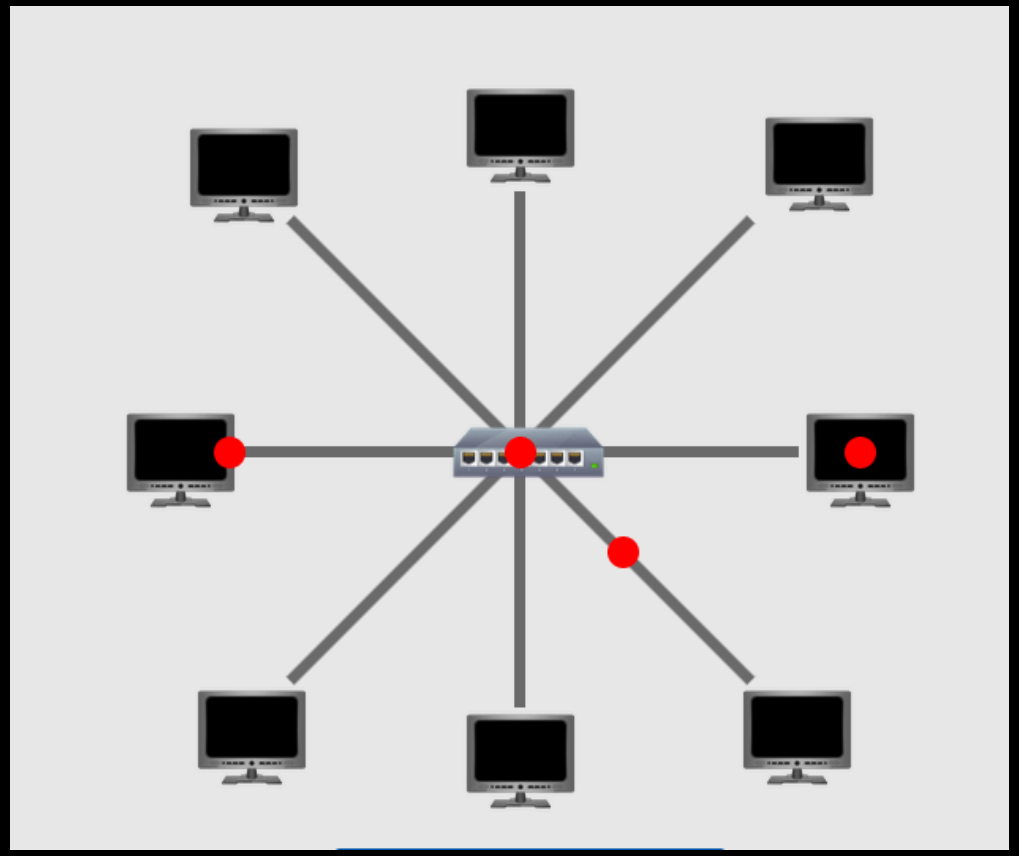


Ring-Topologie: Der Datenkreislauf

Vorteile:	Relative einfache Implementierung/Erweiterung (nur benachbarte Geräte).
	Gute Leistung unter hoher Last (weniger Kollisionen durch Token).
	Kein zentraler Server/Switch zwingend erforderlich (verteilte Steuerung).
	Hohe Zuverlässigkeit bei bidirektionalen Ringen (Backup-Routen).
Nachteile:	Komplexe Fehlersuche (Unterbrechung an einer Stelle kann gesamten Ring beeinträchtigen).
	Geschwindigkeit oft langsamer als Stern (Daten durchlaufen jeden Knoten).
	Sicherheitsprobleme (Datenpaket durchläuft jede Arbeitsstation).
	Hinzufügen/Entfernen von Knoten stört Netzwerkaktivität.
	Bandbreite bei vielen Geräten eingeschränkt, Hardware kann kostspielig sein.

Stern-Topologie: Der zentrale Knotenpunkt

- **Funktionsweise:** Jedes Gerät ist direkt mit einem zentralen Hub verbunden.
- Der zentrale Knotenpunkt leitet Daten vom Sender an den Empfänger weiter.
- **Analogie:** Telefonzentrale oder zentraler Verteilerkasten.



Stern-Topologie: Der zentrale Knotenpunkt

Vorteile:	Hohe Ausfallsicherheit (Ausfall eines Geräts/Verbindung beeinträchtigt Rest des Netzwerks nicht).
	Einfache Fehlersuche (Probleme schnell lokalisierbar).
	Unkomplizierte Erweiterung (neue Geräte leicht hinzufügbar/entfernbar).
	Hohe Datenübertragungsraten (insbesondere mit Switches).
	Flexibilität (unterschiedliche Netzwerktechnologien möglich).
Nachteile:	**Single Point of Failure (SPOF):** Ausfall des zentralen Hubs/Switches legt gesamtes Netzwerk lahm.
	Hoher Verkabelungsaufwand (jede Leitung separat zum Zentrum).
	Höhere Kosten (Verkabelung und zentrale Hardware).
	Hohe Datenbelastung im Zentrum.

Diskussion

- Welche Topologie haben Sie in ihrem Netzwerk zuhause?

- Welche Topologie ist die beste?

5 Minuten

Vergleich der Topologien: Wann welche Struktur?

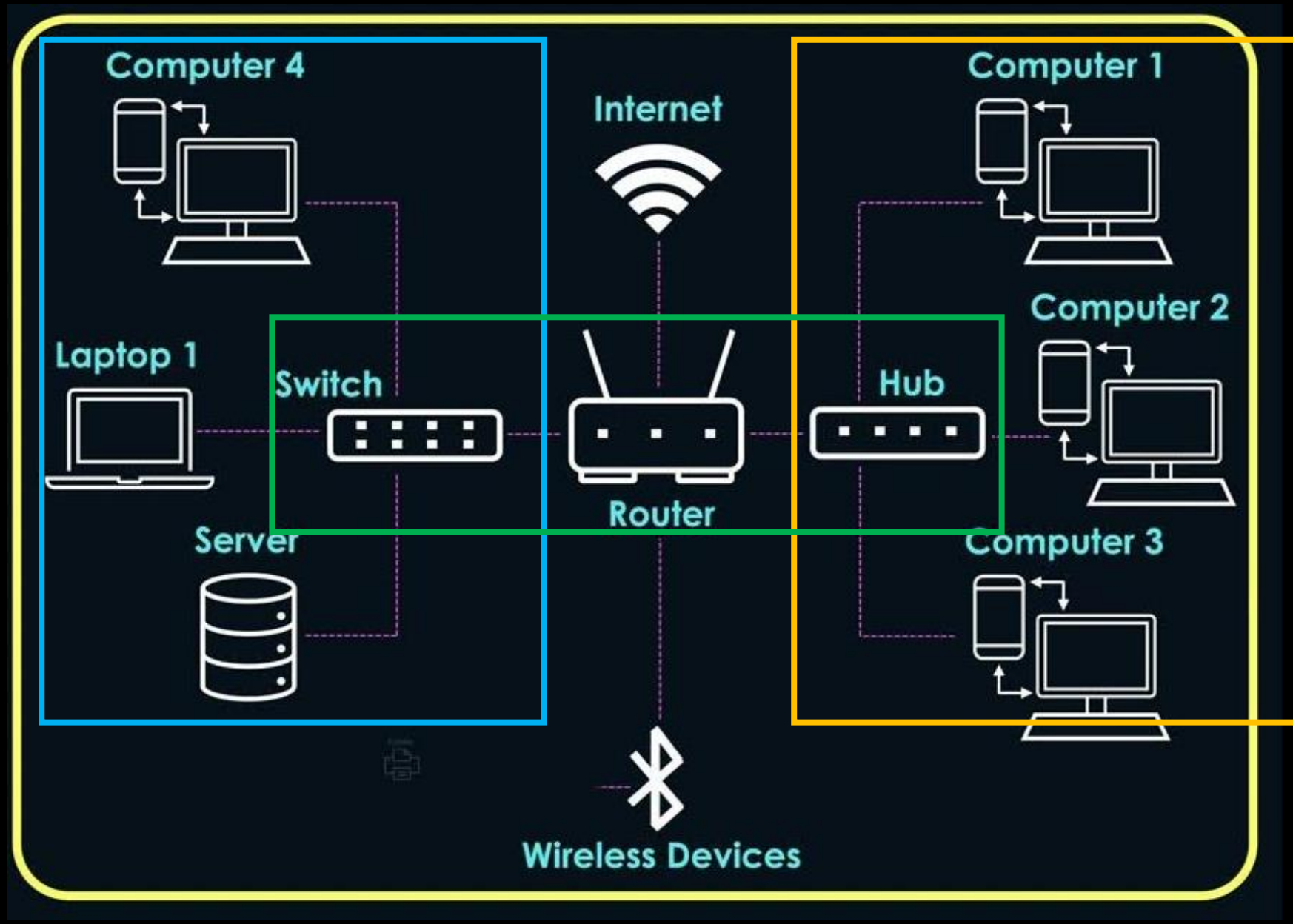
- Die Wahl der Topologie hängt ab von: Reichweite, Geräteanzahl, Sicherheit, Kosten, Zuverlässigkeit.
- **Hybridtopologien:** Kombinieren Elemente verschiedener Topologien (z.B. Stern-Bus).
- **Entwicklungstrend:** Moderne Netzwerke tendieren zu Stern- und Hybridtopologien.

- **Single Point of Failure (SPOF):**
  - Bus: Hauptkabel ist SPOF.
  - Ring: Ausfall eines Knotens/Verbindung kann gesamten Ring beeinträchtigen (außer bei Redundanz).
  - Stern: Zentraler Hub/Switch ist SPOF.

Merkmal	Bus-Topologie	Ring-Topologie	Stern-Topologie
Funktionsweise	Alle Geräte an einem gemeinsamen Kabel; Daten seriell gesendet, von Adressiertem verarbeitet.	Geräte in geschlossenem Kreis verbunden; Daten wandern sequenziell von Knoten zu Knoten (oft mit Token).	Alle Geräte direkt mit zentralem Hub/Switch verbunden; Hub/Switch leitet Daten weiter.
Vorteile	Einfach, kostengünstig, wenig Verkabelung.	Weniger Kollisionen, verteilte Steuerung, hohe Leistung unter Last. Redundanz bei Dual-Ring.	Hohe Ausfallsicherheit (einzelnes Gerät/Kabel), einfache Fehlersuche, leichte Erweiterung, hohe Übertragungsrate.
Nachteile	Single Point of Failure (Kabelbruch), begrenzte Länge/Geräte, Kollisionen, Sicherheitsprobleme.	Schwierige Fehlersuche, Ausfall eines Knotens kann ganzen Ring stören, langsame Geschwindigkeit, Sicherheitsprobleme.	Single Point of Failure (zentraler Hub/Switch), hoher Verkabelungsaufwand, hohe Kosten.
Typische Anwendung	Kleine Netzwerke, ältere industrielle Anwendungen (z.B. CAN-Bus).	Industrielle Automatisierung (wenn Redundanz kritisch ist, z.B. EtherCAT-Varianten).	Moderne LANs, Büros, Unternehmensnetzwerke.

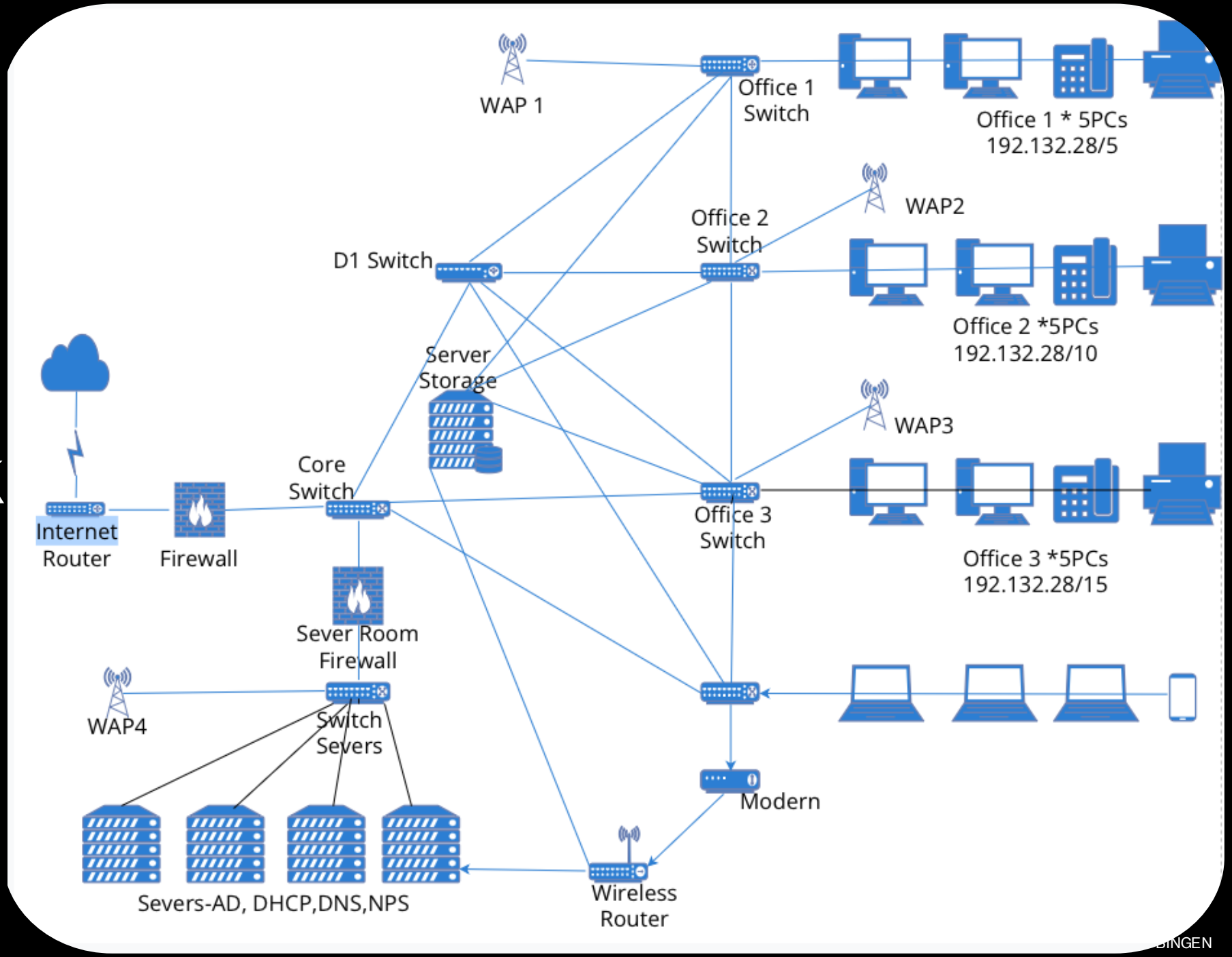
Beispiel

Firmen-Netzwerk



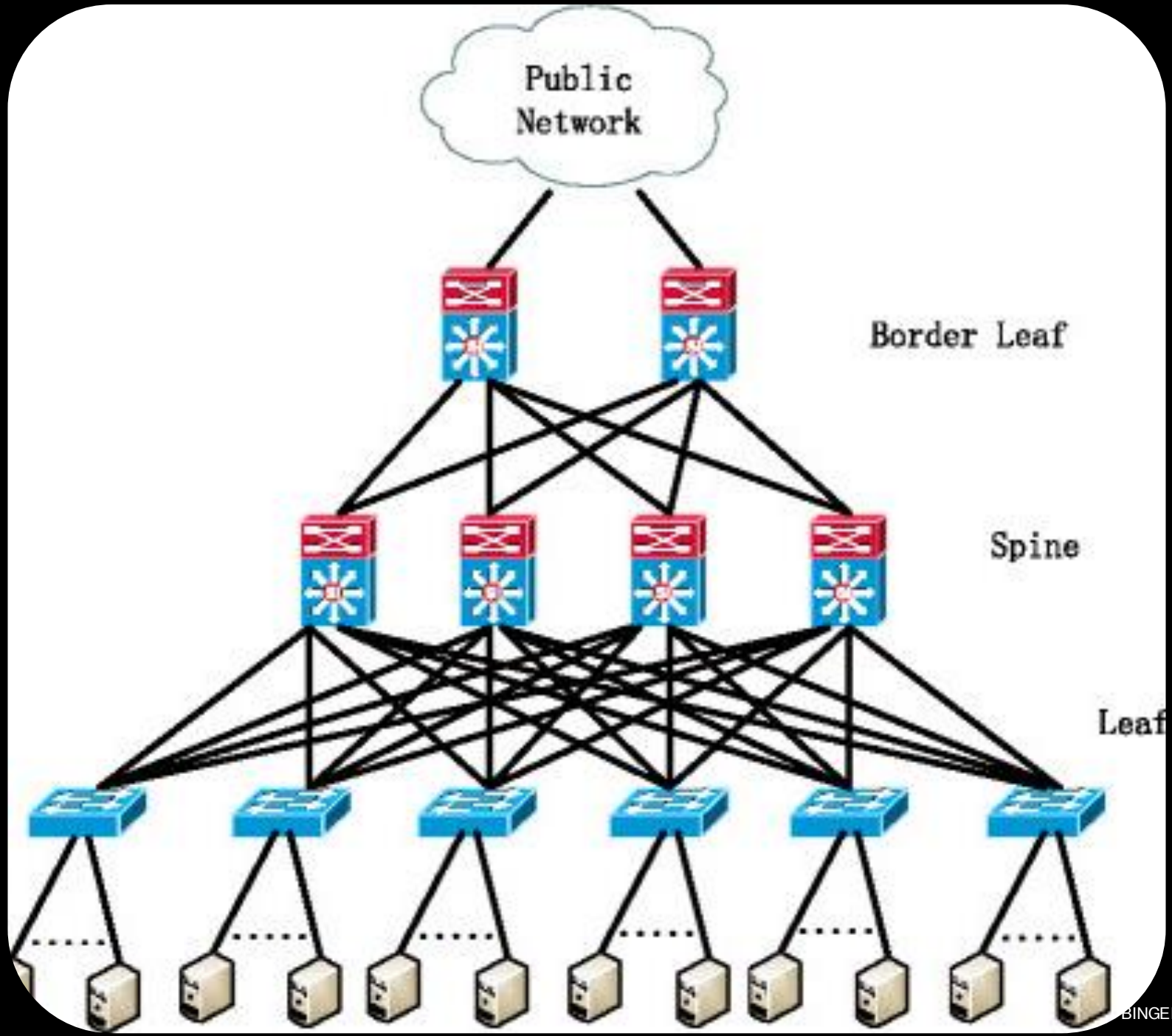
Beispiel

Größeres Firmen-Netzwerk



Beispiel

Leaf Spine Netzwerk



Beispiel

Mother board (Computer)

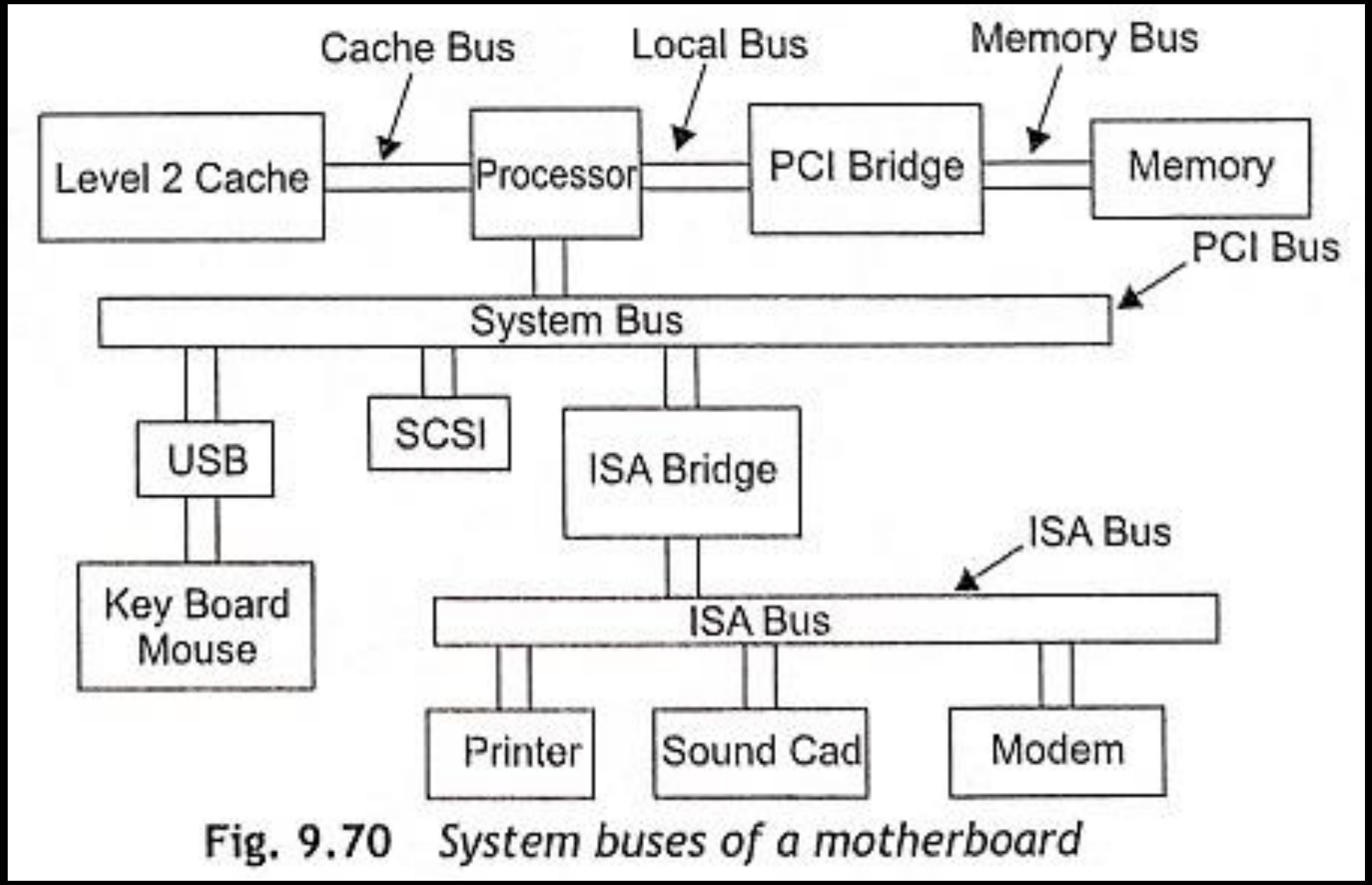
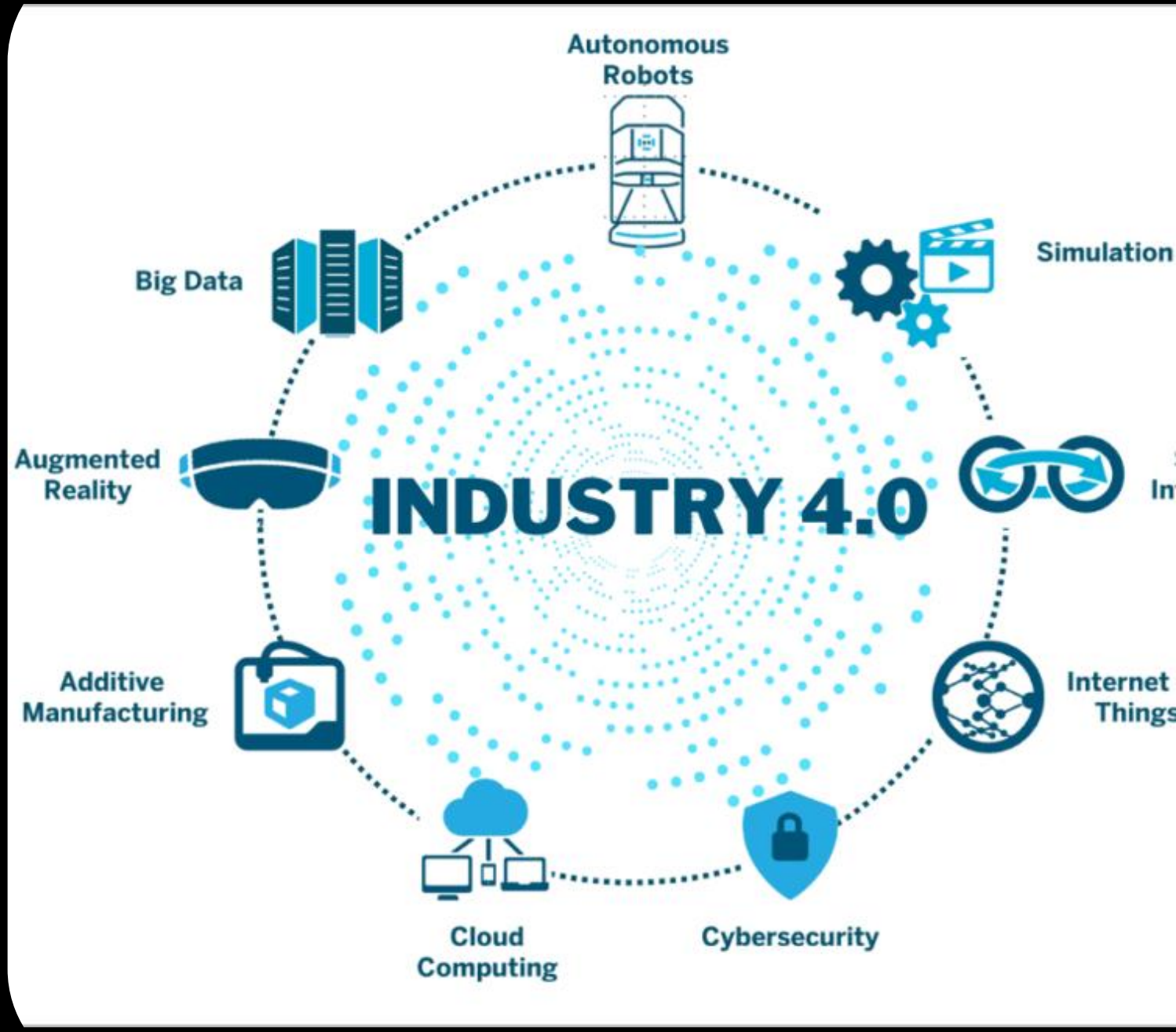


Fig. 9.70 *System buses of a motherboard*

Industrie 4.0 und Vernetzung: Die Zukunft der Kommunikation

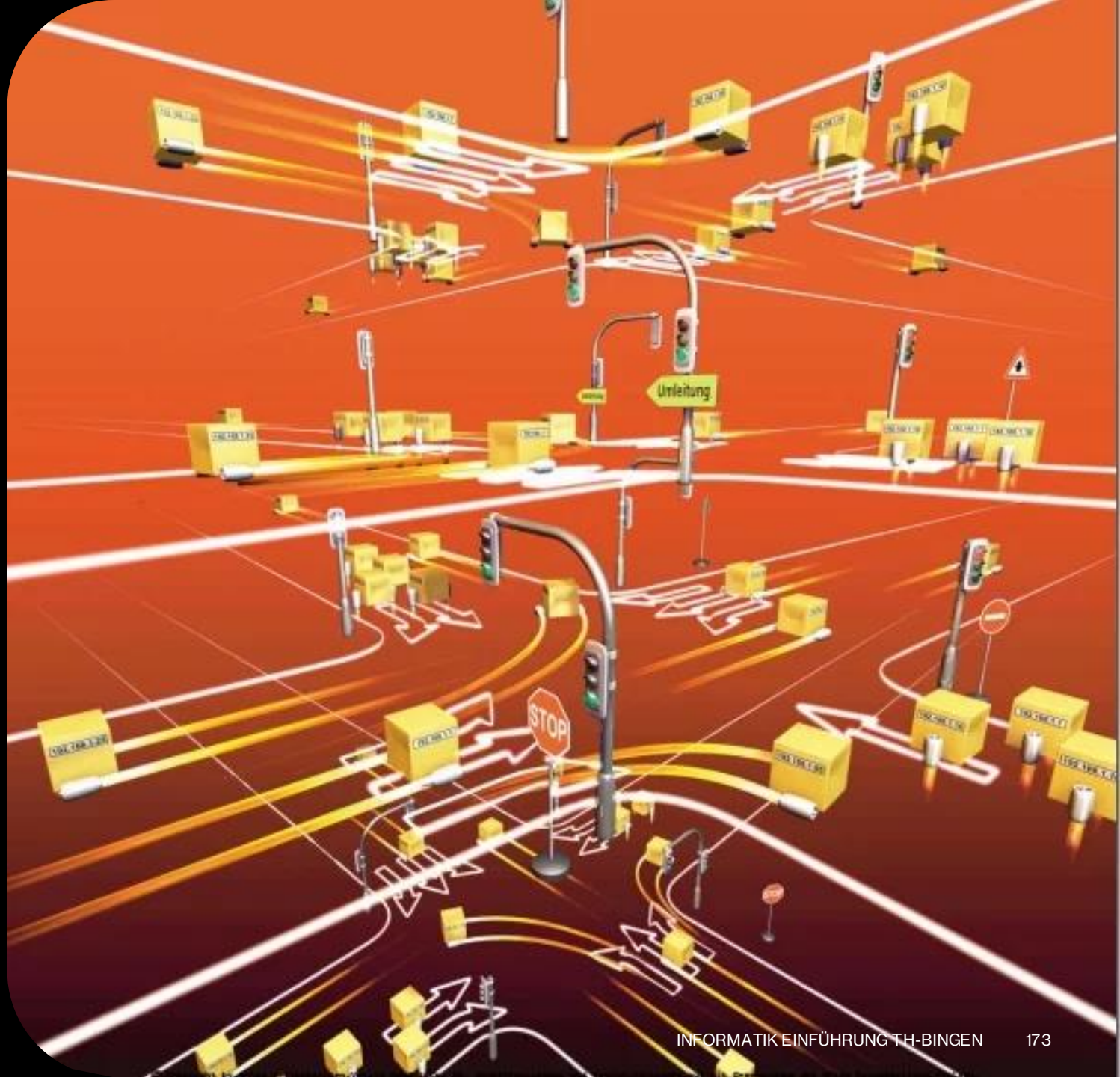
- Digitalisierung und Industrie 4.0 verändern Prozessleittechnik grundlegend.
- Kommunikationssysteme bilden Basis für "Smart Factories" (Maschinen agieren intelligent).
- **Herausforderungen:** Exponentielles Datenwachstum, Echtzeitkommunikation, flexible Produktion.
- **Redundanz:** Kritische System erfordern hohe Ausfallsicherheit.
- **M2M-Kommunikation:** Oftmals reden nur Maschinen zu Maschinen (M2M) → wird nur noch mehr mit dem Ausbau von KI



7.2 Das IP-Netzwerk und OSI-Modell

Einführung: Warum Protokolle & Standards?

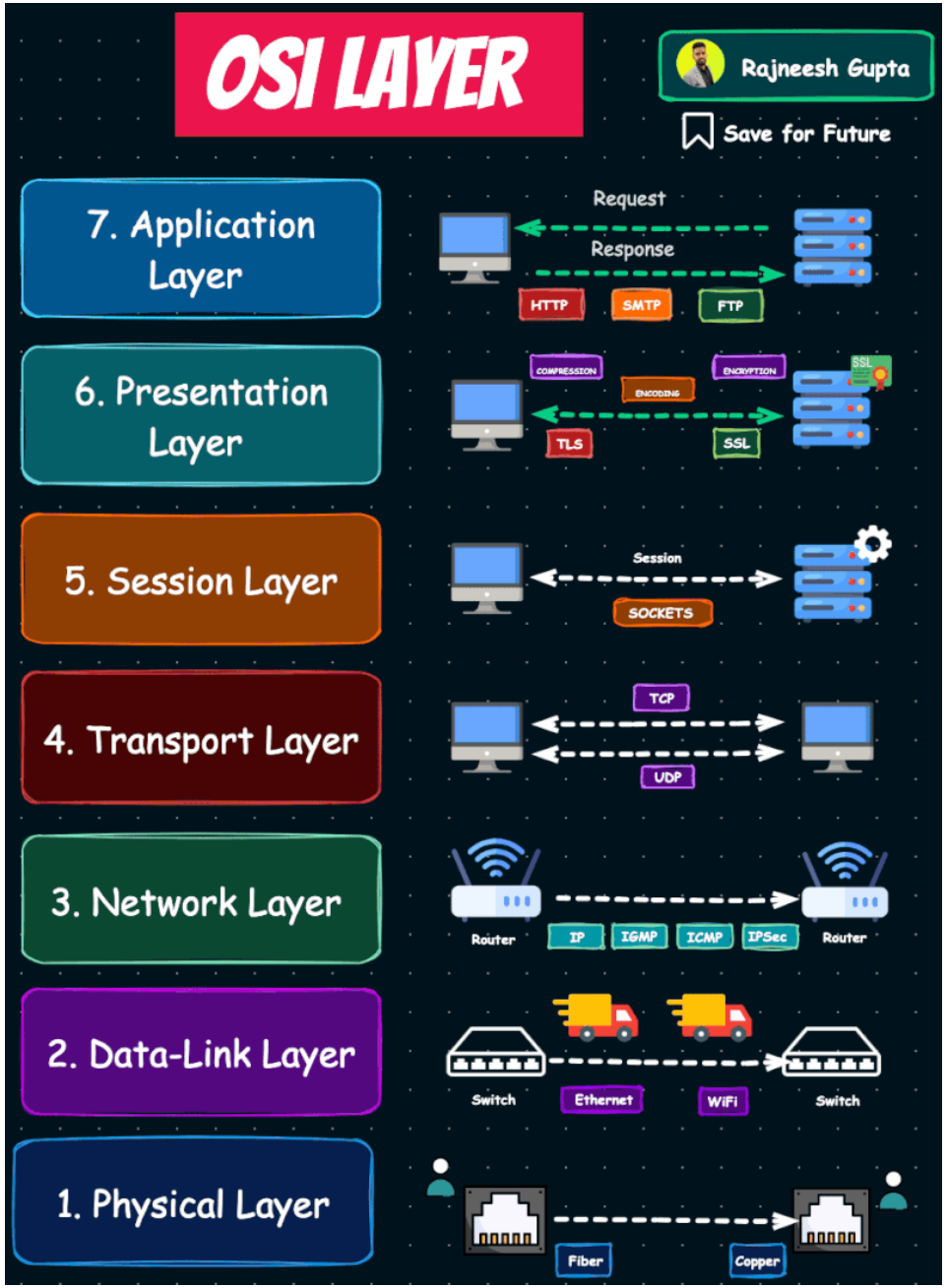
- Das Internet ist **riesig**: Milliarden Geräte weltweit
- Unterschiedliche **Hersteller, Betriebssysteme, Netzwerke**
- Ohne Regeln → **Chaos**, Geräte könnten nicht miteinander reden
- Lösung: **Protokolle & Standards**
- Einheitliche Sprache (wie Englisch für Menschen)
- Sicherstellen, dass Daten **korrekt** und **verständlich** ankommen
- Diese Regeln sind in **Schichten (Layern)** organisiert



Das OSI-Model

- OSI (Open Systems Interconnection)
- Entstand in den 1980ern als **theoretisches Referenzmodell**

- **Aufgeteilt in 7 Schichten:**
  - **Application (Anwendung)** → Programme wie Browser, E-Mail
  - **Presentation (Darstellung)** → Datenformate (z. B. Verschlüsselung, ASCII)
  - **Session (Sitzung)** → Aufrechterhalten der Verbindung
  - **Transport** → Zuverlässigkeit (TCP), Geschwindigkeit (UDP)
  - **Network (Vermittlung)** → IP-Adressen, Routing
  - **Data Link (Sicherung)** → MAC-Adresse, Fehlererkennung
  - **Physical (Physik)** → Kabel, Funk, elektrische Signale





Application Layer	Request / Response	HTTP, FTP, SMTP
Presentation Layer	compression, encoding, encryption	TLS, SSL
Session Layer	Session	Sockets
Transport Layer	Segmentation	TCP, UDP / Reassembly
Network Layer	Packets	IP, ICMP, IPsec / Packets assembly
Data Link Layer	Frames	Ethernet, Wifi / Intra-network communications
Physical Layer	Sending cable	00100111 / Fiber / Receiving cable



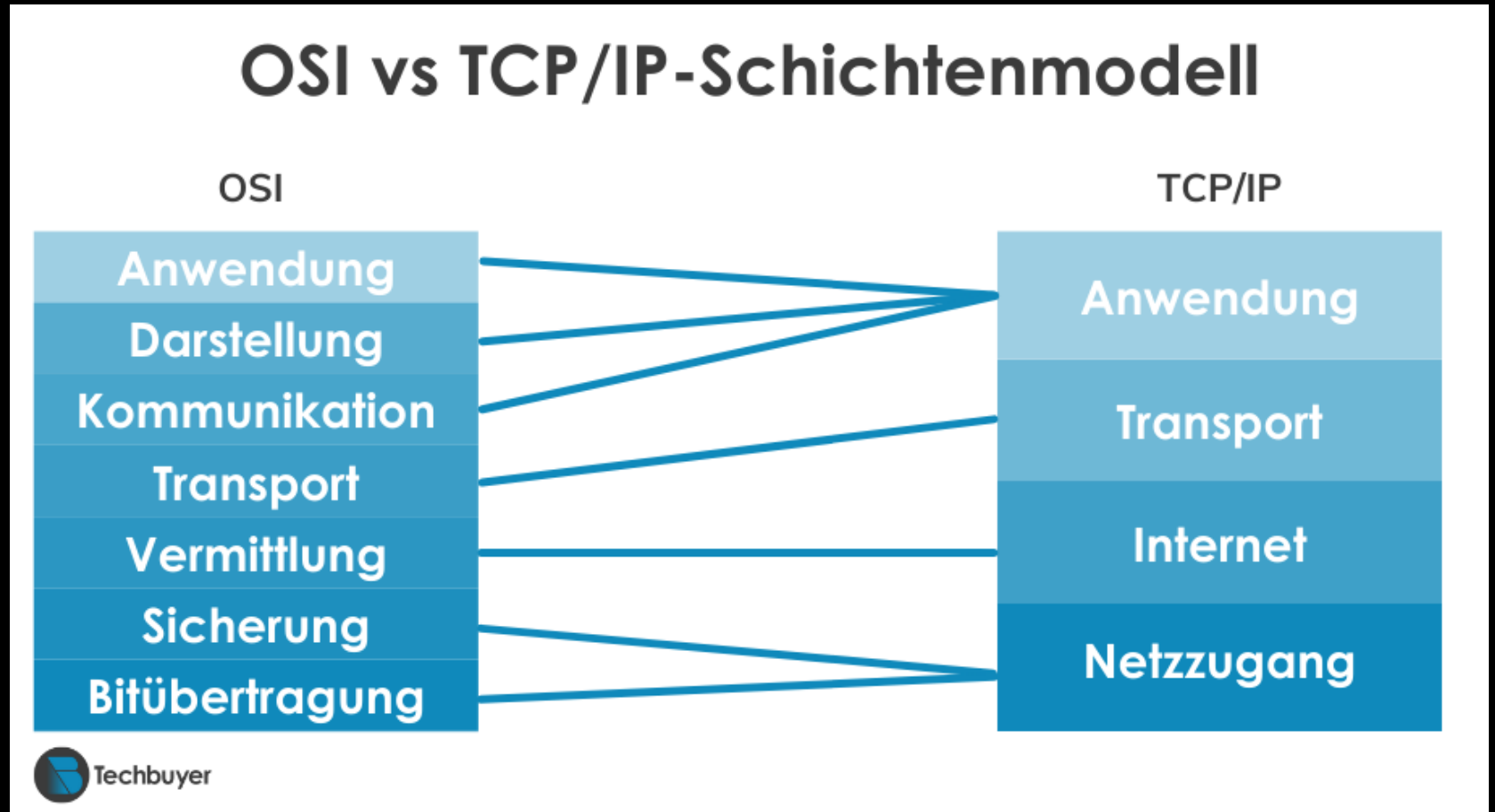
Application Layer	Request / Response	HTTP, FTP, SMTP
Presentation Layer	compression, encoding, encryption	TLS, SSL
Session Layer	Session	Sockets
Transport Layer	Segmentation	TCP, UDP / Reassembly
Network Layer	Packets	IP, ICMP, IPsec / Packets assembly
Data Link Layer	Frames	Ethernet, Wifi / Intra-network communications
Physical Layer	Sending cable	00100111 / Fiber / Receiving cable

Das TCP/IP Modell

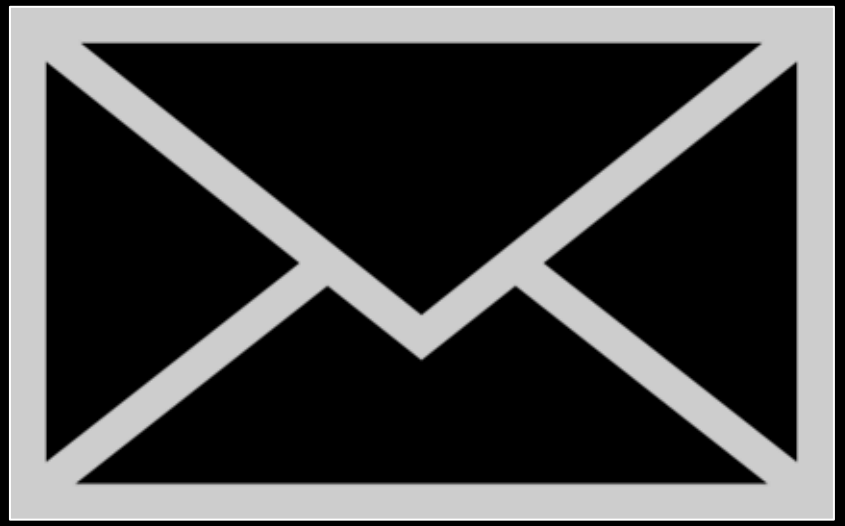
- Entstand parallel aus dem **ARPANET** (Vorläufer des Internets)
- **Pragmatisch**: wurde direkt in reale Netzwerke implementiert

- OSI-Modell = *Theorie* (Lehrbuch, idealisiert)
- TCP/IP = Praxis (Internet, reale Welt)

- Sie arbeiten nicht „nebeneinander“, sondern TCP/IP nutzt Protokolle, die sich in OSI-Schichten einordnen lassen.



Was ist eine IP und MAC-Adresse?



John Sample

MAC → Any Street 42 → Lokales Netzwerk

AA:BB:CC:DD:EE:FF

IP → 12345 Any City
Any State → Netzwerkübergreifend

192.42.42.42

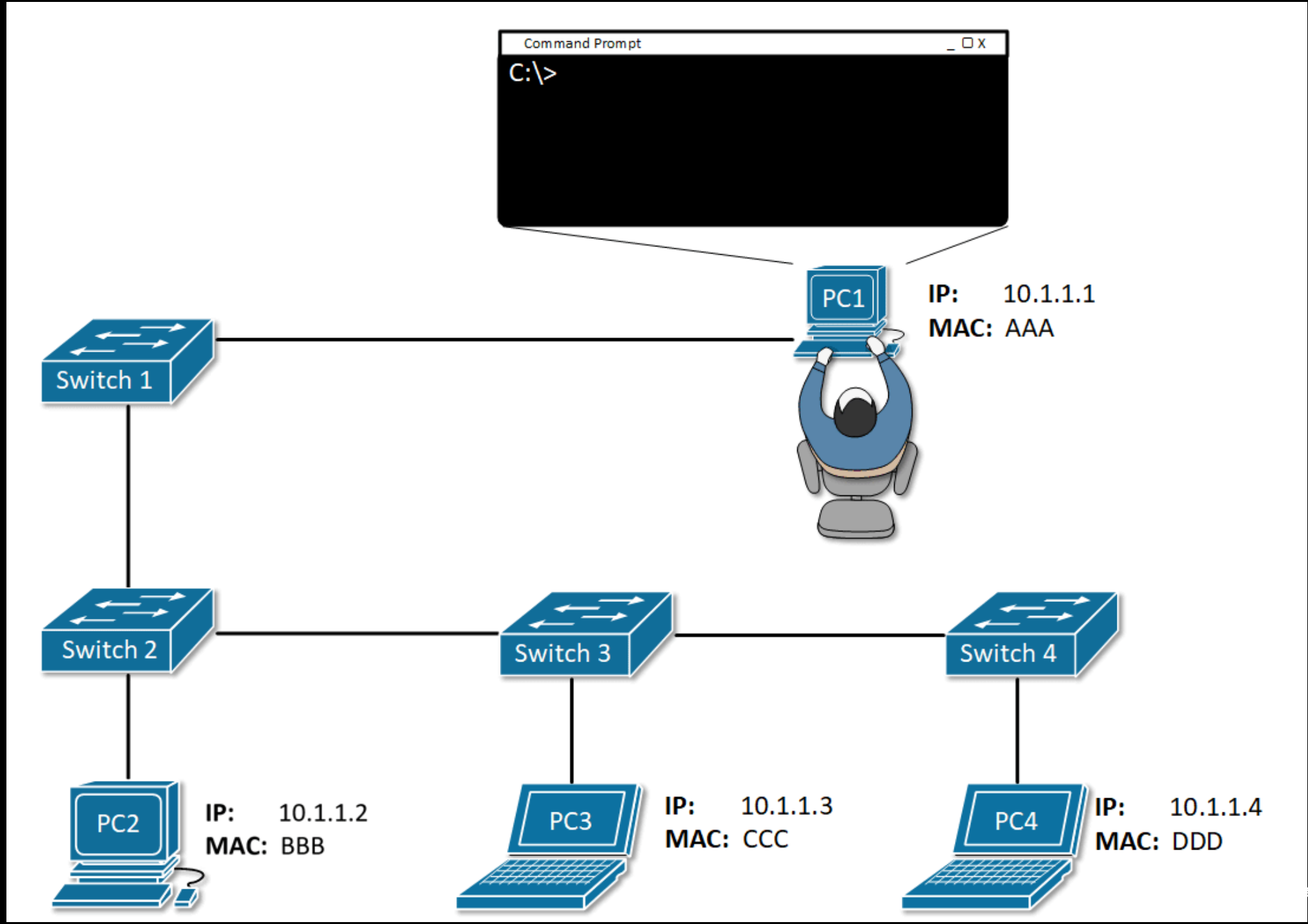
MAC-Adresse (lokale Identität)

- Jedes Gerät hat eine **MAC-Adresse** (Hardware-Adresse)
- Eindeutig, vom Hersteller vergeben (z. B. 00:1A:2B:3C:4D:5E)
- Funktioniert **nur im lokalen Netz** (LAN, WLAN)
- Vergleich: **Name an der Wohnungstür**

IP-Adresse (weltweite Adresse)

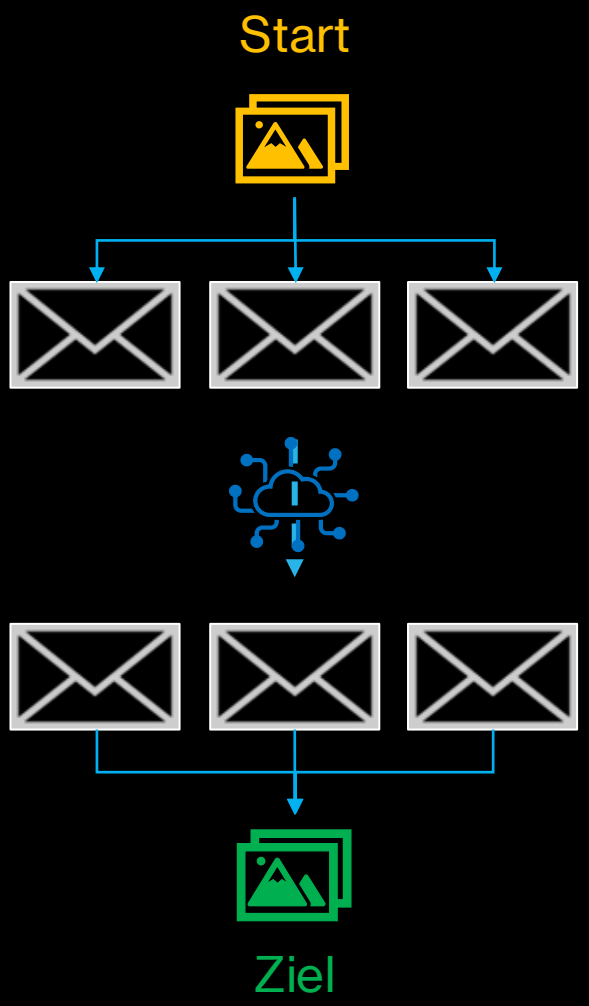
- **IP-Adresse** = Adresse im Netzwerk (vergleichbar mit Hausanschrift)
- Zwei Arten:
  - **Private IPs (lokal, z. B. 192.168.x.x)**
  - **Öffentliche IPs (im Internet sichtbar)**
- Router übersetzt zwischen privater und öffentlicher IP

Vergleich:
MAC = Name auf Klingel
Private IP = Wohnungsnummer
Öffentliche IP = Straßenadresse

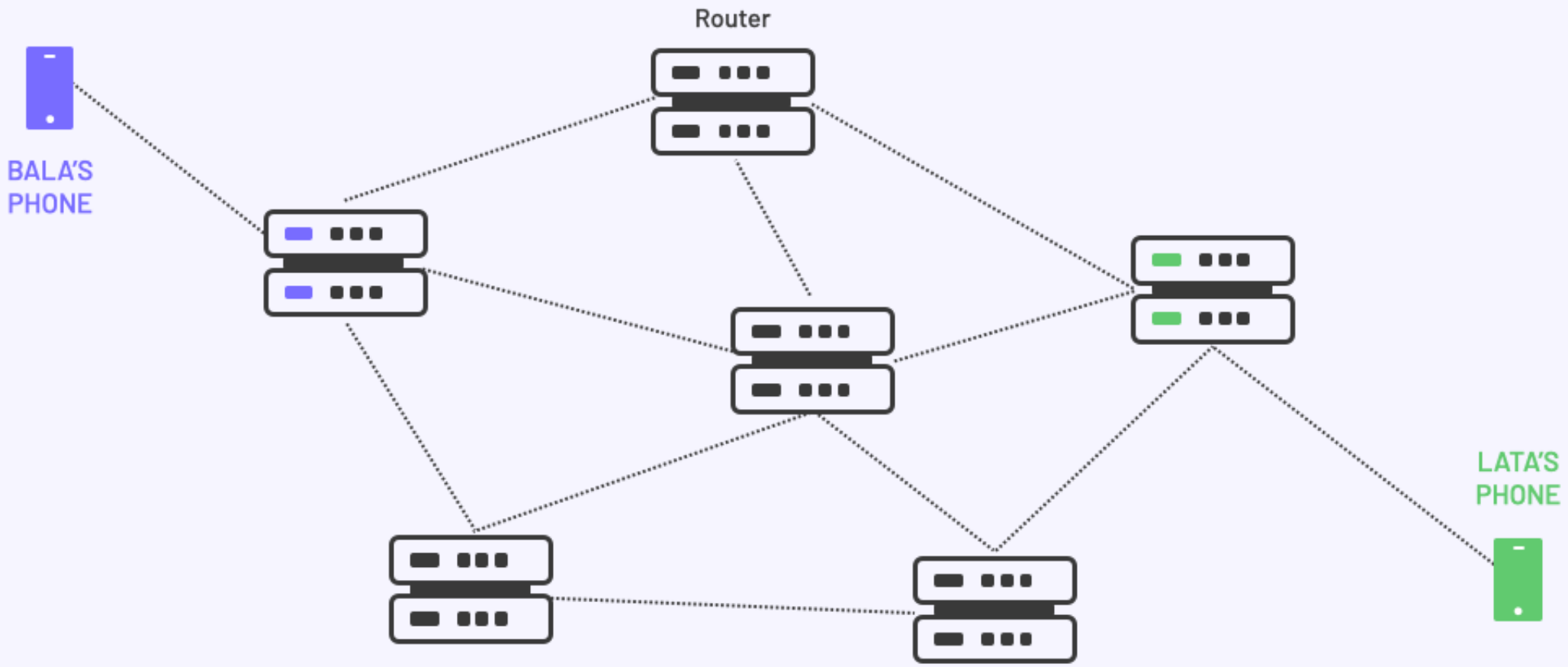

Command Prompt
C:\>
PC1
IP: 10.1.1.1
MAC: AAA
Switch 1
Switch 2
Switch 3
Switch 4
PC2
IP: 10.1.1.2
MAC: BBB
PC3
IP: 10.1.1.3
MAC: CCC
PC4
IP: 10.1.1.4
MAC: DDD

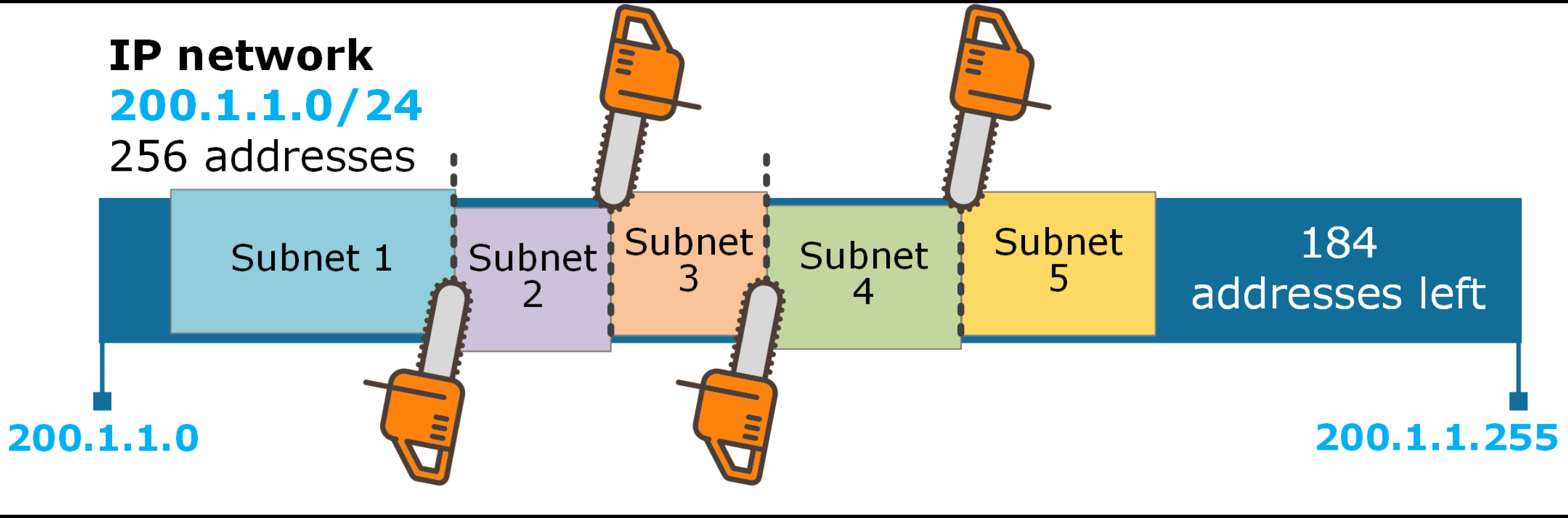
Pakete?

- Daten werden **nicht als Ganzes** geschickt (z. B. ganzer Film, ganze Datei)
- Stattdessen: **Zerlegung in kleine Pakete**
- Vorteile:
  - **Effizienz** – viele Nutzer teilen sich die Leitung
  - **Zuverlässigkeit** – nur verlorene Pakete müssen neu gesendet
  - **Flexibilität** – Pakete können verschiedene Wege nehmen und trotzdem ankommen
- Vergleich:
  - Nicht ein riesiger LKW mit allen Möbeln, sondern **mehrere kleine Lieferwagen**, die unabhängig fahren



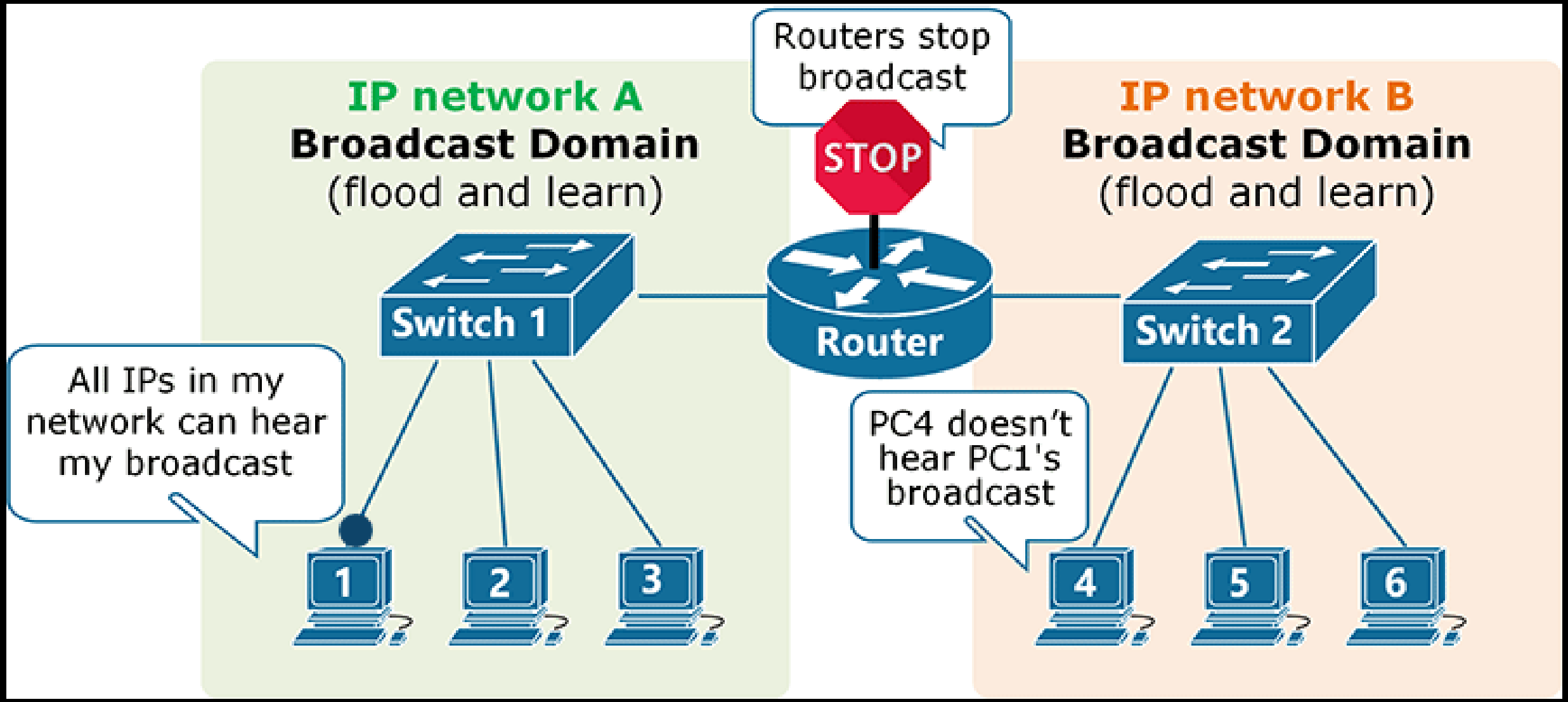
Packet Switching




IP network
200.1.1.0/24
256 addresses
Subnet 1
Subnet 2
Subnet 3
Subnet 4
Subnet 5
184 addresses left
200.1.1.0
200.1.1.255

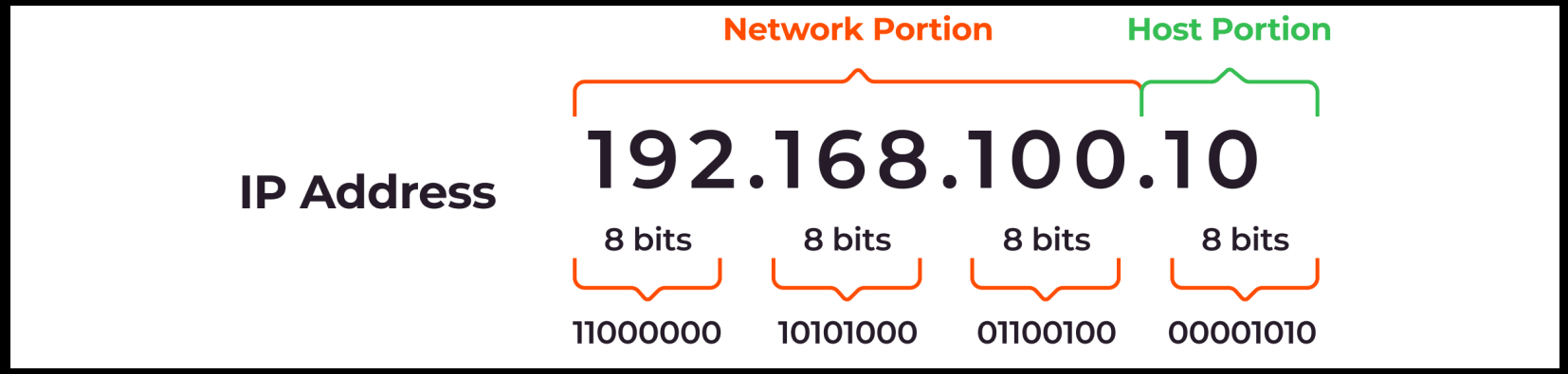
Warum Subnetze?

- Netzwerke können **sehr groß** sein (z. B. Uni, Firma, Internet)
- **Aufteilung in kleinere Netze (Subnetze):**
  - Erhöht **Übersicht & Organisation**
  - Spart IP-Adressen
  - Verbessert **Sicherheit** (Trennung von Bereichen)
- Reduziert **Datenverkehr** (Broadcasts bleiben im Subnetz)
- Vergleich: Eine Stadt wird in **Stadtteile** unterteilt → bessere Orientierung


Routers stop broadcast
STOP
IP network A
Broadcast Domain
(flood and learn)
IP network B
Broadcast Domain
(flood and learn)
Switch 1
Router
Switch 2
All IPs in my network can hear my broadcast
PC4 doesn't hear PC1's broadcast
1
2
3
4
5
6

Subnetze & IP-Adressen

- Eine IP-Adresse besteht aus zwei Teilen:
  - **Netzwerkanteil** → Zu welchem Subnetz gehört das Gerät?
  - **Hostanteil** → Welches Gerät innerhalb dieses Subnetzes?
- Beispiel: 192.168.1.42 /24
  - Netzwerk: 192.168.1.0
  - Host: 42



Wie können wir zwischen Subnetzen kommunizieren?

- **Innerhalb eines Subnetzes:**
  - Geräte sprechen direkt miteinander
  - MAC-Adresse + ARP sorgen dafür, dass Pakete das richtige Gerät erreichen
  - Pakete werden über **Switches** verteilt

- **Zwischen Subnetzen:**
  - MAC alleine reicht nicht, weil das Zielgerät in einem anderen Subnetz ist
  - Paket wird an **Router** geschickt
  - Router leitet es ins richtige Subnetz weiter

Was ist ein Switch?

- **Switch = Verteiler innerhalb eines Netzwerks**
- Aufgabe: **Pakete nur an den richtigen Empfänger weiterleiten**
- Vergleich:
  - Postfach-System in einem Büro → Briefe werden nur an den richtigen Mitarbeiter geschickt, nicht an alle
  - Vermeidet unnötigen Datenverkehr (im Gegensatz zu einem Hub, der alles an alle schickt)

- Arbeitet auf **Layer 2 (Data Link / MAC-Ebene)**



Was ist ein Router

- **Router = Verkehrsmanager zwischen Netzwerken/Subnetzen**
- Aufgabe: **Pakete weiterleiten**, wenn das Ziel in einem anderen Subnetz liegt
  - Router hat Tabellen und lernt aktiv wo sich welche Zieladresse befindet
- Vergleich:
  - Straßenverkehr: Router = Kreuzung + Verkehrsleitzentrale
  - Paketverkehr: Router = Postamt, das Pakete in das richtige Stadtviertel schickt
- Router arbeiten auf **Layer 3 (IP-Ebene)**



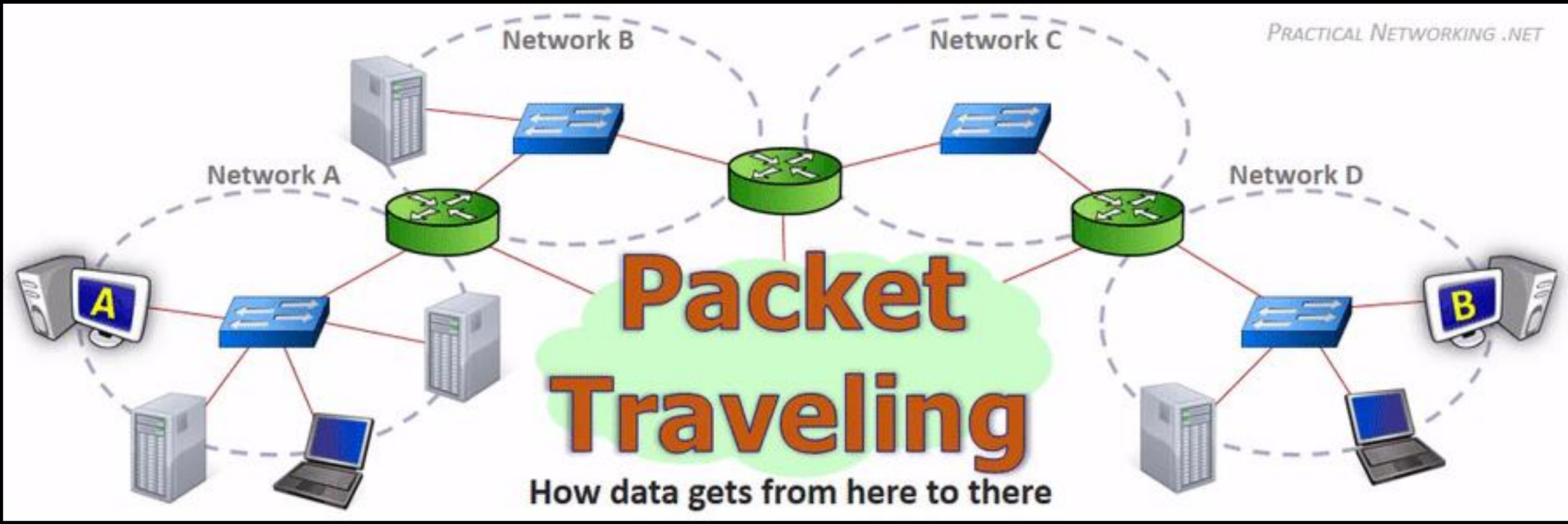
Switch vs. Router

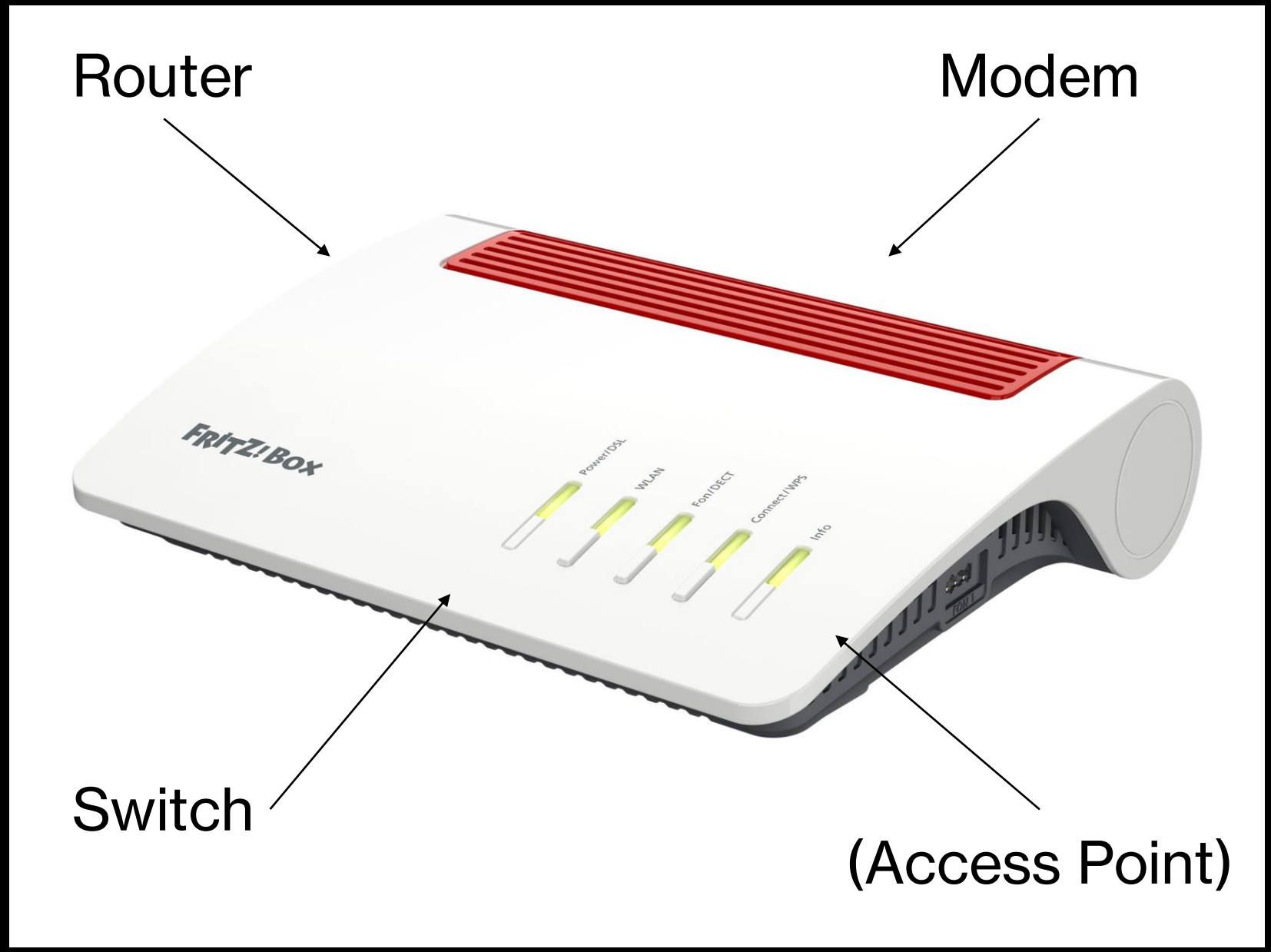
Feature	Switch	Router
Ebene	Layer 2 (MAC)	Layer 3 (IP)
Funktion	Verteilt Daten innerhalb eines Subnetzes	Verbindet unterschiedliche Subnetze / Netzwerke
Entscheidung	Anhand von MAC-Adresse	Anhand von IP-Adresse / Routing-Tabelle
Typischer Einsatz	Büro, Heimnetzwerk, LAN	Internetzugang, Subnetz-Grenzen

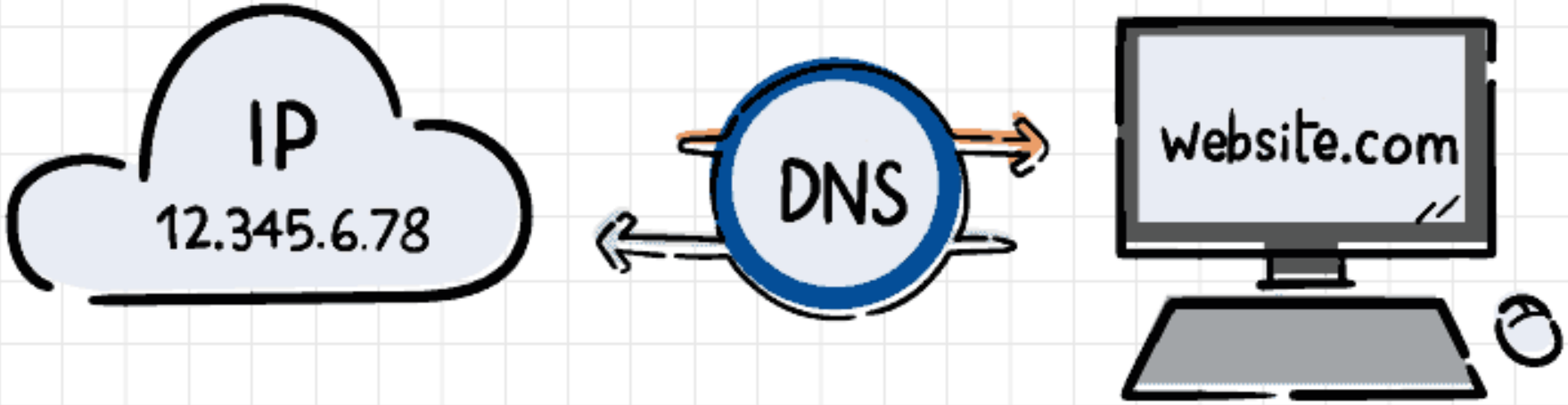
Jetzt wissen wir...

- Wie werden Geräte in Netzwerken identifiziert:
  - MAC → lokales Netz
  - Private IP → netzübergreifend in privaten Netzen
  - Öffentliche IP → netzübergreifend in öffentlichen Netzen (Internet)
- Wie finden sich Geräte untereinander?
  - Im lokalen Netz werden einfach alle nach ihrer MAC gefragt Im öffentlichen Netz übernehmen Router und halten in Tabellen alle gelernten Routen fest

- Wie werden Daten übertragen? → In vielen kleinen Paketen
- Wie behalten wir Ordnung und eine saubere Trennung → Subnetze


Network B
Network C
PRACTICAL NETWORKING .NET
Network A
Network D
A
B
Packet
Traveling
How data gets from here to there


Router
Modem
FRITZ!Box
Power/DSL
WLAN
Fon/DECT
Connect/WPS
Info
Switch
(Access Point)

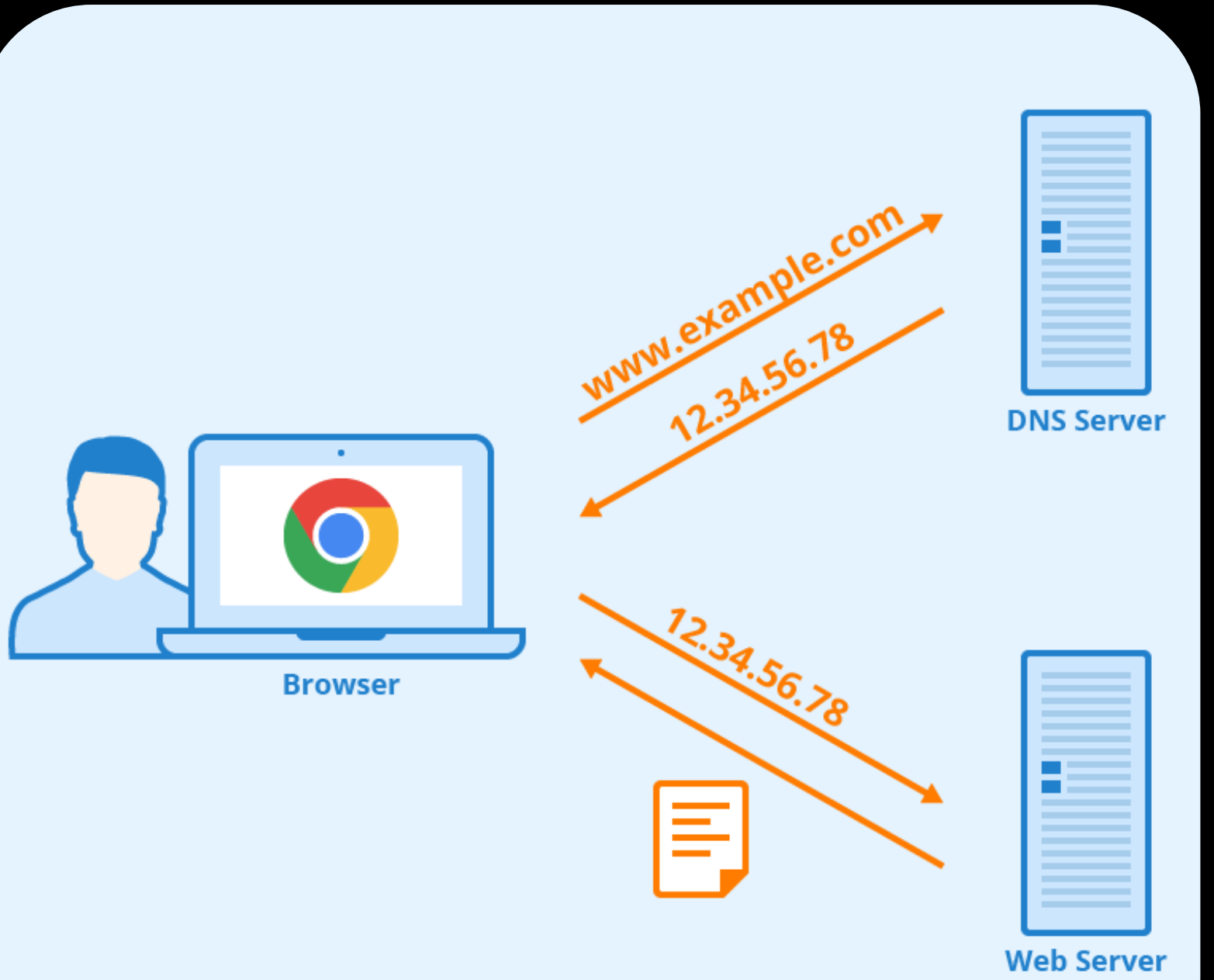


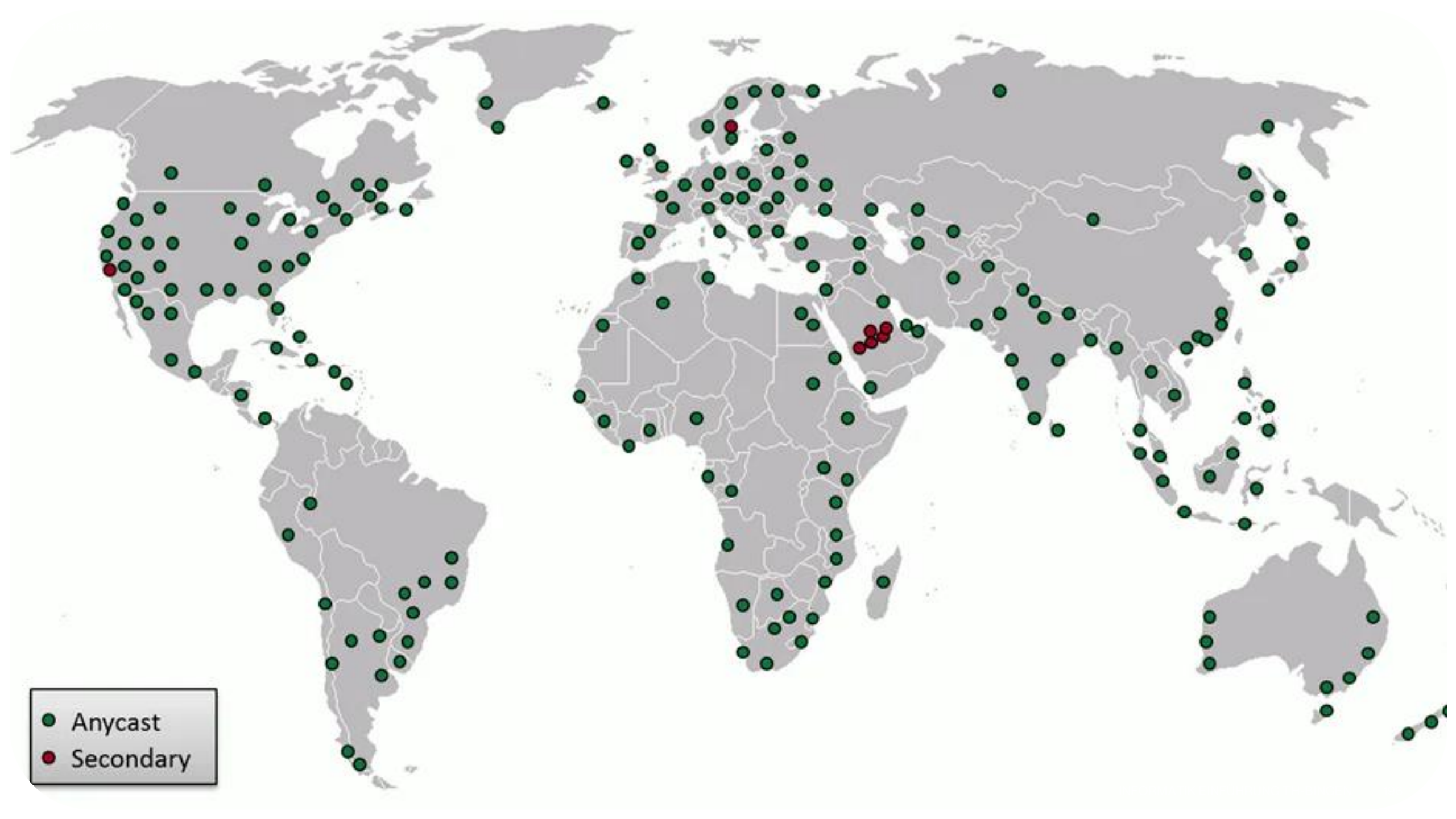
DNS (Domain Name System)

Was ist DNS?

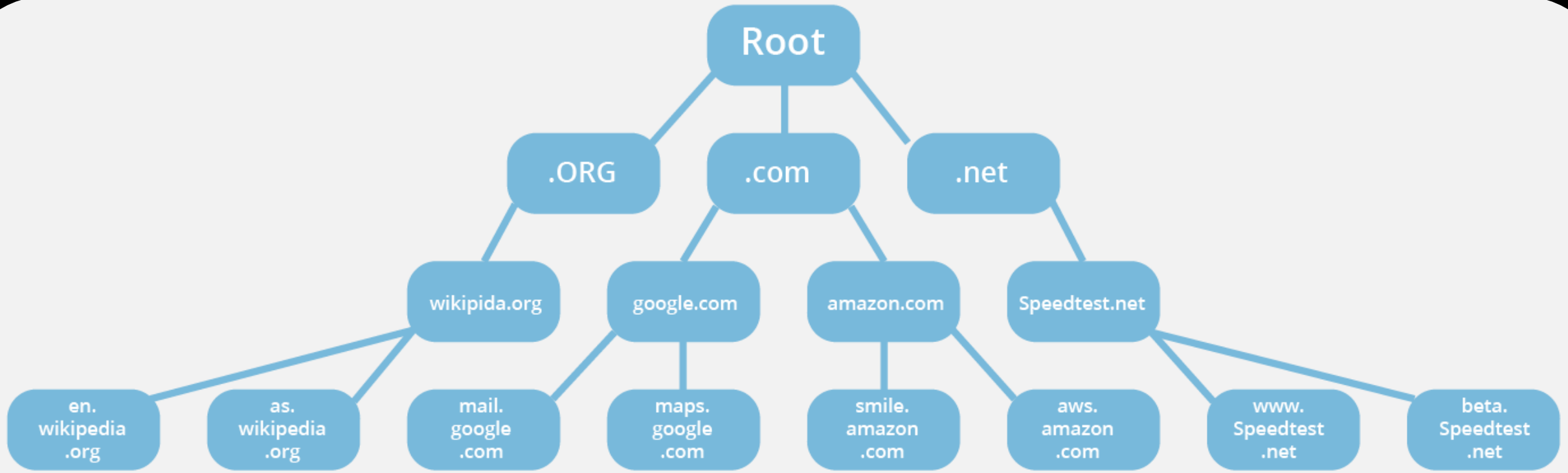
- DNS verknüpft eine IP-Adresse mit einem für Menschen besser leserlichen Namen
  - (z.B. google.com → 142.250.74.206)

- Warum?
  - Menschen können sich nur schwer Zahlen merken
  - IP-Adressen ändern sich regelmäßig
  - Weltweite Verteilung von Adressen ist schwierig


www.example.com
12.34.56.78
DNS Server
Browser
12.34.56.78
Web Server


Anycast
Secondary

DNS-Stammserver



Besserer DNS-Service für mehr Privatsphäre

Nur ein kleiner Tipp am Rande und absolut optional ☺

https://www.cloudflare.com/de-de/learning/dns/what-is-1.1.1.1/



Quiz

Multiple Choice

1 Minute

Was ist die Hauptaufgabe eines Switches?

- A) Pakete zwischen Subnetzen weiterleiten
- B) Pakete gezielt innerhalb eines Subnetzes an die richtige MAC-Adresse zustellen
- C) Routing-Tabellen verwalten
- D) IP-Adressen vergeben

Warum gibt es Subnetze?

- A) Um MAC-Adressen zu ändern
- B) Um Netzwerke zu strukturieren, Datenverkehr zu reduzieren und Sicherheit zu erhöhen
- C) Um Geräte schneller zu machen
- D) Um Internetzugang zu blockieren

Quiz

Wahr/Falsch

1 Minute

- Pakete innerhalb eines Subnetzes brauchen **immer einen Router**, um zum Ziel zu gelangen. 
- Ein Router arbeitet auf Layer 3, also auf der **IP-Ebene**. 
- MAC-Adressen werden für die **Zustellung zwischen Subnetzen** verwendet 
- Subnetze helfen, **Broadcast-Datenverkehr zu reduzieren** 

Diskussion

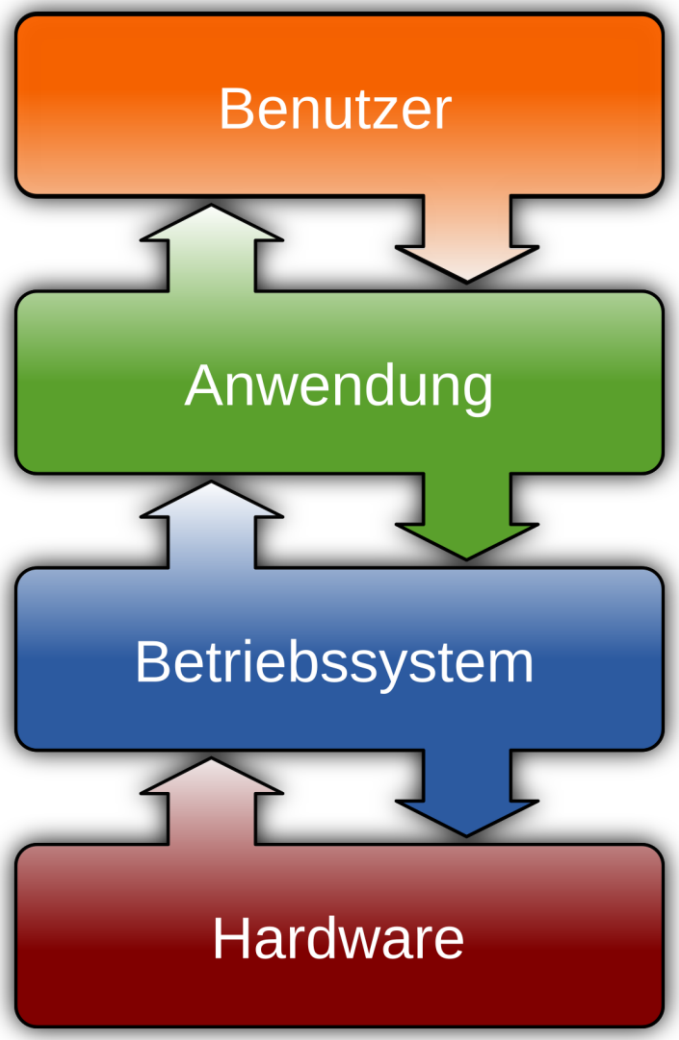
- Erkläre den Unterschied zwischen **MAC-Adresse** und **IP-Adresse**.
- Beschreibe den **Schritt-für-Schritt-Ablauf**, wie ein Paket von Subnetz A zu Subnetz B gelangt.
- Warum sind **Routing-Tabellen** für einen Router wichtig?

5 Minuten

8. Abstrakte Maschinen (APIs, Cloud und die moderne Welt)

Was ist eine Abstrakte Maschine?

- **Definition:** Theoretisches Modell eines Rechners.
  - Definiert Zustände und Operationen.
  - Verbirgt komplexe Hardware-Details, bietet vereinfachte Sicht.
- **Zweck:** Ermöglicht **Portabilität** ("Write once, run anywhere").
- **Struktur:** Speicher und Interpreter.
- **Beispiele:**
  - **Turing-Maschine:** Konzeptionelles Modell mit Band, Kopf, Steuerung.
  - **Java Virtual Machine (JVM):** Führt Java-Bytecode aus, ermöglicht Plattformunabhängigkeit.
  - Programmiersprachen selbst.
- **Bedeutung:** Trennung von Verantwortlichkeiten, Modularität, parallele Entwicklung.

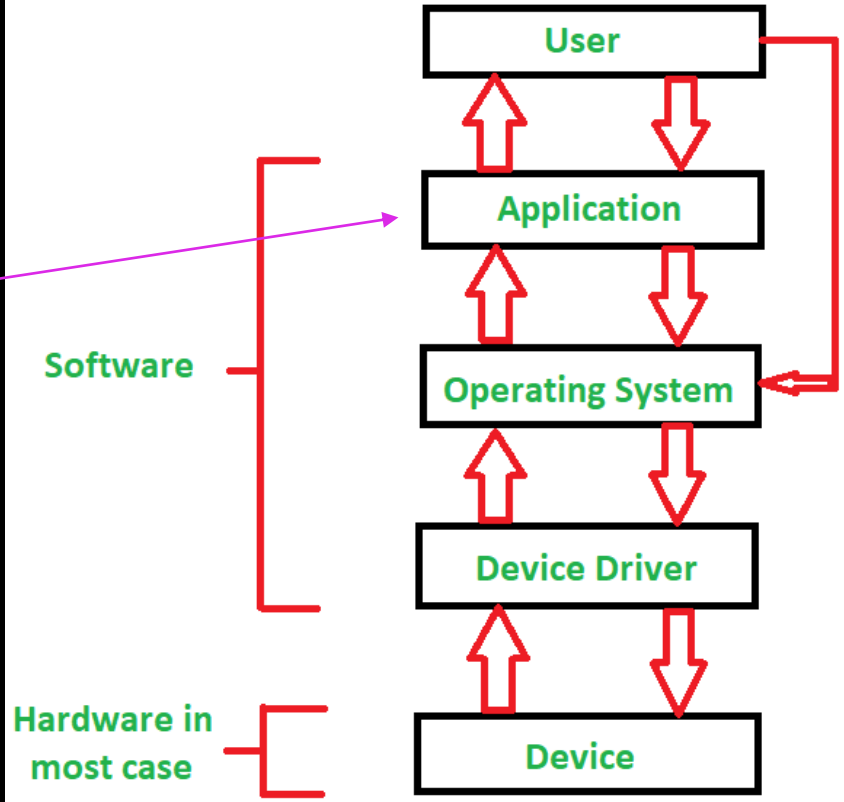


Abstraktionsebenen in der Informatik

- **Hierarchischer Aufbau von Computersystemen:** Jede Schicht baut auf der darunterliegenden auf und abstrahiert sie.
- **Grundlegende Schichten:**
  - **Hardware:** Physische Ebene (Prozessor, Speicher, Geräte).
  - **Betriebssystem (OS):** Baut auf Hardware auf, stellt "abstrakte Maschine" für Anwendungen bereit.
  - **Anwendungsprogramm:** Nutzt Dienste des Betriebssystems.
- **OS als abstrakte Maschine:** Bietet normierte Programmierschnittstelle (API).
- **Vorteile:** Wiederverwendbarkeit, Änderbarkeit, Wartbarkeit, Portabilität, Testbarkeit.
- **Relevanz:** Ermöglicht Spezialisierung und Innovation, wichtig für Prozessleitsysteme.

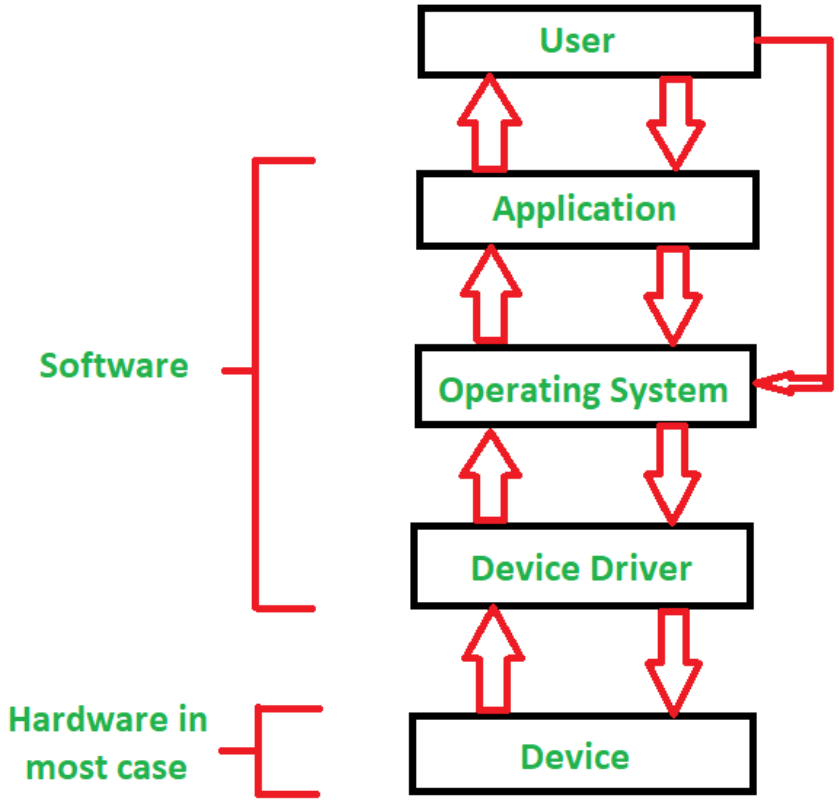
Wie Software auf Hardware ausgeführt wird

- **Ausgangspunkt:** Software in menschenlesbarem Quellcode.
- **Ziel:** Umwandlung in maschinenlesbaren Maschinencode.
- **Ablauf:**
  - **Quellcode schreiben.**
  - **Kompilieren oder Interpretieren:**
    - **Compiler:** Übersetzt *gesamten* Quellcode *vor* Ausführung in Maschinencode; erzeugt ausführbare Datei (z.B. .exe); schnellere Ausführung, Fehlererkennung, Code-Optimierung .
    - **Interpreter:** Übersetzt Quellcode *Zeile für Zeile während* der Ausführung; keine separate ausführbare Datei.
  - **Ausführen:** Maschinencode wird vom Prozessor (CPU) ausgeführt, interagiert mit OS und Treibern.
- **Wahl zwischen Kompilierung/Interpretation:** Technologische Entscheidung mit Auswirkungen auf Performance, Portabilität, Sicherheit, Entwicklungszyklus (z.B. kompilierte Sprachen für Echtzeit-Prozessleitsysteme) .



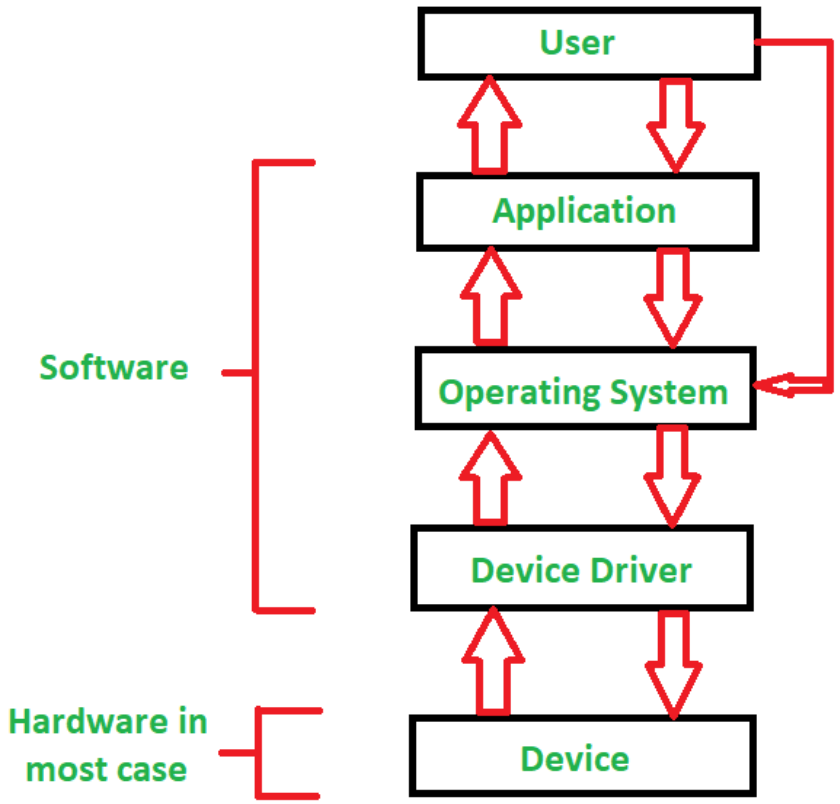
Rolle des Betriebssystems (OS)

- **Definition:** Fundamentale Software, Schnittstelle zwischen Hardware und Software, Basis für Computerbetrieb.
- **Kernaufgaben:**
  - Hardware-Verwaltung (Prozessor, Speicher, Geräte).
  - Software-Verwaltung (Programmüberwachung, Multitasking).
  - Ressourcenverteilung (Rechenzeit, Speicherplatz).
  - Benutzerschnittstelle (grafisch oder Kommandozeile).
  - Programmierschnittstelle (API) für Anwendungszugriff.
  - Schutzstrategien (Isolation, Sicherheit).
  - Fehlerbehandlung und Wiederherstellung.
- **OS als "Virtualisierer":** Verbirgt Hardware-Komplexität, stellt "virtuelle Betriebsmittel" bereit.
- **Bedeutung:** Standardisierung der Softwareentwicklung, Wiederverwendbarkeit, Kompatibilität.
- **Spezialfall:** Realzeit-Betriebssysteme für Steuerungs- und Regelungsaufgaben in der Verfahrenstechnik.



Rolle des Gerätetreibers

- **Definition:** Spezielle Software, die OS-Kommunikation mit Hardware ermöglicht.
- **Hauptfunktionen:**
  - Kommunikationsbrücke zwischen Hardwareabstraktionsschicht (HAL) und Hardware.
  - Übersetzung von OS-Befehlen in hardware-spezifische Anweisungen.
  - Ermöglicht Datenübertragung.
- **Bedeutung:** Ohne Treiber keine Hardware-Nutzung; kapselt Hardware-Komplexität.
- **Sicherheit:** Nur Treiber von Originalherstellern verwenden; digital signierte Treiber bieten zusätzliche Sicherheit.



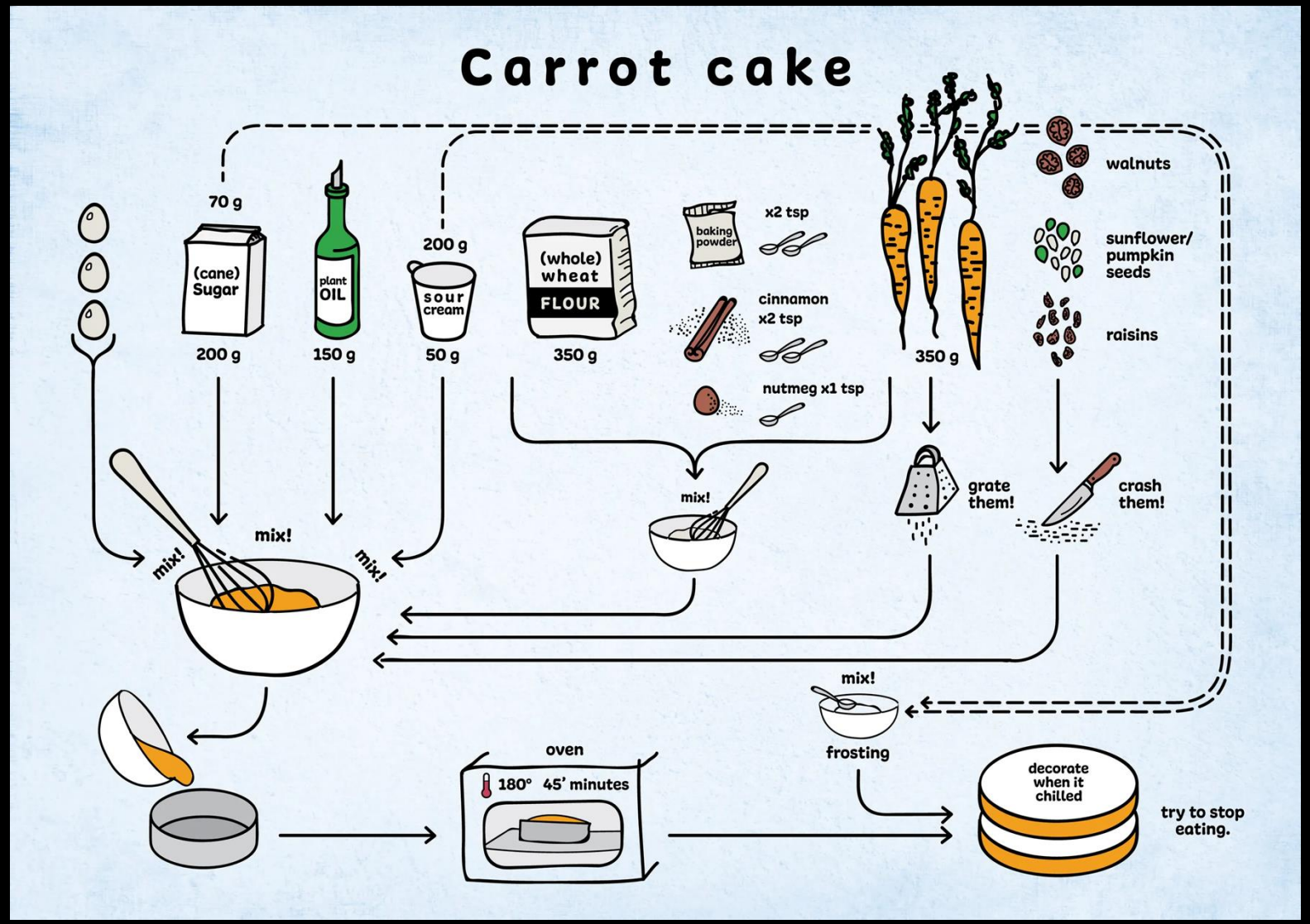
8.1 Turing

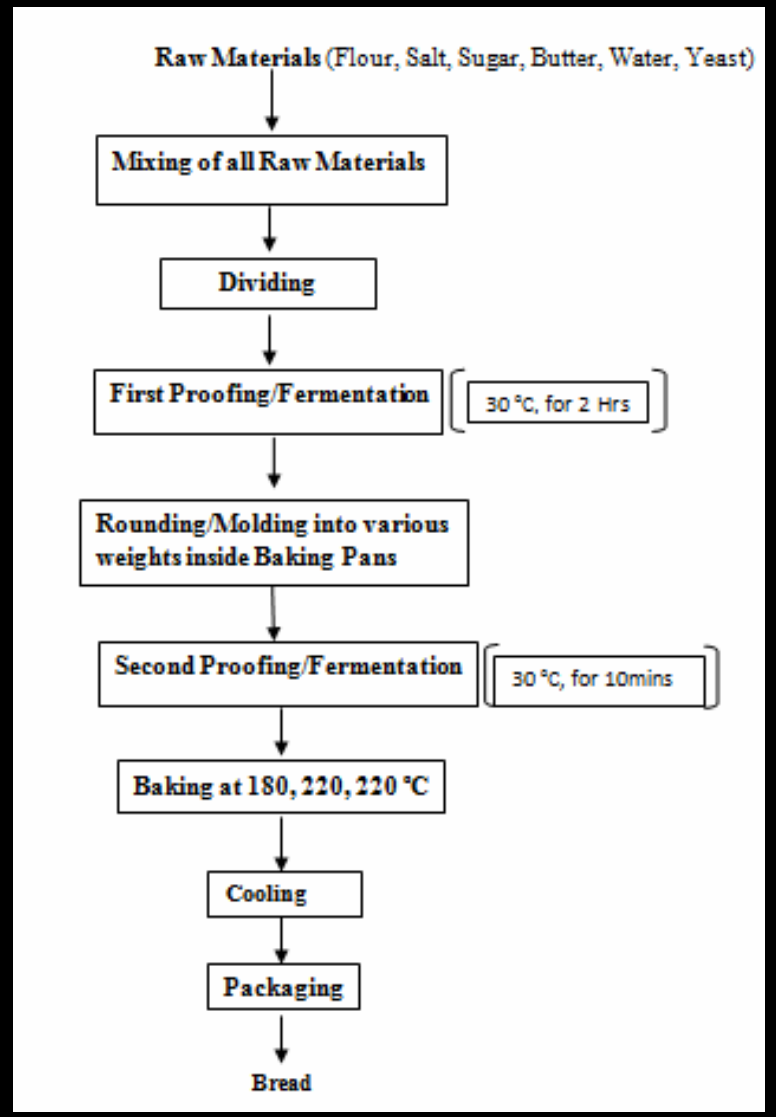
Alan Turing

- Britischer Mathematiker und Informatiker (1912–1954)
- Begründer der theoretischen Informatik
- Pionier der Künstlichen Intelligenz
- Leitideen: Berechenbarkeit, Turing-Maschine, Turing-Test
- Half maßgeblich den 2. Weltkrieg zu beenden (durch das Entschlüsseln der deutschen Enigma Maschine)



Was ist ein Algorithmus?

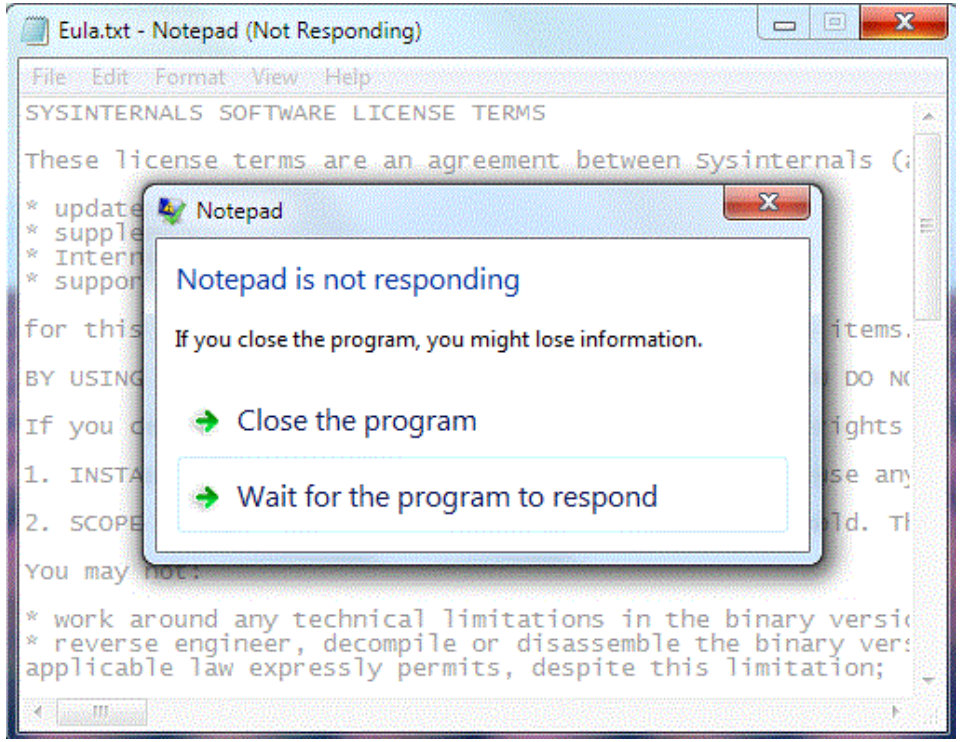




Warum will man Algorithmen theoretisch betrachten?

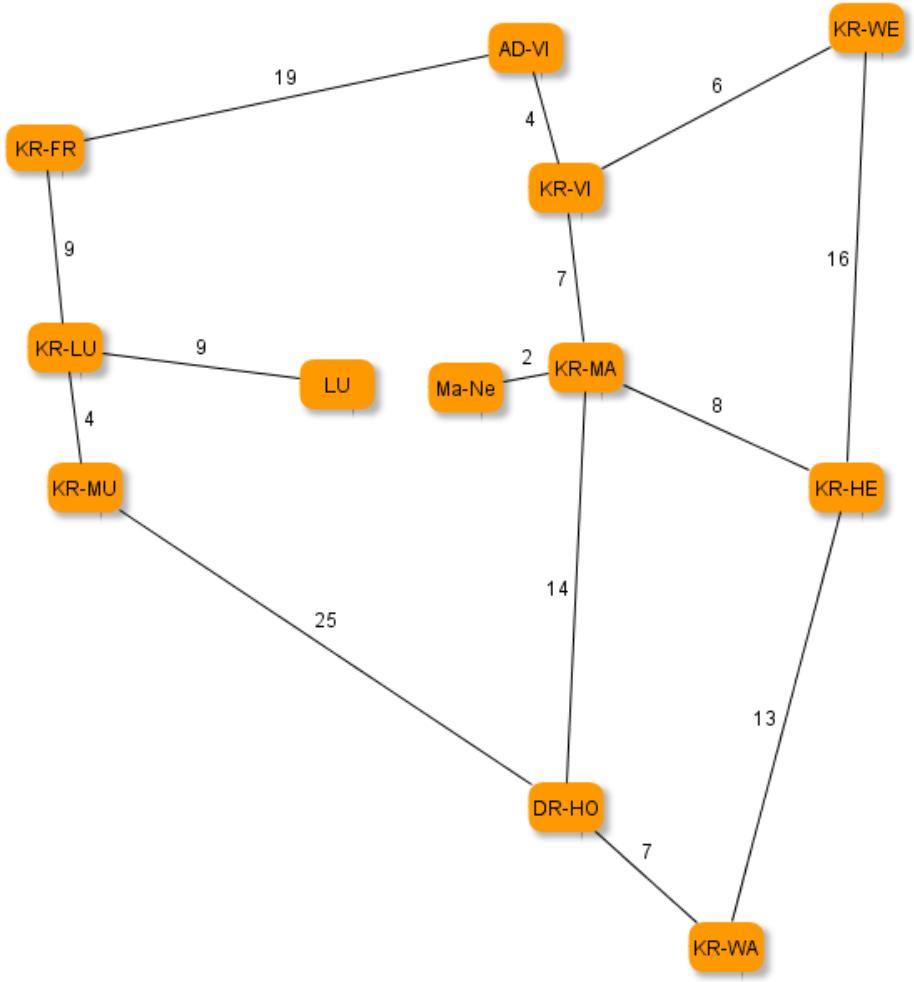
- Es gilt Fragen zu beantworten, wie:
  - Erfüllt mein Algorithmus meine Aufgabe/Fragestellung?
  - Wie lange braucht mein Algorithmus, um das Problem zu lösen (wie viele Schritte, wie viel Zeit)?
  - Terminiert mein Algorithmus oder läuft dieser unendlich lange?

- Mit diesen Informationen lassen sich Algorithmen vergleichen und verbessern



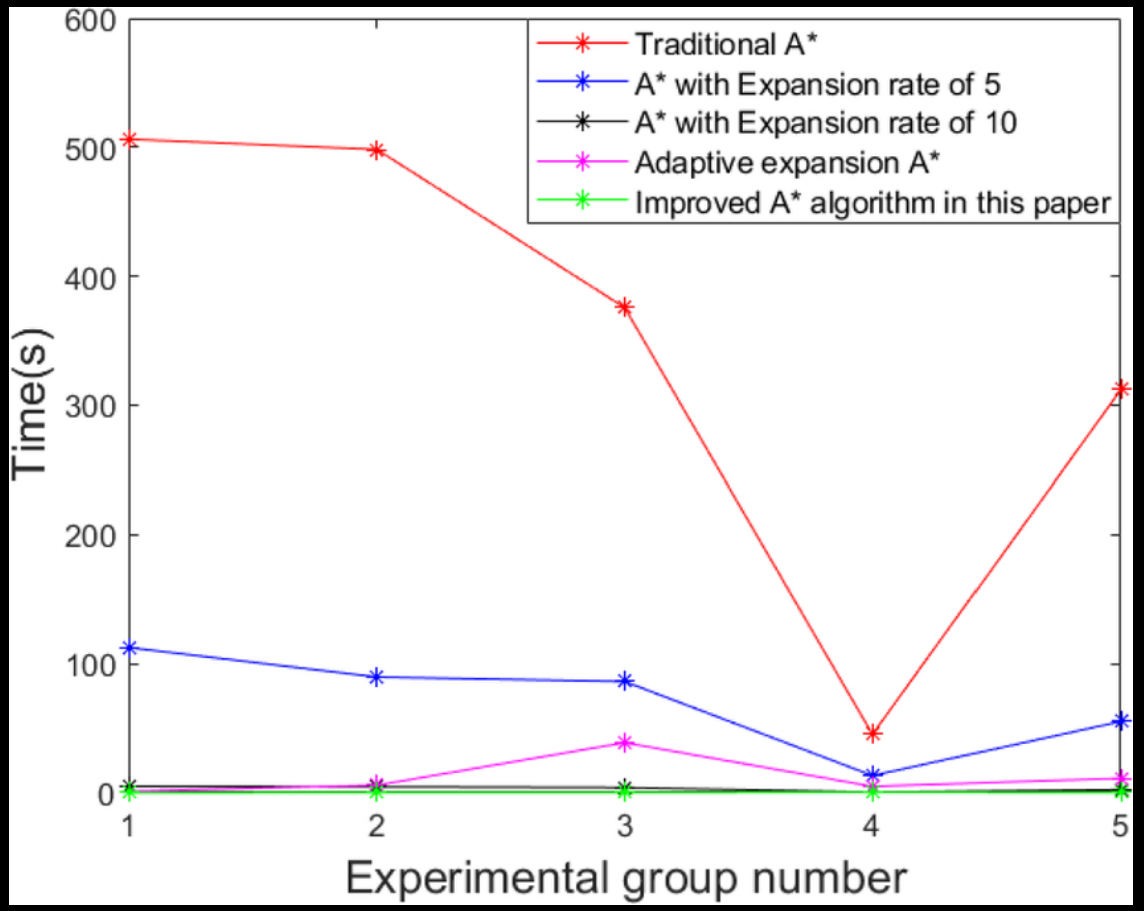
Beispiel Problem Routenplanung

- Wie komme ich am schnellsten von A nach B?
- Es gibt viele verschiedene Algorithmen zur Lösung:
  - Depth-First
  - Breath-First
  - Dijkstra
  - A
  - A*



Unterschiedliche Algorithmen können stark unterschiedlich sein

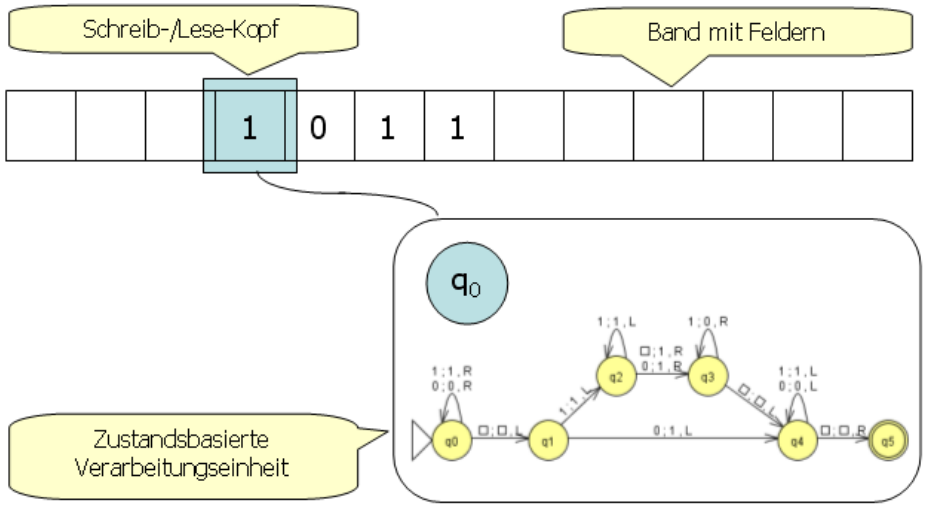
- In diesem Beispiel unterscheiden sich die Algorithmen um bis zu **Faktor 500** wie lange sie brauchen um das Routen Problem zu lösen!



Video

Turing-Maschine

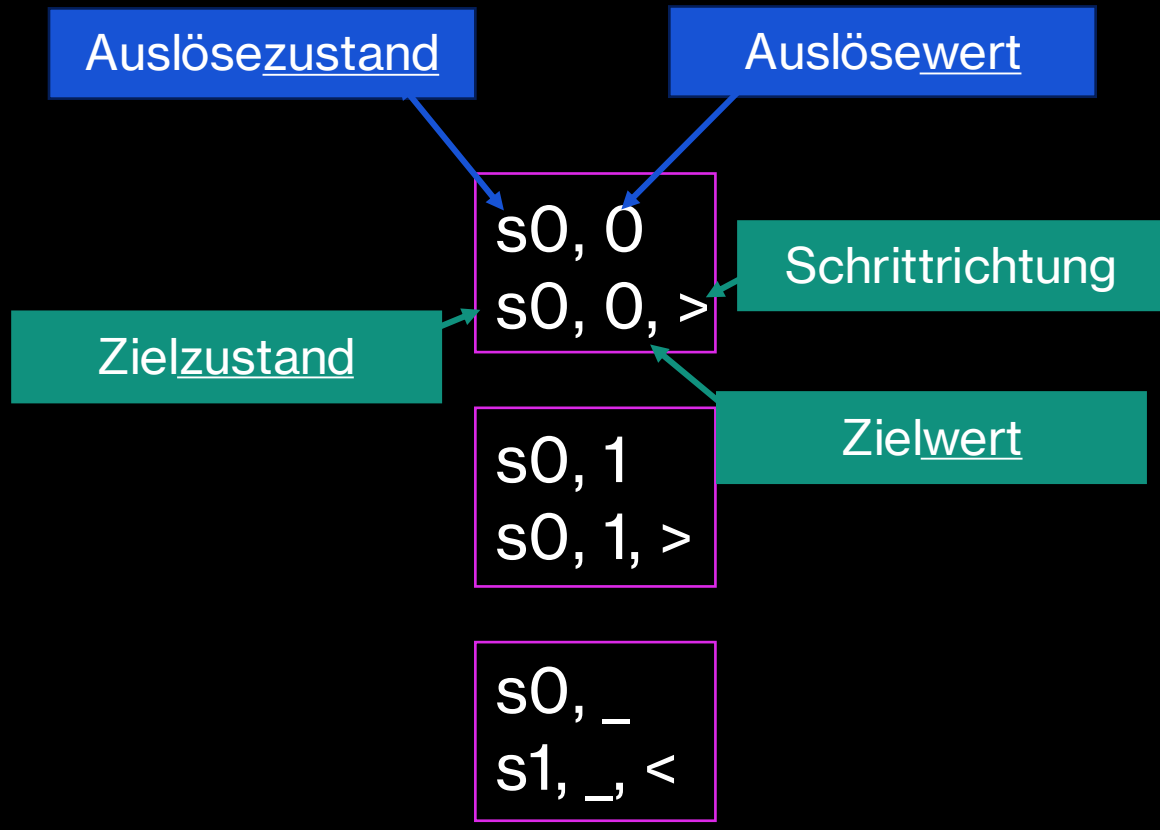
- **Was ist das?**
  - Theoretisches Modell eines Computers
  - Besteht aus: unendliches Band, Lesekopf, endliche Zustände
- **Wozu?**
  - Definiert, was *algorithmisch berechenbar* ist
  - Zeigt Grenzen der Berechenbarkeit (z. B. Halteproblem)
- **Bedeutung:**
  - Grundlage der modernen Informatik
  - Alle heutigen Computer folgen denselben Prinzipien



Bespiel – Addition der Zahl 1 auf eine Binärzahl

```
0 0 0 0      0 0 0 1
      1            1
-------      -------
0 0 0 1      0 0 1 0

1 1 0 1      0 1 1 1
      1            1
-------      -------
1 1 1 0      1 0 0 0
```

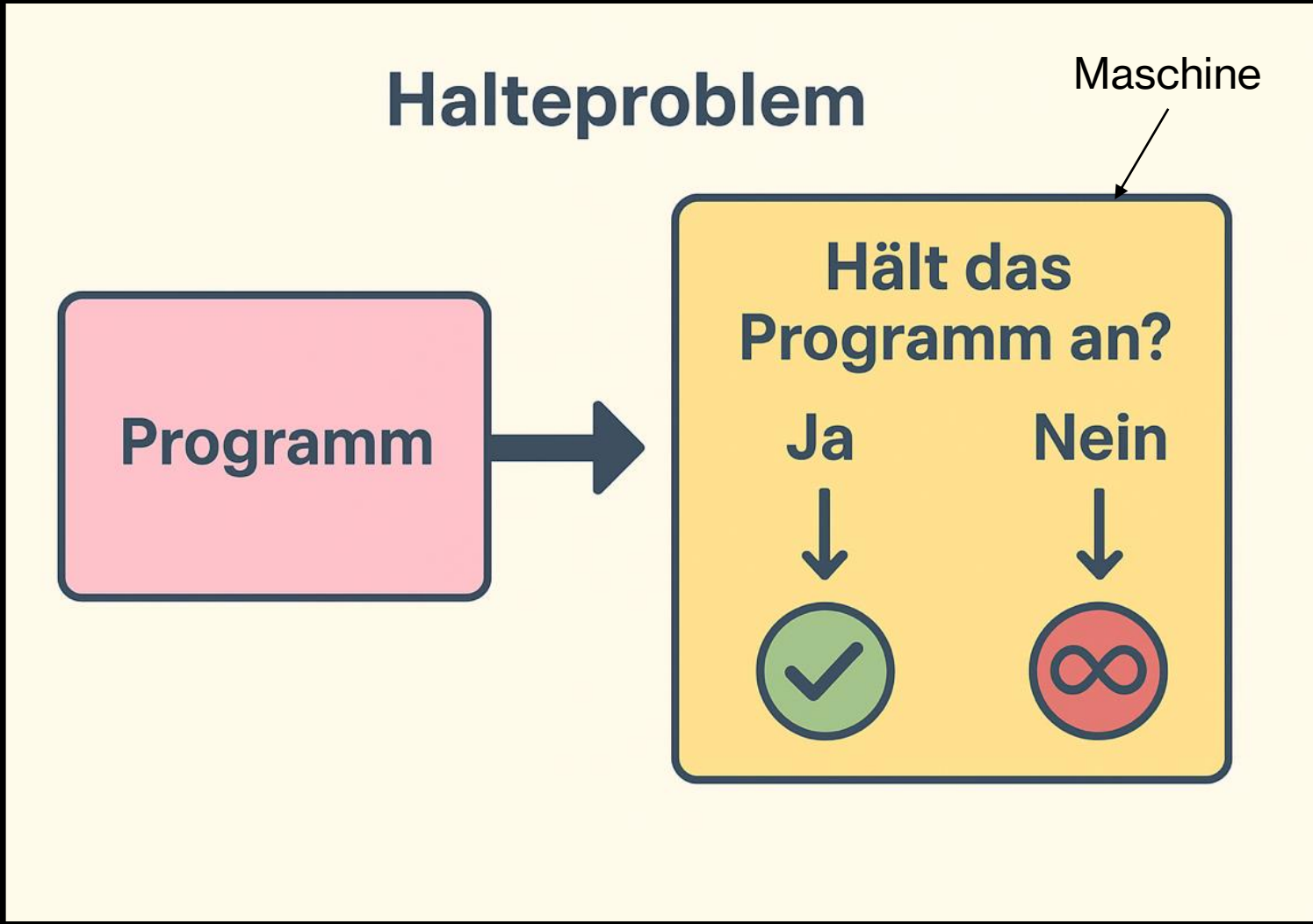


Video zur Erklärung

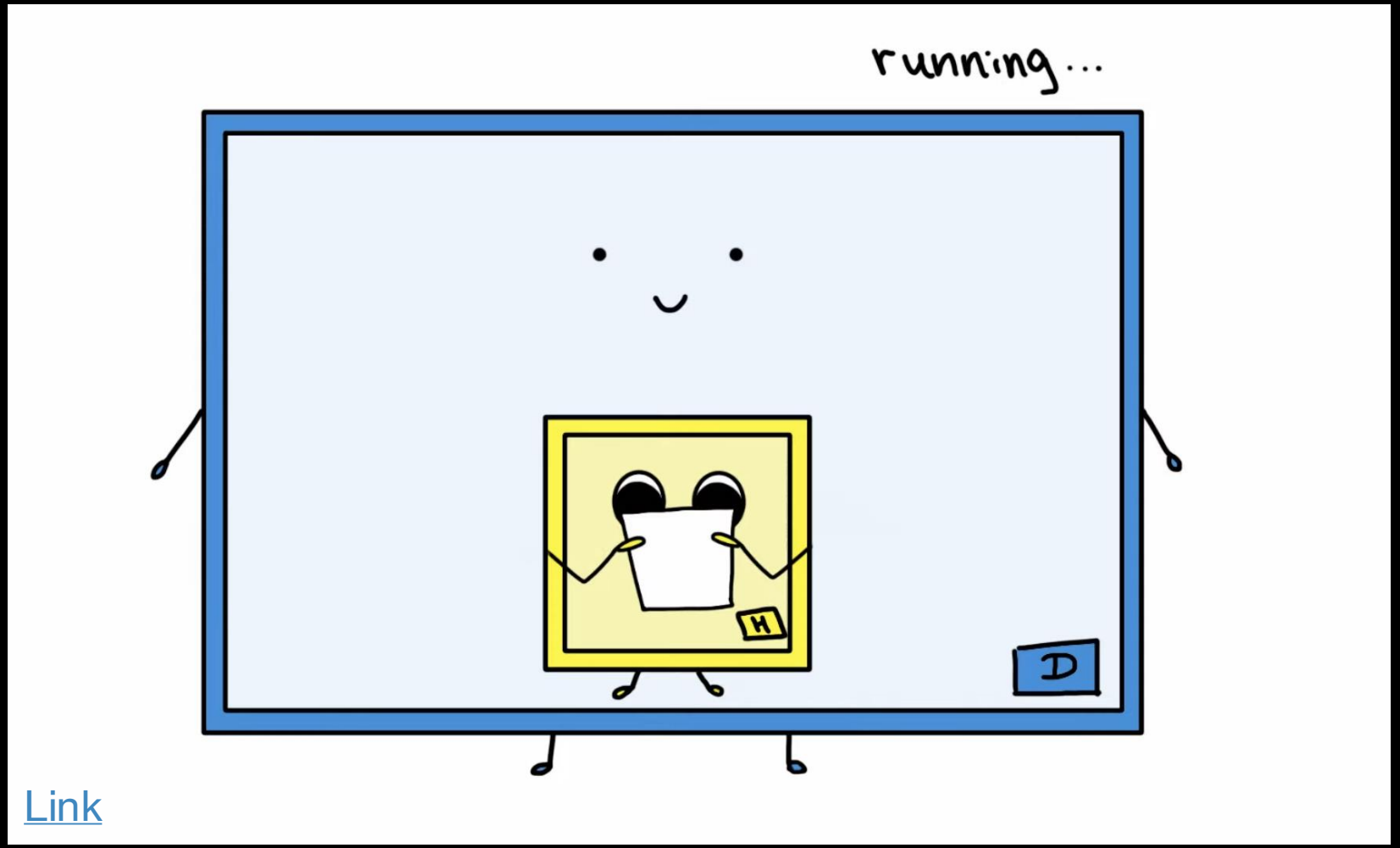
Online Turing Maschine

Das Halteproblem

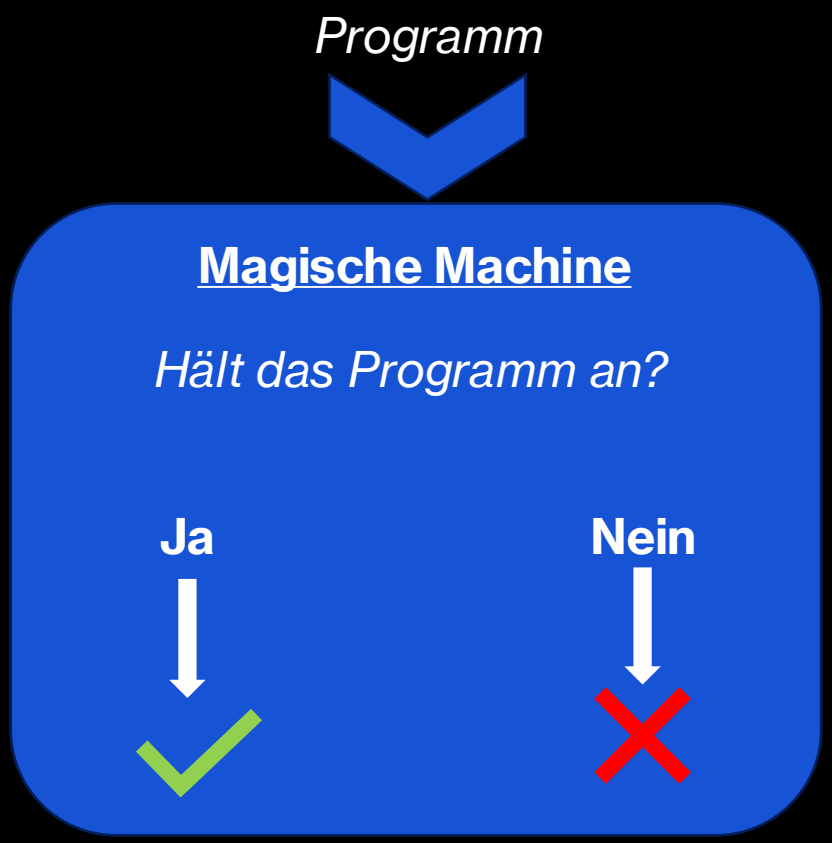
- Zeigt Grenzen der Turing-Maschine auf
- Frage: Hält ein Programm für jede Eingabe irgendwann an?
- Ergebnis: **Kann prinzipiell nicht für alle Programme entschieden werden**
- Verdeutlicht: Computer haben fundamentale Grenzen



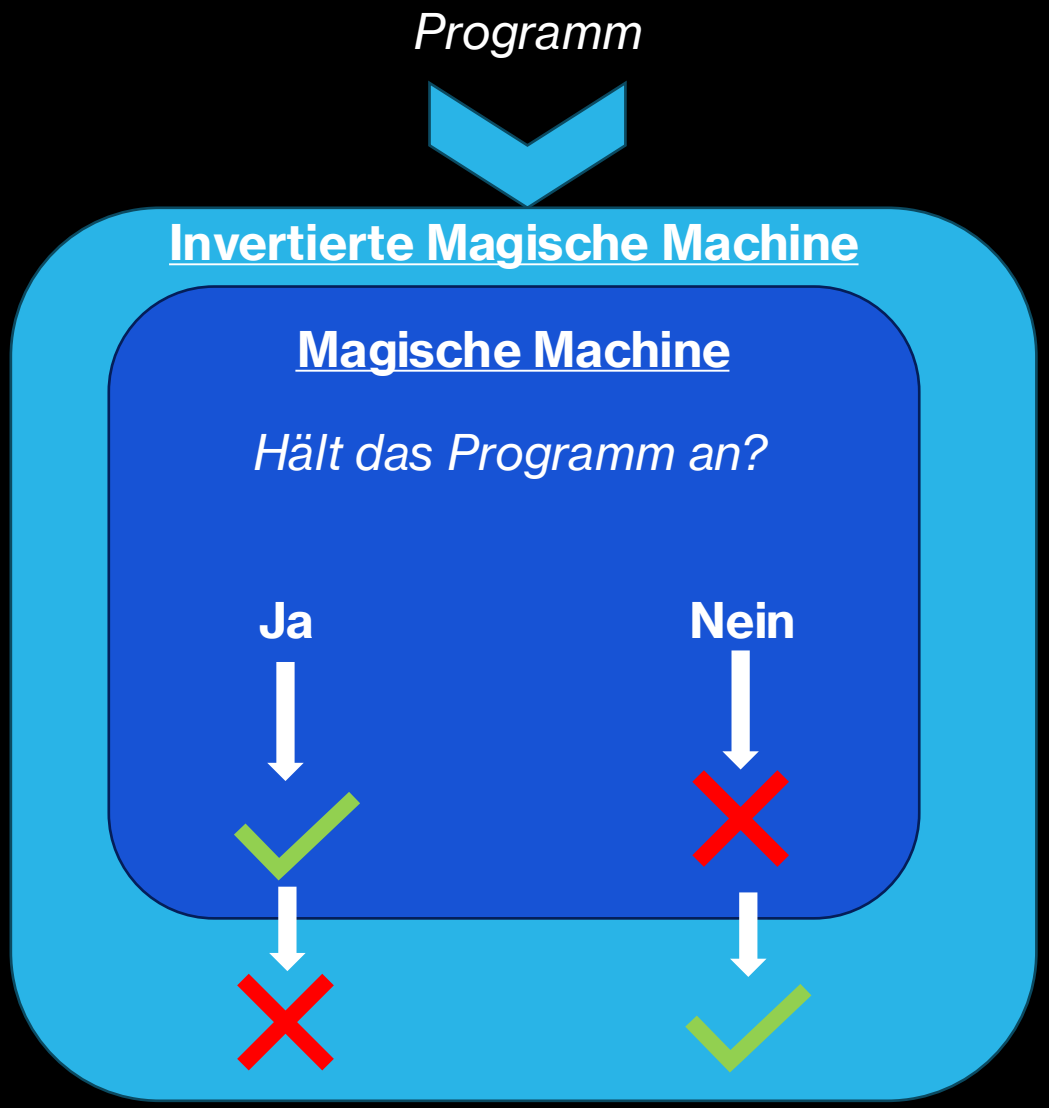
Video zur Verdeutlichung



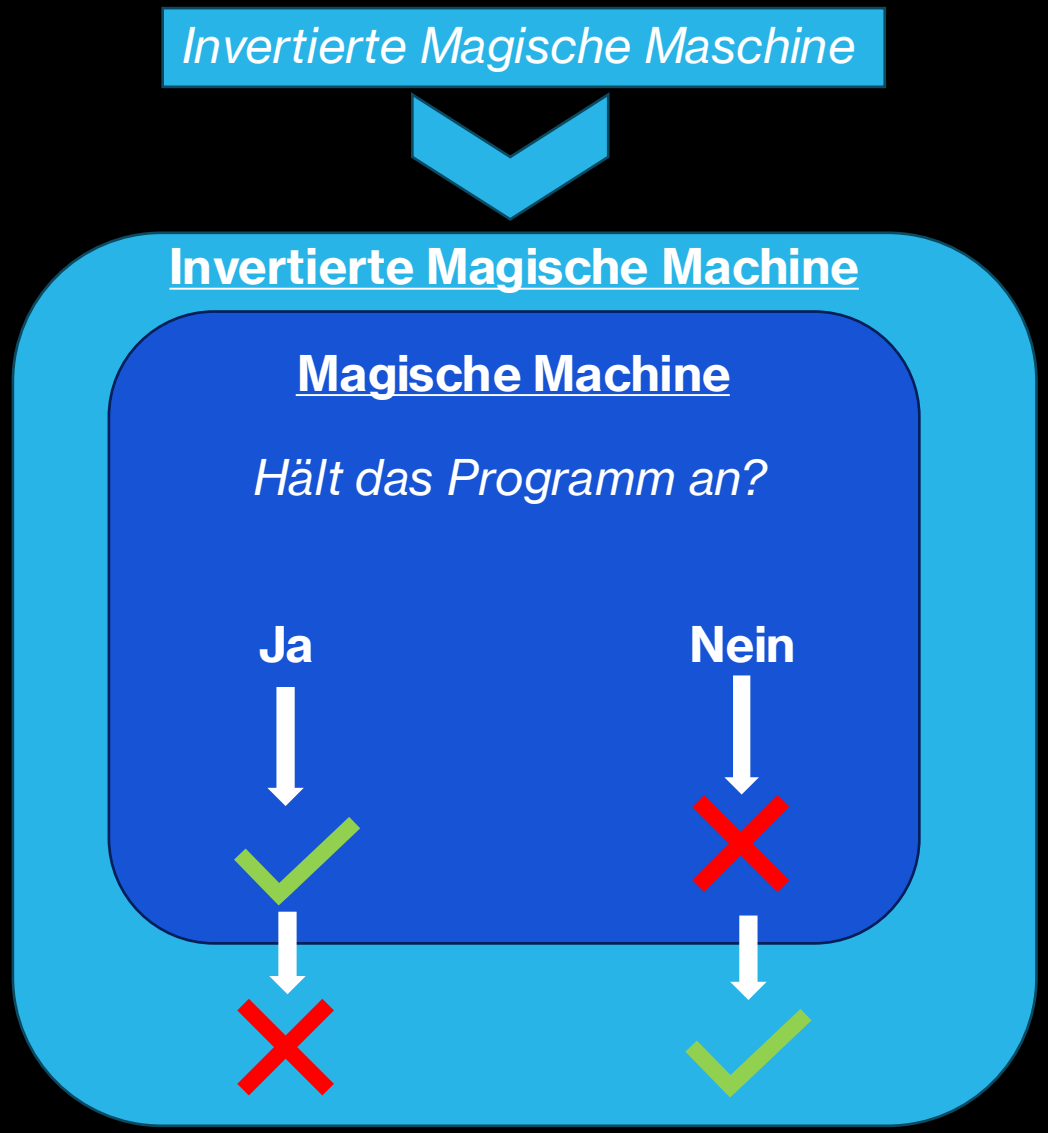
Link



Ausgabe ist nachvollziehbar (deterministisch)



Ausgabe ist nachvollziehbar (deterministisch)



Ausgabe ist nicht nachvollziehbar
(nicht deterministisch)

Die Bedeutung der Turing-Maschine heute

Grundlage für die Analyse von Algorithmen

- z. B. Laufzeit (Big-O) und Speicherbedarf

Komplexitätsklassen

- P, NP, NP-vollständig – Einteilung nach Schwierigkeit

Kryptographie & Sicherheit

- Unlösbare/zu schwere Probleme sichern Verschlüsselung

Verifikation & Compilerbau

- Zustände und Übergänge helfen bei Programmkorrektheit und Übersetzung

Theoretische Grenzen

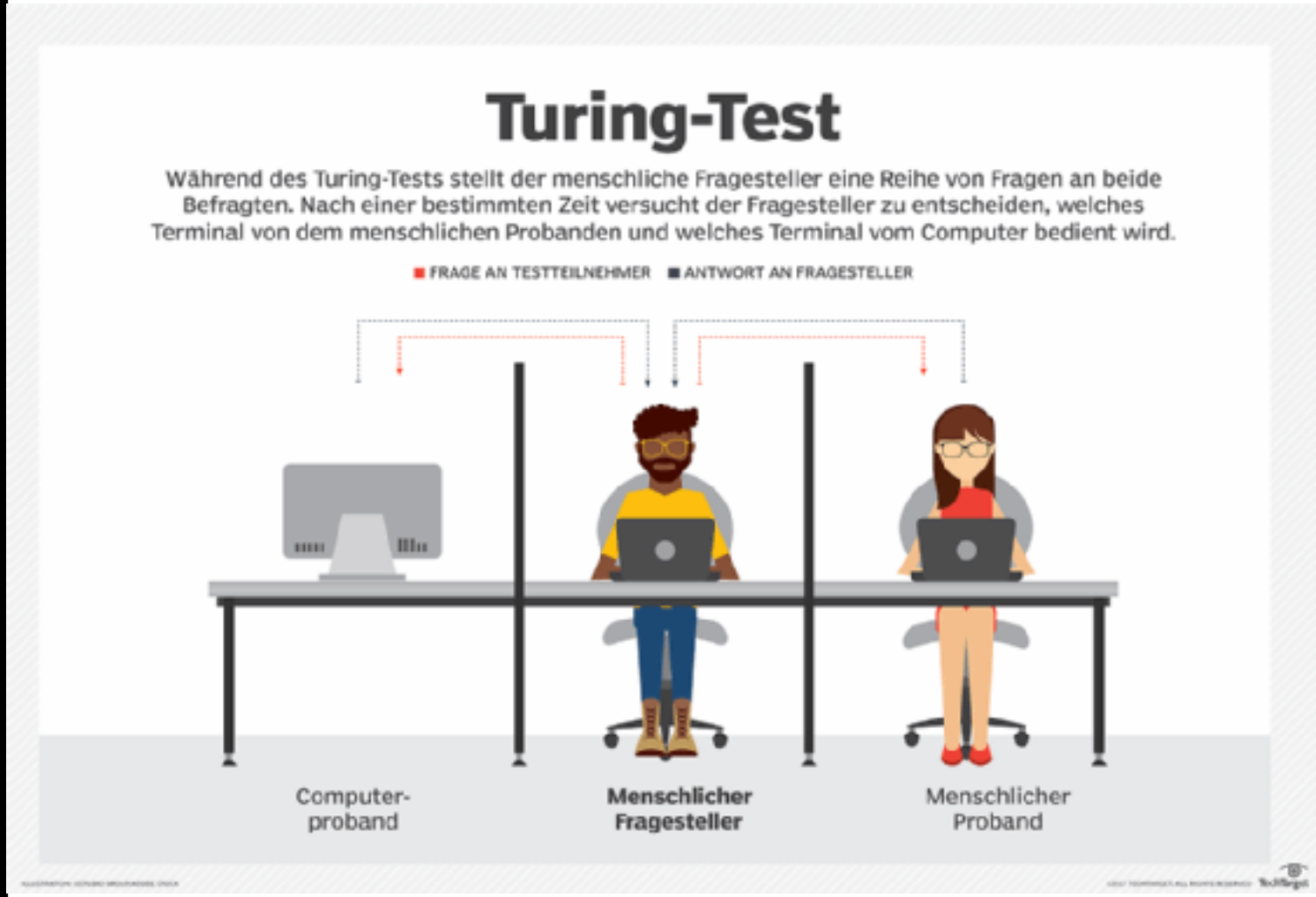
- Halteproblem: zeigt, dass manche Probleme prinzipiell unlösbar sind

Bezug zur Künstlichen Intelligenz

- Alles Simulierbare passt ins Turing-Modell,
- *„Intelligenz“ wird aber mit dem Turing-Test bewertet*

Der Turing-Test

- **Ziel:** Prüfen, ob eine Maschine *wie ein Mensch denken kann*
- Test: Ein Mensch unterhält sich mit Mensch und Maschine
- Kann der Mensch die Maschine nicht erkennen → Turing-Test bestanden
- Wichtig für Künstliche Intelligenz, nicht Berechenbarkeit

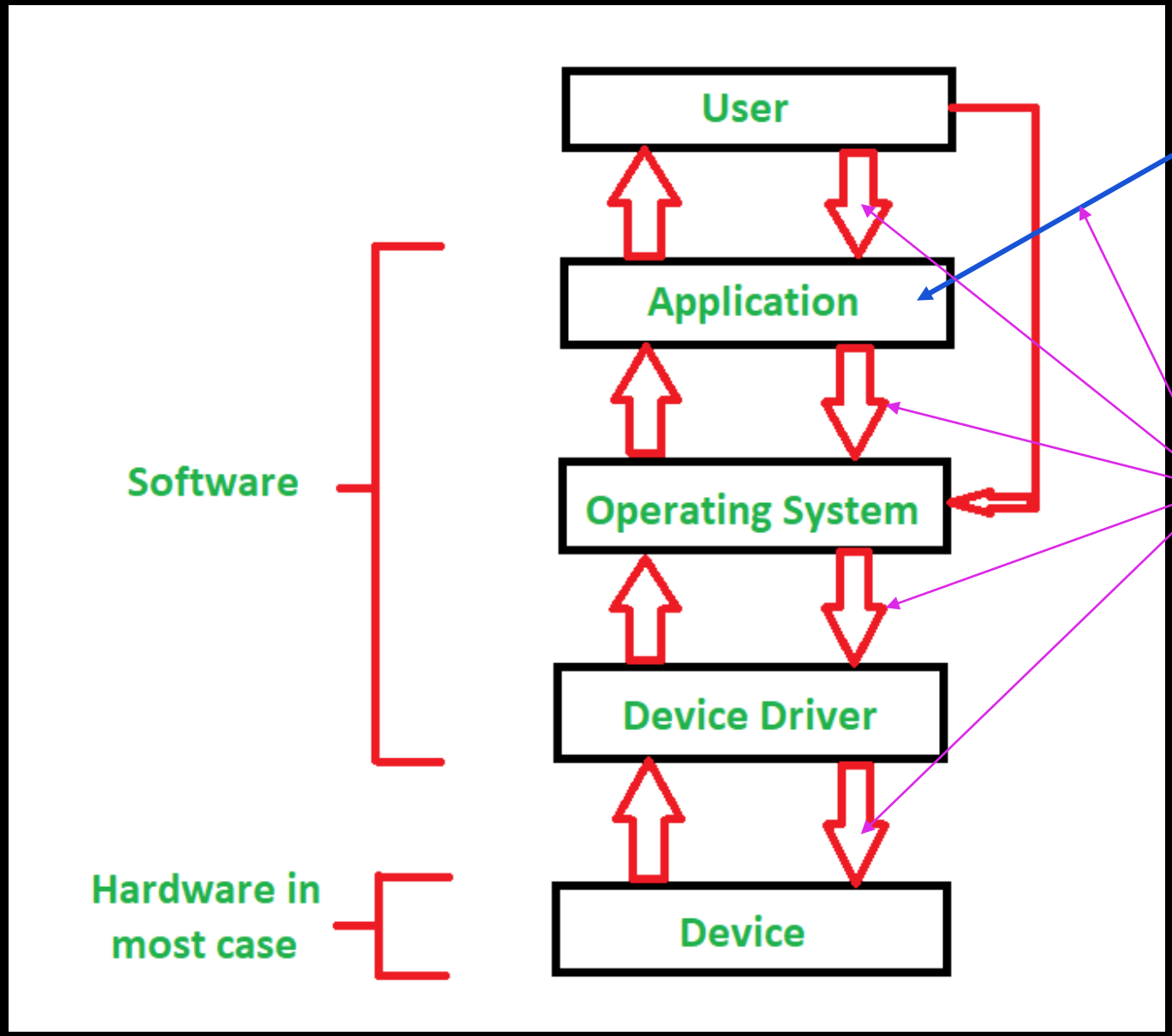


Diskussion

10 Minuten

- **Grundidee der Turing-Maschine**
  - Erklären Sie in eigenen Worten, was eine Turing-Maschine ist und warum sie für die Informatik so wichtig ist.
- **Vergleich**
  - Worin unterscheidet sich die Turing-Maschine vom Turing-Test?
- **Beispielaufgabe**
  - Angenommen, Sie haben ein Programm, das alle geraden Zahlen ausgibt und dann stoppt.
  - → Würde dieses Programm halten? Begründen Sie.
- **Halteproblem – Verständnis**
  - Warum ist es unmöglich, ein allgemeines Programm zu schreiben, das für *alle* Programme korrekt entscheidet, ob sie halten oder nicht?
- **Diskussion**
  - Überlegen Sie: Warum ist die Idee des Halteproblems heute noch für Informatik und KI wichtig?
  - Nennen Sie ein Beispiel aus der Praxis (z. B. Sicherheit, Algorithmus-Analyse, KI).

8.2 APIs


User
Application
Application
Operating System
Device Driver
Device
Software
Hardware in most case
Application
Wie sprechen diese Schichten miteinander?

Was ist eine API?



Kunde
(Entwickler)

Kellner
(API)

Küche
(Programm)

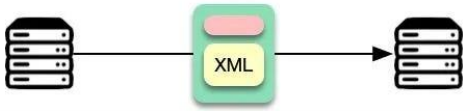
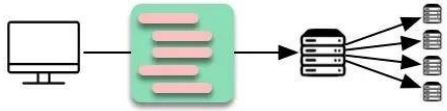
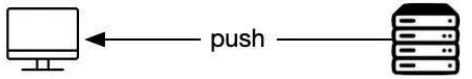
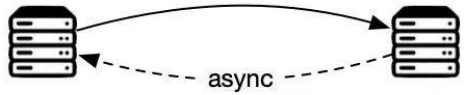
Was ist eine API (Application Programming Interface)?

- **Definition:** Sammlung von Regeln und Protokollen, die Softwareanwendungen die Kommunikation und den Datenaustausch ermöglichen.
- **Analogie:** "Servicevertrag" zwischen zwei Anwendungen.
- **Hauptzweck:**
  - **Modularisierung** von Software.
  - Ermöglicht **Datenaustausch** (oft in Echtzeit).
  - **Vereinfacht Programmierung** für Entwickler.
  - Entscheidend für die **Integration** von Drittprogrammen.
- **Zugriff auf Komponenten:** Datenbanken, Hardware (indirekt), Oberflächen, Anwendungsfunktionen.
- **Dokumentation:** Wichtig für die Nutzung.
- **Bedeutung:** Enabler der "Komponierbarkeit" von Software (Microservices).
- **Wirtschaftlicher Faktor:** Schafft Innovationsmöglichkeiten, Datenmonetarisierung (z.B. Google Maps API).

API-Typen

- Jede Schicht hat spezialisierte APIs:
- **Hardware-Ebene**
  - Device APIs (z. B. GPU-Treiber, Sensor-APIs)
  - Firmware/Embedded APIs
- **Betriebssystem-Ebene**
  - System-APIs (z. B. Windows API, POSIX)
  - Library-APIs (z. B. libc, DLLs, Frameworks)
- **Middleware/Plattform-Ebene**
  - Runtime APIs (z. B. Java API, .NET API)
  - Cloud/Service APIs (z. B. Kubernetes API, DB-APIs)
- **Web-Ebene**
  - REST APIs
  - GraphQL
  - SOAP
  - WebSocket APIs

Top 6 Most Popular API Architecture Styles — ByteByteGo.com

Style	Illustration	Use Cases
SOAP		XML-based for enterprise applications
RESTful		Resource-based for web servers
GraphQL		Query language reduce network load
gRPC		High performance for microservices
WebSocket		Bi-directional for low-latency data exchange
Webhook		Asynchronous for event-driven application

API-Beispiele aus dem Alltag

- **Google Maps API:** Einbindung von Kartenfunktionen, Routenplanung in Websites/Apps.
- **Wetter-APIs (z.B. OpenWeatherMap API):** Abruf aktueller Wetterdaten für eigene Anwendungen.
- **PayPal API:** Integration von Zahlungsabwicklung in Online-Shops.
- **DHL API:** Integration von Sendungsverfolgung oder Versandkostenberechnung.









Übung

- Abrufen von Wetter Daten mittels API (via open-meteo)

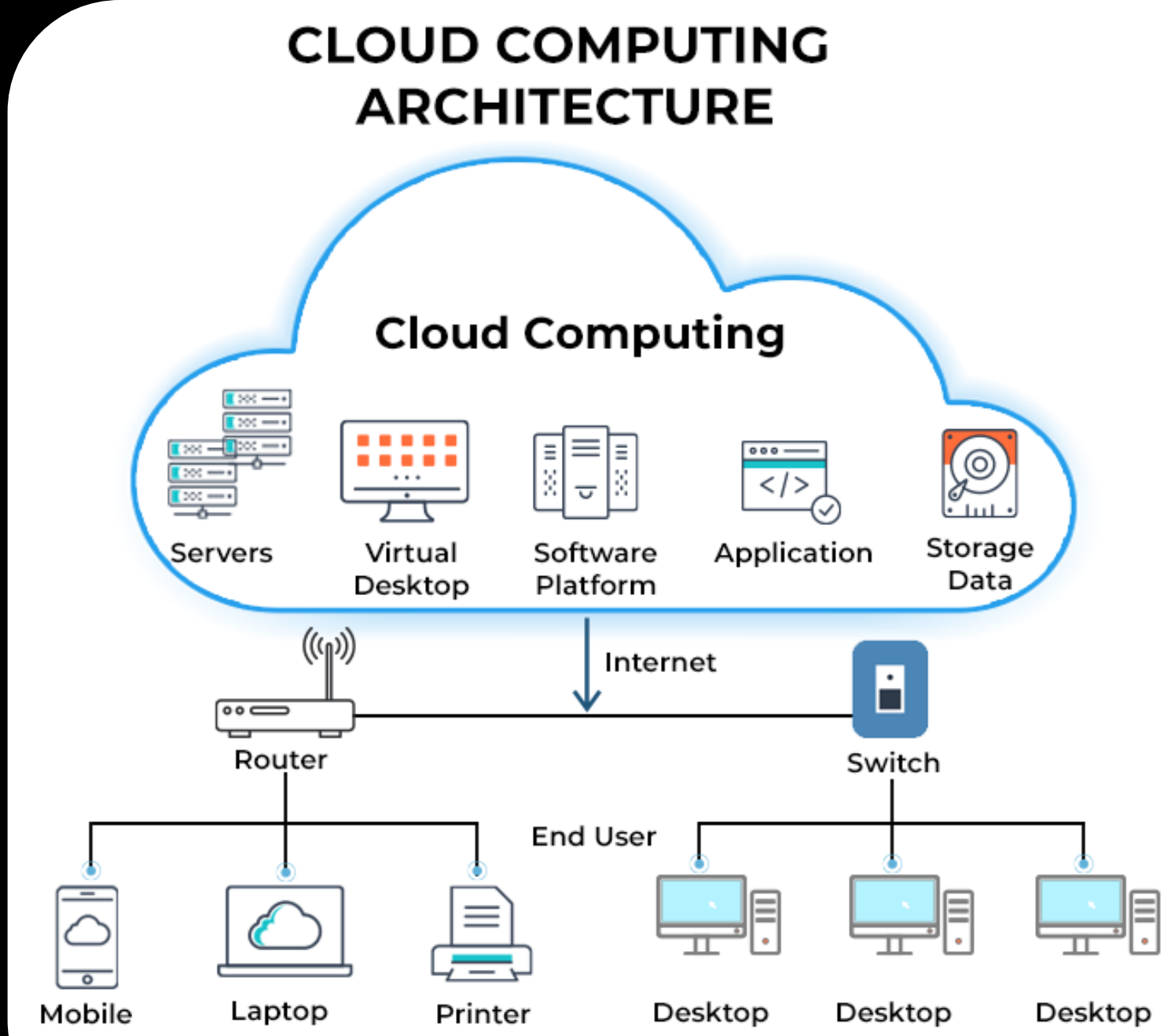
- Code: https://github.com/rummens/th-bingen-python-intro/blob/main/Beispiele/api_weather_example.py

5 Minuten

8.3 Cloud

Was bedeutet „Cloud“?

- „Cloud“ ist die Abkürzung für **Cloud Computing**.
- Gemeint ist: IT-Ressourcen (z. B. Rechenleistung, Speicher, Programme) werden **nicht** mehr **lokal** auf einem eigenen Computer oder Server betrieben, sondern über das **Internet bereitgestellt**.
- Ein anschauliches Bild: Früher musste man seinen eigenen Stromgenerator haben, heute bezieht man Strom aus dem Netz. Genauso ist es bei IT: Man muss keine eigenen Server besitzen, sondern nutzt sie „aus der Wolke“.



Centorrino Technologies

The history of Cloud

Beyond the physical world

1967
IBM introduces a virtualised operating system.

1985
Data storage tapes become commercial available.

The dawn of the internet

1991
The World Wide Web is launched.

1994
The word "cloud" makes it debut.

Cloud is commercialised

1999
The birth of Software as a Service (SaaS)

2006
Beginning of Infrastructure as a service (SaaS)

Computing power could be accessed on demand.

Birth of the hyperscalers

2007
The birth of productivity tools.
Microsoft and Google launch productivity and collaboration tools, creating the first Software as a Service models.

2010
Microsoft Azure enteres the IaaS cloud market.

Hybrid cloud emerges

2010s
Containers and kubernetes enable businesses to develop, run and move applications fromone cloud to another.

2012
The first hybrid cloud management platform is launched.

The hybrid work, AI and IoT era

2020
Remote working and learning becomes the norm

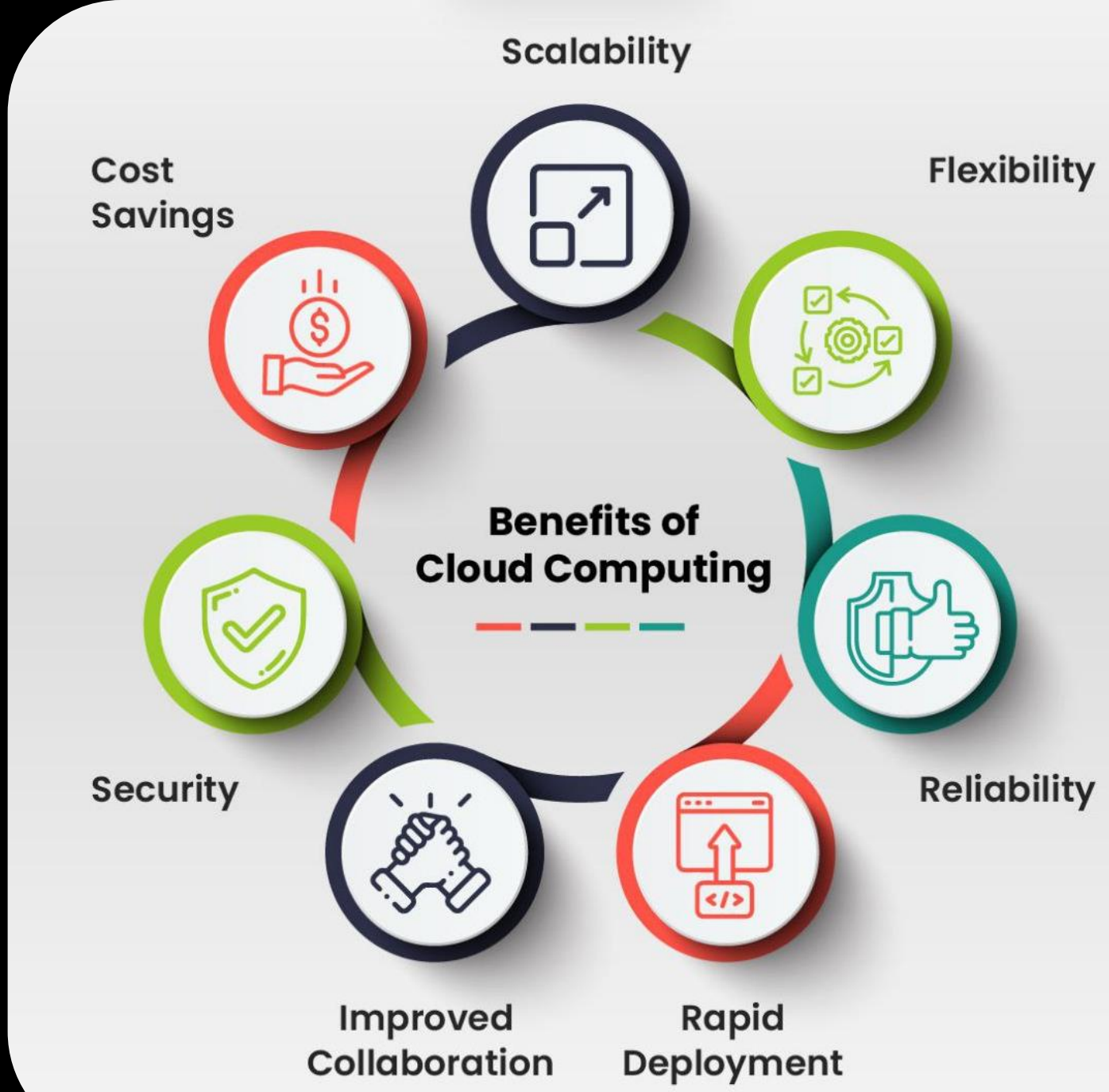
Cloud spending increases by 35%.

2022 to present
ChatGPT and generative AI emerges.

Edge computing and Internet of things (IoT) advances.

Warum war die Cloud so erfolgreich?

- **Kostenvorteile:** Unternehmen müssen keine teure Hardware mehr kaufen und warten.
- **Flexibilität:** Man kann Rechenleistung und Speicher innerhalb von Minuten hoch- oder runterskalieren („wie ein Gashahn aufdrehen“).
- **Globale Verfügbarkeit:** Dienste sind weltweit erreichbar, unabhängig vom Standort.
- **Innovationstempo:** Neue Technologien (z. B. KI, Big Data) sind sofort nutzbar, ohne dass man selbst Spezial-Hardware kaufen muss.
- **Sicherheit & Wartung:** Cloud-Anbieter übernehmen Updates, Backups und Schutz vor Ausfällen – Aufgaben, die für viele Unternehmen allein kaum zu stemmen wären.





Cloud Computing – Nachteile & Herausforderungen

- **Abhängigkeit vom Anbieter (Vendor Lock-in)**
  - Wechsel schwierig & teuer durch proprietäre Systeme
- **Datenschutz & Sicherheit**
  - Daten liegen oft außerhalb der eigenen Kontrolle, Gesetze variieren
- **Kostenfallen**
  - Laufende Mietkosten können langfristig höher sein als eigene Hardware
- **Internetabhängigkeit**
  - Ohne stabile Verbindung eingeschränkt nutzbar
- **Technische & organisatorische Hürden**
  - Aufwändige Migration
  - Schulungsbedarf für Mitarbeitende
  - Rechtliche Anforderungen (z. B. Datenhaltung in der EU)

Cloud im Alltag

- **Privatnutzer:** Fotos in Google Fotos oder iCloud, Streaming über Netflix oder Spotify – alles Cloud.
- **Industrie/Verfahrenstechnik:** Prozessdaten aus Sensoren werden in Echtzeit in die Cloud geladen und dort analysiert. So lassen sich Produktionsprozesse optimieren oder Ausfälle frühzeitig erkennen.







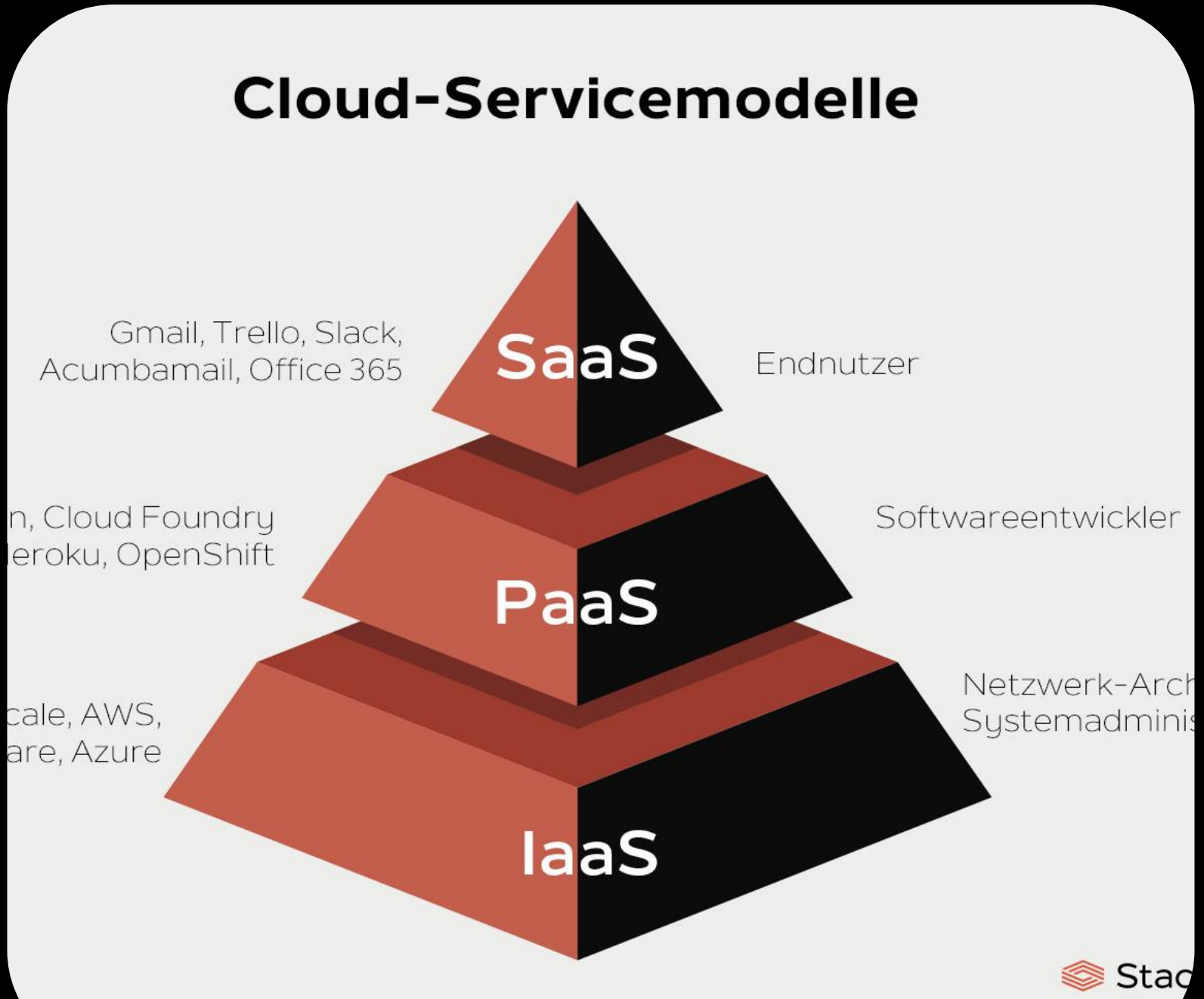
Google Photos



Cloud-Dienstmodelle

- **Unterscheidung:** Grad der Kontrolle und des Managements durch den Nutzer
- **Infrastructure as a Service (IaaS):**
  - **Was:** Grundlegende IT-Infrastruktur (virtuelle Server, Speicher, Netzwerke)
  - **Kontrolle:** Höchste Kontrolle über OS, Anwendungen, Middleware
  - **Analogie:** Miete eines leeren Grundstücks mit Anschlüssen
  - **Beispiel:** Azure Virtual Machines, Google Compute Engine
- **Platform as a Service (PaaS):**
  - **Was:** Plattform für Entwicklung, Ausführung, Verwaltung von Anwendungen (inkl. OS, Laufzeiten, Datenbanken)
  - **Kontrolle:** Fokus auf Anwendungsentwicklung, Anbieter verwaltet Infrastruktur
  - **Analogie:** Miete einer Wohnung mit Grundinstallationen
  - **Beispiel:** Google App Engine, AWS Elastic Beanstalk
- **Software as a Service (SaaS):**
  - **Was:** Komplette Anwendungssoftware über das Internet, vom Anbieter gehostet und verwaltet
  - **Kontrolle:** Nutzer greift auf fertige Anwendung zu, Anbieter kümmert sich um alles
  - **Analogie:** Wohnen im Hotel
  - **Beispiel:** Microsoft 365, Salesforce, Google Docs
- **Bedeutung:** Ausdruck unterschiedlicher "Abstraktions-Kompromisse", strategische Entscheidung für Unternehmen

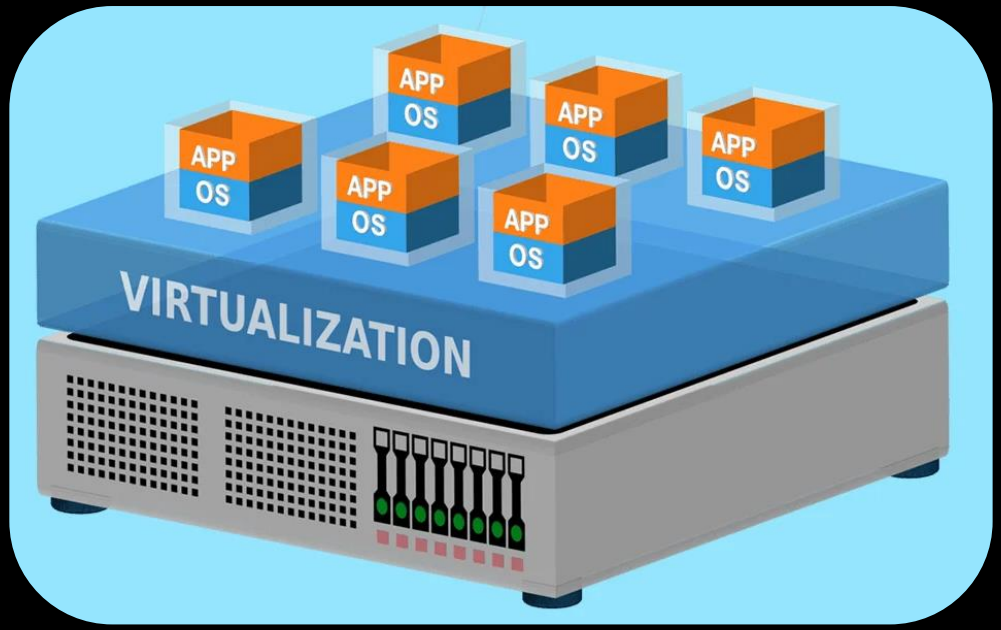
Cloud-Servicemodelle



Weniger Kontrolle

Mehr Kontrolle

Virtuelle Maschine – Die Cash Cow in Cloud



- **Definition:** Softwarebasierter Computer, verhält sich wie physischer Computer
  - Führt eigenes Betriebssystem und eigene Anwendungen aus
  - Läuft auf einem physischen Host-Server
- **Funktionsweise:** Basiert auf einem **Hypervisor**
  - Hypervisor teilt physische Ressourcen des Host-Servers auf VMs auf
- **Wesentliches Merkmal: Isolation** – jede VM ist vom Hostsystem und anderen VMs isoliert
- **Beziehung zur abstrakten Maschine:** Eine VM ist eine konkrete Implementierung einer abstrakten Maschine
- **Bedeutung:** Ermöglicht Betrieb verschiedener OS gleichzeitig auf einer Maschine
- **Relevanz:** Serverkonsolidierung (reduziert Kosten, Energie, Wartung), erhöhte Sicherheit, schnelle Wiederherstellung

Vorteile und Anwendungsfälle von VMs

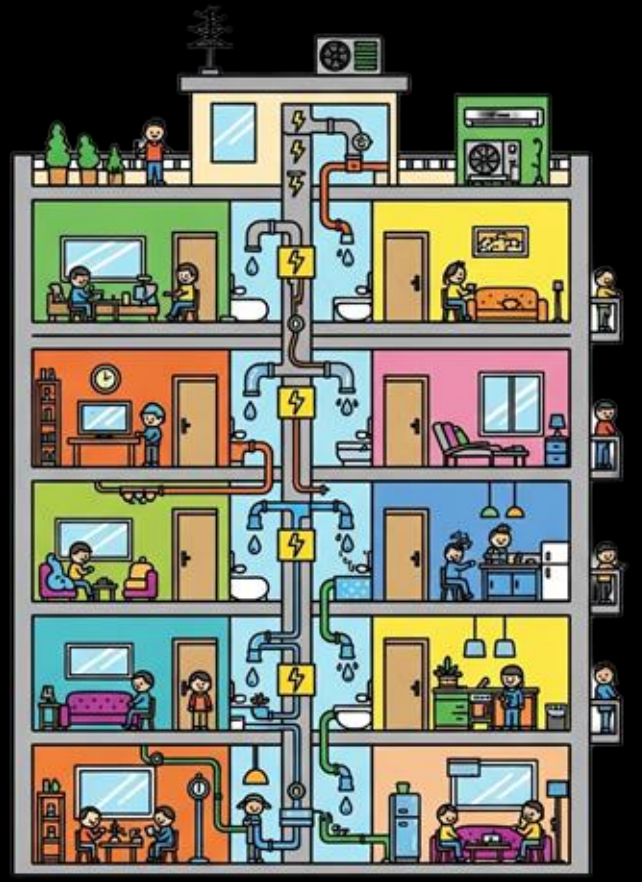
- **Vorteile:**
  - **Ressourcenoptimierung:** Effiziente Nutzung physischer Server
  - **Isolation & Sicherheit:** Probleme in einer VM beeinträchtigen andere nicht; sichere Malware-Untersuchung
  - **Flexibilität & Portabilität:** Leicht zwischen Hypervisoren/Maschinen verschiebbar
  - **Reduzierte Ausfallzeiten:** Einfache Backups und schnelle Wiederherstellung
  - **Skalierbarkeit:** Anwendungen leicht skalierbar
- **Anwendungsfälle:**
  - Softwareentwicklung und -tests (isolierte Umgebungen)
  - Cloud Computing (skalierbares Ressourcenmanagement)
  - Serverkonsolidierung (Kostenreduktion)
  - Ausführung von Legacy-Anwendungen
  - Bildung und Ausbildung

How can I imagine a VM and a Container?

Virtual Machines (VMs): Your Own Private House

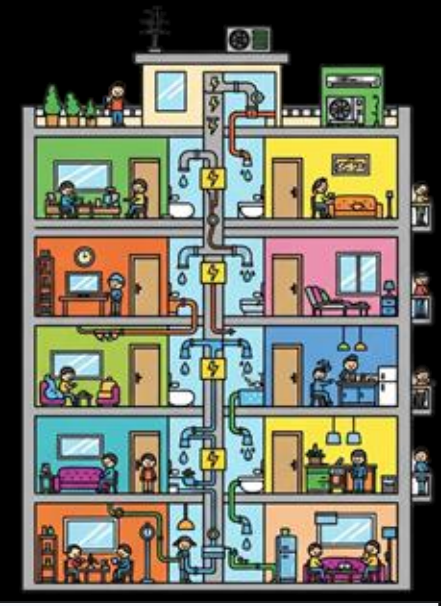


Containers: An Apartment in a Modern Building



What is the difference between a VM and a Container?





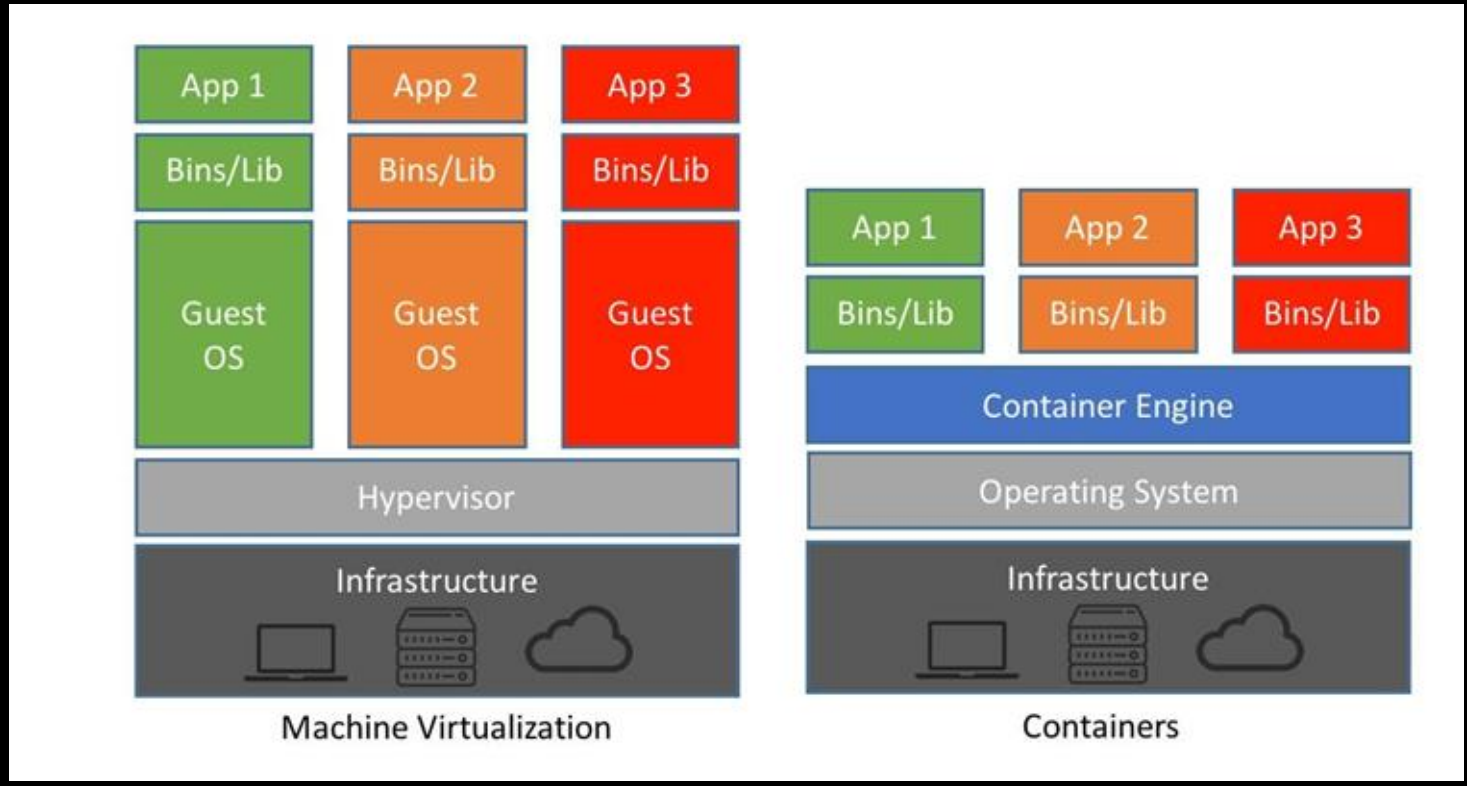
Concept	Each VM is a **complete, independent house** with its own foundation and utilities (OS).	Containers are **apartments** within a building, sharing core structure (host OS kernel).
Definition	**Full digital copies of a computer system**, each running its own operating system.	**Lightweight, portable packages** that run applications by leveraging the host system's resources and shared OS.
Startup Speed	**Minutes** (like building/booting up an entire house).	**Seconds** (like moving into an already built apartment).
Resource Use	**Higher** (each "house" has its own dedicated utilities).	**Lower** (by sharing the building's core utilities).
Flexibility	Can build each "house" with **any operating system**, offering maximum isolation.	While sharing the building's foundation, apartments run efficiently on the **same platform**.
Best For	**Legacy applications**, highly **secure, regulated workloads**, or complete separation.	**Modern applications**, quick **scaling**, and fostering **agility** in development.

What are Containers from a technical point of view?

Lightweight, **portable** units of software

... that package an application **along with its dependencies** and runtime environment

... enabling **consistency** across different environments.



Vorteile und Anwendungsfälle von Containern

- **Vorteile:**
  - **Ressourceneffizienz:** Weniger Overhead als VMs, da kein eigenes Betriebssystem pro Container benötigt wird.
  - **Schneller Start:** Container starten in Sekunden, ideal für dynamische Workloads.
  - **Portabilität:** „Läuft überall“ – Container enthalten Anwendung + Abhängigkeiten.
  - **Konsistenz:** Gleiche Umgebung von Entwicklung bis Produktion → weniger „läuft bei mir“-Probleme.
  - **Skalierbarkeit & Automatisierung:** Perfekt kombinierbar mit Orchestrierung (z. B. Kubernetes).
- **Anwendungsfälle:**
  - Microservices-Architekturen (Anwendungen in kleine, flexible Dienste zerlegen).
  - DevOps & CI/CD (schnelle, wiederholbare Software-Builds & Deployments).
  - Cloud-native Anwendungen (optimal für Cloud-Umgebungen entwickelt).
  - Hybride Multi-Cloud-Szenarien (einheitliche Plattform über verschiedene Clouds hinweg).
  - Testumgebungen & Experimentieren (schnell neue Versionen ausprobieren).

Zusammenfassung & Ausblick

- **Kernbotschaften**
  - **Abstraktion:** Schlüsselkonzept zur Komplexitätsbeherrschung und Modularisierung.
  - **Zusammenspiel:** Hardware, Betriebssystem (OS), Compiler und Treiber arbeiten hierarchisch zusammen.
  - **APIs:** Standardisierte Kommunikationswege für Software-Interaktion und Dienstleistungsaustausch.
  - **VMs, Cloud, Container:** Moderne Infrastrukturkonzepte für flexible und kosteneffiziente Softwarebereitstellung und -verwaltung.
- **Relevanz für Verfahrenstechnik**
  - **Automatisierung & Prozessleitsysteme:** Verständnis für Design, Implementierung und Wartung smarter Anlagen.
  - **Datenanalyse & Big Data:** Verarbeitung großer Sensordatenmengen, oft in der Cloud.
  - **Internet of Things (IoT) in der Industrie:** Vernetzung von Geräten und Anlagen, basierend auf APIs und Containern.
  - **Simulationen & Digitale Zwillinge:** Effiziente Ausführung komplexer Modelle und Entwicklung digitaler Zwillinge.
  - **Sicherheit & Ausfallsicherheit:** Absicherung von Systemen gegen Cyberbedrohungen, Gewährleistung der Ausfallsicherheit kritischer Infrastrukturen.

Quiz

Multiple Choice

1 Minute

Was ist die Hauptaufgabe eines Betriebssystems?

a) Den Quellcode in Maschinencode übersetzen.

b) Die Hardware-Ressourcen verwalten und Programme überwachen.

c) Eine Website im Internet anzeigen.

d) Mathematische Berechnungen durchführen.

Welches Merkmal trifft am ehesten auf einen Container zu im Vergleich zu einer Virtuellen Maschine?

a) Er virtualisiert die Hardware-Ebene vollständig.

b) Er benötigt ein eigenes, vollständiges Betriebssystem.

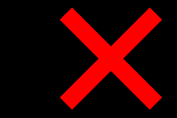
c) Er teilt sich den Kernel des Host-Betriebssystems.

d) Er ist primär für Legacy-Anwendungen gedacht.

Quiz

Wahr/Falsch

1 Minute

- APIs werden nur für die Kommunikation zwischen Hardware und Betriebssystem genutzt. 
- Container starten in der Regel schneller als virtuelle Maschinen. 
- Cloud Computing bedeutet, dass IT-Ressourcen ausschließlich im eigenen Rechenzentrum betrieben werden. 
- Eine Turing-Maschine ist ein theoretisches Modell, das beschreibt, was Computer grundsätzlich berechnen können. 

Diskussion

- Nennen Sie zwei Vorteile der Nutzung einer **API**
- Wie nennt man die Software, die dem Betriebssystem hilft, mit einer spezifischen Hardware-Komponente zu kommunizieren?
- Was ist der Unterschied zwischen IaaS, PaaS und SaaS Lösungen?
- Wo könnte man Cloud Technologien einsetzen?

5 Minuten

9. Kryptographie und Blockchain

Was ist Kryptographie?

- **Kryptographie:** Wissenschaft des Schutzes digitaler Informationen in unkontrollierten Umgebungen (z.B. Internet)
- **Hauptziele der Kryptographie:**
  - **Vertraulichkeit:** Nur autorisierte Personen können Daten lesen.
  - **Integrität:** Daten bleiben vollständig und unverändert (keine Manipulation).
  - **Authentizität:** Eindeutige Herkunft von Daten oder Identität des Absenders.
- **Relevanz für Verfahrenstechniker:**
  - Studium basiert auf Naturwissenschaften, nicht primär Informatik.
  - Kryptographie ist entscheidend für die Sicherheit industrieller Systeme.
  - **Anwendungsbereiche in der Verfahrenstechnik:**
    - **Datenintegrität in Prozessleitsystemen:** Schutz vor Manipulation von Sensordaten, Steuerbefehlen in IACS (z.B. chemische Anlagen).
    - **Sichere Lieferketten:** Manipulationssichere Rückverfolgbarkeit von Rohstoffen und Produkten (Chemie, Pharma).
    - **Schutz geistigen Eigentums:** Nachweis von Urheberschaft und Unveränderlichkeit von Patenten, Rezepturen.
    - **IoT-Sicherheit in der Industrie:** Absicherung vernetzter Maschinen und Sensoren vor Cyberangriffen.

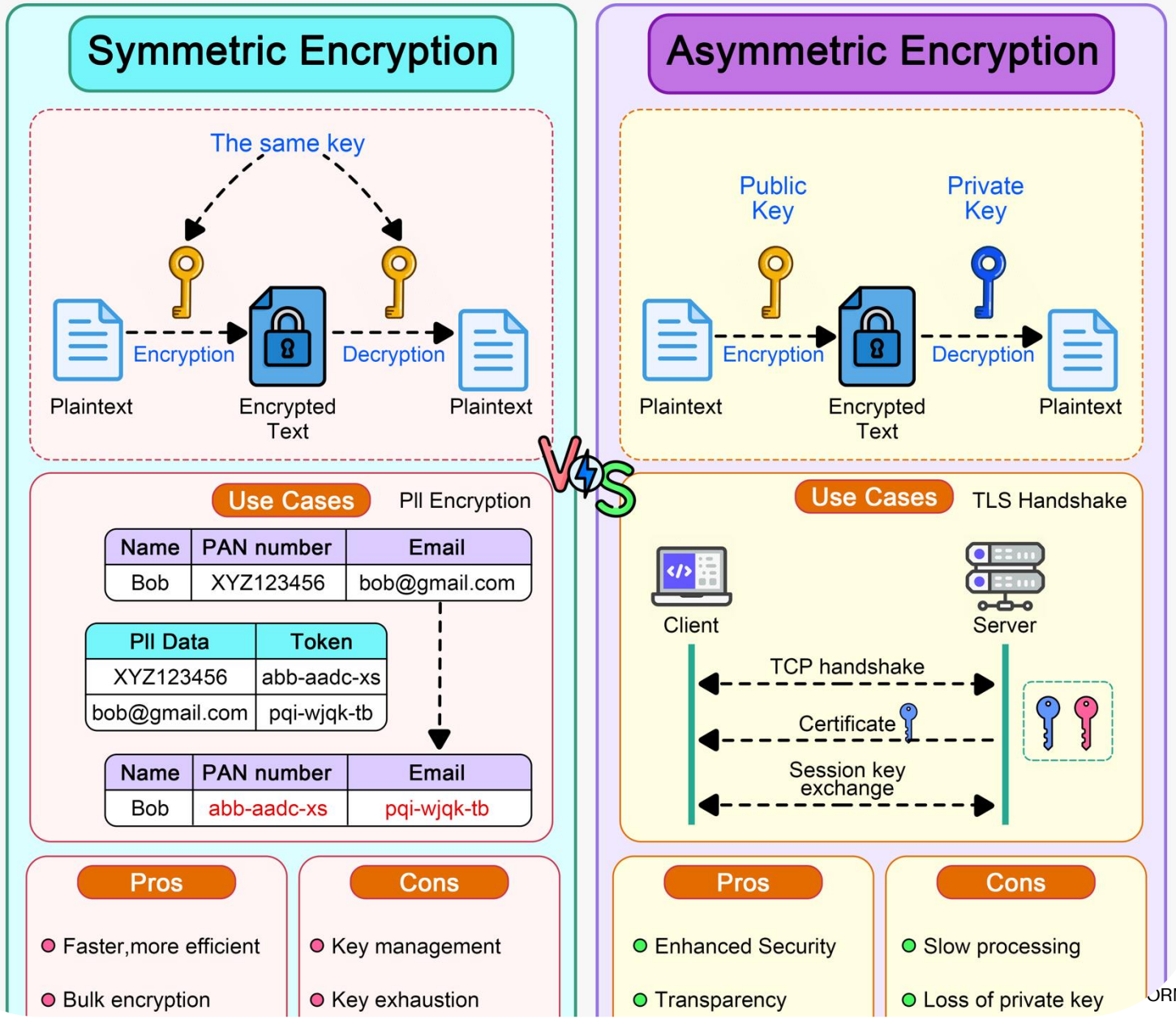
Kryptographische Grundkonzepte (Verschlüsselung)

- **Symmetrische Verschlüsselung**
- **Prinzip:** Ein einziger, geheimer Schlüssel für Ver- und Entschlüsselung.
  - Klartext + Schlüssel + Algorithmus = Chiffretext.
  - Chiffretext + Schlüssel + Algorithmus = Klartext.
- **Vorteile:** Schnell und effizient, ideal für große Datenmengen.
- **Nachteile:** Sichere Schlüsselverteilung ist eine Herausforderung.
- **Beispiele:** AES (Festplatten, WLAN), VPNs, TLS/SSL.

- **Asymmetrische Verschlüsselung (Public-Key-Kryptographie)**
- **Prinzip:** Verwendet ein Schlüsselpaar: öffentlicher und privater Schlüssel.
  - **Öffentlicher Schlüssel:** Kann offen geteilt werden (zum Verschlüsseln).
  - **Privater Schlüssel:** Muss geheim gehalten werden (zum Entschlüsseln).
- **Funktionsweise:**
  - Sender verschlüsselt mit *öffentlichem Schlüssel* des Empfängers.
  - Empfänger entschlüsselt mit eigenem privatem Schlüssel.
- **Vorteile:** Keine vorherige sichere Schlüsselverteilung nötig.
- **Nachteile:** Rechenintensiver und langsamer als symmetrische Verfahren.
- **Anwendungen:** Schlüsselaustausch, digitale Signaturen, sichere E-Mails (PGP, S/MIME), HTTPS.

Sysmmetric vs Asymmetric Encryption

ByteByteGo

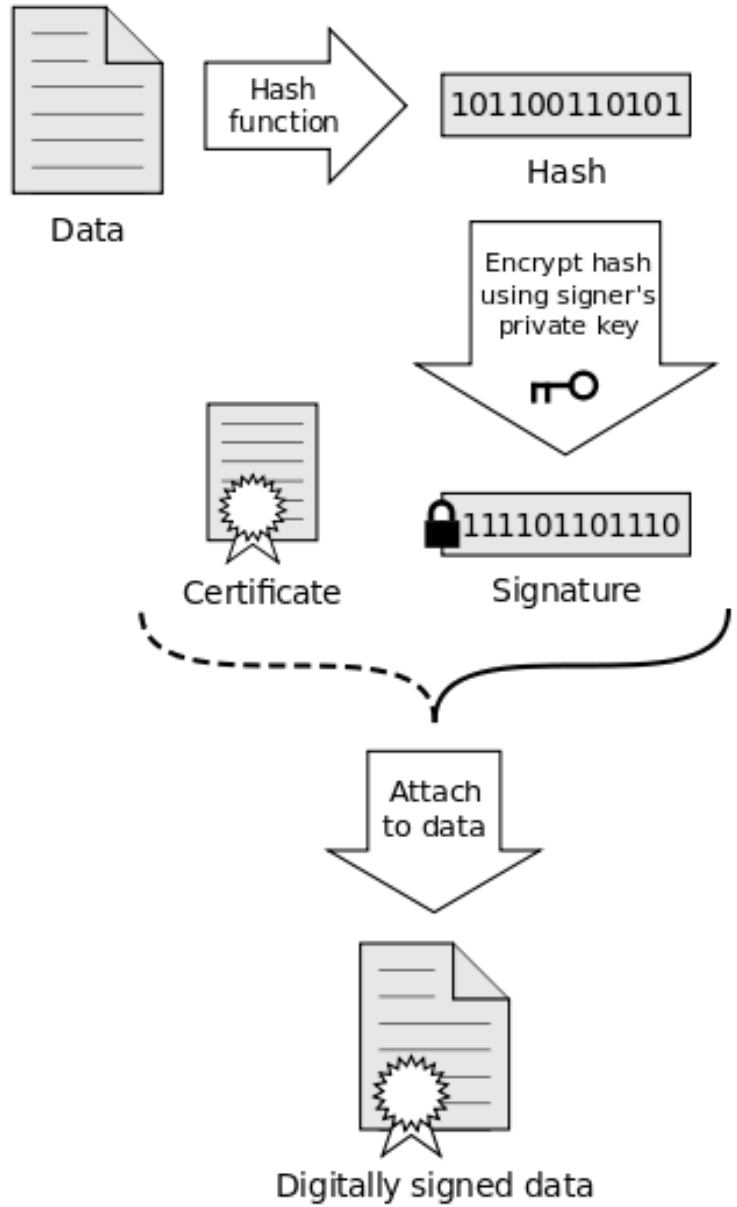


Kryptographische Grundkonzepte (Integritätsprüfung)

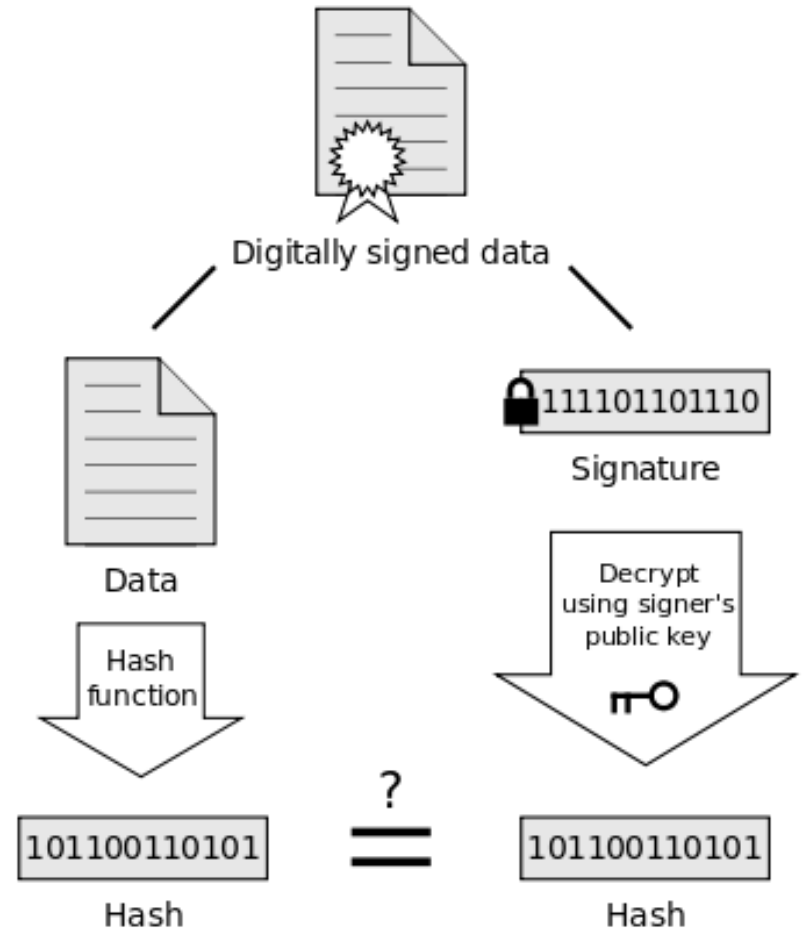
- **Hash-Funktionen**
- **Prinzip:** Wandeln beliebige Daten in einen Hashwert (Fingerabdruck) fester, kurzer Länge um.
- **Wichtige Eigenschaften:**
  - **Einweg-Funktion:** Originalwert kann nicht aus Hashwert rekonstruiert werden.
  - **Kollisionsresistenz:** Extrem schwierig, zwei unterschiedliche Eingaben mit gleichem Hashwert zu finden.
  - **Deterministisch:** Gleiche Eingabe erzeugt immer gleichen Hashwert.
  - **Geschwindigkeit:** Nicht zu schnell, um Brute-Force-Angriffe zu erschweren.
- **Anwendungen:**
  - Verifizierung von Passworteingaben (Speicherung von Hashes, nicht Passwörtern).
  - Kontrolle der Datenintegrität (Vergleich von Hashes vor und nach Übertragung).
  - Identifizierung von Duplikaten.

- **Digitale Signaturen**
- **Prinzip:** Kryptographisches Verfahren basierend auf asymmetrischer Kryptographie.
- **Ziele:**
  + **Authentizität:** Nachricht stammt nachweislich von bestimmtem Absender.
  + **Integrität:** Nachricht wurde nicht nachträglich verändert.
  + **Nichtabstreitbarkeit:** Absender kann Signatur nicht leugnen.
- **Funktionsweise (vereinfacht):**
  + Nachricht wird gehasht (Fingerabdruck).
  + Hashwert wird mit *privatem Schlüssel* des Absenders verschlüsselt = digitale Signatur.
  + Originalnachricht + digitale Signatur werden gesendet.
  + Empfänger berechnet selbst Hash der Nachricht.
  + Empfänger entschlüsselt Signatur mit *öffentlichem Schlüssel* des Absenders.
  + Stimmen beide Hashwerte überein, ist Signatur gültig.
- **Wichtig:** Digitale Signatur gewährleistet *nicht* Vertraulichkeit der Nachricht.
- **Anwendungen:** Rechtlich gültige Dokumente, Software-Updates, E-Mails.

Signing



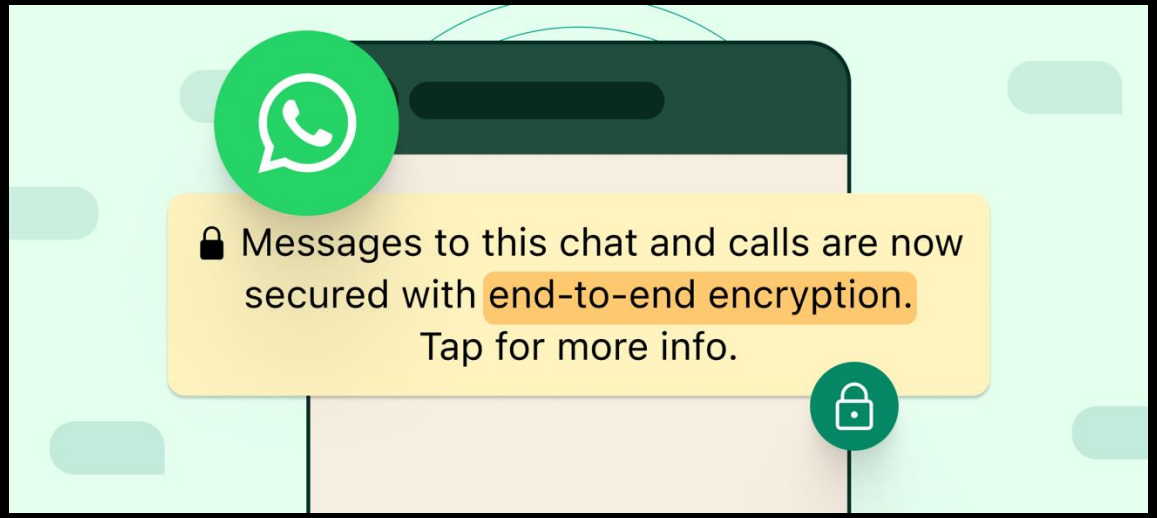
Verification

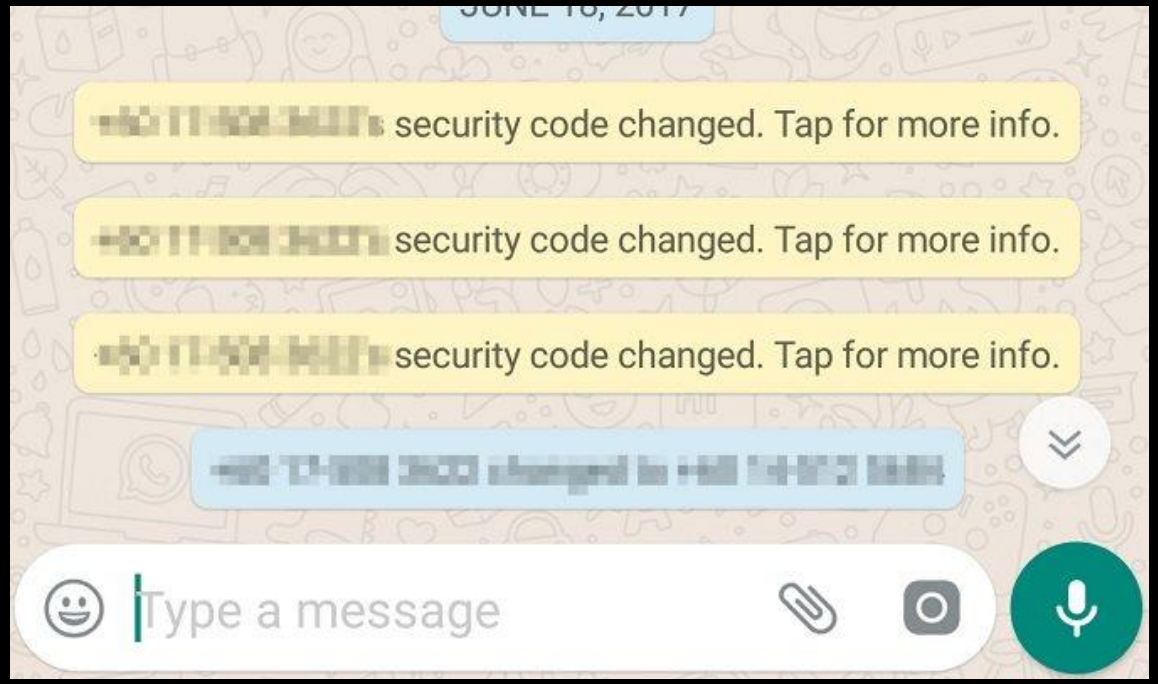


If the hashes are equal, the signature is valid.

9.1 Der RSA-Algorithmus



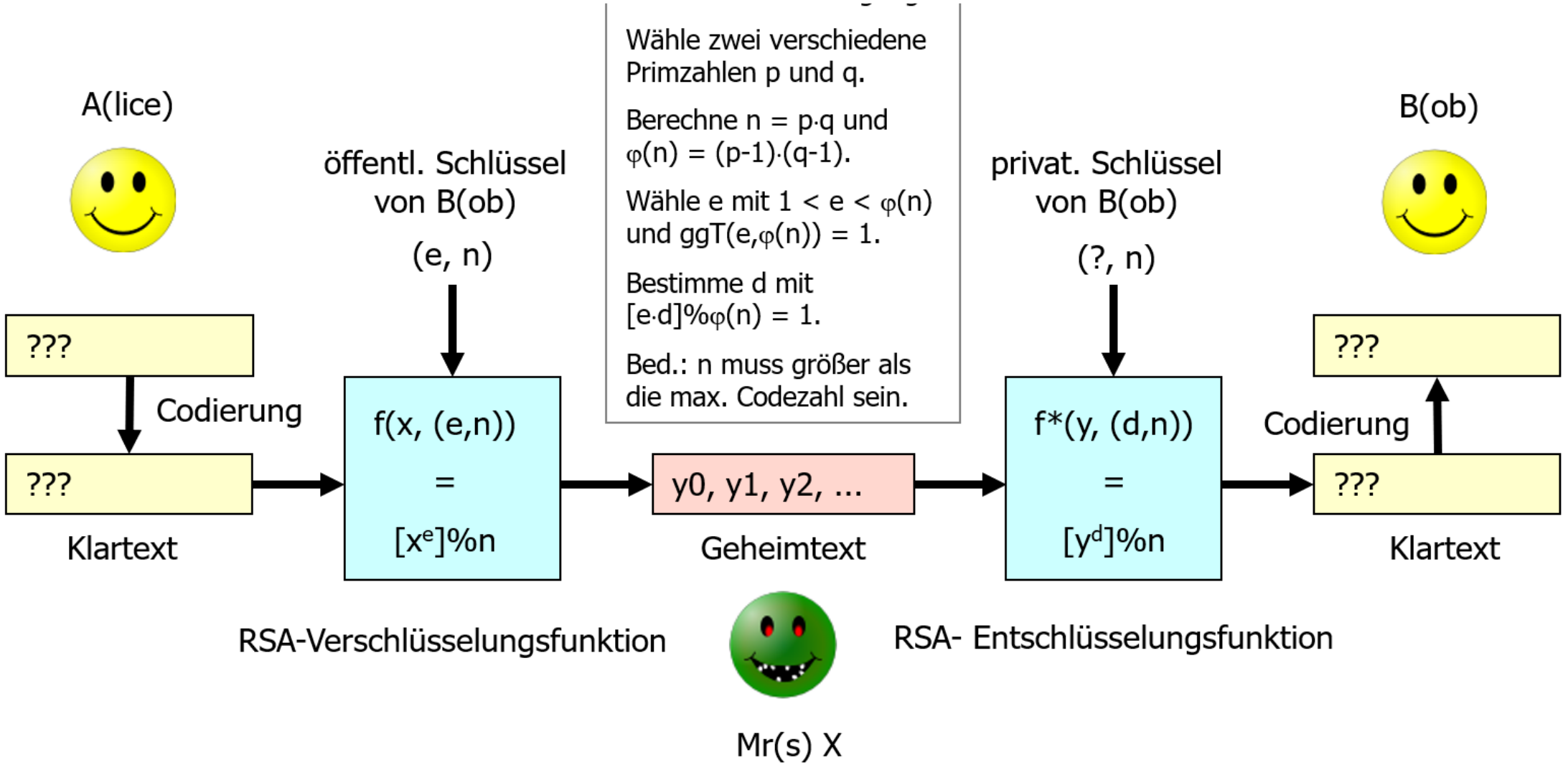




** WhatsApp und Signal benutzen andere Algorithmen. Beispiel dient nur der Verdeutlichung*

Grundprinzip des RSA-Algorithmus

- **Erfinder:** Rivest, Shamir und Adleman (1977).
- **Asymmetrisches Verfahren:** Nutzt öffentliches und privates Schlüsselpaar.
- **Vereinfachte Funktionsweise:**
  - **Alice (Empfänger) erzeugt Schlüssel:** Wählt zwei sehr große Primzahlen p und q. Berechnet n = p * q und m = (p-1) * (q-1). Wählt a (teilerfremd zu m).
  - **Öffentlicher Schlüssel:** (n, a) wird veröffentlicht.
  - **Privater Schlüssel:** b (berechnet aus a und m) wird geheim gehalten.
  - **Bob (Sender) verschlüsselt:** Nachricht x wird mit Alices öffentlichem Schlüssel (n, a) verschlüsselt: y = x^a mod n.
  - **Alice entschlüsselt:** Empfängt y und entschlüsselt mit ihrem privaten Schlüssel b: x = y^b mod n.
- **Eigenschaft:** Nicht monoalphabetisch (keine feste Zuordnung von Zeichen).



https://www.cryptool.org/de/cto/rsa-step-by-step/

Die Bedeutung großer Primzahlen für die Sicherheit von RSA

- **Sicherheitsgrundlage:** Beruht auf der Schwierigkeit, das Produkt zweier sehr großer Primzahlen in seine Faktoren zu zerlegen (Faktorisierungsproblem).
  - Multiplikation ist einfach, Faktorisierung ist extrem aufwendig (Einwegfunktion).
- **Praktische Größe:** Primzahlen sind mehrere Hundert Stellen lang, n typischerweise 2048 oder 4096 Bit.
  - Faktorisierung für klassische Computer rechnerisch unmöglich (Jahre/Jahrhunderte).
- **"Starke Primzahlen":** Müssen bestimmte Eigenschaften aufweisen, um Faktorisierungsmethoden zu erschweren.
- **Konsequenz:** Wenn n in p und q zerlegt werden könnte, wäre RSA kompromittiert.

9.2 Die Zukunft der Kryptographie: Quantencomputer und Post-Quanten-Kryptographie

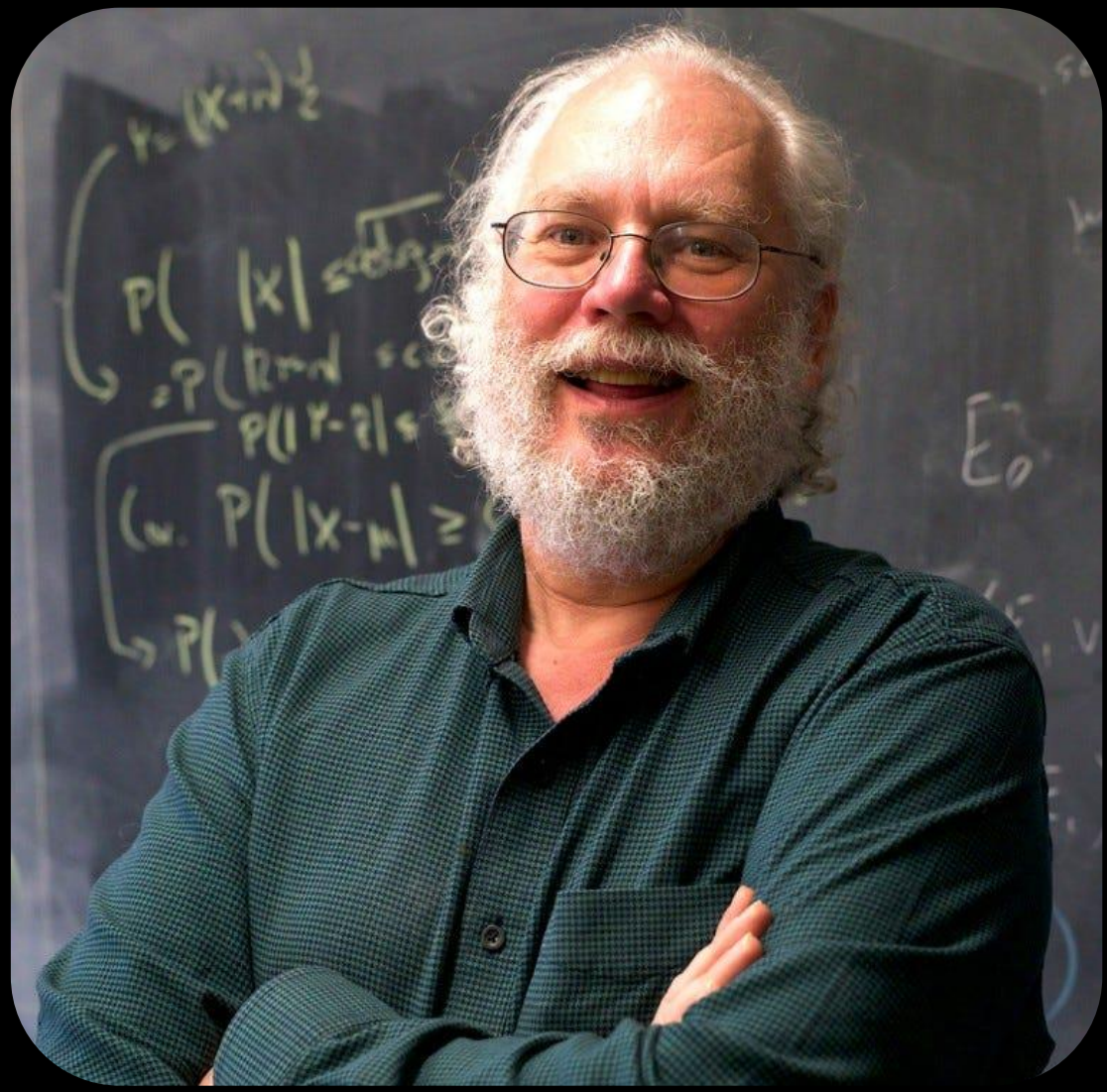
Einführung in Quantencomputer

- **Funktionsweise:** Basieren auf Quantenmechanik.
- **Qubits:** Können 0, 1 oder beides gleichzeitig sein (Superposition).
- **Vorteil:** Lösen bestimmte mathematische Aufgaben exponentiell schneller als klassische Computer.
- **Aktueller Stand:** Noch im Forschungsstadium, benötigen extrem niedrige Temperaturen, hat hohe Fehlerraten aber Prototypen mit Tausenden von Qubits existieren.



Der Shor-Algorithmus

- **Entwickler:** Peter Shor (1994).
- **Fähigkeit:** Faktorisiert große Zahlen exponentiell schneller als klassische Algorithmen.
- **Bedrohung:** Stellt die Sicherheit von *RSA* und *Elliptic Curve Cryptography (ECC)* fundamental in Frage.
  - Leistungsfähiger Quantencomputer könnte heutige RSA-Schlüssel knacken.
- **Zeitrahmen:** Kryptographisch relevante Quantencomputer in 10-20 Jahren erwartet, Fortschritte könnten dies verkürzen.
- **"Store Now, Decrypt Later":** Daten, die heute verschlüsselt werden, könnten in Zukunft entschlüsselt werden.



Post-Quanten-Kryptographie (PQC)

- **Ziel:** Neue kryptographische Methoden, die auch gegen Angriffe von Quantencomputern resistent sind.
  - Basieren auf mathematischen Problemen, die selbst für Quantencomputer schwer zu lösen sind.
- **Beispiele für Ansätze:** Gitterbasierte Kryptographie (CRYSTALS-Kyber, CRYSTALS-Dilithium), multivariate Polynome, Hash-Funktionen, fehlerkorrigierende Codes.
- **Standardisierung:** NIST hat erste PQC-Verfahren ausgewählt (CRYSTALS-Kyber, CRYSTALS-Dilithium, FALCON, SPHINCS+).
- **BSI-Empfehlung:** Zunächst hybrider Einsatz mit klassischen Verfahren ("Kryptoagilität").
- **Abgrenzung zur Quantenkryptographie (QKD):**
  - PQC: Algorithmen, die auf klassischen Computern laufen und quantenresistent sind.
  - QKD: Nutzt physikalische Eigenschaften von Photonen für sicheren Schlüsselaustausch, noch nicht praxisreif.
- **Herausforderung:** Migration zu PQC ist komplex, erfordert Integration in bestehende Systeme und fortlaufende Anpassung.

9.3 Kryptowährungen und Blockchain: Praktische Anwendungen der Kryptographie

Blockchain-Grundlagen

- **Definition:** Dezentrales, digitales Kontenbuch (Distributed Ledger Technology, DLT).
  - Speichert Informationen dauerhaft, transparent und vertrauenswürdig ohne zentrale Instanz.
  - Vergleichbar mit einem öffentlichen, unveränderlichen Kassenbuch.
- **Struktur:** Kette von Blöcken.
  - Jeder Block enthält Transaktionen, Zeitstempel und kryptographischen Hash des *vorherigen* Blocks.
- **Kernprinzipien:**
  - **Unveränderlichkeit (Immutability):** Daten in einem Block können nicht nachträglich verändert werden, ohne die gesamte Kette zu beeinflussen.
  - **Dezentralisierung:** Transaktionen werden in einem Netzwerk von Computern (Knoten) gespeichert, keine zentrale Kontrolle.
  - **Sicherheit durch Kryptographie:** Kryptographische Hashes verknüpfen Blöcke manipulationssicher.

Wie funktioniert die Blockchain-Technologie? Ein Beispiel.



Unternehmen/Person A **sendet Geld** an Unternehmen/Person B



Eine **Kopie des Blocks** wird anschließend an alle Computer im Blockchain-Netz geschickt.



Anschließend wird der Block an die **Block-Kette** ("Blockchain") angehangen. Diese wird damit zu einer **transparenten und permanenten Aufzeichnung** der Transaktion.



Alle Informationen der Transaktion von A werden **einem Datenblock hinzugefügt**.



Das Netzwerk **validiert** den Block mithilfe von Verschlüsselungstechniken und **bestätigt die Transaktion als gültig**.



Das Geld wird von A zu B **überwiesen**.

Kryptowährungen

- **Definition:** Digitale Währungen, die Kryptographie zur Sicherung von Transaktionen nutzen.
  - Dezentral verwaltet, keine zentrale Ausgabebehörde.
  - Peer-to-Peer-System (direkte Übertragung zwischen Nutzern).
- **Bitcoin:** Älteste und bekannteste Kryptowährung (seit 2009).
- **Transaktionen:** In Blockchain aufgezeichnet, pseudonymisiert (nicht direkt mit Namen verknüpft).
- **Speicherung:** Digitale Geldbörsen (Wallets).
  - **Hot Wallets:** Online, bequem, höheres Risiko.
  - **Cold Wallets (Hardware-Wallets):** Offline, sicherer.

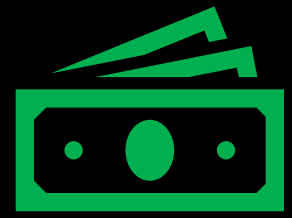




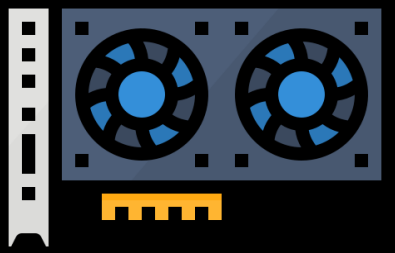




Konsensmechanismen



- **Zweck:** Sicherstellung der Einigung aller Teilnehmer auf den aktuellen Zustand der Blockchain.
- **Bekannte Mechanismen:**
  - **Proof-of-Work (PoW):**
    - Basis von Bitcoin.
    - "Miner" lösen komplexe Rechenaufgaben, um Blöcke zu validieren.
    - Ressourcen- und energieintensiv, aber hohe Sicherheit.
  - **Proof-of-Stake (PoS):**
    - Alternative zu PoW, weniger rechenintensiv.
    - Wahrscheinlichkeit der Blockerzeugung hängt vom Anteil (Stake) an der Kryptowährung ab.



CoinPro.ch

Proof of Work VS Proof of Stake





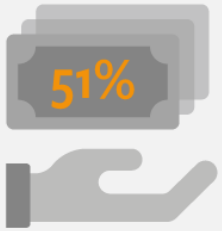
Um einen neuen Block zur Kette hinzuzufügen, müssen Miner ein kompliziertes technisches Rätsel durch den Einsatz der Rechenleistung ihres Computers lösen.



Ein Konkurrenzkampf ist im System nicht vorgesehen. Denn für die Auswahl der Blockhersteller zeichnet ein Algorithmus verantwortlich. Dieser wählt Nutzer auf Basis ihres Einsatzes aus.



Um der Blockchain einen bösartigen Block hinzuzufügen, benötigen Sie einen Computer mit einer leistungsfähigeren Rechenleistung als 51 % des gesamten Netzwerks.



Um dem System einen bösartigen Block hinzuzufügen, müssen Nutzer mindestens 51 % der Kryptowährungs-Anteile besitzen, die aktuell im Netzwerk in Umlauf sind.



Der erste Miner in der Blockchain, der ein solches Rätsel löst, erhält für die geleistete Arbeit eine Belohnung im System.



Nutzer erhalten keine Belohnung für das Erstellen eines Blocks. Dafür aber erheben Blockersteller eine Transaktionsgebühr.

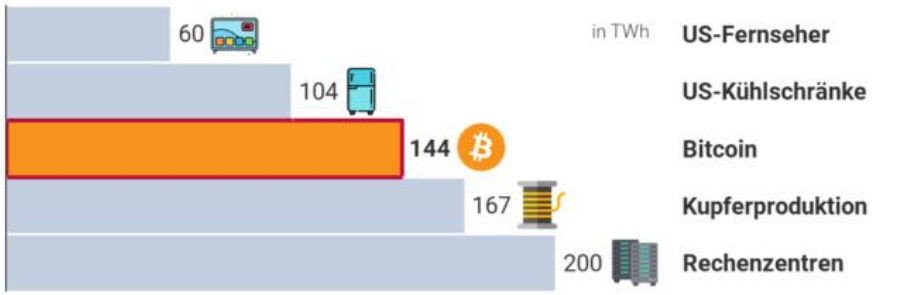
WIEVIEL ENERGIE VERBRAUCHT BITCOIN?

Der **Cambridge Bitcoin Electricity Consumption Index (CBECI)** schätzt, **wieviel elektrische Energie das Bitcoin-Netzwerk verbraucht**.

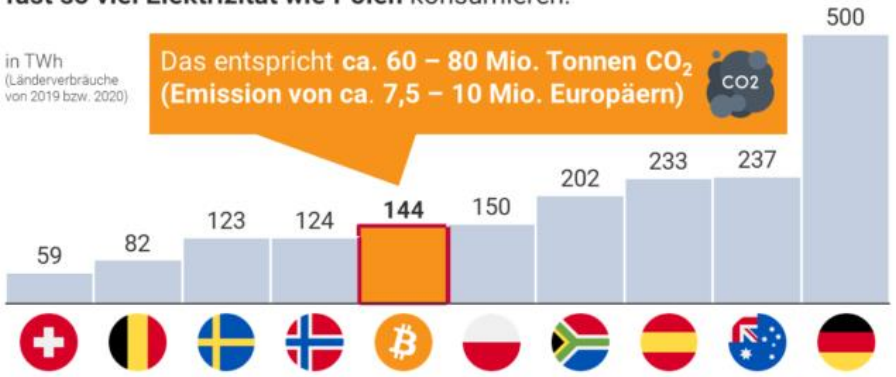
Der **geschätzte jährliche Stromverbrauch** liegt bei ca. **144** Terawatt-stunden

Dies entspricht einem **Anteil des weltweiten Elektrizitätsverbrauchs** von ca. **0,65 %**

Das wäre **mehr Energie als alle US-Kühlschränke** benötigen und ca. **70 % des Stromverbrauchs aller Rechenzentren** weltweit.



Wäre Bitcoin ein Land, dann würde es **mehr Strom als Schweden** bzw. **fast so viel Elektrizität wie Polen** konsumieren.



Datenquelle: Cambridge Centre for Alternative Finance; ccaf.io/cbeci/; Stand: 21.02.2022.

Mehr Infos beim Cambridge Centre for Alternative Finance: ccaf.io

PROF. DR. FABIAN LANG

RED FLAGS FOR CRYPTO SCAMS

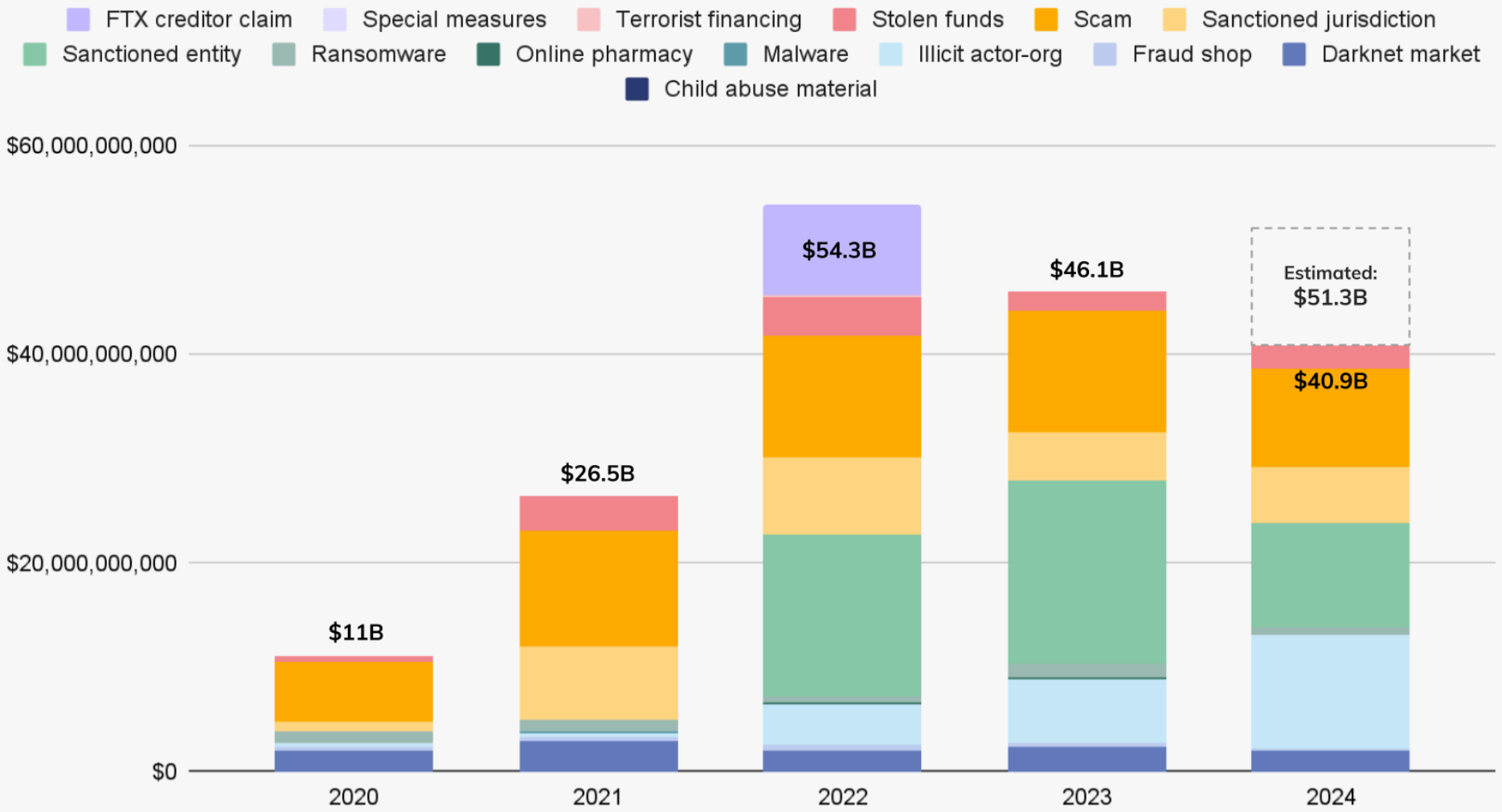
- UNSOLICITED CONTACT
- GUARANTEED HIGH RETURNS
- LACK OF TRANSPARENCY



- PRESSURE TACTICS
- REQUESTS FOR UPFRONT PAYMENTS
- UNREGISTERED OR UNREGULATED PLATFORMS

Total cryptocurrency value received by illicit addresses

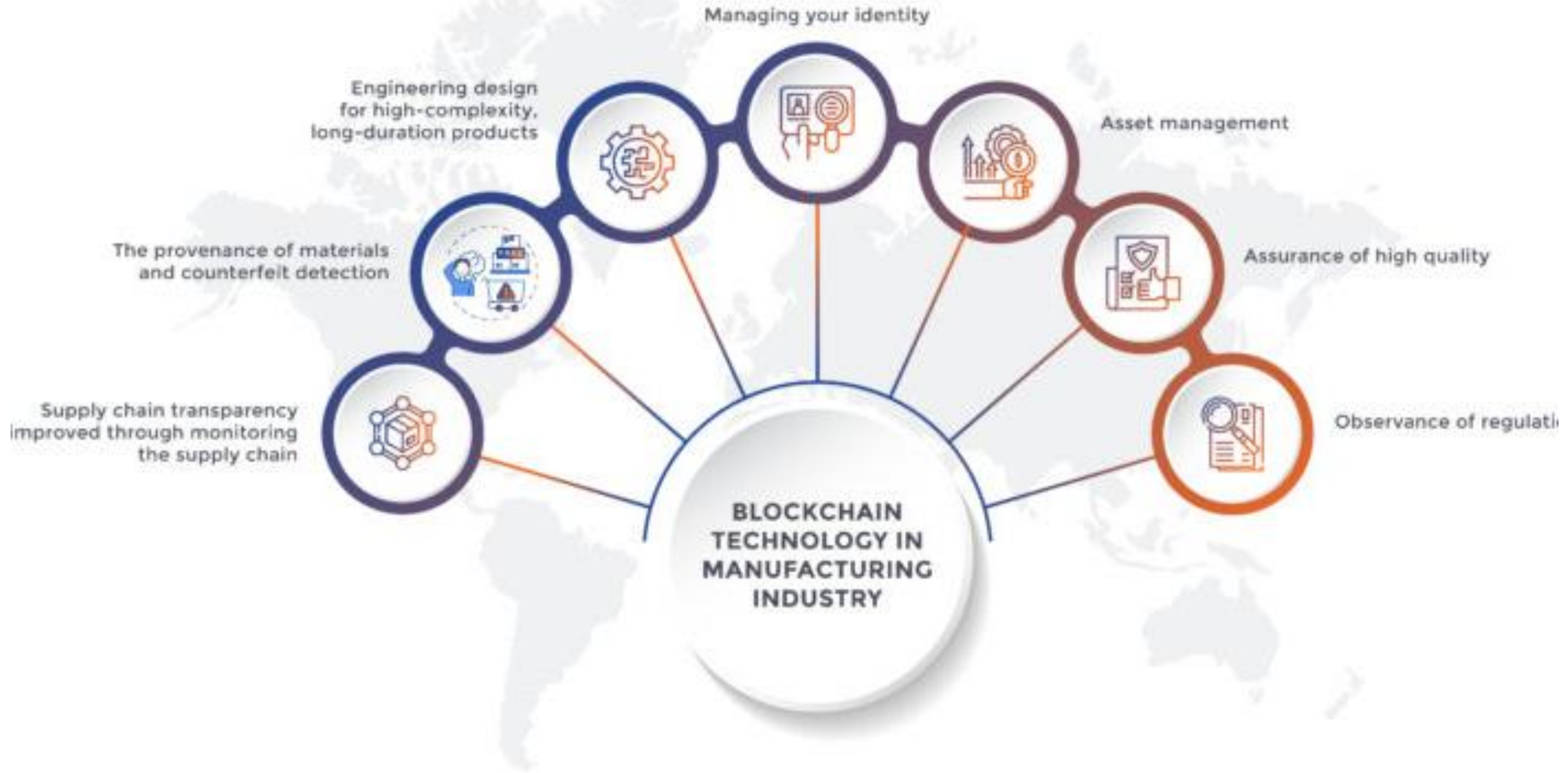
2020 - 2024



© 2025 Chainalysis

Blockchain-Anwendungen in der Industrie und Verfahrenstechnik

- **Vertrauensschaffende Infrastruktur:** Für dezentrale Systeme in komplexen industriellen Umgebungen.
- **Lieferkettenmanagement:**
  - **Transparenz und Rückverfolgbarkeit:** Manipulationssichere Aufzeichnung jeder Transaktion (Herkunft, Transportbedingungen von Chemikalien, Medikamenten).
  - **Betrugs- und Fehlerprävention:** Reduziert Betrug durch manipulationssicheres Ledger.
  - **Automatisierte Zahlungen (Smart Contracts):** Algorithmen, die sich selbst ausführen, wenn Bedingungen erfüllt sind (z.B. automatische Zahlung bei Warenlieferung).
- **Datenintegrität in industriellen Prozessen und Automatisierung:**
  - **IoT-Integration:** Sichere und unveränderliche Speicherung großer Datenmengen von IoT-Geräten.
  - **Smart Contracts für Automatisierung:** Autonome, algorithmisch ausgeführte Transaktionen zwischen Maschinen (z.B. Elektrofahrzeug verhandelt mit Ladestation).
  - **Prozessoptimierung:** Reduzierung von Verwaltungskosten durch transparente Prozessdaten.
- **Schutz geistigen Eigentums:**
  - **Lizenzierung mit Smart Contracts:** Automatische Steuerung von Lizenzgebühren und Nutzungsbedingungen.
  - **Marken- und Patentregistrierung:** Effizientere und transparentere Registrierung, Nachweis der Nutzung.
  - **Nachweis des Eigentums:** Manipulationssichere Datenbank für Urheberrechtsdaten.


Managing your identity
Engineering design for high-complexity, long-duration products
Asset management
The provenance of materials and counterfeit detection
Assurance of high quality
Supply chain transparency improved through monitoring the supply chain
Observance of regulati
BLOCKCHAIN TECHNOLOGY IN MANUFACTURING INDUSTRY

Quiz

Multiple Choice

1 Minute

Was ist das Hauptziel der Integrität in der Kryptographie?

a) Sicherstellen, dass nur berechtigte Personen Daten lesen können.

b) Garantie, dass Daten nach der Übertragung vollständig und unverändert sind.

c) Nachweis der Identität des Absenders.

d) Beschleunigung der Datenübertragung.

Welcher Algorithmus stellt eine große Bedrohung für die Sicherheit des RSA-Verfahrens durch Quantencomputer dar?

a) Grover-Algorithmus

b) Shor-Algorithmus

c) AES

d) SHA-256

Quiz

Wahr/Falsch

1 Minute

- Symmetrische Verschlüsselung verwendet immer zwei verschiedene Schlüssel – einen öffentlichen und einen privaten. 
- Asymmetrische Verschlüsselung ist langsamer als symmetrische Verschlüsselung, bietet aber Vorteile bei der Schlüsselverteilung. 
- Eine Blockchain kann nachträglich leicht verändert werden, wenn genügend Teilnehmer im Netzwerk zustimmen. 
- Post-Quanten-Kryptographie (PQC) nutzt neue mathematische Verfahren, die auch gegen Angriffe von Quantencomputern sicher sind. 

Diskussion

- Das RSA-Verfahren basiert auf der Schwierigkeit, sehr große Zahlen zu faktorisieren. Warum ist es wichtig, dass diese Zahlen *Primzahlen* sind und nicht einfach beliebige große Zahlen?

- Nennen Sie zwei konkrete Anwendungsbeispiele, wie Blockchain-Technologie die Datenintegrität in einer chemischen Produktionsanlage verbessern könnte.

5 Minuten

Sonstige Themen und Klausur

Sonstige Themen

- Was wollt ihr noch wissen?

Klausur

- 90 Minuten

- Ihr dürft euch ein Thema (auswählen) aus Kapitel 5-9, welches nicht drankommt.

- Wollt ihr noch eine Fragerunde im neuen Jahr?